

Indice

1	Introducción.....	5
1.1	El dominio del problema.....	5
1.1.1	Software de nivel industrial.....	5
1.1.2	El software es costoso.....	6
1.1.3	Tardío e In-fiabile.....	7
1.1.4	Mantenimiento y Re-trabajo.....	8
1.2	Los Desafíos de la Ingeniería de Software.....	10
1.2.1	Escala.....	10
1.2.2	Calidad y Productividad.....	12
1.2.3	Consistencia y Repetibilidad.....	14
1.2.4	Cambio.....	15
1.3	El Enfoque de la Ingeniería de Software.....	15
1.3.1	Proceso de Desarrollo por Fases.....	16
	Análisis de Requisitos.....	17
	Diseño de Software.....	17
	Codificación.....	18
	Pruebas.....	18
1.3.2	Gestión del Proceso.....	19
1.4	Resumen.....	20
2	Procesos de software.....	22
2.1	Software Process.....	22
2.1.1	Procesos y Modelos de Proceso.....	22
2.1.2	Procesos de Software de Componentes.....	23
2.1.3	Enfoque ETVX para la Especificación de Procesos.....	25
2.2	Características Deseables de un Proceso de Software.....	26
2.2.1	Predictibilidad.....	26
2.2.2	Soporte para la capacidad de prueba y el mantenimiento.....	27
2.2.3	Soporte para el cambio.....	28
2.2.4	Detección temprana de defectos.....	29
2.2.5	Mejora del Proceso y Retroalimentación.....	30
2.3	Modelos de Proceso de Desarrollo de Software.....	31
2.3.1	Modelo en Cascada.....	31
2.3.2	Prototipado.....	33
2.3.3	Desarrollo Iterativo.....	35
2.3.4	Modelo de Timeboxing.....	37
2.4	Otros Procesos de Software.....	40
2.4.1	Proceso de Gestión de Proyectos.....	41
2.4.2	El Proceso de Inspección.....	42
	Planificación.....	43
	Preparación y repaso previo (overview).....	43
	Reunión de revisión grupal.....	44
	Corrección y seguimiento.....	45
	Roles y responsabilidades.....	45
2.4.3	Proceso de Administración de Configuración de Software.....	46
	Funcionalidad de la CM.....	47
	Mecanismos de CM.....	48
	Proceso de CM.....	49
2.4.4	Proceso de Gestión de Cambios en los Requerimientos.....	51
2.4.5	Proceso de Gestión del Proceso.....	53
3	Análisis y Especificación de Requerimientos de Software.....	57

3.1	Requerimientos de Software.....	58
3.1.1	Necesidad del SRS.....	58
	Errores en los requerimientos y su impacto.....	59
	Impacto en costos.....	60
	Requerimientos cambiantes y re-trabajo.....	61
	Conclusión.....	62
3.1.2	Proceso de Requerimientos.....	62
3.2	Análisis del Problema.....	65
3.2.1	Enfoque Informal.....	66
3.2.2	Modelado de Flujo de Datos.....	66
	Diagramas de Flujo de Datos y Diccionario de Datos.....	67
	Verificación de un DFD y su diccionario de datos.....	70
	El Método de Análisis Estructurado.....	71
	Etapas del método.....	71
	Recomendaciones de construcción.....	73
	Fortalezas y limitaciones.....	73
3.2.3	Modelado Orientado a Objetos.....	73
	Conceptos Básicos y Notación.....	74
3.2.4	Prototipado.....	75
3.3	Especificación de Requisitos.....	78
3.3.1	Características de un SRS.....	79
3.3.2	Componentes de un SRS.....	81
	Requisitos Funcionales.....	81
	Requisitos de Rendimiento.....	82
3.3.3	Lenguaje de Especificación.....	83
3.4	Especificación Funcional con Casos de Uso.....	84
3.5	Validación.....	84
3.6	Métricas.....	86
3.6.1	Tamaño — Puntos de Función.....	86
4	Arquitectura de Software.....	91
4.1	El Rol de la Arquitectura de Software.....	91
	Principales usos de la descripción arquitectónica.....	92
4.2	Vistas de la Arquitectura.....	93
4.3	Vista de Componentes y Conectores.....	95
4.3.1	Componentes.....	95
4.3.2	Conectores.....	96
5	Planificación de un Proyecto de Software.....	98
5.1	Planificación del Proceso.....	99
5.2	Estimación del Esfuerzo.....	99
5.2.1	Incertidumbres en la Estimación del Esfuerzo.....	100
5.2.2	Construcción de Modelos de Estimación del Esfuerzo.....	101
5.2.3	Un Enfoque de Estimación de Abajo Hacia Arriba.....	103
5.2.4	Modelo COCOMO.....	105
5.3	Planificación y Dotación de Personal del Proyecto.....	107
5.3.1	Planificación General.....	108
5.3.2	Planificación Detallada.....	109
5.4	Plan de Gestión de la Configuración del Software.....	111
5.5	Planeamiento de Control de Calidad.....	112
5.5.1	Ciclo de Inyección y Eliminación de Defectos.....	112
5.5.2	Enfoques para la Gestión de la Calidad.....	114
5.5.3	Plan de Calidad.....	115
5.6	Gestión de Riesgos.....	116

5.6.1	Conceptos de Gestión de Riesgos.....	116
5.6.2	Evaluación de Riesgos.....	117
	Identificación de riesgos.....	117
	Análisis y priorización de riesgos.....	118
	Evaluación subjetiva.....	119
5.6.3	Control de Riesgos.....	119
	Dinamismo y monitoreo de riesgos.....	120
5.6.4	Un Enfoque Práctico de Gestión de Riesgos.....	120
5.7	Plan de Monitoreo del Proyecto.....	121
5.7.1	Mediciones.....	121
5.7.2	Monitoreo y Seguimiento del Proyecto.....	122
6	Diseño Orientado a Funciones.....	124
6.1	Principios de Diseño.....	124
6.2	Conceptos a Nivel de Módulo.....	126
6.2.1	Acoplamiento.....	126
6.2.2	Cohesión.....	127
7	Diseño Orientado a Objetos.....	131
7.1	Análisis OO y Diseño OO.....	131
7.2	Conceptos OO.....	133
7.3	Conceptos de Diseño.....	133
7.3.1	Acoplamiento.....	134
7.3.2	Cohesión.....	135
8	Diseño Detallado.....	137
8.1	Diseño Detallado y PDL.....	137
8.1.1	PDL.....	137
8.1.2	Diseño de lógica o de algoritmos.....	138
8.1.3	Modelado de estado de las clases.....	141
8.2	Verificación.....	143
8.2.1	Revisiones de diseño (Design Walkthroughs).....	143
8.2.2	Revisión Crítica del Diseño.....	144
8.2.3	Verificadores de Consistencia.....	145
8.3	Métricas.....	145
8.3.1	Complejidad Ciclomática.....	146
8.3.2	Vínculos de datos.....	147
8.3.3	Métrica de Cohesión.....	148
9	Codificación.....	149
9.1	Principios y guías de programación.....	150
9.1.1	Errores comunes de codificación.....	151
	Fugas de memoria (Memory Leaks).....	151
	Liberar un recurso ya liberado.....	151
	Desreferenciación de NULL.....	152
	Falta de direcciones únicas.....	153
	Errores de sincronización.....	153
9.1.2	Programación Estructurada.....	153
9.1.3	Ocultamiento de Información.....	156
9.2	Proceso de Codificación.....	157
9.2.1	Un Proceso de Codificación Incremental.....	157
9.2.2	Desarrollo Guiado por Pruebas (Test Driven Development).....	159
9.2.3	Programación en Pareja (Pair Programming).....	160
9.2.4	Control del Código Fuente y Construcción.....	161
9.3	Refactorización.....	163
9.3.1	Conceptos Básicos.....	163

9.3.2 Un ejemplo.....	165
9.3.3 Malos olores.....	165
9.3.4 Refactorizaciones comunes.....	166
Mejorando los métodos.....	166
Mejorando las clases.....	167
Mejorando las jerarquías.....	168
9.4 Verificación.....	169
9.4.1 Inspección de código (Code Inspections).....	170
10 Pruebas (Testing).....	172
10.1 Fundamentos de las pruebas.....	172
10.1.1 Error, Falla y Fallo.....	172
10.1.2 Oráculos de Prueba.....	174
10.1.3 Casos de Prueba y Criterios de Prueba.....	175
Selección de Casos de Prueba y Criterios.....	176
Propiedades de un Criterio de Prueba.....	176
Limitaciones Prácticas.....	176
Relación entre Criterios.....	177
Conclusión.....	177
10.1.4 Psicología de las pruebas.....	177
10.2 Pruebas de caja negra (Black-Box Testing).....	178
10.2.1 Partición en Clases de Equivalencia.....	179
10.2.2 Análisis de Valores Límite.....	180
10.2.3 Graficado Causa-Efecto.....	181
10.2.4 Pruebas por pares.....	183
10.2.5 Casos especiales.....	185
10.2.6 Pruebas basadas en estados.....	186
10.3 Pruebas de caja blanca.....	188
10.3.1 Criterios basados en el flujo de control.....	189

Ingeniería del Software

La Ingeniería de Software es la aplicación de un enfoque sistemático, disciplinado, y cuantificable al desarrollo, operación, y mantenimiento del software.

Software

Colección de programas, procedimientos, y la documentación y datos asociados que determinan la operación de un sistema de computación.

1 Introducción

1.1 El dominio del problema

En ingeniería de software no tratamos con programas que la gente desarrolla para ilustrar un concepto o como pasatiempo (a lo que aquí llamamos sistemas de estudiante). En cambio, el dominio de problemas se centra en software que resuelve necesidades reales de ciertos usuarios, en contextos donde sistemas más grandes o incluso negocios completos pueden depender de ese software, y donde los fallos pueden provocar pérdidas significativas, ya sea de forma directa o indirecta.

A este tipo de software lo llamamos software de nivel industrial.

Comencemos primero hablando de la diferencia clave entre el software de estudiante y el software de nivel industrial.

1.1.1 Software de nivel industrial

Un sistema desarrollado por un estudiante está pensado principalmente con fines de demostración; por lo general, no se utiliza para resolver un problema real de alguna organización. En consecuencia, nada verdaderamente importante depende de que funcione correctamente. Y como nada de peso depende de ese software, la existencia de “bugs” (defectos o fallos) no es un gran problema. Por eso, normalmente no se diseña pensando en aspectos de calidad como portabilidad, robustez, fiabilidad o usabilidad.

Además, el software de estudiante suele ser utilizado por el mismo desarrollador que lo creó, así que no hay necesidad de documentación, y nuevamente los errores no son críticos, ya que el propio usuario–desarrollador puede corregirlos en cuanto los detecta.

En cambio, un sistema de software de nivel industrial se construye para resolver un problema real de un cliente, y es utilizado por la organización del cliente para operar alguna parte de su negocio (entendiendo “negocio” en un sentido muy amplio: puede ser administrar inventarios, finanzas, monitorear pacientes, controlar el tráfico aéreo, etc.). Dicho de otro modo, actividades importantes dependen de que el sistema funcione correctamente. Y una falla en este tipo de sistemas puede tener un impacto enorme: pérdidas económicas o comerciales, molestias para los usuarios, e incluso pérdida de bienes o vidas humanas.

Por lo tanto, este software necesita ser de alta calidad en propiedades como confiabilidad, robustez y facilidad de uso.

Este requisito de alta calidad tiene muchas implicaciones:

- **Primero**, el software debe probarse exhaustivamente antes de ponerse en uso. Esta necesidad de pruebas rigurosas aumenta considerablemente el costo. En un proyecto de software industrial, entre un 30% y un 50% del esfuerzo total puede dedicarse a las pruebas (mientras que en un proyecto de estudiante, ¡incluso un 5% podría considerarse mucho!).
- **Segundo**, desarrollar software de alta calidad requiere dividir el proceso en fases, de manera que la salida de cada una sea revisada y evaluada para eliminar defectos lo antes posible. Esta necesidad de dividir el problema y detectar errores temprano implica más documentación, más estándares y más procesos. Todo esto aumenta el esfuerzo necesario para construir el software, por lo que la productividad al desarrollar software industrial suele ser mucho menor que la de un software estudiantil.

El software de nivel industrial también presenta otras características que no existen en los sistemas de estudiante. Generalmente, para un mismo problema, los requisitos detallados de lo que debe hacer el software aumentan considerablemente. Además de los requisitos de calidad, se suman necesidades de respaldo y recuperación, tolerancia a fallos, cumplimiento de estándares, portabilidad, etc. Todo esto, por lo general, hace que el sistema sea más complejo y más grande.

De hecho, para un mismo problema, el software industrial puede tener un tamaño al menos dos veces mayor que el sistema de estudiante. Si asumimos una productividad cinco veces menor y un tamaño dos veces mayor, construir un software de nivel industrial puede requerir unas **10 veces más esfuerzo** que un software de estudiante para el mismo problema. La “regla práctica” de Brooks también dice que el software industrial puede costar unas 10 veces más que el software estudiantil [25].

La industria del software está interesada principalmente en desarrollar software de nivel industrial, y el área de ingeniería de software se enfoca en cómo construir este tipo de sistemas.

En el resto del libro, cuando usemos la palabra *software*, nos estaremos refiriendo a *software de nivel industrial*.

El IEEE define software como: “*el conjunto de programas informáticos, procedimientos, reglas, y la documentación y datos asociados*” [91]. Esta definición deja claro que el software no son solo programas, sino que también incluye toda la documentación y datos relacionados. Esto implica que la disciplina que se ocupa de desarrollar software no debe centrarse únicamente en crear programas, sino en producir todos los elementos que lo componen.

1.1.2 El software es costoso

El software de nivel industrial es muy caro, principalmente porque su desarrollo es extremadamente intensivo en mano de obra. Para darnos una idea de los costos implicados, consideremos el estado actual de la práctica en la industria.

La métrica más utilizada para medir el tamaño del software en la industria es el número de líneas de código (LOC) o miles de líneas de código (KLOC) entregadas.

Dado que el principal costo de producir software es el trabajo humano, el **costo de desarrollo** se mide generalmente en **persona-meses** de esfuerzo invertido. Asimismo, la **productividad en la industria** suele expresarse en **LOC (o KLOC) por persona-mes**.

En el desarrollo de nuevo código, la productividad típica en la industria oscila entre **300 y 1.000 LOC por persona-mes**. Es decir, a lo largo de todo el ciclo de desarrollo, un programador promedio produce entre 300 y 1.000 LOC por mes.

Las empresas de software suelen cobrar al cliente para el que desarrollan a partir de **100.000 USD por persona-año**, o más de **8.000 USD por persona-mes** (aproximadamente 50 USD por hora). Con las cifras actuales de productividad, esto significa que **cada línea de código entregada cuesta entre 8 y 25 USD**. Dicho de otro modo, una sola línea de código puede costar entre 8 y 25 dólares con los niveles actuales de costos y productividad.

Incluso proyectos pequeños pueden fácilmente alcanzar **50.000 LOC**. Con esta productividad, un proyecto así costaría entre **0,5 y 1,25 millones de USD**.

Con la potencia actual de las computadoras, un software de este tamaño puede ejecutarse perfectamente en una estación de trabajo o un pequeño servidor. Esto implica que un software que puede costar más de un millón de dólares puede correr sobre un hardware cuyo precio no pasa de unas decenas de miles de dólares. Esto demuestra claramente que **el costo del hardware es solo una fracción del costo del software** que ejecuta.

Este ejemplo muestra que el software no solo es muy caro, sino que suele ser el **componente principal** del costo total de un sistema automatizado, siendo el hardware un elemento mucho menor. Esto se refleja en el gráfico clásico de inversión hardware–software mostrado en la Figura 1.1 [17].

Como se observa en la Figura 1.1, en los primeros tiempos el costo del hardware solía dominar el costo total del sistema. Pero, a medida que el precio del hardware ha ido disminuyendo y continúa bajando, y que su potencia se duplica aproximadamente cada 2 años (la Ley de Moore), permitiendo ejecutar sistemas de software cada vez más grandes, **el costo del software ha pasado a ser el factor dominante en los sistemas**.

1.1.3 Tardío e In-fiabile

A pesar de los importantes avances en las técnicas de desarrollo de software, esta sigue siendo un área débil. En una encuesta realizada a más de 600 empresas, más del **35%** informó tener algún proyecto de desarrollo relacionado con computadoras que clasificaron como “*runaway*” [131].

Un *runaway* no es simplemente un proyecto que se retrasa un poco o que supera ligeramente el presupuesto, sino uno en el que **el presupuesto y el cronograma están completamente fuera de control**. El problema se ha vuelto tan grave que incluso ha generado una industria dedicada a resolverlo: existen empresas consultoras que asesoran sobre cómo recuperar estos proyectos, y una de ellas obtuvo más de **30 millones de dólares** en ingresos provenientes de más de 20 clientes [131].

De forma similar, se han documentado numerosos casos de **falta de fiabilidad** en el software: programas que no hacen lo que deberían hacer o que hacen cosas que no deberían. En una encuesta

de defensa, se informó que **más del 70% de todas las fallas de equipos** se debieron al software. Y esto en sistemas que también cuentan con componentes eléctricos, hidráulicos y mecánicos.

Este dato deja claro que **otras disciplinas de la ingeniería han avanzado mucho más que la ingeniería de software**, y que, en un sistema que combina productos de varias ramas de la ingeniería, el software suele ser el componente más débil. El fallo de un vuelo temprano del programa Apolo se atribuyó al software. De forma parecida, la falla en una prueba de lanzamiento de un misil en India se debió a problemas de software. Incluso muchos bancos han perdido millones de dólares debido a errores e imprecisiones en su software [122].

Un apunte sobre la **causa de la falta de fiabilidad en el software**: las fallas de software son distintas de las que ocurren, por ejemplo, en sistemas mecánicos o eléctricos. En estos últimos, los productos fallan por cambios en sus propiedades físicas o eléctricas debido al envejecimiento.

En cambio, un producto de software **nunca se desgasta con el tiempo**. Sus fallos se producen por *bugs* o errores introducidos durante el diseño o el desarrollo. Así, aunque un sistema de software pueda fallar después de funcionar correctamente durante un tiempo, **el error que provoca esa falla ya estaba presente desde el principio**; simplemente, se ejecutó en el momento del fallo. Esto es muy diferente a otros sistemas, en los que un fallo suele significar que, en algún momento antes de producirse, surgió un problema (por desgaste) que antes no existía.

1.1.4 Mantenimiento y Re-trabajo

Una vez que el software se entrega y se pone en funcionamiento, entra en la fase de mantenimiento. ¿Por qué es necesario el mantenimiento si el software no envejece?

El mantenimiento del software no se debe a que sus componentes se desgasten y necesiten reemplazo, sino a que normalmente permanecen **errores residuales** en el sistema que deben corregirse a medida que se descubren. Se cree comúnmente que, con el estado actual de la tecnología, prácticamente **todo el software desarrollado contiene errores o bugs**. Muchos de ellos solo aparecen después de que el sistema ha estado en operación, incluso durante mucho tiempo.

Estos errores, una vez detectados, deben eliminarse, lo que implica cambiar el software. A esto se le llama **mantenimiento correctivo**.

Incluso sin *bugs*, el software suele cambiar con frecuencia. La principal razón es que el software a menudo debe **actualizarse y mejorarse** para incluir más funcionalidades y ofrecer más servicios. Esto también requiere modificaciones.

Se ha argumentado que, una vez desplegado, el software opera en un **entorno cambiante**. Por lo tanto, las necesidades que motivaron su desarrollo también cambian para adaptarse a ese nuevo entorno. Esto obliga a que el software se adapte a las nuevas necesidades. A su vez, el software modificado cambia el entorno, lo que genera nuevos cambios. Este fenómeno se conoce como **ley de la evolución del software** y el mantenimiento que ocasiona se llama **mantenimiento adaptativo**.

Aunque el mantenimiento no se considera parte directa del desarrollo, es una actividad crucial en la vida de un producto de software. Si consideramos todo el ciclo de vida, **el costo del mantenimiento**

suele superar el costo del desarrollo. La proporción mantenimiento/desarrollo se ha estimado en 80:20, 70:30 o 60:40.

El trabajo de mantenimiento se basa en software existente, mientras que el desarrollo crea software nuevo. Por ello, el mantenimiento gira en torno a **entender el software existente**, lo que incluye no solo el código, sino también la documentación relacionada.

Al modificar el software, el mantenedor debe comprender bien **los efectos del cambio**, ya que es fácil introducir efectos secundarios no deseados. Para comprobar que las partes no modificadas siguen funcionando igual, se realiza **pruebas de regresión** (*regression testing*), que consisten en ejecutar casos de prueba antiguos para asegurarse de que no se han introducido nuevos errores.

En resumen, el mantenimiento implica:

- Comprender el código y la documentación existente
- Analizar el impacto del cambio
- Modificar código y documentación
- Probar las nuevas partes
- Reprobar las partes antiguas

Como muchas veces durante el desarrollo no se piensa en las necesidades de los futuros mantenedores, suele haber poca documentación de apoyo, lo que, junto con la complejidad de la tarea, convierte al mantenimiento en la **actividad más costosa** de la vida del software.

El mantenimiento es un tipo de cambio que se realiza **después** de completar el desarrollo y desplegar el software. Sin embargo, existen otros cambios que generan **re-trabajo** (*rework*) durante el propio desarrollo.

Uno de los mayores problemas, especialmente en sistemas grandes y complejos, es que los **requisitos** del software no se entienden del todo. Para especificarlos completamente, todas las funcionalidades, interfaces y restricciones deben definirse antes de empezar el desarrollo. Esto obliga tanto a clientes como desarrolladores a **imaginar cómo debería comportarse el software terminado**, algo difícil de hacer, sobre todo en proyectos grandes.

En la práctica, el desarrollo comienza cuando se cree que los requisitos están bien definidos. Pero con el tiempo y una mejor comprensión del sistema, los clientes suelen descubrir requisitos adicionales no especificados al inicio. Esto lleva a cambios de requisitos, que obligan a modificar también el diseño y el código, generando re-trabajo.

Además, no es solo por requisitos mal entendidos: en proyectos largos, las **necesidades del cliente cambian con el tiempo**. Los requerimientos originales reflejan el contexto actual en el que se inició el proyecto, pero, al cambiar el contexto, cambian también las necesidades. Obviamente, el cliente quiere que el sistema final satisfaga sus necesidades más recientes, lo que provoca más cambios y re-trabajo.

Se estima que el re-trabajo representa **entre el 30% y el 40% del costo de desarrollo** [22]. Es decir, de todo el esfuerzo invertido en desarrollo, entre un tercio y casi la mitad se dedica a rehacer trabajo debido a cambios. Por eso, los cambios y el re-trabajo son uno de los principales contribuyentes a la **crisis del software**.

A diferencia de otros problemas ya mencionados, el re-trabajo no siempre refleja deficiencias técnicas del desarrollo: muchas veces es el propio cliente quien impulsa los cambios debido a la evolución de sus necesidades.

1.2 Los Desafíos de la Ingeniería de Software

Ahora que tenemos una mejor comprensión del dominio del problema con el que trata la ingeniería de software, orientemos nuestra discusión hacia la propia Ingeniería de Software.

La ingeniería de software se define como el enfoque sistemático para el desarrollo, operación, mantenimiento y retiro del software [91]. En este libro nos enfocaremos principalmente en el desarrollo.

El uso del término **enfoque sistemático** para el desarrollo de software implica que se utilizan metodologías para desarrollarlo que son **repetibles**. Es decir, si las metodologías son aplicadas por diferentes grupos de personas, se producirá un software similar. En esencia, el objetivo de la ingeniería de software es acercar el desarrollo de software a la ciencia y la ingeniería y alejarlo de los enfoques ad-hoc para el desarrollo, cuyos resultados no son predecibles, pero que han sido usados ampliamente en el pasado y que aún se siguen utilizando para desarrollar software.

Como se mencionó, el software de tipo industrial está destinado a resolver algún problema del cliente. (Usamos el término **cliente** en un sentido muy general, para referirnos a las personas cuyas necesidades deben ser satisfechas por el software). Por lo tanto, el problema es desarrollar (de manera sistemática) un software que satisfaga las necesidades de algunos usuarios o clientes. Este problema fundamental con el que trata la ingeniería de software se muestra en la Figura 1.2.

Aunque el problema básico es desarrollar de manera sistemática un software que satisfaga al cliente, existen algunos factores que afectan los enfoques seleccionados para resolver el problema. Estos factores son las fuerzas primarias que impulsan el progreso y el desarrollo en el campo de la ingeniería de software. Nosotros consideramos estos como los **principales desafíos de la ingeniería de software** y aquí discutiremos algunos de los más importantes.

1.2.1 Escala

Un factor fundamental con el que la ingeniería de software debe lidiar es el tema de la **escala**; el desarrollo de un sistema muy grande requiere un conjunto de métodos muy diferente al que se usa para desarrollar un sistema pequeño. En otras palabras, los métodos que se utilizan para desarrollar sistemas pequeños, en general, no escalan a sistemas grandes.

Un ejemplo ilustra este punto. Consideremos el problema de contar personas en una sala frente a realizar un censo de un país. Ambos son esencialmente problemas de conteo. Pero los métodos utilizados para contar personas en una sala (probablemente ir fila por fila o columna por columna) simplemente no funcionarían para realizar un censo. Se deberá usar un conjunto de métodos distinto para llevar adelante un censo, y este problema requerirá considerablemente más gestión, organización y validación, además del simple conteo.

De manera similar, los métodos que se pueden usar para desarrollar programas de unos pocos cientos de líneas no se pueden esperar que funcionen cuando se necesita desarrollar software de unos pocos cientos de miles de líneas. Se debe usar un conjunto diferente de métodos para desarrollar software de gran tamaño. Cualquier proyecto grande implica el uso de **ingeniería** y de **gestión de proyectos**. En proyectos de software, por ingeniería entendemos los métodos, procedimientos y herramientas que se utilizan. En proyectos pequeños, pueden usarse métodos informales tanto para el desarrollo como para la gestión. Sin embargo, para proyectos grandes, ambos tienen que ser mucho más formales, como se muestra en la Figura 1.3.

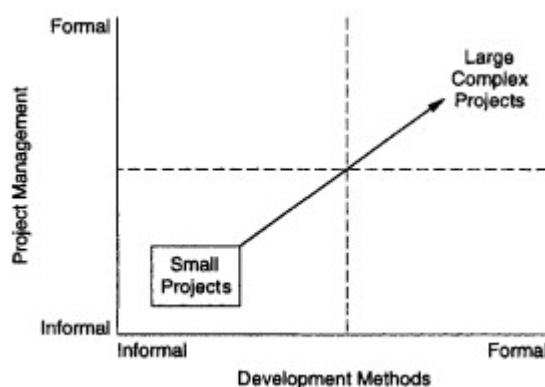


Figura 1.3 : El problema de la escala.

Tal como muestra la figura, al tratar con un proyecto pequeño de software, la capacidad de ingeniería requerida es baja (todo lo que necesitas saber es programar y hacer un poco de pruebas) y el requerimiento de gestión de proyectos también es bajo. Sin embargo, cuando la escala cambia a grande, para resolver esos problemas adecuadamente es esencial avanzar en ambas direcciones: los métodos de ingeniería utilizados para el desarrollo deben ser más formales, y la gestión del proyecto de desarrollo también debe ser más formal.

Por ejemplo, si juntamos a 50 programadores brillantes (que saben cómo desarrollar programas pequeños correctamente) sin una gestión y procedimientos de desarrollo formales, y les pedimos que desarrollen un sistema de control de inventario en línea para un fabricante de automóviles, es muy poco probable que produzcan algo útil. Para ejecutar el proyecto con éxito, se debe utilizar un método adecuado de ingeniería del sistema y el proyecto debe estar fuertemente gestionado para asegurarse de que los métodos efectivamente se están siguiendo y de que el costo, el cronograma y la calidad están bajo control.

No existe una definición universalmente aceptada de qué es un proyecto "pequeño" y qué es un proyecto "grande", y las escalas cambian claramente con el tiempo. Sin embargo, de manera informal, podemos usar órdenes de magnitud y decir que un proyecto es **pequeño** si su tamaño es menor a 10 KLOC, **mediano** si el tamaño es menor a 100 KLOC (y mayor a 10), **grande** si el tamaño es menor a un millón de LOC, y **muy grande** si el tamaño es de muchos millones de LOC.

Para darnos una idea del tamaño de algunos productos de software reales, en la Tabla 1.1 se muestran los tamaños aproximados de algunos productos conocidos.

Size (KLOC)	Software	Languages
980	gcc	ansic, cpp, yacc
320	perl	perl, ansic, sh
305	teTeX	ansic, perl
200	openssl	ansic, cpp, perl
200	Python	python, ansic
100	apache	ansic, sh
90	cvs	ansic, sh
65	sendmail	ansic
60	xfig	ansic
45	gnuplot	ansic, lisp
38	openssh	ansic
30,000	Red Hat Linux	ansic, cpp
40,000	Windows XP	ansic, cpp

Tabla 1.1 : Tamaño en KLOC de algunos productos conocidos.

1.2.2 Calidad y Productividad

Una disciplina de ingeniería, casi por definición, está guiada por parámetros prácticos de **costo**, **cronograma** y **calidad**. Una solución que demande enormes recursos y muchos años puede no ser aceptable. De la misma manera, una solución de baja calidad, incluso si tiene un costo bajo, puede no ser de mucha utilidad. Como todas las disciplinas de ingeniería, **la ingeniería de software está impulsada por estos tres factores principales: costo, cronograma y calidad.**

El costo de desarrollar un sistema corresponde al costo de los recursos utilizados para el sistema, lo cual, en el caso del software, está dominado por el costo de la mano de obra, ya que el desarrollo es en gran medida una actividad intensiva en trabajo humano. Por lo tanto, el costo de un proyecto de software suele medirse en **persona-meses**; es decir, el costo se considera el número total de persona-meses invertidos en el proyecto. (Los persona-meses pueden convertirse a un monto en dólares multiplicándolos por el costo promedio en dólares —incluyendo costos indirectos como hardware y herramientas— de un persona-mes).

El **cronograma** es un factor importante en muchos proyectos. Las tendencias del mercado exigen que el *time-to-market* de un producto se reduzca; es decir, que el ciclo desde el concepto hasta la entrega sea más corto. Para el software, esto significa que debe desarrollarse más rápido.

La **productividad**, medida en términos de salida (*KLOC*) por persona-mes, puede capturar adecuadamente tanto las preocupaciones de costo como de cronograma. Si la productividad es mayor, está claro que el costo en términos de persona-meses será menor (el mismo trabajo se puede realizar con menos persona-meses). De manera similar, si la productividad es mayor, la posibilidad de desarrollar el software en menos tiempo mejora: un equipo con alta productividad terminará un trabajo en menos tiempo que un equipo del mismo tamaño con baja productividad. (El tiempo real que tomará el proyecto, por supuesto, también depende del número de personas asignadas al mismo). En otras palabras, la productividad es un factor clave en todos los negocios, y el deseo de alta productividad dicta, en gran medida, cómo se hacen las cosas.

El otro gran factor que guía cualquier disciplina de producción es la **calidad**. Hoy en día, la calidad es un mantra central, y las estrategias de negocio se diseñan en torno a ella. Claramente, **desarrollar**

software de alta calidad es otro objetivo fundamental de la ingeniería de software. Sin embargo, mientras que el costo es un concepto generalmente bien entendido, la calidad en el contexto del software necesita una discusión más profunda. Aquí utilizamos el estándar internacional de calidad de productos de software como base.



Figura 1.4: Atributos de calidad del software

Según el modelo de calidad adoptado por este estándar, la calidad del software se compone de **seis atributos principales** (llamados características), tal como se muestra en la Figura 1.4. Estos **seis atributos** tienen características detalladas que se consideran las básicas y que pueden (y deben) medirse usando métricas adecuadas. A nivel superior, para un producto de software, estos atributos se definen de la siguiente manera:

- **Funcionalidad:** La capacidad de proporcionar funciones que satisfacen necesidades explícitas e implícitas cuando se utiliza el software.
- **Confiabilidad:** La capacidad de mantener un nivel de desempeño especificado.
- **Usabilidad:** La capacidad de ser entendido, aprendido y utilizado.
- **Eficiencia:** La capacidad de ofrecer un rendimiento adecuado en relación con la cantidad de recursos utilizados.
- **Mantenibilidad:** La capacidad de ser modificado con fines de corrección, mejora o adaptación.
- **Portabilidad:** La capacidad de adaptarse a diferentes entornos especificados sin necesidad de acciones o medios adicionales más allá de los previstos en el producto.

Las características de cada atributo brindan mayor detalle. Por ejemplo, la **usabilidad** incluye entendibilidad, capacidad de aprendizaje y operabilidad; la **mantenibilidad** incluye capacidad de cambio, capacidad de prueba y estabilidad; mientras que la **portabilidad** incluye adaptabilidad, instalabilidad, etc. La **funcionalidad** incluye adecuación (si se proporciona el conjunto apropiado de funciones), exactitud (los resultados son correctos) y seguridad. Es importante notar que, en esta clasificación, la **seguridad** se considera una característica de la funcionalidad y se define como: “la capacidad de proteger información y datos de manera que personas o sistemas no autorizados no puedan leerlos ni modificarlos, y que personas o sistemas autorizados no vean denegado su acceso”.

Existen dos consecuencias importantes de que la calidad tenga múltiples dimensiones. Primero, la calidad del software no puede reducirse a un solo número (o un solo parámetro). Y segundo, el concepto de calidad es específico de cada proyecto. En un proyecto ultra-sensible, la confiabilidad puede ser de máxima importancia y no así la usabilidad; mientras que en un paquete comercial de juegos para PC, la usabilidad puede ser la prioridad principal y no la confiabilidad. Por lo tanto, para cada proyecto de desarrollo de software debe especificarse un objetivo de calidad antes de

comenzar el desarrollo, y la meta del proceso de desarrollo debe ser satisfacer ese objetivo de calidad.

A pesar de que existen muchos factores de calidad, la **confiabilidad** suele aceptarse como el criterio de calidad más importante. Dado que la falta de confiabilidad del software se debe a la presencia de defectos en el mismo, una medida de la calidad es el número de defectos en el software entregado por unidad de tamaño (generalmente miles de líneas de código, o KLOC). Con este criterio principal de calidad, el objetivo es reducir lo más posible el número de defectos por KLOC. Las mejores prácticas actuales en ingeniería de software han logrado reducir la densidad de defectos a menos de 1 por KLOC.

Es importante señalar que, para usar esta definición de calidad, debe estar claramente definido qué se considera un **defecto**. Un defecto puede ser un problema en el software que lo hace colapsar, un problema que provoca que una salida no se alinee correctamente, o incluso un error de ortografía en una palabra. La definición exacta de lo que se considera un defecto dependerá claramente del proyecto o de los estándares de la organización que lo desarrolla (generalmente es esto último).

1.2.3 Consistencia y Repetibilidad

Ha habido muchos casos en los que se desarrolló software de alta calidad con una productividad muy elevada. Pero, han existido muchos más casos en los que se desarrolló software con baja calidad o baja productividad. Un desafío clave al que se enfrenta la ingeniería de software es cómo asegurar que los resultados exitosos puedan repetirse, y que pueda existir cierto grado de consistencia tanto en la calidad como en la productividad.

Podemos decir que una organización que desarrolla un sistema con alta calidad y una productividad razonable, pero que no es capaz de mantener esos niveles de calidad y productividad en otros proyectos, no conoce verdaderamente la buena ingeniería de software. Uno de los objetivos de los métodos de la ingeniería de software es que sistema tras sistema pueda ser producido con alta calidad y productividad. Es decir, que los métodos que se usan sean repetibles a lo largo de los proyectos, lo que conduce a una consistencia en la calidad del software producido.

Una organización dedicada al desarrollo de software no solo quiere alta calidad y productividad, sino que quiere obtenerlas de manera consistente. En otras palabras, una organización de desarrollo de software desea producir software de calidad consistente con productividad consistente. La consistencia en el desempeño es un factor importante para cualquier organización; le permite predecir el resultado de un proyecto con una precisión razonable, y mejorar sus procesos para producir productos de mayor calidad e incrementar su productividad. Sin consistencia, incluso estimar el costo de un proyecto se vuelve difícil.

Lograr la consistencia es un problema importante que la ingeniería de software debe abordar. Como puede imaginarse, este requisito de consistencia obliga a seguir ciertos procedimientos estandarizados para desarrollar software. No existen metodologías aceptadas de manera global y diferentes organizaciones utilizan distintas. Sin embargo, dentro de una organización, la consistencia se logra utilizando sus metodologías elegidas de manera uniforme. Marcos como **ISO9001** y el **Modelo de Madurez de Capacidades (CMM)** alientan a las organizaciones a estandarizar metodologías, usarlas de forma consistente y mejorarlas con base en la experiencia. Discutiremos este tema un poco más en el próximo capítulo.

1.2.4 Cambio

Hemos mencionado antes cómo el mantenimiento y el re-trabajo son muy costosos y cómo forman parte integral del dominio de problemas con los que trata la ingeniería de software. En el mundo actual, el cambio en los negocios es muy rápido. A medida que los negocios cambian, requieren que el software que los respalda también cambie. En general, a medida que el mundo cambia más rápido, el software también debe cambiar más rápido.

El cambio rápido tiene un impacto especial en el software. Como el software es fácil de modificar debido a la ausencia de propiedades físicas que dificulten el cambio, la expectativa hacia el software para adaptarse es mucho mayor.

Por lo tanto, uno de los desafíos de la ingeniería de software es **acomodar y aceptar el cambio**. Como veremos, se utilizan distintos enfoques para manejar el cambio. Pero el cambio es hoy uno de los principales motores de la ingeniería de software. Los enfoques que logran producir software de alta calidad con alta productividad pero que no pueden aceptar ni adaptarse al cambio tienen muy poca utilidad actualmente, ya que solo pueden resolver un número reducido de problemas que son resistentes al cambio.

1.3 El Enfoque de la Ingeniería de Software

Ahora entendemos el dominio del problema y los factores básicos que impulsan la ingeniería de software. Podemos considerar la **alta calidad y productividad (Q&P)** como el objetivo fundamental que **debe lograrse de manera consistente** para **problemas a gran escala** y **bajo la dinámica de los cambios**. La Q&P alcanzada durante un proyecto dependerá claramente de muchos factores, pero las tres fuerzas principales que gobiernan la Q&P son las **personas**, los **procesos** y la **tecnología**, a menudo llamadas el **Triángulo de Hierro**, como se muestra en la Figura 1.5.



Figura 1.5: Triangulo de hierro

Por lo tanto, **para lograr una alta Q&P se debe usar buena tecnología, se deben aplicar buenos procesos o métodos, y las personas que realicen el trabajo deben estar debidamente capacitadas**. En la ingeniería de software, el enfoque está principalmente en los **procesos**, a lo que anteriormente nos referimos como un enfoque sistemático en la definición dada. Como los procesos constituyen el corazón de la ingeniería de software (con herramientas y tecnología que brindan apoyo para ejecutar los procesos de manera eficiente), en este libro nos centraremos principalmente en los procesos. El proceso es lo que nos lleva desde las necesidades del usuario hasta el software que satisface dichas necesidades.

El enfoque básico de la ingeniería de software es separar el **proceso de desarrollo de software** del **producto desarrollado** (es decir, el software). La premisa es que, en gran medida, el proceso de software determina la calidad del producto y la productividad alcanzada. Por lo tanto, para abordar el dominio del problema y enfrentar con éxito los desafíos que afronta la ingeniería de software, se debe enfocar en el proceso de software. El diseño de procesos de software adecuados y su control se convierten entonces en un objetivo clave de la investigación en ingeniería de software.

Es este enfoque en el **proceso** lo que distingue a la ingeniería de software de la mayoría de las otras disciplinas de la computación. La mayoría de las otras disciplinas de la computación se enfocan en algún tipo de producto—algoritmos, sistemas operativos, bases de datos, etc.—mientras que la ingeniería de software se centra en el **proceso para producir los productos**. Es, esencialmente, el equivalente en software de la “ingeniería de manufactura”.

Aunque discutiremos más sobre los procesos en el próximo capítulo, aquí hablaremos brevemente de dos aspectos clave: **el proceso de desarrollo y la gestión del proceso de desarrollo**.

1.3.1 Proceso de Desarrollo por Fases

Un proceso de desarrollo consiste en varias fases, cada fase terminando con una salida definida. Las fases se realizan en un orden especificado por el modelo de proceso que se está siguiendo. La razón principal de tener un proceso por fases es que divide el problema de desarrollar software en realizar con éxito un conjunto de fases, cada una manejando una preocupación diferente del desarrollo de software.

Esto asegura que el costo del desarrollo sea más bajo de lo que hubiera sido si todo el problema se abordara de una sola vez. Además, un proceso por fases permite una verificación adecuada de calidad y de progreso en algunos puntos definidos durante el desarrollo (al final de las fases). Sin esto, uno tendría que esperar hasta el final para ver qué software ha sido producido. Claramente, esto no funcionará para sistemas grandes.

Por lo tanto, para manejar la complejidad, el seguimiento del proyecto y la calidad, todos los procesos de desarrollo consisten en un conjunto de fases. Un proceso de desarrollo por fases es central para el enfoque de ingeniería de software para resolver la crisis del software.

Se han propuesto varios modelos de proceso para desarrollar software. De hecho, la mayoría de las organizaciones que siguen un proceso tienen su propia versión. Discutiremos algunos de los modelos comunes en el próximo capítulo. En general, sin embargo, podemos decir que cualquier resolución de problemas en software debe consistir en:

- la **especificación de requisitos** para entender y declarar claramente el problema,
- el **diseño** para decidir un plan de solución,
- la **codificación** para implementar la solución planificada, y
- la **prueba** para verificar los programas.

Para problemas pequeños, estas actividades pueden no hacerse explícitamente, los límites de inicio y fin de estas actividades pueden no estar claramente definidos, y no se puede mantener un registro

escrito de las actividades. Sin embargo, los enfoques sistemáticos requieren que cada una de estas cuatro actividades de resolución de problemas se realice formalmente.

De hecho, para sistemas grandes, cada actividad en sí misma puede ser extremadamente compleja, y se necesitan metodologías y procedimientos para realizarlas de manera eficiente y correcta. Aunque diferentes modelos de proceso llevarán a cabo estas fases de manera diferente, ellas existen en todos los procesos. Discutiremos diferentes modelos de proceso en el próximo capítulo. Aquí discutimos brevemente estas fases básicas; cada una de ellas será tratada en más detalle a lo largo del libro (hay al menos un capítulo para cada una de estas fases).

Análisis de Requisitos

El análisis de requisitos se realiza para **entender el problema** que el sistema de software debe resolver.

El énfasis en el análisis de requisitos está en **identificar qué se necesita del sistema, no cómo el sistema alcanzará sus objetivos**. Para sistemas complejos, incluso determinar qué se necesita es una tarea difícil. El objetivo de esta actividad es documentar los requisitos en un **documento de especificación de requisitos de software**.

Hay dos actividades principales en esta fase:

1. **Análisis del problema** → comprender el problema, su contexto y los requisitos del nuevo sistema.
2. **Especificación de requisitos** → detallar lo comprendido en un documento formal.

Entender los requisitos de un sistema que aún no existe es complicado y requiere pensamiento creativo. El problema se vuelve más complejo porque un sistema automatizado ofrece posibilidades que no existirían de otra manera. En consecuencia, incluso los propios usuarios pueden no saber con claridad cuáles son sus necesidades.

Una vez analizado el problema y comprendidos los aspectos esenciales, los requisitos deben especificarse en un documento de requisitos, el cual debe incluir:

- todos los **requisitos funcionales** y de **rendimiento**;
- los formatos de entradas y salidas;
- todas las **restricciones de diseño** existentes (políticas, económicas, ambientales, de seguridad, etc.).

En otras palabras, además de la funcionalidad requerida, todos los factores que puedan afectar el diseño y el correcto funcionamiento del sistema deben especificarse en el documento de requisitos. Un **manual preliminar de usuario**, que describa las interfaces principales, suele formar parte del documento de requisitos.

Diseño de Software

El propósito de la fase de diseño es **planificar una solución** al problema especificado en el documento de requisitos. Esta fase es el primer paso en el paso del **dominio del problema** al

dominio de la solución. Dicho de otra manera: comenzando con lo que se necesita, el diseño nos lleva hacia cómo satisfacer esas necesidades.

El diseño de un sistema es quizá el factor más crítico que afecta la **calidad del software**, ya que tiene un gran impacto en las fases posteriores, especialmente en **pruebas** y **mantenimiento**.

La actividad de diseño suele producir tres salidas:

1. **Diseño de arquitectura** → define el sistema como un conjunto de componentes y cómo interactúan entre sí.
2. **Diseño de alto nivel** → identifica los módulos a construir y sus especificaciones.
3. **Diseño detallado** → define la lógica interna de cada módulo.

En resumen:

- En **arquitectura**, el foco está en qué grandes componentes se necesitan.
- En **diseño de alto nivel**, el foco está en qué módulos se necesitan.
- En **diseño detallado**, el foco está en cómo implementar esos módulos en software.

Una **metodología de diseño** es un enfoque sistemático para crear un diseño aplicando un conjunto de técnicas y guías. La mayoría de las metodologías se centran en el **diseño de alto nivel**.

Codificación

Una vez que el diseño está completo, la mayoría de las decisiones principales sobre el sistema ya se han tomado. Sin embargo, muchos detalles sobre la codificación —que dependen del lenguaje de programación elegido— no están especificados en el diseño.

El objetivo de la fase de codificación es **traducir el diseño en código** en un lenguaje de programación. El propósito es implementar el diseño de la mejor manera posible.

La fase de codificación afecta profundamente tanto a las pruebas como al mantenimiento. **Un código bien escrito** puede reducir significativamente el esfuerzo en pruebas y mantenimiento. Dado que los costos de pruebas y mantenimiento son mucho más altos que los de codificación, el objetivo debe ser reducir el esfuerzo posterior.

Por ello, durante la codificación el énfasis debe estar en **desarrollar programas fáciles de leer y entender**, no simplemente fáciles de escribir. La **simplicidad y claridad** deben ser prioridades.

Pruebas

Las pruebas son la principal medida de **control de calidad** durante el desarrollo de software. Su función básica es **detectar defectos** en el software.

Durante el análisis de requisitos y el diseño, la salida es un documento textual y no ejecutable. Después de la codificación, ya existen programas ejecutables que pueden probarse. Esto implica que las pruebas deben descubrir no solo errores introducidos durante la codificación, sino también errores de requisitos y de diseño.

El objetivo de las pruebas es, por lo tanto, **detectar errores de requisitos, diseño y codificación**.

El proceso suele seguir estas etapas:

- **Pruebas unitarias** → verificar módulos o componentes individuales.
- **Pruebas de integración** → verificar interacciones entre módulos.
- **Pruebas de sistema** → verificar que se cumplen todos los requisitos.
- **Pruebas de aceptación** → demostrar al cliente, con datos reales, que el sistema funciona.

Las pruebas son una actividad extremadamente crítica y que consume mucho tiempo. Requieren **planificación adecuada**, usualmente mediante un **plan de pruebas**, que define actividades, cronograma, recursos y guías.

El plan de pruebas incluye:

- condiciones a probar,
- unidades a verificar,
- cómo se integrarán los módulos.

Luego, para cada unidad de prueba se produce un **documento de especificación de casos de prueba**, con los casos y los resultados esperados.

Durante la ejecución, se comparan los resultados obtenidos con los esperados.

La salida final de la fase de pruebas es el **informe de pruebas** y el **informe de errores**, donde se detallan los casos ejecutados, resultados, errores encontrados y las acciones tomadas para corregirlos.

1.3.2 Gestión del Proceso

Como se mencionó antes, un proceso de desarrollo por fases es central para el enfoque de la ingeniería de software. Sin embargo, un proceso de desarrollo no especifica cómo asignar recursos a las diferentes actividades del proceso. Tampoco especifica cosas como el cronograma de actividades, cómo dividir el trabajo dentro de una fase, cómo asegurar que cada fase se realice correctamente, o cuáles son los riesgos del proyecto y cómo mitigarlos.

Sin una gestión adecuada de estos aspectos relacionados con el proceso, es poco probable que se puedan cumplir los objetivos de costo y calidad. Estas cuestiones relacionadas con la **gestión del proceso de desarrollo de un proyecto** se manejan mediante **la gestión de proyectos**.

Las actividades de gestión suelen girar en torno a un **plan**. Un plan de software forma la base que se usa intensamente para **monitorear y controlar** el proceso de desarrollo del proyecto. Esto convierte a la planificación en la actividad más importante de la gestión de proyectos. Se puede afirmar con seguridad que, sin una adecuada planificación, es muy poco probable que un proyecto de software cumpla sus objetivos. A la planificación del proyecto le dedicaremos un capítulo completo.

Gestionar un proceso requiere **información** sobre la cual basar las decisiones de gestión. De lo contrario, ni siquiera preguntas esenciales como: *¿se está cumpliendo el cronograma del proyecto?*, *¿cuál es el grado de sobre costo?*, *¿se están cumpliendo los objetivos de calidad?* pueden responderse.

Además, la información subjetiva es solo un poco mejor que no tener ninguna (ejemplo:

—P: ¿Qué tan cerca están de terminar?

—R: Ya casi lo logramos).

Por eso, para gestionar un proceso de manera efectiva, se necesita **información objetiva**. Para esto se usan las **métricas de software**.

Las **métricas de software** son medidas **cuantificables** que pueden usarse para medir diferentes características de un sistema de software o del proceso de desarrollo. Existen dos tipos:

- **Métricas de producto:** cuantifican características del producto en desarrollo (el software).
- **Métricas de proceso:** cuantifican características del proceso usado para desarrollar el software. Estas buscan medir aspectos como la productividad, costos y necesidades de recursos, efectividad de las medidas de control de calidad, y el efecto de las técnicas y herramientas de desarrollo.

Las **métricas y la medición** son aspectos necesarios para gestionar un proyecto de software. Para un monitoreo efectivo, la gestión necesita información sobre el proyecto: cuánto ha progresado, cuánto desarrollo se ha realizado, qué tan retrasado está respecto al cronograma, y cuál es la calidad alcanzada hasta el momento.

Con esta información, pueden tomarse decisiones sobre el proyecto. Sin métricas adecuadas para cuantificar esta información, habría que basarse en opiniones subjetivas, que a menudo son poco confiables y van en contra de los principios de la ingeniería.

Por lo tanto, podemos decir que la **gestión basada en métricas** también es un componente clave en la estrategia de la ingeniería de software para alcanzar sus objetivos.

Aunque nos hemos enfocado en la **gestión del proceso de desarrollo de un proyecto**, existen otros aspectos de la gestión de procesos de software. Algunos de estos se discutirán en el próximo capítulo.

1.4 Resumen

El costo del software ahora constituye la principal parte del costo de un sistema de cómputo. Actualmente, el software es **extremadamente costoso** de desarrollar y con frecuencia es **poco confiable**.

En este capítulo, hemos discutido algunos temas sobre el software y la ingeniería de software:

1. El dominio del problema de la ingeniería de software es el **software de nivel industrial**.
2. Este software no es solo un conjunto de programas, sino que incluye programas, datos asociados y documentación.

El software de nivel industrial es:

- Costoso y difícil de construir,
 - Costoso de mantener debido a cambios y reprocesamiento,
 - Con altos requisitos de calidad.
3. La **ingeniería de software** es la disciplina que busca proporcionar métodos y procedimientos para desarrollar software de nivel industrial de manera sistemática.

Las principales fuerzas que impulsan a la ingeniería de software son: **el problema de escala, la calidad y productividad (Q&P), la consistencia y el cambio.**

Alcanzar una alta Q&P de forma consistente, en problemas de gran escala y donde los cambios ocurren de manera continua, es el principal desafío de la ingeniería de software.

4. El **enfoque fundamental** de la ingeniería de software para lograr sus objetivos es separar el **proceso de desarrollo** de los **productos**.

La ingeniería de software se centra en el proceso porque la calidad de los productos desarrollados y la productividad alcanzada dependen en gran medida de él.

Para enfrentar los retos de la ingeniería de software, este proceso de desarrollo es **por fases**.

Otro enfoque clave es **gestionar el proceso de forma eficaz y proactiva usando métricas**.

2 Procesos de software

Como vimos en el capítulo anterior, el concepto de proceso está en el corazón del enfoque de la ingeniería de software. Según el diccionario Webster, el término “proceso” significa “*un método particular de hacer algo, que generalmente implica una serie de pasos u operaciones*”. En el contexto de la ingeniería de software, la expresión proceso de software se refiere a los métodos mediante los cuales se desarrolla el software.

Un proceso de software es un conjunto de actividades, junto con restricciones de orden entre ellas, de manera que si las actividades se realizan correctamente y respetando dicho orden, se obtiene el resultado deseado.

El resultado básico que se busca, como se mencionó antes, es lograr alta calidad y alta productividad en el desarrollo del software.

En este capítulo se profundizará en el concepto de procesos de software, se analizarán los componentes que conforman un proceso de software y se presentarán algunos modelos de procesos que han sido propuestos para estructurar y mejorar el desarrollo de software.

2.1 Proceso de Software

En una organización cuyo negocio principal es el desarrollo de software, normalmente existen muchos procesos ejecutándose de manera simultánea. Muchos de estos no están directamente relacionados con la ingeniería de software, aunque sí influyen en el desarrollo del software. Estos se consideran procesos no pertenecientes a la ingeniería de software. Ejemplos de ellos son los procesos de negocio, los procesos sociales y los procesos de capacitación. Si bien estos afectan la actividad de desarrollo, están fuera del alcance de la ingeniería de software.

El proceso que aborda los aspectos técnicos y de gestión del desarrollo de software se denomina proceso de software. Claramente, para desarrollar software es necesario realizar muchos tipos distintos de actividades. Todas estas actividades en conjunto constituyen el proceso de software.

Dado que los diferentes tipos de actividades suelen ser realizadas por personas distintas y con objetivos diferentes, es más adecuado considerar el proceso de software como un conjunto de procesos componentes, cada uno formado por un tipo particular de actividad.

Cada uno de estos procesos componentes tiene un objetivo específico, aunque todos cooperan entre sí para alcanzar el objetivo general de la ingeniería de software, que es desarrollar software de alta calidad de manera eficiente.

En esta sección se definirán los principales procesos componentes de un proceso de software y se explicará cuáles son sus objetivos.

2.1.1 Procesos y Modelos de Proceso

Un proyecto exitoso es aquel que cumple con las expectativas en los tres objetivos principales: costo, cronograma y calidad (donde la funcionalidad o las características del producto se consideran parte de la calidad). Por lo tanto, al planificar y ejecutar un proyecto de software, las decisiones se toman principalmente con el fin de reducir costos o tiempos de desarrollo, o mejorar la calidad.

Los proyectos de software utilizan un proceso para organizar la ejecución de las tareas necesarias para alcanzar dichos objetivos.

La **especificación del proceso** de un proyecto define las tareas que deben realizarse y el **orden** en que deben ejecutarse. El **proceso real** ocurre cuando el proyecto se lleva a cabo en la práctica. Aunque la especificación del proceso y su ejecución son conceptos distintos, en este contexto se los considerará equivalentes y se usará el término **proceso** para referirse a ambos.

En la realidad, sin embargo, no siempre es fácil seguir estrictamente el proceso planificado. A menudo, el proceso que realmente se sigue difiere del que se había especificado. Las razones para esta diferencia pueden ir desde la **falta de disciplina o de comprensión de la importancia del proceso**, hasta el simple hecho de que “**las viejas costumbres son difíciles de cambiar**”. Asegurar que un proyecto siga efectivamente el proceso definido es un desafío importante para las organizaciones dedicadas al desarrollo de software.

Un **modelo de proceso** define un proceso general, normalmente dividido en **etapas**, especificando el orden en que deben ejecutarse y las condiciones o restricciones asociadas. La idea central detrás de un modelo de proceso es que, en las situaciones para las cuales el modelo es aplicable, su uso debería conducir a **menores costos, mayor calidad o menor tiempo de desarrollo**.

En otras palabras, un proceso es un **medio para alcanzar los objetivos** de alta calidad, bajo costo y corto tiempo de ciclo; y un modelo de proceso proporciona una **guía genérica** para desarrollar un proceso adecuado para un proyecto específico.

El proceso de un proyecto puede basarse en un modelo de proceso general, pero no se trata simplemente de copiarlo. Cada proyecto debe **adaptar (o “ajustar”)** el modelo a sus características particulares y necesidades específicas. Esta personalización del modelo recibe el nombre de **tailoring** (adaptación o ajuste).

Cuando un proceso se ejecuta en un proyecto, se **producen productos de software**, siendo el más importante el **software final**. El proceso define los pasos, el proyecto los ejecuta, y a lo largo de esa ejecución se generan resultados intermedios y finales.

En resumen, el proceso **limita los grados de libertad** de un proyecto al establecer qué actividades deben realizarse y en qué orden, de manera que se obtenga el camino más **eficiente** desde las necesidades del usuario hasta el software que las satisface.

Por esta razón, el **proceso** es el principal **motor del proyecto** y tiene una **gran influencia en sus resultados esperados**, motivo por el cual el **enfoque de la ingeniería de software** se centra fuertemente en el estudio y mejora de los **procesos**.

2.1.2 Procesos de Software de Componentes

Hemos mencionado que el proceso de desarrollo es el proceso central que especifica las tareas que deben realizarse en un proyecto. La planificación y programación de dichas tareas, así como el seguimiento de su ejecución, pertenecen al ámbito del proceso de gestión del proyecto.

Por lo tanto, existen claramente dos componentes principales dentro de un proceso de software:

- el proceso de desarrollo, y
- el proceso de gestión del proyecto,

El proceso de desarrollo define las actividades de desarrollo y aseguramiento de la calidad que deben llevarse a cabo, mientras que el proceso de gestión especifica cómo planificar y controlar dichas actividades para cumplir con los objetivos de costo, cronograma, calidad y otros.

Durante el proyecto, se producen diversos productos intermedios y finales, compuestos a su vez por muchos elementos (por ejemplo, el código fuente final puede estar formado por varios archivos fuente). Estos elementos evolucionan continuamente a medida que avanza el proyecto, generando distintas versiones.

Para garantizar que el software en desarrollo utilice las versiones correctas de estos elementos, se requieren procesos adecuados para controlar su evolución. Dado que los procesos de desarrollo no suelen enfocarse en los cambios y su evolución, se utiliza un proceso adicional llamado proceso de control de configuración del software (*software configuration control process*).

El objetivo principal de este proceso es gestionar los cambios de manera que la integridad de los productos no se vea comprometida a pesar de las modificaciones. A veces, los cambios en los requisitos se manejan de forma independiente mediante un proceso de gestión de cambios en los requisitos (*requirements change management process*).

Estos tres procesos —desarrollo, gestión del proyecto y control de configuración— se centran en los proyectos y los productos, y se los puede considerar como parte de los procesos de ingeniería del producto, ya que su objetivo principal es producir el software deseado.

Si consideramos al proceso de software como una entidad estática, estos tres componentes serían suficientes. Sin embargo, el proceso de software es en realidad una entidad dinámica, ya que debe adaptarse continuamente a nuestra mejor comprensión del desarrollo de software y a la aparición de nuevas tecnologías y herramientas.

Por ello, se requiere un proceso adicional: el proceso de gestión del proceso (*process management process*).

El objetivo básico de este proceso es mejorar el proceso de software. Por “mejorar”, entendemos aumentar la capacidad del proceso para producir productos de mayor calidad a menor costo. Para lograr esto, se analiza el proceso actual, generalmente mediante el estudio de proyectos pasados que se desarrollaron siguiendo dicho proceso.

El ciclo de comprender el proceso existente, analizar sus características, identificar oportunidades de mejora e implementar los cambios necesarios forma parte del proceso de gestión del proceso.

La relación entre estos principales componentes se muestra en la Figura 2.1. Estos procesos no solo difieren en el tipo de actividades que comprenden, sino también en las personas involucradas en su ejecución:

- Las actividades de desarrollo son realizadas por programadores, diseñadores y testers.
- Las actividades de gestión del proyecto son llevadas a cabo por el equipo de gestión del proyecto.
- Las actividades de control de configuración son responsabilidad del grupo conocido como controlador de configuración.
- Las actividades de gestión del proceso son ejecutadas por el grupo de procesos de ingeniería de software (*Software Engineering Process Group – SEPG*).

Más adelante en este capítulo se discutirá brevemente cada uno de estos procesos, así como el proceso de inspección, utilizado para el control de calidad de los distintos productos de trabajo.

Sin embargo, en el resto del libro nos centraremos principalmente en los procesos relacionados con la ingeniería del producto, ya que la gestión del proceso es un tema avanzado y queda fuera del alcance de este texto.

A lo largo del libro se analizarán las diferentes fases del proceso de desarrollo y las metodologías utilizadas para ejecutarlas.

En lo que resta del libro, el término proceso de software se usará para referirse a los procesos de ingeniería del producto, salvo que se indique lo contrario.

2.1.3 Enfoque ETVX para la Especificación de Procesos

Un proceso tiene un conjunto de fases (o pasos), y cada fase realiza una tarea bien definida que lleva al proyecto hacia el cumplimiento de sus objetivos. Para reducir costos, un proceso debe intentar detectar los defectos en la fase en la que son introducidos. Esto requiere que exista algún tipo de verificación al final de cada paso, lo cual a su vez exige que haya una salida claramente definida de cada fase que pueda ser verificada de alguna manera. En otras palabras, no es aceptable decir que la salida de una fase es una idea o un pensamiento en la mente de alguien; la salida debe ser una entidad formal y tangible.

Tales salidas de un proceso de desarrollo, que no son el producto final, suelen llamarse productos de trabajo (*work products*). En el desarrollo de software, un producto de trabajo puede ser el documento de requerimientos, el documento de diseño, el código, un prototipo, entre otros.

Esta restricción de que la salida de cada paso sea algún producto de trabajo verificable sugiere que el proceso debería tener un número pequeño de pasos. Tener demasiados pasos implica producir demasiados productos de trabajo o documentos. Por esta razón, en un nivel general, un proceso suele consistir en unos pocos pasos, cada uno con un objetivo claro y que produce un documento que puede verificarse. La forma en que se realiza la actividad de una fase o paso en particular generalmente se aborda mediante metodologías específicas para esa actividad. A lo largo del libro se discutirán diversas metodologías para diferentes actividades.

Como un proceso típicamente contiene una secuencia de pasos, el siguiente tema a tratar es cuándo debe iniciarse y finalizar una fase. Esto se hace frecuentemente especificando los *criterios de entrada* y *criterios de salida* de una fase.

Los criterios de entrada de una fase especifican las condiciones que la entrada de la fase debe cumplir para iniciar sus actividades. Los criterios de salida especifican las condiciones que el producto de trabajo de la fase debe cumplir para dar por terminadas sus actividades. Los criterios de entrada y salida establecen, por lo tanto, las condiciones bajo las cuales comenzar o detener una actividad. Debe quedar claro que los criterios de entrada de una fase deben ser consistentes con los criterios de salida de la fase anterior.

Además de los criterios de entrada y salida, las entradas y salidas de cada paso también deben estar claramente especificadas. Dado que se pueden introducir errores en cada etapa, cada una debería terminar con algún tipo de verificación de sus actividades, y esto también debe definirse claramente. La especificación de un paso con sus entradas, salidas y criterios de entrada y salida se muestra en la Figura 2.2.

Este enfoque para la especificación de procesos se llama **enfoque ETVX** (por *Entry criteria, Task, Verification, and eXit criteria* —criterios de entrada, tarea, verificación y criterios de salida—).

Además de los criterios de entrada y salida, un paso también debe generar cierta información que ayude a una adecuada gestión del proceso. Esto requiere que un paso produzca información que proporcione **visibilidad** sobre el estado del proceso. Esa información puede luego utilizarse para tomar las acciones adecuadas, cuando sea necesario, para mantener el proceso bajo control. El flujo de información desde un paso y el ejercicio del control también se muestra en la Figura 2.2.

2.2 Características Deseables de un Proceso de Software

Aún no hemos especificado ningún proceso. ¿Es adecuado usar cualquier proceso? En esta sección discutiremos algunas de las características deseables que debe tener un proceso de software. Dado que un proceso puede ser utilizado por muchos proyectos, necesita poseer características que vayan más allá de simplemente satisfacer los objetivos de un proyecto en particular. A continuación, se analizan algunas de las más importantes.

2.2.1 Predictibilidad

La **predictibilidad** de un proceso determina qué tan exactamente se puede prever el resultado de seguir dicho proceso en un proyecto antes de que este haya finalizado.

La predictibilidad puede considerarse una propiedad fundamental de cualquier proceso. De hecho, si un proceso no es predecible, su utilidad es limitada. Veamos por qué.

Una forma de estimar el costo podría ser decir: “Este proyecto A es muy similar al proyecto B que hicimos hace 2 años, por lo tanto, el costo de A será muy parecido al de B.”

Sin embargo, incluso este método simple supone que el proceso que se usará para desarrollar el proyecto A será el mismo que se utilizó para el proyecto B, y que al seguirlo nuevamente se obtendrán resultados similares a los de la primera vez. Es decir, se asume que el proceso es **predecible**. Si no lo fuera, no habría garantía de que un proyecto similar, desarrollado mediante el mismo proceso, tuviera un costo parecido.

La situación con la **calidad** es similar. La base fundamental para predecir la calidad es que esta depende en gran medida del proceso utilizado para desarrollar el producto. A partir de esto, la calidad de un nuevo proyecto puede estimarse observando la calidad de los productos desarrollados previamente con el mismo proceso.

De hecho, la gestión efectiva de las actividades de control de calidad depende en gran parte de la predictibilidad del proceso.

Por ejemplo, para un control de calidad efectivo, una estrategia es estimar qué tipos y cantidades de errores se detectarán en cada etapa del desarrollo, y luego usar esa información para determinar si las actividades de aseguramiento de calidad se están realizando correctamente.

Esto solo puede hacerse si el proceso es predecible; basándose en la experiencia previa con el proceso, uno puede estimar la distribución esperada de errores para el proyecto actual.

De lo contrario, ¿cómo podría alguien decir si detectar 10 errores por cada 100 líneas de código (LOC) durante las pruebas es algo “aceptable”?

Si el proceso es predecible y la experiencia pasada indica que normalmente se detectan alrededor de 10 errores por cada 100 LOC durante las pruebas, entonces probablemente las pruebas se realizaron adecuadamente.

Pero si, en cambio, la experiencia previa muestra que normalmente se detectan solo 2 errores por cada 100 LOC, entonces sería necesario examinar más cuidadosamente el proceso de prueba del proyecto actual.

Debe quedar claro que, si queremos usar la experiencia pasada para **controlar costos y garantizar calidad**, debemos emplear un proceso **predecible**. Con baja predictibilidad, la experiencia obtenida de proyectos anteriores tiene poco valor.

Un proceso predecible también se dice que está **bajo control estadístico**.

Un proceso está **bajo control estadístico** si, al seguirlo de la misma manera, se obtienen resultados **similares** —con cierta variación, pero causada principalmente por factores aleatorios y no por problemas del proceso en sí—.

Esto se muestra en la Figura 2.3, donde el eje y representa alguna propiedad de interés (como calidad o productividad) y el eje x representa los proyectos.

La línea oscura muestra el valor esperado de esa propiedad para el proceso.

El control estadístico implica que, la mayoría de las veces, el valor real de la propiedad estará dentro de un rango alrededor del valor esperado.

(Los **gráficos de control** ofrecen una manera formal de definir estos límites. Para una discusión sobre cómo establecer límites óptimos, véase [101].)

Debe quedar claro que, si se espera desarrollar software de alta calidad y bajo costo de manera consistente, es necesario contar con un proceso bajo control estadístico.

Un proceso predecible es un requisito esencial para garantizar **buena calidad y bajo costo**.

Esto no significa que sea imposible producir software de alta calidad y bajo costo sin un proceso así —un equipo de personas muy talentosas podría lograrlo—, pero lo importante es que **sin un proceso predecible esos resultados no pueden repetirse**.

Por lo tanto, si se busca obtener calidad de forma **consistente** en múltiples proyectos, contar con un proceso predecible es indispensable.

Dado que la ingeniería de software se interesa por métodos generales que puedan aplicarse al desarrollo de distintos sistemas, **un proceso predecible constituye la columna vertebral de los métodos de la ingeniería de software**.

2.2.2 Soporte para la capacidad de prueba y el mantenimiento

Ya hemos visto que, a lo largo de la vida útil del software, **los costos de mantenimiento suelen superar a los costos de desarrollo**. Por lo tanto, si queremos reducir el costo total del software —o lograr una *optimalidad global* en términos de costo, en lugar de una *optimalidad local* limitada solo al desarrollo—, el objetivo del desarrollo debe ser **reducir el esfuerzo de mantenimiento**. En otras palabras, uno de los objetivos más importantes de un proyecto de desarrollo debe ser **producir software que sea fácil de mantener**, y el proceso utilizado debe garantizar esta mantenibilidad.

Incluso durante el desarrollo, con frecuencia se le da una gran importancia a la **codificación**.

Sabemos que un proceso consta de varias fases —requisitos, diseño, codificación y pruebas—. Un ejemplo de distribución del esfuerzo entre estas fases podría ser:

Requisitos 10%
Diseño 10%
Codificación 30%
Pruebas 50%

Los números exactos pueden variar según la organización y la naturaleza del proceso, pero podemos hacer algunas observaciones importantes.

La primera es que **la codificación representa solo alrededor de un tercio del esfuerzo total de desarrollo**, lo cual contradice la noción ingenua de que desarrollar software consiste principalmente en escribir programas y que la programación es la actividad principal.

Otra forma de medir el esfuerzo dedicado a la programación es analizar cómo los programadores emplean su tiempo en una organización de software.

Un estudio realizado en **Bell Labs** mostró la siguiente distribución:

Escribir programas 13%
Leer programas y manuales 16%
Comunicación laboral 32%
Otras actividades (incluyendo personales) 39%

Estos datos muestran claramente que **programar no es la actividad principal** en la que los programadores pasan su tiempo. Incluso si ignoramos el tiempo dedicado a “otras” actividades, el tiempo efectivo dedicado a escribir programas sigue siendo **menos del 25%**. En otro estudio citado por **Boehm**, se encontró que los programadores pasan **menos del 20% de su tiempo programando**.

La segunda observación importante sobre la distribución del esfuerzo es que **las pruebas consumen la mayor parte de los recursos durante el desarrollo**.

Esto contradice la práctica común, que tiende a considerar las pruebas como una actividad secundaria y poco planificada. Subestimar el esfuerzo necesario para las pruebas lleva a asignar **recursos insuficientes**, lo que a su vez produce **software poco confiable o retrasos en el cronograma**.

En resumen, el objetivo del proceso **no debe ser reducir el esfuerzo de diseño y codificación**, sino **disminuir los costos de prueba y mantenimiento**.

Tanto las pruebas como el mantenimiento dependen en gran medida de la **calidad del diseño y del código**, y esos costos pueden reducirse significativamente si el software se diseña y se programa pensando en facilitar las pruebas y el mantenimiento.

Por eso, durante las fases iniciales del desarrollo, las principales preguntas deberían ser: **“¿Se puede probar fácilmente?”** y **“¿Se puede modificar fácilmente?”**.

2.2.3 Soporte para el cambio

El software cambia por una variedad de razones. En el **Capítulo 1**, destacamos que el **cambio es una característica fundamental** del dominio del problema. Aquí nos enfocamos en los **cambios debidos a modificaciones en los requisitos**. Aunque el cambio siempre ha sido parte de la vida, **en el mundo actual los cambios son muchos más y ocurren mucho más rápido**. A medida que las organizaciones y los negocios evolucionan, el software que los sustenta también debe cambiar. Por lo tanto, cualquier modelo de desarrollo que produzca software y haga que los cambios sean difíciles de implementar **no será adecuado en muchas situaciones**.

Además de modificar un software ya existente y en funcionamiento —lo cual podría considerarse fuera del proceso de desarrollo—, los **cambios también ocurren mientras el desarrollo está en curso**. Después de todo, las **necesidades del cliente pueden cambiar durante el proyecto**, y si este tiene una duración considerable, **es de esperar que se produzcan modificaciones significativas**.

Además de los cambios impulsados por necesidades del negocio, **también pueden surgir cambios simplemente porque las personas cambian de opinión** al reflexionar más sobre las posibilidades y alternativas. Así, una parte del sistema puede ser desarrollada y mostrada a los usuarios, quienes al verla pueden descubrir que lo que habían solicitado **no era exactamente lo que necesitaban**, o que **requieren algo más o algo diferente**.

En otras palabras, **el cambio es algo habitual**, y **un proceso que pueda manejar los cambios con facilidad es altamente deseable**.

2.2.4 Detección temprana de defectos

La idea de que la programación es la actividad central durante el desarrollo de software proviene, en gran parte, de que la programación se considera una tarea difícil y, a veces, un “arte”. Como consecuencia de este tipo de pensamiento, también surge la creencia de que los errores ocurren principalmente durante la programación, ya que es la actividad más compleja del desarrollo y ofrece muchas oportunidades para cometer fallos.

Hoy en día está claro que **los errores pueden ocurrir en cualquier etapa del desarrollo**. Un ejemplo de la distribución de errores según la fase es:

Requisitos 20%

Diseño 30%

Codificación 50%

Como se puede ver, **los errores aparecen a lo largo de todo el proceso de desarrollo**. Sin embargo, **el costo de corregirlos varía según la fase en la que se detecten y se corrijan**. La **Figura 2.4** muestra el costo relativo de corregir errores de requisitos dependiendo de cuándo se detectan.

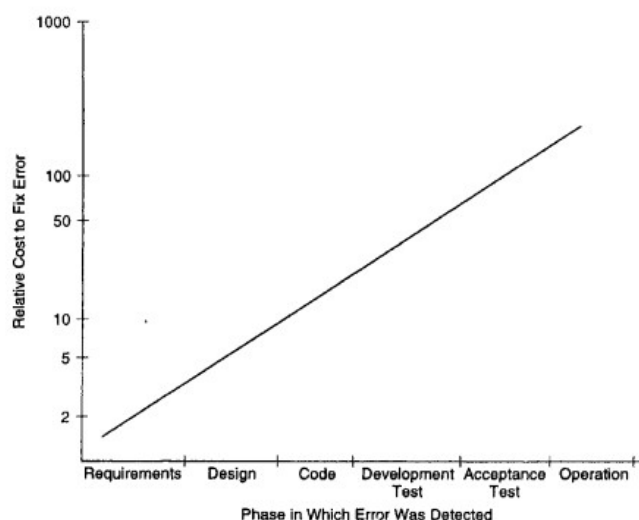


Figure 2.4: Costo de corregir errores.

Como era de esperarse, **cuanto más se retrasa la detección de un error, más costoso es corregirlo**. Según la figura, **un error cometido durante la fase de requisitos puede costar hasta 100 veces más corregirlo si se detecta recién en la fase de pruebas de aceptación**, en lugar de hacerlo en la fase de requisitos.

La razón de esto es bastante evidente: si existe un error en los requisitos, entonces el diseño y el código se basarán en esos requisitos erróneos. Corregir el problema después de que el código ya está hecho implica **modificar tanto el diseño como el código**, lo que **incrementa significativamente el costo**.

El punto principal de esta discusión es que **debemos intentar detectar los errores en la misma fase en la que ocurren**, y no esperar hasta la etapa de pruebas para descubrirlos. **La detección y corrección de errores debe ser un proceso continuo a lo largo de todo el desarrollo**.

En términos de fases del desarrollo, esto significa que **deberíamos verificar el resultado de cada fase antes de comenzar la siguiente** (de ahí la “V” del modelo ETVX). En otras palabras, **el proceso debe incluir actividades de control de calidad distribuidas a lo largo de todo el ciclo de desarrollo y dentro de cada fase**.

Una **actividad de control de calidad (QC)** es aquella cuyo principal propósito es **identificar y eliminar defectos**. Tener tareas de QC a lo largo del desarrollo es un objetivo que el proceso debe apoyar.

Sin embargo, **es aún mejor si el proceso también ayuda a prevenir defectos**. Existe un consenso general de que **las técnicas de control de calidad actuales no pueden detectar todos los defectos introducidos** (de lo contrario, no habría errores en los programas que se corrigen en versiones posteriores).

Por lo tanto, **para reducir tanto el número de defectos residuales al momento de la entrega como el costo total de su corrección**, una estrategia evidente es **prevenir la introducción de errores desde el principio**.

Esto requiere que **el proceso de desarrollo esté diseñado de tal manera que se introduzcan menos defectos**. El método generalmente utilizado para lograr esta prevención es **aprovechar la experiencia de proyectos anteriores para mejorar los métodos con los que se realizan las actividades**. Este tema será tratado con mayor detalle en un capítulo posterior.

2.2.5 Mejora del Proceso y Retroalimentación

Como se mencionó anteriormente, un proceso no es una entidad estática. Mejorar la calidad y reducir el costo de los productos son objetivos fundamentales de cualquier disciplina de ingeniería. En el contexto del software, dado que la productividad (y, por lo tanto, el costo de un proyecto) y la calidad están determinadas en gran medida por el proceso, para cumplir con los objetivos de mejora de calidad y reducción de costos, **el proceso de software debe mejorarse**.

Tener la mejora del proceso como un objetivo fundamental requiere que el proceso de software sea un **proceso de bucle cerrado** (*closed-loop process*). Es decir, el proceso debe mejorarse basándose en experiencias previas, y **cada proyecto realizado usando el proceso existente debe retroalimentar información** que facilite esta mejora.

Como se indicó antes, esta actividad de analizar y mejorar el proceso se realiza principalmente en el **componente de gestión del proceso** dentro del proceso de software. Sin embargo, para respaldar esta actividad, la información de varios otros procesos debe fluir hacia el proceso de gestión. En otras palabras, para apoyar esta actividad, **otros procesos también deben participar activamente**.

La mejora del proceso también es un objetivo dentro de un proyecto grande, donde la retroalimentación de las primeras partes del proyecto puede utilizarse para mejorar la ejecución del resto. Este tipo de retroalimentación resulta especialmente adecuado cuando se utiliza un modelo de proceso de desarrollo iterativo, ya que la retroalimentación de una iteración se usa para mejorar la ejecución de las iteraciones posteriores.

2.3 Modelos de Proceso de Desarrollo de Software

En el proceso de desarrollo de software nos centramos en las actividades directamente relacionadas con la **producción del software**, por ejemplo, diseño, codificación y pruebas.

Dado que el proceso de desarrollo especifica las principales actividades de desarrollo y de **control de calidad** que deben realizarse en el proyecto, el **proceso de desarrollo** forma realmente el **núcleo del proceso de software**. El proceso de gestión se define en función del proceso de desarrollo.

Debido a la importancia del proceso de desarrollo, se han propuesto **diversos modelos**. En esta sección discutiremos algunos de los modelos más importantes.

2.3.1 Modelo en Cascada

El modelo de proceso más simple es el **modelo en cascada**, que establece que las fases se organizan en un orden lineal. El modelo fue propuesto originalmente por Royce, aunque han surgido variaciones del mismo según la naturaleza de las actividades y el flujo de control entre ellas.

En este modelo, un proyecto comienza con un **análisis de viabilidad**. Una vez que se demuestra exitosamente la viabilidad del proyecto, comienza el **análisis de requerimientos** y la **planificación del proyecto**. El **diseño** inicia después de completar el análisis de requerimientos, y la **codificación** comienza después de terminar el diseño. Una vez completada la programación, el código se integra y se **realizan las pruebas**. Tras la finalización exitosa de las pruebas, el sistema se instala. Posteriormente, se lleva a cabo la operación regular y el mantenimiento del sistema. El modelo se muestra en la Figura 2.5.

La fase de análisis de requerimientos se menciona como “análisis y planificación”. La **planificación** es una actividad crítica en el desarrollo de software. Un buen plan se basa en los requerimientos del sistema y debe realizarse antes de que comiencen las fases posteriores. Sin embargo, en la práctica, no se necesitan requerimientos detallados para planificar. Por lo tanto, la planificación generalmente se superpone con el análisis de requerimientos, y un plan está listo antes de que comiencen las fases posteriores. Este plan constituye una entrada adicional para todas las fases siguientes.

El **orden lineal de las actividades** tiene algunas consecuencias importantes. Primero, para identificar claramente el final de una fase y el inicio de la siguiente, se debe emplear algún **mecanismo de certificación** al final de cada fase. Esto generalmente se realiza mediante algún medio de **verificación y validación** que asegure que la salida de una fase sea consistente con su entrada (que es la salida de la fase anterior) y que la salida de la fase sea consistente con los requerimientos generales del sistema.

La necesidad de certificación implica que cada fase debe tener una **salida definida** que pueda ser evaluada y certificada. Es decir, cuando se completan las actividades de una fase, debe existir un **producto** producido por esa fase. Las salidas de las fases iniciales suelen llamarse **productos de trabajo** y generalmente están en forma de documentos, como el documento de requerimientos o el documento de diseño. Para la fase de codificación, la salida es el código. Aunque el conjunto de documentos que debe producirse en un proyecto depende de cómo se implemente el proceso, los siguientes documentos generalmente forman un conjunto razonable que debería producirse en cada proyecto:

- Documento de requerimientos
- Plan del proyecto
- Documentos de diseño (arquitectura, sistema, detallado)
- Plan de pruebas e informes de prueba
- Código final
- Manuales de software (por ejemplo, usuario, instalación, etc.)

Además de estos productos de trabajo, hay varios otros documentos que se producen en un proyecto típico. Estos incluyen **informes de revisión**, que son el resultado de las revisiones realizadas sobre los productos de trabajo, así como **informes de estado** que resumen el estado del proyecto de manera regular. Pueden producirse muchos otros informes para mejorar la ejecución del proyecto o la presentación de informes del proyecto.

Una de las principales ventajas de este modelo es su **simplicidad**. Conceptualmente es **directo y divide la gran tarea de construir un sistema de software en una serie de fases claramente separadas, cada fase abordando una preocupación lógica distinta**. También es **fácil de administrar** en un entorno contractual: a medida que se completa cada fase y se produce su producto de trabajo, el cliente entrega cierta cantidad de dinero a la organización desarrolladora.

El modelo en cascada, aunque ampliamente usado, tiene algunas **limitaciones importantes**. Algunas de las principales limitaciones son:

1. **Asume que los requerimientos de un sistema pueden fijarse (baseline) antes de que comience el diseño**. Esto es posible para sistemas diseñados para automatizar un sistema manual existente. Pero **para sistemas nuevos, determinar los requerimientos es difícil ya que el usuario ni siquiera conoce los requerimientos**. Por lo tanto, tener requerimientos inmutables es poco realista para estos proyectos.
2. **Fijar los requerimientos generalmente requiere elegir el hardware** (porque forma parte de la especificación de requerimientos). Un proyecto grande podría tardar varios años en completarse. Si el hardware se selecciona temprano, debido a la rapidez con que cambia la tecnología de hardware, **es probable que el software final utilice tecnología de hardware a punto de volverse obsoleta**. Esto claramente no es deseable para sistemas de software tan costosos.
3. **Sigue un enfoque “big bang”**, es decir, **todo el software se entrega de una vez al final**. Esto implica riesgos elevados, ya que el usuario no sabe hasta el final qué está recibiendo.

Además, si el proyecto se queda sin dinero a mitad de camino, no habrá software; es decir, tiene una proposición de valor de “todo o nada”.

4. **Es un proceso basado en documentos**, que requiere documentos formales al final de cada fase.

A pesar de estas limitaciones, el modelo en cascada ha sido el modelo de proceso más utilizado. Es **adecuado para proyectos rutinarios** donde los requerimientos se comprenden bien. Es decir, si la organización desarrolladora está familiarizada con el dominio del problema y los requerimientos del software son bastante claros, el modelo en cascada funciona bien.

2.3.2 Prototipado

El **objetivo** de un proceso de desarrollo basado en **prototipos** es contrarrestar las dos primeras limitaciones del modelo en cascada.

La idea básica es que, en lugar de congelar los requerimientos antes de comenzar con el diseño o la codificación, **se construye un prototipo desechable** para ayudar a comprender los requerimientos. Este prototipo se desarrolla en base a los requerimientos conocidos hasta el momento. **Su desarrollo obviamente pasa por etapas de diseño, codificación y prueba, pero cada una de estas fases se realiza de manera menos formal y menos exhaustiva.**

Mediante el uso del prototipo, el cliente puede obtener una **sensación real del sistema**, ya que las interacciones con el mismo le permiten comprender mejor los requerimientos del sistema deseado. Esto **da como resultado requerimientos más estables**, que cambian con menos frecuencia.

El **prototipado** es una idea muy atractiva para sistemas grandes y complejos para los cuales no existe un proceso manual ni un sistema previo que sirva para determinar los requerimientos. En tales casos, permitir que el cliente “juegue” con el prototipo proporciona información valiosa (aunque intangible) que ayuda a determinar los requerimientos del sistema. También es un método efectivo para **demostrar la viabilidad** de un enfoque particular. Esto puede ser necesario en sistemas nuevos, donde no está claro si se pueden cumplir ciertas restricciones o si es posible desarrollar algoritmos adecuados para implementar los requerimientos. En ambos casos, el uso de prototipos **reduce los riesgos** asociados con el proyecto.

El modelo de proceso del enfoque basado en prototipos se muestra en la Figura 2.6.

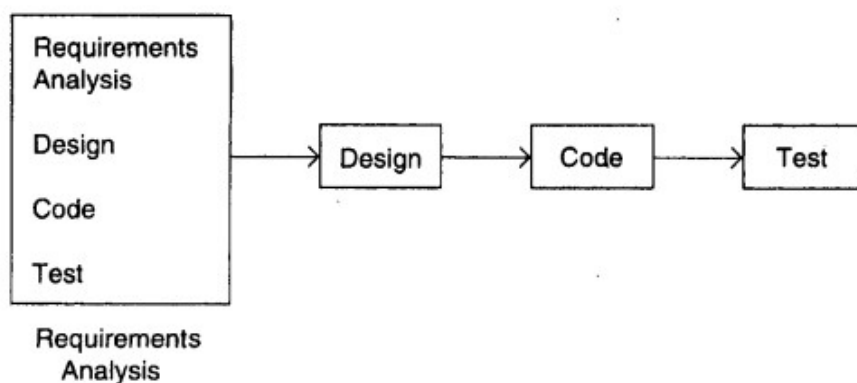


Figura 2.6: El modelo de prototipo

Un proceso de desarrollo que utiliza **prototipos desechables** típicamente procede de la siguiente manera :

El desarrollo del prototipo comienza cuando se ha elaborado una versión preliminar del documento de especificación de requerimientos.

En esta etapa, existe una comprensión razonable del sistema, de sus necesidades y de cuáles de ellas son poco claras o propensas a cambiar.

Después de desarrollar el prototipo, se les da a los **usuarios finales y clientes** la oportunidad de usarlo y experimentar con él.

Basándose en su experiencia, ellos brindan **retroalimentación** a los desarrolladores sobre el prototipo: qué está correcto, qué debe modificarse, qué falta, qué sobra, etc.

Con base en esa retroalimentación, el prototipo se modifica para incorporar algunos de los cambios sugeridos que puedan hacerse fácilmente, y luego los usuarios y clientes vuelven a probar el sistema.

Este ciclo se repite hasta que, según el juicio de los desarrolladores y analistas, el **beneficio de seguir haciendo cambios** y obteniendo retroalimentación **ya no justifica el costo y el tiempo** que esto implica.

Con base en la retroalimentación final, los requerimientos iniciales se modifican para producir la **especificación final de requerimientos**, que luego se utiliza para desarrollar el sistema en su versión de producción.

Para que el prototipado con fines de **análisis de requerimientos** sea viable, **su costo debe mantenerse bajo**.

Por lo tanto, solo se incluyen en el prototipo aquellas funcionalidades que aporten un **valor útil a partir de la experiencia del usuario**.

El manejo de excepciones, la recuperación de errores y la conformidad con ciertos estándares o formatos generalmente no se incluyen.

Como el prototipo será **desechado**, no tiene sentido implementar aquellas partes del sistema que ya se comprenden bien.

El desarrollo se enfoca en las características **que no se entienden completamente**.

El enfoque es **“rápido y sucio”**, priorizando el desarrollo veloz por encima de la calidad.

Dado que el prototipo se va a descartar, **solo se produce la documentación mínima necesaria** durante el proceso.

Por ejemplo, **no se necesitan documentos de diseño, planes de prueba ni especificaciones de casos de prueba** durante la creación del prototipo.

Otra medida importante para reducir costos es **disminuir las pruebas**, ya que las pruebas suelen representar una gran parte del gasto en el desarrollo regular de software.

Mediante estas técnicas de reducción de costos, es posible mantener el costo del prototipo por debajo de un **pequeño porcentaje del costo total** del desarrollo.

A menudo, el prototipado no se utiliza porque se teme que incremente los costos.

Sin embargo, en algunos casos, el costo del desarrollo **sin prototipo puede ser mayor** que con él.

Hay dos razones principales para esto:

Primero, la experiencia adquirida al desarrollar el prototipo puede **reducir el costo de las fases posteriores** cuando se construye el software real.

Segundo, en muchos proyectos, los requerimientos cambian constantemente, especialmente cuando el desarrollo es largo.

Sabemos que **los cambios tardíos en los requerimientos** aumentan significativamente el costo del proyecto.

Al prolongar la fase de análisis de requerimientos (ya que el desarrollo del prototipo lleva tiempo), **los requerimientos se “congelan” más adelante, cuando ya están más definidos y estables.**

Además, como el cliente y los usuarios ganan experiencia con el sistema, es más probable que los **requerimientos definidos después del prototipo estén más cerca de los requerimientos reales.**

Esto reduce aún más los cambios posteriores y, por lo tanto, **disminuye los costos asociados a las modificaciones.**

En resumen, el costo del desarrollo **después del prototipo** puede ser considerablemente menor que el costo sin prototipado, y ya vimos cómo es posible mantener **bajo el costo del prototipo en sí.**

El **prototipado es muy adecuado para proyectos donde los requerimientos son difíciles de determinar** y la confianza en los requerimientos iniciales es baja.

En tales proyectos, el modelo en cascada exigiría congelar los requerimientos para poder continuar con el desarrollo, incluso cuando estos no son estables.

Esto lleva a **cambios y retrabajo** durante el desarrollo.

En cambio, los requerimientos definidos después de la experiencia con el prototipo tienden a ser **más estables.**

En general, en proyectos donde los requerimientos **no están bien comprendidos al inicio**, usar el modelo de proceso basado en prototipado puede ser **el método más efectivo** para desarrollar el software.

Es una técnica excelente para **reducir ciertos tipos de riesgos** asociados al proyecto.

Discutiremos más sobre el prototipado cuando hablemos de la **especificación de requerimientos** y de la **gestión de riesgos.**

2.3.3 Desarrollo Iterativo

El modelo de proceso de desarrollo iterativo contrarresta la tercera limitación del modelo en cascada y trata de combinar los beneficios tanto del prototipado como del modelo en cascada.

La idea básica es que el software debe desarrollarse en **incrementos**, cada incremento agregando cierta capacidad funcional al sistema hasta que el sistema completo esté implementado.

En cada paso se pueden hacer extensiones y modificaciones de diseño.

Una **ventaja** de este enfoque es que puede resultar en **mejores pruebas**, porque **probar cada incremento es probablemente más fácil que probar todo el sistema de una sola vez**, como ocurre en el modelo en cascada.

Además, al igual que en el prototipado, **los incrementos brindan retroalimentación al cliente**, lo cual es útil para determinar los requisitos finales del sistema.

El **modelo de mejora iterativa** es un ejemplo de este enfoque.

En el primer paso de este modelo, se realiza una **implementación inicial simple** de un subconjunto del problema total.

Este subconjunto incluye algunos de los aspectos clave del problema que son fáciles de entender e implementar, y que forman un sistema útil y funcional.

Se crea una **lista de control del proyecto** (*project control list*) que contiene, en orden, todas las tareas que deben realizarse para obtener la implementación final.

Esta lista da una idea de qué tan avanzado está el proyecto en un momento dado respecto al sistema final.

Cada paso consiste en **remover la siguiente tarea de la lista**, diseñar la implementación para esa tarea seleccionada, **codificarla y probarla**, realizar un **análisis del sistema parcial** obtenido después de ese paso y **actualizar la lista** según el resultado del análisis.

Estas tres fases se llaman: **fase de diseño**, **fase de implementación** y **fase de análisis**.

El proceso se itera hasta que la lista de control del proyecto esté vacía, momento en el cual estará disponible la implementación final del sistema.

El modelo de mejora iterativa se muestra en la Figura 2.7.

La lista de control del proyecto guía los pasos de iteración y mantiene el registro de todas las tareas que deben realizarse.

Con base en el análisis, una de las tareas de la lista puede incluir el **rediseño de componentes defectuosos** o incluso el **rediseño del sistema completo**.

Sin embargo, el rediseño del sistema generalmente ocurre solo en las etapas iniciales.

En las etapas posteriores, el diseño ya estará más estabilizado y habrá menos posibilidad de rediseño.

Cada entrada en la lista es una tarea que debe completarse en un solo paso del proceso de mejora iterativa y debe ser lo suficientemente simple como para entenderse completamente.

Seleccionar las tareas de esta manera minimiza las posibilidades de error y reduce el trabajo de rediseño.

Las fases de diseño e implementación de cada paso pueden realizarse de manera **descendente (top-down)** o usando alguna otra técnica.

El **modelo en espiral** es otro modelo iterativo que se ha propuesto [18].

Como su nombre lo sugiere, las actividades en este modelo pueden organizarse como una **espiral** que tiene muchos ciclos, como se muestra en la Figura 2.8 [18].

Cada ciclo en la espiral comienza con la **identificación de los objetivos** de ese ciclo, las diferentes **alternativas posibles** para lograrlos y las **restricciones** que existen.

El siguiente paso en el ciclo es **evaluar estas alternativas** en función de los objetivos y restricciones.

El foco de esta evaluación está basado en la **percepción de riesgos** del proyecto.

El siguiente paso es **desarrollar estrategias que resuelvan las incertidumbres y riesgos**.

Este paso puede involucrar actividades como **benchmarking**, **simulación** y **prototipado**.

Luego, el software se desarrolla teniendo en cuenta los riesgos. Finalmente, se **planifica la siguiente etapa**.

Un uso efectivo del modelo iterativo se observa frecuentemente en el **desarrollo de productos**, en el cual los propios desarrolladores proveen las especificaciones y, por lo tanto, tienen mucho control sobre qué especificaciones se incluyen en el sistema y cuáles se dejan fuera.

Generalmente, se libera una versión del producto que contiene cierta funcionalidad.

Con base en la **retroalimentación de los usuarios** y la experiencia con esa versión, junto con

cambios tecnológicos o comerciales, se genera una lista de nuevas características y capacidades deseables.

Estas características forman la base para la **mejora del software** y se incluyen en la siguiente versión.

En otras palabras, la **primera versión contiene capacidades básicas**, y las versiones posteriores **agregan más funcionalidades**.

En el **desarrollo de software a medida**, donde el cliente debe proporcionar y aprobar las especificaciones, este modelo de proceso se ha vuelto extremadamente popular, a pesar de algunas dificultades para aplicarlo en este contexto.

La razón principal es la misma: como los negocios cambian muy rápidamente hoy en día, **nunca se conocen realmente todos los “requisitos completos” del software**, y existe la necesidad constante de **agregar nuevas capacidades** para adaptarse a las situaciones cambiantes.

Además, **los clientes no quieren invertir demasiado tiempo y dinero sin ver resultados**.

En el escenario empresarial actual, es preferible ver **retornos continuos** de la inversión realizada.

El modelo iterativo permite esto: **después de cada iteración se entrega un software funcional**.

El enfoque iterativo para el desarrollo de software se utiliza ampliamente hoy en día.

Muchos enfoques contemporáneos de desarrollo, como la **programación extrema (Extreme Programming)** y las **metodologías ágiles (Agile)**, consideran el desarrollo iterativo como una **estrategia fundamental** para el desarrollo de software moderno.

El **Rational Unified Process (RUP)** también emplea un proceso iterativo.

2.3.4 **Modelo de Timeboxing**

Para acelerar el desarrollo, puede emplearse **paralelismo entre diferentes iteraciones**.

Es decir, una nueva iteración comienza antes de que el sistema producido por la iteración actual sea liberado; por lo tanto, **el desarrollo de una nueva versión ocurre en paralelo con el desarrollo de la versión actual**.

Al comenzar una iteración antes de que la anterior haya terminado, **es posible reducir el tiempo promedio de entrega entre iteraciones**.

Sin embargo, para poder ejecutar iteraciones en paralelo, **cada iteración debe estar estructurada adecuadamente y los equipos deben organizarse de manera apropiada**.

El modelo de timeboxing propone un enfoque para lograr esto.

En el modelo de timeboxing, la **unidad básica de desarrollo es una “caja de tiempo” (time box)**, que **tiene una duración fija**.

Dado que la duración está fijada, un factor clave para seleccionar los requisitos o funcionalidades que se construirán en una caja de tiempo es lo que puede caber dentro de esa duración fija.

Esto contrasta con los enfoques iterativos tradicionales, donde **primero se selecciona la funcionalidad y luego se determina el tiempo necesario para entregarla**.

El timeboxing cambia la perspectiva del desarrollo: **el cronograma se convierte en innegociable y en un compromiso de máxima prioridad**.

Cada caja de tiempo se divide en una secuencia de etapas, al igual que en el modelo en cascada.

Cada etapa realiza una tarea claramente definida para la iteración y produce un resultado también claramente definido.

El modelo también requiere que la duración de cada etapa (es decir, el tiempo que lleva completar

la tarea de esa etapa) sea aproximadamente la misma.

Además, exige que exista un equipo dedicado para cada etapa.

Esto significa que el equipo de una etapa solo realiza tareas de esa etapa —las tareas de otras etapas son realizadas por sus respectivos equipos—.

Esto es muy diferente de otros modelos iterativos, donde generalmente el mismo equipo realiza todas las tareas del proyecto o de la iteración.

Tener iteraciones con cajas de tiempo, etapas de igual duración y equipos dedicados permite implementar un pipeline (tubería o canalización) de diferentes iteraciones.

(Pipelining es un concepto proveniente del hardware, donde diferentes instrucciones se ejecutan en paralelo: la ejecución de una nueva instrucción comienza una vez que la primera etapa de la instrucción anterior ha terminado).

Supongamos una caja de tiempo con duración T y compuesta por n etapas — $S_1, S_2, \dots, S_n$ —, donde cada etapa S_i es ejecutada por un equipo dedicado.

El equipo de cada etapa tiene T/n de tiempo disponible para completar su tarea dentro de la caja de tiempo.

Cuando el equipo de la etapa i completa sus tareas para la caja de tiempo k , entrega el resultado al equipo que ejecuta la etapa $i + 1$, y luego comienza a trabajar en su etapa para la siguiente caja de tiempo $k + 1$.

Usando la salida del equipo S_i , el equipo S_{i+1} comienza su trabajo para esa caja de tiempo.

Para cuando la primera caja de tiempo se aproxima a su finalización, ya hay $n - 1$ cajas de tiempo en diferentes etapas de ejecución.

Aunque la primera entrega ocurre después de T , cada entrega siguiente ocurre cada T/n unidades de tiempo, entregando software que fue desarrollado durante un ciclo de duración T .

Como ejemplo, consideremos una caja de tiempo que consiste en tres etapas:

1. Especificación de requisitos
2. Construcción (build)
3. Despliegue (deployment)

La etapa de requisitos es ejecutada por su equipo de analistas y finaliza con una lista priorizada de requisitos a implementar en esa iteración, junto con un diseño de alto nivel.

El equipo de construcción desarrolla el código para implementar esos requisitos y realiza las pruebas.

Luego, el código probado se entrega al equipo de despliegue, que ejecuta pruebas previas al despliegue e instala el sistema en producción.

Estas tres etapas son tales que pueden completarse en tiempos aproximadamente iguales dentro de una iteración.

Con una caja de tiempo de tres etapas, el proyecto progresa así:

- Cuando el equipo de requisitos termina los requisitos de la timebox-1, se los entrega al equipo de construcción para que desarrolle el software.
- El equipo de requisitos luego comienza a preparar los requisitos para la timebox-2.

- Cuando el desarrollo de la timebox-1 se completa, el código se entrega al equipo de despliegue, el equipo de construcción pasa a trabajar en la timebox-2, y el equipo de requisitos empieza con la timebox-3.

Esta ejecución en pipeline del proceso de timeboxing se muestra en la Figura 2.9 [99].

Con una caja de tiempo de tres etapas, como máximo tres iteraciones pueden estar en progreso simultáneamente.

Si la caja de tiempo tiene una duración de T días, la primera entrega de software ocurrirá después de T días.

Las entregas posteriores, sin embargo, se realizarán cada $T/3$ días.

Por ejemplo, si la duración T de la caja de tiempo es de 9 semanas (y cada etapa dura 3 semanas):

- la primera entrega ocurre después de 9 semanas del inicio del proyecto,
- la segunda después de 12 semanas,
- la tercera después de 15 semanas, y así sucesivamente.

Esto contrasta con una ejecución lineal de iteraciones, donde la primera entrega sería después de 9 semanas, la segunda después de 18 semanas, la tercera después de 27 semanas, etc.

En este proyecto hay tres equipos trabajando: el de requisitos, el de construcción y el de despliegue. La actividad de cada equipo en el pipeline de tres etapas se muestra en la Figura 2.10 [99].

Debe quedar claro que la duración de cada iteración no se ha reducido.

El trabajo total realizado en una caja de tiempo y el esfuerzo invertido en ella también permanecen iguales:

la misma cantidad de software se entrega al final de cada iteración, ya que la caja de tiempo atraviesa las mismas etapas.

Entonces, si el esfuerzo y el tiempo por iteración siguen siendo los mismos, ¿cuál es el costo de reducir el tiempo de entrega?

El verdadero costo de este tiempo reducido está en los recursos utilizados por este modelo.

Con timeboxing, existen equipos dedicados para diferentes etapas, y el tamaño total del equipo del proyecto es la suma de los equipos de cada etapa.

Esta es la principal diferencia con la situación en la que un solo equipo realiza todas las etapas y trabaja en la misma iteración.

Por ejemplo, consideremos un desarrollo iterativo con tres etapas, como en el ejemplo anterior:

- Supongamos que se necesitan 2 personas durante 2 semanas para los requisitos,
- 4 personas durante 2 semanas para la construcción,
- y 3 personas durante 2 semanas para las pruebas y el despliegue.

Si las iteraciones se ejecutan de forma secuencial, el equipo del proyecto será de 4 personas (el tamaño máximo necesario para una etapa):

- durante las primeras 2 semanas, 2 personas harán principalmente los requisitos,
- luego las 4 trabajarán en la construcción,

- y finalmente 3 harán el despliegue.

Si este mismo proyecto se ejecuta usando el modelo de timeboxing, habrá 3 equipos separados:

- un equipo de requisitos de 2 personas,
- un equipo de construcción de 4 personas,
- y un equipo de despliegue de 3 personas.

Por lo tanto, el tamaño total del equipo es $2 + 4 + 3 = 9$ personas, más del doble del tamaño máximo del equipo en la ejecución secuencial.

Es precisamente este aumento en el tamaño del equipo lo que permite incrementar el rendimiento (throughput) y reducir el tiempo promedio de entrega.

Por lo tanto, el timeboxing ofrece un enfoque para utilizar más personal con el fin de reducir el tiempo de entrega.

Es bien sabido que con los métodos tradicionales de ejecución de proyectos no se puede reducir sustancialmente el tiempo del ciclo simplemente agregando más personas.

Sin embargo, mediante el modelo de timeboxing, se puede utilizar más personal de manera organizada, de modo que mediante la ejecución paralela de diferentes etapas sea posible entregar el software más rápidamente.

En otras palabras, proporciona una forma de acortar los tiempos de entrega mediante el uso de más recursos humanos.

El timeboxing es adecuado para proyectos que requieren desarrollar una gran cantidad de funcionalidades en poco tiempo, sobre una arquitectura estable y usando tecnologías estables. Estas funcionalidades deben ser tales que exista cierta flexibilidad para agruparlas y construir sistemas significativos en cada iteración que aporte valor a los usuarios.

El modelo no es adecuado para proyectos donde sea difícil dividir el desarrollo total en múltiples iteraciones de duración similar, ni para aquellos donde las iteraciones requieran diferentes etapas, o donde no haya flexibilidad para combinar funcionalidades en entregas coherentes.

Aquí solo se ha descrito el modelo básico del proceso, sin entrar en detalle sobre el impacto de etapas desiguales, excepciones durante la ejecución, o cuestiones de gestión del proyecto.

Para más información sobre el modelo, así como un ejemplo detallado de su aplicación en un proyecto comercial real, se remite al lector a la referencia correspondiente.

2.4 Otros Procesos de Software

Aunque el proceso de desarrollo es el proceso central dentro de los procesos de software, se necesitan otros procesos para ejecutar correctamente el proceso de desarrollo y para alcanzar las características deseadas de los procesos de software.

Existen procesos para cada una de las actividades dentro del proceso de desarrollo, por ejemplo, el proceso de diseño, el proceso de prueba, etc. Estos procesos suelen denominarse **metodologías**, y los discutiremos en sus respectivos capítulos.

Aquí abordaremos aquellos procesos que abarcan todo el proyecto y que no están asociados a una tarea específica dentro del proceso de desarrollo. Discutiremos algunos de los procesos más importantes que intervienen en el desarrollo de software.

2.4.1 Proceso de Gestión de Proyectos

Una gestión adecuada es una parte integral del desarrollo de software. Un proyecto grande de desarrollo de software involucra a muchas personas trabajando durante un largo período de tiempo. Hemos visto que un proceso de desarrollo normalmente divide el problema de desarrollar software en un conjunto de fases. Para cumplir con los objetivos de costo, calidad y cronograma, los recursos deben asignarse correctamente a cada actividad del proyecto, y el progreso de las distintas actividades debe ser monitoreado, tomando medidas correctivas si es necesario. Todas estas actividades forman parte del **proceso de gestión de proyectos**.

El proceso de gestión de proyectos especifica todas las actividades que deben realizarse por parte de la dirección del proyecto para asegurar que los objetivos de costo y calidad se cumplan. Su tarea básica es garantizar que, una vez elegido un proceso de desarrollo, este se implemente de manera óptima. El enfoque está en aspectos como la planificación del proyecto, la estimación de recursos y cronograma, y el monitoreo y control del proyecto. En otras palabras, la tarea principal es planificar la implementación detallada del proceso para el proyecto en cuestión y luego asegurarse de que dicho plan se cumpla. En un proyecto grande, un proceso de gestión adecuado es esencial para el éxito.

Las actividades en el proceso de gestión de un proyecto pueden agruparse ampliamente en tres fases: **planificación, monitoreo y control, y análisis de terminación**.

La gestión de proyectos comienza con la planificación, que quizás sea la actividad más crítica de la gestión. El objetivo de esta fase es desarrollar un plan para el desarrollo del software mediante el cual los objetivos del proyecto puedan cumplirse de manera exitosa y eficiente.

Un plan de software generalmente se produce antes de que comience la actividad de desarrollo y se actualiza a medida que este avanza y se dispone de datos sobre el progreso del proyecto. Durante la planificación, las actividades principales son: estimación de costos, determinación de cronogramas y hitos, asignación de personal, planes de control de calidad y planes de monitoreo y control. La **planificación del proyecto** es, sin duda, la actividad de gestión más importante, y constituye la base para el monitoreo y el control. Más adelante en el libro se dedicará un capítulo completo a la planificación de proyectos.

La fase de **monitoreo y control** del proceso de gestión es la más larga en términos de duración; abarca la mayor parte del proceso de desarrollo. Incluye todas las actividades que la gestión del proyecto debe realizar mientras el desarrollo está en curso para garantizar que los objetivos del proyecto se cumplan y que el desarrollo avance según el plan (y lo actualice si es necesario). Como el costo, el cronograma y la calidad son los principales factores impulsores, la mayoría de las actividades de esta fase giran en torno a monitorear los factores que los afectan.

Monitorear los **riesgos potenciales del proyecto**, que podrían impedir el cumplimiento de los objetivos, es otra actividad importante durante esta fase. Y si la información obtenida a través del monitoreo sugiere que los objetivos podrían no cumplirse, en esta fase se toman las acciones necesarias ejerciendo un control adecuado sobre las actividades de desarrollo.

Monitorear un proceso de desarrollo requiere contar con información adecuada sobre el proyecto. Esta información es típicamente obtenida por el proceso de gestión a partir del proceso de desarrollo. Como se mostró anteriormente en la Figura 2.2, la implementación de un modelo de proceso de desarrollo debe ser tal que cada paso del proceso proporcione la información que el proceso de gestión necesita para ese momento. Es decir, el proceso de desarrollo proporciona la

información que el proceso de gestión requiere. Sin embargo, la **interpretación** de esa información forma parte del monitoreo y control.

Mientras que el monitoreo y control se extienden durante toda la duración del proyecto, la última fase del proceso de gestión —**análisis de terminación**— se realiza cuando el proceso de desarrollo ha concluido.

La razón principal para realizar el análisis de terminación es **obtener información sobre el proceso de desarrollo y aprender del proyecto con el fin de mejorar el proceso**. Esta fase también se conoce como **análisis post mortem**. En el desarrollo iterativo, este análisis puede realizarse después de cada iteración para proporcionar retroalimentación que mejore la ejecución de las iteraciones siguientes.

No se discutirá más en detalle en este libro; para un ejemplo de un informe post mortem se remite al lector a.

La relación temporal entre el proceso de gestión y el proceso de desarrollo se muestra en la **Figura 2.12**. Esta relación idealizada muestra que la planificación se realiza antes de que comience el desarrollo, y el análisis de terminación después de que este finaliza. Como muestra la figura, durante el desarrollo, desde las distintas fases del proceso de desarrollo, fluye información cuantitativa hacia la fase de monitoreo y control del proceso de gestión, que utiliza dicha información para ejercer control sobre el proceso de desarrollo.

2.4.2 El Proceso de Inspección

El **objetivo principal** del proceso de inspección es **detectar defectos** en los productos de trabajo.

Las inspecciones de software fueron propuestas por primera vez por **Fagan**. En sus inicios, las inspecciones se centraban en el código, pero con el tiempo su uso se ha extendido también a otros productos de trabajo. En otras palabras, el proceso de inspección **se utiliza a lo largo de todo el proceso de desarrollo**.

Las inspecciones de software son hoy en día una **práctica reconocida en la industria**, con abundantes datos que respaldan que ayudan a mejorar la calidad y también la productividad. Existen libros dedicados al tema que describen en gran detalle cómo deben llevarse a cabo las inspecciones.

Una **inspección** es una revisión de un producto de trabajo de software realizada por un grupo de colegas siguiendo un proceso claramente definido.

El **objetivo básico** de las inspecciones es **mejorar la calidad del producto de trabajo identificando defectos**. Sin embargo, las inspecciones **también mejoran la productividad**, ya que **permiten detectar defectos de manera temprana y rentable**.

Algunas de las características de las inspecciones son:

- Una inspección es realizada por personal técnico para personal técnico.
- Es un proceso estructurado con **roles definidos** para los participantes.
- El enfoque está en **identificar problemas**, no en resolverlos.
- Los datos de la revisión se **registran y utilizan** para monitorear la efectividad del proceso de inspección.

Como las inspecciones son realizadas por un grupo de personas, pueden aplicarse a **cualquier producto de trabajo**, algo que no puede hacerse con las pruebas (testing).

La principal ventaja de esto es que los defectos introducidos en los productos de trabajo de las etapas tempranas del ciclo de vida, o en los productos generados por otros procesos (como el proceso de gestión de proyectos o el proceso de gestión de la configuración), pueden **detectarse en ese mismo producto**, evitando así el costo mucho mayor de descubrir los defectos en etapas posteriores.

Las inspecciones son realizadas por un **equipo de revisores** (o inspectores), que incluye al autor, y uno de ellos actúa como **moderador**.

El moderador tiene la responsabilidad general de asegurar que la revisión se realice de manera adecuada y que se sigan todos los pasos del proceso de revisión.

La mayoría de los métodos de inspección son similares, con **variaciones menores**.

Aquí discutiremos el proceso de inspección utilizado por una **organización comercial**. Las diferentes etapas de este proceso son:

planificación, preparación y visión general, reunión grupal de revisión, y retrabajo y seguimiento.

Estas etapas generalmente se ejecutan en **orden lineal**. A continuación, discutiremos cada una de estas fases.

Planificación

El **objetivo** de la fase de planificación es **prepararse para la inspección**. El **autor del producto** de trabajo se asegura de que el producto esté listo para la inspección. El **moderador** verifica que los **criterios de entrada** sean satisfechos por el producto de trabajo. Los criterios de entrada para diferentes productos de trabajo serán diferentes. Por ejemplo, para el código, un criterio de entrada es que el código compile correctamente y que las herramientas de análisis estático disponibles hayan sido aplicadas. El **equipo** de revisión (inspección) también **se forma en esta fase**.

El paquete que necesita ser distribuido al equipo de revisión es preparado. El paquete incluye el producto de trabajo que será revisado, las especificaciones para ese producto de trabajo, listas de verificación y estándares relevantes. Las especificaciones para el producto de trabajo son con frecuencia la salida de la fase anterior y son necesarias para verificar la corrección del producto de trabajo actual. Por ejemplo, cuando se debe revisar un diseño de alto nivel, entonces el paquete debe incluir también la especificación de requerimientos, sin la cual comprobar la corrección del diseño puede no ser posible.

Preparación y repaso previo (overview)

En esta fase, **el paquete para la revisión es entregado a los revisores**. El **moderador** puede organizar **una reunión inicial**, si es necesario, en la cual el **autor puede ofrecer una breve descripción general** del producto y señalar cualquier área especial que necesite ser examinada cuidadosamente. En esta reunión también se pueden presentar el objetivo y la descripción general del proceso de inspección. La reunión es opcional y puede omitirse. En ese caso, el moderador entrega una copia del paquete de revisión grupal a los revisores.

La **tarea principal** en esta fase es que cada **revisor realice una auto-revisión del producto de trabajo**. Durante la auto-revisión, un **revisor examina todo el producto de trabajo y registra todos los posibles defectos que encuentre** en el registro de auto-preparación. A menudo, los revisores marcan el defecto directamente sobre el producto de trabajo. Los revisores también registran el tiempo que

dedican a la auto-revisión. Puede utilizarse un formulario estándar para el registro de auto-preparación; un ejemplo de formulario se muestra en la Figura 2.13.

Listas de verificación, guías y estándares relevantes pueden ser utilizados durante la revisión. Las listas de verificación que especifican el tipo de defectos que se deben buscar son particularmente útiles. Idealmente, la auto-revisión debe hacerse en un solo período de tiempo continuo. El tiempo recomendado es de menos de dos horas, es decir, el producto de trabajo debe ser lo suficientemente pequeño como para ser examinado completamente en menos de dos horas. Esta fase del proceso de revisión termina cuando todos los revisores han realizado adecuadamente su auto-revisión y completado los registros de auto-revisión.

Reunión de revisión grupal

El propósito básico de la reunión de revisión grupal es elaborar la lista final de defectos, basada en la lista inicial de defectos reportada por los revisores y los nuevos defectos encontrados durante la discusión en la reunión. El criterio de entrada para esta fase es que el moderador esté satisfecho de que todos los revisores estén listos para la reunión. Los principales resultados de esta fase son el registro de defectos y el informe resumido de defectos.

El moderador primero verifica si todos los revisores están preparados. Esto se hace mediante un breve examen de los datos de esfuerzo y de defectos en los registros de auto-revisión para confirmar que se ha dedicado suficiente tiempo y atención a la preparación. Cuando la preparación no es adecuada, la revisión grupal se pospone hasta que todos los participantes estén completamente preparados.

Si todo está listo, se realiza la reunión de revisión grupal. El moderador está a cargo de la reunión y debe asegurarse de que la reunión se mantenga enfocada en su propósito básico de identificación de defectos y no se convierta en una sesión general de lluvia de ideas o en ataques personales hacia el autor.

La reunión se lleva a cabo de la siguiente manera. Un miembro del equipo (llamado lector) revisa el producto de trabajo línea por línea (o cualquier otra unidad pequeña conveniente) y parafrasea cada línea al equipo. A veces no se hace parafraseo y el equipo simplemente revisa el producto línea por línea. En cualquier línea, si algún revisor tiene algún problema previo o encuentra un problema nuevo durante la reunión mientras escucha a los demás, el revisor lo señala. Puede haber una discusión sobre el problema planteado. El autor acepta el problema como un defecto o aclara por qué no lo es. Después de la discusión se llega a un acuerdo y un miembro del equipo de revisión (llamado escriba) registra los defectos identificados en el registro de defectos. Al final de la reunión, el escriba lee los defectos registrados en el registro para una revisión final por parte de los miembros del equipo.

Se debe notar que durante todo el proceso de revisión, solo se identifican defectos. No es objetivo del grupo identificar soluciones, eso lo hace el autor posteriormente.

El registro final de defectos es el registro oficial de los defectos identificados en la inspección y también puede usarse para dar seguimiento a los defectos hasta su cierre. Sin embargo, para analizar la efectividad de una revisión, solo se necesita información a nivel resumido, para lo cual se prepara un informe resumido. El informe resumido describe el producto de trabajo, el esfuerzo total invertido y su desglose en las diferentes actividades del proceso de revisión, el número total de defectos encontrados por cada categoría y el tamaño. Si también se registraron los tipos de defectos,

entonces el número de defectos en cada categoría puede registrarse en el resumen. Un informe resumido parcialmente completado de la revisión de un plan de gestión de proyectos se muestra en la Figura 2.14 [97].

El informe resumido se explica por sí mismo. El número total de defectos menores encontrados fue 19, y el número total de defectos mayores encontrados fue 3. Es decir, la densidad de defectos encontrada es $16/14 = 1.2$ defectos menores por página y $3/14 = 0.2$ defectos mayores por página. Según la experiencia, ambos valores están dentro del rango observado anteriormente; por lo tanto, se puede asumir que la revisión se realizó correctamente. El equipo de revisión tenía 4 miembros, y cada uno dedicó 2.5 horas a la revisión individual y la reunión de revisión duró 2.5 horas. Esto significa que la tasa de cobertura durante la preparación y revisión fue de $14/2.5 = 5.6$ páginas por hora, lo que según la experiencia pasada también parece aceptable.

Si las modificaciones requeridas para corregir los defectos y abordar los problemas son pocas, el estado de la revisión grupal es "aceptada". Si las modificaciones requeridas son muchas, puede ser necesaria una reunión de seguimiento por parte del moderador o una nueva revisión para verificar si los cambios se han incorporado correctamente. El moderador recomienda lo que se debe hacer. Además, también puede hacer recomendaciones respecto a las revisiones en las etapas siguientes (por ejemplo, en una revisión de diseño detallado puede recomendar qué módulos de código deben someterse a inspecciones).

Corrección y seguimiento

En esta fase, el autor corrige todos los defectos señalados durante la inspección. El autor puede rehacer el producto de trabajo, si eso es lo que recomendó el moderador. El autor revisa las correcciones con el moderador o en una nueva revisión, dependiendo de la decisión tomada en la reunión de revisión grupal. El escriba se asegura de que el informe de la revisión grupal y las actas de las reuniones sean comunicados al equipo de revisión grupal.

Roles y responsabilidades

El proceso de inspección es un proceso estructurado en el que diferentes personas tienen distintas responsabilidades. Los roles clave en una revisión grupal son los de moderador, lector, escriba, autor y revisor. Estos son roles lógicos y una persona puede tener múltiples roles, con las restricciones de que el autor no puede ser moderador ni lector, y el moderador no puede ser lector. Esto implica que el tamaño mínimo del equipo de revisión grupal es de tres personas: el autor, el moderador y el lector. Estas tres personas también son revisores y pueden asignar el rol de escriba a otra persona. Las responsabilidades de estos roles deben quedar claras a partir del proceso de inspección. Aquí se resumen brevemente las principales actividades del moderador y de los revisores.

El moderador probablemente tiene el rol más importante durante una revisión grupal. Tiene la responsabilidad general de asegurar que la revisión se realice correctamente. El moderador debe haber recibido capacitación formal sobre cómo conducir revisiones o tener experiencia participando en algunas revisiones. Las responsabilidades del moderador incluyen:

- Programar la reunión de revisión grupal.

- Al inicio de la reunión de revisión, asegurarse de que todos los participantes estén preparados y hayan entregado su registro de auto-preparación, o reprogramar la revisión grupal.
- Conducir la revisión grupal de manera ordenada y eficiente.
- Asegurarse de que la reunión se mantenga enfocada en la tarea principal de identificación de defectos.
- Hacer seguimiento de cada problema hasta su resolución o asegurarse de que sea registrado por otra persona.
- Asegurarse de que se completen los informes de la revisión grupal.

Durante la reunión, el moderador debe asegurarse de que todos los participantes contribuyan de manera efectiva, que todos sean escuchados, que haya acuerdo sobre los hallazgos de la revisión y que el nivel de interés no disminuya. Una responsabilidad clave es garantizar que durante la reunión el enfoque permanezca en la identificación de problemas y no se desvíe hacia la resolución de problemas, y que todos los revisores permanezcan enfocados en encontrar defectos en el producto de trabajo y no en buscar fallas en el autor. En general, la conducción ordenada y amistosa de la reunión es responsabilidad del moderador. Después de la reunión, el moderador debe asegurarse de que todos los participantes estén satisfechos, que los informes de revisión estén completos y que se hayan tomado las acciones de seguimiento.

Un **revisor es principalmente responsable de encontrar defectos**. Generalmente, todos los miembros del equipo de revisión grupal son revisores. Los defectos se encuentran ya sea mediante revisión individual o durante la reunión de revisión grupal. Los puntos principales para un revisor son:

- Estar preparado para la revisión grupal.
- Ser objetivo; enfocarse en los problemas y no en las personas.
- Concentrarse en los problemas (ofrecer soluciones solo después de la revisión grupal).
- Si algo no está claro, no dudar en detener el progreso hasta entenderlo.
- Cuando se demuestre que está equivocado, seguir adelante.

2.4.3 Proceso de Administración de Configuración de Software

En un proyecto de software, los cambios ocurren de manera continua: cambios debido a la evolución de los productos de trabajo a medida que avanza el proyecto, cambios debido a la detección y corrección de defectos (bugs), y cambios debido a modificaciones en los requisitos. Todos estos se reflejan como cambios en los archivos que contienen código fuente, datos o documentación. La **gestión de configuración (CM)** o gestión de configuración de software (SCM) es la **disciplina que permite controlar de manera sistemática los cambios que ocurren durante el desarrollo**. El IEEE define SCM como “el proceso de identificar y definir los elementos del sistema, controlar el cambio de estos elementos a lo largo de su ciclo de vida, registrar e informar el estado de los elementos y las solicitudes de cambio, y verificar la completitud y corrección de los elementos”.

Aunque los tres tipos de cambios existen, los **cambios por evolución del producto y los cambios por corrección de errores** pueden considerarse, en cierto sentido, como una **parte natural del proyecto** que debe manejarse **incluso si los requisitos no cambian**. Por otro lado, los cambios en los requisitos tienen una dinámica diferente. Los pasos adicionales que deben realizarse para los cambios en los requisitos se tratarán como un proceso separado después de discutir el proceso de CM.

La gestión de configuración de software es un proceso independiente del proceso de desarrollo, en gran medida porque la mayoría de los modelos de desarrollo consideran la visión macro y no los cambios en archivos individuales. De alguna manera, el proceso de desarrollo se incorpora bajo el proceso de control de configuración, de modo que los cambios se permiten de manera controlada, como se muestra en la Figura 2.15 para un modelo de desarrollo tipo cascada. Cabe destacar que SCM controla directamente solo los productos de un proceso e influye de manera indirecta en las actividades que producen dichos productos.

La CM es esencial para cumplir uno de los objetivos básicos de un proyecto: la entrega de un producto de software de alta calidad al cliente. ¿Qué es este “software” que se entrega? Como mínimo, contiene los distintos archivos fuente u objetos que componen el código fuente u objeto, los scripts para construir el sistema funcional a partir de estos archivos y la documentación asociada. Durante el transcurso del proyecto, los archivos cambian, generando diferentes versiones. En esta situación, ¿cómo asegura un gerente de programa que las versiones apropiadas del código fuente se combinen sin omitir ningún archivo y que las versiones correctas de los documentos, consistentes con el código final, sean entregadas? Esto se garantiza mediante una adecuada gestión de configuración.

Funcionalidad de la CM

Para comprender mejor la gestión de configuración (CM), consideremos algunas de las funcionalidades que un proyecto requiere del proceso de CM. Aunque los requerimientos específicos de un proyecto respecto a su proceso de CM dependen de la naturaleza del proyecto, aquí se describen algunas funciones que generalmente son necesarias.

- **Obtener la última versión de un programa.** Supongamos que un programa debe modificarse. Claramente, la modificación debe realizarse sobre la copia más reciente de ese programa; de lo contrario, los cambios realizados anteriormente podrían perderse. Un proceso de CM adecuado debe garantizar que la última versión de un archivo pueda obtenerse fácilmente.
- **Deshacer un cambio o volver a una versión específica.** Puede suceder que se realice un cambio en un programa y luego sea necesario revertirlo. De manera similar, podría haberse aplicado una modificación a varios programas para implementar una solicitud de cambio, y más tarde decidirse que todo ese cambio debe deshacerse. El proceso de CM debe permitir que esto se realice sin dificultades.
- **Prevenir cambios o eliminaciones no autorizadas.** Un programador podría decidir modificar algunos programas, solo para descubrir después que el cambio tiene efectos secundarios negativos. El proceso de CM se asegura de que los cambios no aprobados no estén permitidos.
- **Reunir todas las fuentes, documentos y otra información del sistema actual.** Todos los archivos fuente y los archivos relacionados son necesarios para liberar el producto. El proceso de CM debe proporcionar esta funcionalidad. Además, a veces se necesitan todas las fuentes y archivos relacionados de un sistema en funcionamiento para su reinstalación.

Estas son algunas de las necesidades básicas que un proceso de CM debe satisfacer. Existen también otros requerimientos más avanzados, como manejar actualizaciones concurrentes o mantener invariantes.

Mecanismos de CM

El propósito principal de la **Gestión de la Configuración (CM)** es proporcionar varios mecanismos que puedan soportar la funcionalidad necesaria que un proyecto requiere para manejar los tipos de escenarios discutidos anteriormente, los cuales surgen debido a cambios.

Los mecanismos comúnmente usados para proveer la funcionalidad necesaria incluyen los siguientes:

- Identificación de configuración y establecimiento de líneas base
- Control o gestión de versiones
- Control de acceso

Como se discutió anteriormente, el software que se desarrolla no es un monolito.

Un **Elemento de Configuración de Software (SCI)**, o simplemente *elemento*, es un documento o artefacto que se coloca explícitamente bajo control de configuración y que puede considerarse como una unidad básica de modificación.

A medida que el proyecto avanza, se realizan cientos de cambios a estos elementos de configuración.

Sin combinar periódicamente versiones apropiadas de estos elementos en un estado del sistema, se volverá muy difícil obtener el sistema a partir de las diferentes versiones de los múltiples SCIs.

Por esta razón, se establecen **líneas base (baselines)**.

Una línea base, una vez establecida, captura un estado lógico del sistema y forma la base de los cambios posteriores [14].

Una línea base también forma un punto de referencia en el desarrollo de un sistema.

Una línea base es esencialmente una disposición de un conjunto de SCIs.

Es decir, una línea base es un conjunto de SCIs y la relación entre ellos.

Por ejemplo, una **línea base de requerimientos** puede consistir en muchos SCIs de requerimientos (por ejemplo, cada requerimiento es un SCI) y cómo estos SCIs se relacionan en la línea base de requerimientos (por ejemplo, en qué orden aparecen).

Debe notarse que los SCIs gestionados por el SCM no son independientes entre sí, y existen dependencias entre varios SCIs.

Se dice que un SCI X depende de otro SCI Y si un cambio en Y podría requerir un cambio en X para que X siga siendo correcto o para que las líneas base permanezcan consistentes [147].

Una solicitud de cambio, por lo tanto, podría requerir que se realicen cambios en ciertos SCIs; la dependencia de otros SCIs con respecto a los que se están cambiando podría requerir que esos otros SCIs también necesiten ser modificados.

Claramente, la dependencia entre los SCIs debe ser comprendida y documentada adecuadamente.

El **control de versiones** es un tema clave para la CM [14, 12, 147], y existen muchas herramientas disponibles para ayudar a gestionar las distintas versiones de los programas.

Sin tal mecanismo, muchas de las funciones requeridas por la CM no pueden ser soportadas.

El control de versiones ayuda a preservar versiones anteriores de los programas cada vez que estos

son modificados.

Los sistemas de CM comúnmente usados, como **SCCS**, **CVS** (www.cvshome.org) y **VSS** (msdn.microsoft.com/vstudio/previous/ssafe), se enfocan fuertemente en el control de versiones.

La mayoría de los sistemas de CM también proporcionan medios para el **control de acceso**.

Para entender la necesidad del control de acceso, veamos el **ciclo de vida de un SCI**.

Típicamente, mientras un SCI está en desarrollo y no es visible para otros SCIs, se considera que está en el **estado de trabajo** (*working state*).

Un SCI en estado de trabajo no está bajo control de SCM y puede modificarse libremente.

Una vez que el desarrollador considera que el SCI es lo suficientemente estable como para ser usado por otros, el SCI se somete a **revisión**, y el elemento entra en el estado “*en revisión*”.

Una vez que un elemento está en este estado, se considera “congelado”, y cualquier cambio hecho a una copia privada que el desarrollador haya mantenido no es reconocido.

Después de una revisión exitosa, el SCI se incorpora en una **biblioteca**, tras lo cual el elemento está formalmente bajo SCM.

El propósito básico de esta revisión es asegurarse de que el elemento sea de calidad satisfactoria y necesario para otros, aunque la naturaleza exacta de la revisión dependerá de la naturaleza del SCI y de la práctica real del SCM.

Por ejemplo, la revisión podría implicar verificar si el elemento cumple con sus especificaciones o si ha sido correctamente probado de manera unitaria.

Si el elemento no es aprobado, puede devolverse al desarrollador, y el SCI vuelve al estado de trabajo.

Este “**ciclo de vida**” de un elemento, desde la perspectiva del SCM, se muestra en la Figura 2.16 [147].

Una vez que un SCI está en la biblioteca, cualquier modificación debe ser controlada, ya que otros podrían estar usando ese elemento.

Por lo tanto, el acceso a los elementos en la biblioteca está controlado.

Para realizar un cambio aprobado, el SCI se **extrae** (*checked out*) de la biblioteca, se realiza el cambio, la modificación se revisa y luego el SCI se **vuelve a ingresar** (*checked in*) a la biblioteca.

Cuando se registra una nueva versión, la versión anterior no se reemplaza, y tanto la versión antigua como la nueva pueden coexistir en la biblioteca —a menudo de manera lógica, manteniendo un solo archivo junto con información sobre los cambios para recrear la versión anterior—.

Este aspecto del SCM a veces se denomina **gestión de biblioteca** (*library management*) y se realiza con la ayuda de herramientas.

Proceso de CM

El **proceso de Gestión de la Configuración (CM)** define el conjunto de actividades que deben realizarse para controlar los cambios.

Como ocurre con la mayoría de las actividades de gestión de proyectos, la primera etapa del proceso de CM es la **planificación**.

Luego, el proceso debe ejecutarse, generalmente utilizando algunas herramientas.

Finalmente, dado que cualquier plan de CM requiere cierta disciplina por parte del personal del proyecto —en cuanto a almacenar los elementos en los lugares adecuados y realizar los cambios correctamente—, el **monitoreo del estado** de los elementos de configuración y la **realización de auditorías de CM** son, por lo tanto, otras actividades del proceso de CM.

La **planificación de la gestión de la configuración** implica identificar los elementos de configuración y especificar los procedimientos que se usarán para controlar e implementar los cambios en dichos elementos.

Identificar los elementos de configuración es una actividad fundamental en cualquier tipo de CM [12, 89, 147].

Ejemplos típicos de **elementos de configuración** incluyen:

- Especificaciones de requerimientos,
- Documentos de diseño,
- Código fuente,
- Planes de prueba, guiones de prueba, procedimientos de prueba, datos de prueba,
- Estándares usados en el proyecto (como estándares de codificación o de diseño),
- El plan de aceptación,
- Documentos como el plan de CM y el plan del proyecto,
- Documentación de usuario (como el manual del usuario),
- Documentos como el material de capacitación,
- Documentos contractuales (incluyendo herramientas de soporte como un compilador o herramientas internas),
- Registros de calidad (registros de revisiones, registros de pruebas),
- Registros de CM (registros de liberación, registros de seguimiento de estado).

Cualquier producto suministrado por el cliente o elemento adquirido que forme parte de la entrega (llamado “*producto de software incluido*”) también es un elemento de configuración.

Dado que normalmente hay muchos elementos en un proyecto, **la manera en que serán organizados** también se decide en la fase de planificación.

Típicamente, la **estructura de directorios** que se utilizará para almacenar los distintos elementos se define en el plan.

Para facilitar una denominación adecuada de los elementos de configuración, las **convenciones de nombres** para los elementos de CM se deciden durante las etapas de planificación de CM.

Además de los estándares de nombres, también debe planificarse la **numeración de versiones**.

Cuando un elemento de configuración cambia, el elemento antiguo no se reemplaza con la nueva copia; en cambio, se mantiene la copia antigua y se crea una nueva.

Este enfoque genera múltiples versiones de un mismo elemento, por lo que se necesitan **políticas para la asignación de números de versión**.

Si se está utilizando una herramienta de CM, a veces la herramienta maneja la numeración de versiones.

De lo contrario, debe hacerse explícitamente dentro del proyecto.

El **resultado de esta fase** es el **plan de CM**.

El **controlador de configuración (CC)** es responsable de la implementación del plan de CM.

Dependiendo del tamaño del sistema en desarrollo, su función puede ser de tiempo parcial o de

tiempo completo.

En ciertos casos, donde hay equipos grandes o donde dos o más equipos o grupos están involucrados en el desarrollo de las mismas o diferentes partes del software o de sistemas que interactúan, puede ser necesario contar con una **Junta de Control de Configuración (CCB)**.

Esta junta incluye representantes de cada uno de los equipos.

Un CCB (o un CC) se considera esencial para la CM [89], y el plan de CM debe definir claramente los **roles y responsabilidades** del CC/CCB.

Estas responsabilidades también dependerán del tipo de sistema de archivos y de la naturaleza de las herramientas de CM que se utilicen.

Para que un proceso de CM funcione bien, las personas del proyecto deben usarlo de acuerdo con el plan de CM y seguir sus políticas y procedimientos.

Sin embargo, las personas cometen errores.

Y si por error un SCI se extravía, o se violan las políticas de control de acceso, la **integridad del producto** puede perderse.

Para minimizar errores y detectarlos tempranamente, puede realizarse un **chequeo regular del estado de los SCIs**.

También puede llevarse a cabo una **auditoría de configuración** periódica para asegurar que la integridad del sistema de CM no esté siendo violada.

La auditoría también puede verificar que los cambios en los SCIs debido a solicitudes de cambio (que se discutirán a continuación) se hayan hecho correctamente y que dichas solicitudes se hayan implementado.

Además de verificar el estado de los elementos, también debe controlarse el **estado de las solicitudes de cambio** (discutidas más adelante).

Para lograr este objetivo, se examinan las solicitudes de cambio que se hayan recibido desde la última operación de monitoreo de estado de CM.

Para cada solicitud de cambio, el estado del elemento mencionado en los registros de la solicitud de cambio se compara con el estado real.

También pueden realizarse verificaciones para asegurar que todos los elementos modificados pasen por su **ciclo de vida completo** (es decir, el diagrama de estados) antes de incorporarse a la línea base.

El **controlador de configuración** o el **gerente del proyecto** realizan la planificación de CM.

Esta comienza solo cuando el proyecto ha sido iniciado y cuando el entorno operativo y las especificaciones de requerimientos están claramente documentados.

2.4.4 Proceso de Gestión de Cambios en los Requerimientos

Los **requerimientos cambian**.

Y los cambios en los requerimientos pueden ocurrir en cualquier momento durante la vida de un proyecto (o incluso después de ella).

Cuanto más avanzado esté el ciclo de vida cuando los requerimientos cambian, más severo será el impacto sobre el proyecto.

En lugar de desear que no haya cambios, o esperar que de alguna manera los requerimientos iniciales sean “tan buenos” que no se requieran modificaciones, es mejor que el **gerente del proyecto** se prepare para manejar las solicitudes de cambio a medida que surjan.

Los **cambios no controlados en los requerimientos** pueden tener un efecto muy negativo en el

costo, el cronograma y la calidad del proyecto.

Los cambios en los requerimientos pueden representar **hasta un 40% del costo total** [22].

Debido al gran impacto potencial que los cambios en los requerimientos pueden tener sobre el proyecto, con frecuencia se utiliza un **proceso separado** para tratarlos.

El **proceso de gestión de cambios** define el conjunto de actividades que se realizan cuando hay nuevos requerimientos o cambios en los requerimientos existentes (a ambos los llamaremos *cambios en los requerimientos*).

Aunque aquí nos enfocamos en los cambios de requerimientos, este proceso **también puede aplicarse a otros cambios importantes, como modificaciones de diseño o correcciones de errores mayores** en un sistema ya en despliegue.

Aquí discutimos un proceso de gestión de cambios en los requerimientos basado en uno utilizado en una organización comercial.

El proceso de gestión de cambios tiene los siguientes pasos:

- Registrar los cambios
- Realizar un análisis de impacto sobre los productos de trabajo
- Estimar el impacto en el esfuerzo y el cronograma
- Revisar el impacto con las partes interesadas correspondientes
- Retrabajar los productos de trabajo

Un cambio se inicia mediante una **solicitud de cambio** (*change request*).

Se mantiene un **registro de solicitudes de cambio** para hacer seguimiento de ellas.

Cada entrada en el registro contiene un **número de solicitud de cambio**, una **breve descripción del cambio**, el **efecto del cambio**, el **estado de la solicitud de cambio** y las **fechas clave**.

El efecto de una solicitud de cambio se evalúa realizando un **análisis de impacto**.

El análisis de impacto implica identificar los productos de trabajo y los elementos de configuración que necesitan ser modificados y evaluar la magnitud del cambio en cada uno; reexaminar los riesgos del proyecto revisando el plan de gestión de riesgos; y evaluar las implicaciones generales de los cambios en las estimaciones de **esfuerzo y cronograma**.

Una vez que un cambio ha sido revisado y aprobado, se implementa, es decir, se realizan los cambios en todos los elementos afectados.

El **seguimiento de la implementación** de una solicitud de cambio puede ser gestionado por el **proceso de gestión de configuración**, discutido anteriormente.

Un peligro de los cambios en los requerimientos es que, aunque cada cambio individual no sea grande por sí mismo, **a lo largo de la vida del proyecto el impacto acumulado de los cambios puede ser considerable**.

Por lo tanto, además de estudiar y rastrear el impacto de los cambios individuales, también debe **monitorearse el impacto acumulado de los cambios**.

Para los cambios acumulativos, se utiliza el **registro de cambios**.

Para facilitar este análisis, el registro suele mantenerse en forma de **hoja de cálculo (spreadsheet)**.

2.4.5 Proceso de Gestión del Proceso

Un **proceso de software** no es una entidad estática: debe **cambiar para mejorar**, de modo que los productos producidos mediante ese proceso sean de **mayor calidad y menor costo**.

Como hemos visto, mejorar la **calidad** y la **productividad** son metas fundamentales de la ingeniería.

Para lograr estos objetivos, el proceso de software debe mejorarse continuamente, ya que la calidad y la productividad dependen en gran medida del proceso.

Como se mencionó antes, mejorar la calidad y productividad del proceso es el **principal objetivo del proceso de gestión del proceso**.

Debe enfatizarse que la **gestión del proceso** es muy diferente de la **gestión de proyectos**.

En la gestión del proceso, el enfoque está en **mejorar el proceso**, lo que a su vez mejora la calidad y la productividad generales de los productos que se desarrollan con él.

En la gestión de proyectos, el enfoque está en **ejecutar el proyecto actual** y asegurar que sus objetivos se cumplan.

La duración temporal de interés en la gestión de proyectos suele ser la duración del proyecto, mientras que la gestión del proceso opera en una escala de tiempo mucho mayor, ya que **cada proyecto se considera un punto de datos** para el proceso.

La gestión del proceso es un tema avanzado, fuera del alcance de este libro.

Se recomienda al lector interesado consultar el libro de **Humphrey**.

Aquí solo discutiremos brevemente algunos aspectos.

Para mejorar su proceso de software, una organización necesita primero **entender el estado actual del proceso**, y luego **desarrollar un plan para mejorarlo**.

Generalmente se acepta que los cambios en un proceso deben introducirse **en pequeños incrementos**, y que no es factible **revolucionar completamente un proceso**.

La razón es que se necesita tiempo para **interiorizar y seguir verdaderamente** los nuevos métodos que se introduzcan.

Y solo cuando esos nuevos métodos se implementen correctamente serán visibles sus efectos.

Introducir demasiados métodos nuevos en el proceso de software haría muy difícil la tarea de implementar el cambio.

Si aceptamos que los cambios en un proceso deben introducirse en **pequeños incrementos**, la siguiente pregunta es:

de un conjunto grande de posibles mejoras, ¿**en qué orden** deberían realizarse las actividades de mejora?

O, ¿**qué pequeño cambio** debería introducirse primero?

Esto depende del **estado actual del proceso**.

Por ejemplo, si el proceso es muy primitivo, no tiene sentido proponer **métricas sofisticadas de control de proyectos** como estrategia de mejora, ya que incorporarlas en un proceso tan básico no es sencillo.

Por otro lado, si el proceso ya utiliza muchos modelos básicos, tal paso podría ser el adecuado para seguir mejorándolo.

Por lo tanto, decidir qué actividades emprender para mejorar el proceso depende del estado actual del proceso.

Una vez que se realiza alguna mejora, **el estado del proceso cambia**, y puede surgir un nuevo conjunto de posibilidades.

Este concepto de introducir cambios en pequeños incrementos basados en el estado actual del proceso se refleja en el **marco de trabajo del Modelo de Madurez de Capacidades (CMM, Capability Maturity Model)**.

El CMM proporciona una **hoja de ruta general para la mejora del proceso**.

A continuación se da una breve descripción del marco CMM.

Un ejemplo de implementación del CMM en una organización puede encontrarse en [96].

La **capacidad del proceso de software** describe el rango de resultados esperados que pueden lograrse siguiendo el proceso.

La capacidad del proceso de una organización determina qué puede esperarse de ella en términos de **calidad y productividad**.

El objetivo de la mejora del proceso es **mejorar su capacidad**.

Un **nivel de madurez** es un punto de evolución bien definido hacia la consecución de un proceso de software maduro [134].

Basado en evidencia empírica obtenida al examinar los procesos de muchas organizaciones, el CMM propone **cinco niveles de madurez** bien definidos para un proceso de software:

1. Inicial (nivel 1)
2. Repetible
3. Definido
4. Gestionado
5. Optimizando (nivel 5)

El marco CMM establece que, dado que la mejora del proceso se incorpora mejor en **pequeños incrementos**, los procesos avanzan **de su nivel actual al siguiente nivel superior** cuando se mejoran.

Así, durante el curso de la mejora del proceso, un proceso **progresará nivel por nivel** hasta alcanzar el nivel 5.

Esto se muestra en la Figura 2.17.

El CMM proporciona **características de cada nivel**, que pueden usarse para **evaluar el nivel actual del proceso** de una organización.

Como el avance se da de un nivel al siguiente, las características también sugieren las **áreas en las que el proceso debe mejorar** para poder subir de nivel.

Esencialmente, para cada nivel, el modelo especifica las áreas donde la mejora puede ser absorbida y traer los **mayores beneficios**.

En conjunto, esto proporciona una **hoja de ruta para la mejora continua del proceso**.

El proceso **inicial (nivel 1)** es esencialmente un proceso **ad hoc**, sin métodos formalizados para ninguna actividad.

Faltan controles básicos de proyecto que aseguren que las actividades se realizan correctamente y que se sigue el plan del proyecto.

En situaciones de crisis, los planes del proyecto y los procesos de desarrollo se abandonan en favor de un enfoque de *codificar y probar*.

El éxito en organizaciones de este tipo depende únicamente de la calidad y capacidad de los individuos.

La capacidad del proceso es **impredecible**, ya que el proceso cambia constantemente.

Las organizaciones en este nivel pueden beneficiarse más mejorando la **gestión de proyectos**, la **aseguración de la calidad** y el **control de cambios**.

En un proceso **repetible (nivel 2)**, existen **políticas** para gestionar proyectos de software y **procedimientos** para implementarlas.

Es decir, la **gestión de proyectos** está bien desarrollada en este nivel.

Algunas características de un proceso en este nivel son:

los compromisos de proyecto son realistas y se basan en experiencias pasadas con proyectos similares;

el costo y el cronograma se rastrean y los problemas se resuelven cuando surgen;

existen mecanismos formales de **control de configuración**;

y los estándares de proyectos de software están definidos y se siguen.

En esencia, los resultados obtenidos con este proceso pueden repetirse, ya que la planificación y el seguimiento del proyecto son formales.

En el nivel **definido (nivel 3)**, la organización ha **estandarizado un proceso de software**, que está debidamente documentado.

Existe un **grupo de procesos de software** dentro de la organización que **posee y gestiona el proceso**.

En el proceso, cada paso está cuidadosamente definido, con **criterios de entrada y salida verificables**, **metodologías para realizar el paso** y **mecanismos de verificación** para su salida.

En este nivel, tanto los procesos de desarrollo como los de gestión son **formales**.

En el nivel **gestionado (nivel 4)**, existen **objetivos cuantitativos** para los procesos y productos.

Se recopilan datos de los procesos de software, que se utilizan para **construir modelos** que caracterizan el proceso.

Por lo tanto, la **medición** juega un papel importante en este nivel.

Debido a los modelos construidos, la organización tiene una buena comprensión de la capacidad y las deficiencias del proceso.

Los resultados de un proceso así pueden **predecirse en términos cuantitativos**.

En el nivel **optimizando (nivel 5)**, el enfoque de la organización está en la **mejora continua del proceso**.

Se recopilan y analizan rutinariamente datos para identificar áreas que pueden fortalecerse para mejorar la calidad o la productividad.

Se introducen **nuevas tecnologías y herramientas**, y se mide su efecto en el desempeño del proceso.

Se aplican las **mejores prácticas de ingeniería y gestión de software** en toda la organización.

Este marco CMM puede utilizarse para **mejorar el proceso**.

La mejora requiere primero **evaluar el nivel actual del proceso**.

A partir de ese nivel, se conocen —gracias al marco— las áreas en las que pueden obtenerse los mayores beneficios.

Por ejemplo, para mejorar un proceso en el nivel 1 (o pasar del nivel 1 al nivel 2), deben **formalizarse las actividades de gestión de proyectos y de control de cambios**.

El marco completo de CMM proporciona más detalles sobre qué áreas específicas deben fortalecerse para avanzar en los niveles de madurez.

Esto se logra especificando las **áreas clave de proceso** de cada nivel de madurez, las cuales pueden usarse para determinar qué áreas deben reforzarse para subir de nivel.

Algunas de las áreas clave de proceso de los distintos niveles se muestran en la Figura 2.18 [134].

Aunque el marco CMM especifica **qué áreas del proceso deben mejorarse** para aumentar su madurez, **no especifica cómo** debe hacerse esa mejora.

Es, esencialmente, un **marco de referencia**, no una guía detallada.

Su propósito es **orientar la actividad de mejora del proceso** a lo largo de los niveles de madurez, de modo que la mejora se introduzca en incrementos y las acciones estén siempre **claramente enfocadas**.

Muchas organizaciones han usado con éxito este marco para mejorar sus procesos.

Es una **fuerza impulsora clave** para la mejora de procesos.

Un ejemplo detallado de cómo una organización que sigue el CMM ejecuta sus proyectos puede encontrarse en [96].

3 Análisis y Especificación de Requerimientos de Software

La complejidad y el tamaño de los sistemas de software están aumentando continuamente.

A medida que la escala cambia hacia sistemas de software más grandes y complejos, surgen nuevos problemas que no existían en sistemas más pequeños (o que eran de menor importancia), lo que conduce a una redefinición de las prioridades de las actividades que intervienen en el desarrollo de software.

Los requerimientos de software son una de esas áreas, a las que se les daba poca importancia en los primeros días del desarrollo de software, ya que el énfasis estaba en la codificación y el diseño. La suposición tácita era que los desarrolladores entendían claramente el problema cuando se les explicaba, generalmente de manera informal.

A medida que los sistemas se hicieron más complejos, se volvió evidente que los objetivos de todo el sistema no podían ser comprendidos con facilidad. De ahí surgió la necesidad de un análisis de requerimientos más riguroso. Hoy en día, para los grandes sistemas de software, el análisis de requerimientos es quizás la actividad más difícil e intratable; también es muy propensa a errores. Muchos creen que la disciplina de la ingeniería de software es más débil en esta área crítica.

Parte de la dificultad se debe al alcance de esta actividad. El proyecto de software se inicia a partir de las necesidades del cliente. Al comienzo, estas necesidades están en la mente de distintas personas dentro de la organización del cliente. El analista de requerimientos debe identificar estos requerimientos hablando con dichas personas y comprendiendo sus necesidades.

En situaciones donde el software busca automatizar un proceso actualmente manual, muchas de las necesidades pueden entenderse observando la práctica actual. Pero no existen métodos similares para sistemas en los que los procesos manuales no existen, o para las “nuevas funcionalidades” que suelen agregarse al automatizar un proceso existente. Para esos sistemas, el problema de los requerimientos se complica aún más por el hecho de que las necesidades y los requerimientos del sistema pueden no ser conocidos ni siquiera por el propio usuario —deben ser visualizados y creados.

Por lo tanto, identificar requerimientos implica necesariamente especificar lo que algunas personas tienen en sus mentes (o lo que llegará a sus mentes cuando lo visualicen). Como la información en sus mentes no está formalmente declarada ni organizada, la entrada a la fase de especificación de requerimientos de software es, por naturaleza, informal e imprecisa, y es probable que esté incompleta. Cuando se deben recopilar entradas de múltiples personas, estas también tienden a ser inconsistentes.

La fase de requerimientos traduce las ideas en la mente de los clientes (la entrada) en un documento formal (la salida de la fase de requerimientos). Así, el resultado de la fase es un conjunto de requerimientos especificados con precisión, que idealmente sean completos y consistentes, mientras que la entrada no posee ninguna de estas propiedades.

Es evidente que el proceso de especificación de requerimientos no puede ser totalmente formal; cualquier proceso de traducción formal que produzca una salida formal debe tener una entrada precisa y no ambigua. Esta es la razón por la cual la actividad de requerimientos de software no

puede ser completamente automatizada, y cualquier método para identificarlos puede ser, como máximo, un conjunto de guías.

En este capítulo discutiremos qué son los requerimientos, por qué la especificación de requerimientos es importante, cómo se analizan y especifican los requerimientos, cómo se validan y algunas métricas que se pueden aplicar a los requerimientos. Finaliza con una discusión sobre el **SRS** (Software Requirements Specification) de los casos de estudio utilizados en el libro.

3.1 Requerimientos de Software

El **IEEE** define un requerimiento como:

(1) Una *condición o capacidad que un usuario necesita para resolver un problema o alcanzar un objetivo*;

(2) Una *condición o capacidad que debe cumplir o poseer un sistema... para satisfacer un contrato, estándar, especificación u otro documento formalmente impuesto*. [91]

Es importante notar que en los **requerimientos de software** estamos tratando con los requerimientos del sistema propuesto, es decir, *las capacidades que el sistema —que aún no se ha desarrollado— debería tener*. Es precisamente porque se trata de especificar un sistema que no existe todavía que el problema de los requerimientos se vuelve complicado.

El objetivo de la actividad de requerimientos es producir la **Especificación de Requerimientos de Software (SRS)**, la cual describe **qué debe hacer el software propuesto**, sin describir **cómo lo hará**.

Producir el SRS es más fácil de decir que de hacer. Una limitación básica es que las necesidades del usuario cambian constantemente, a medida que cambia el entorno en el cual el sistema debe funcionar. Aunque es inevitable aceptar que algunos cambios en los requerimientos ocurrirán, aún existen razones de peso por las cuales debe hacerse un trabajo minucioso en la fase de requerimientos, de modo que el SRS resultante sea de alta calidad y relativamente estable.

3.1.1 Necesidad del SRS

El origen de la mayoría de los sistemas de software está en las **necesidades de los clientes**. El sistema de software es creado por los **desarrolladores**, y finalmente será usado por los **usuarios finales**. Así, existen tres partes principales interesadas en un nuevo sistema:

- el **cliente**,
- el **desarrollador**,
- y los **usuarios**.

De algún modo, los requerimientos del sistema que satisfagan las necesidades del cliente y las preocupaciones de los usuarios deben ser comunicados al desarrollador.

El problema es que:

- el cliente usualmente **no entiende de software ni del proceso de desarrollo**,
- y el desarrollador a menudo **no comprende el problema ni el área de aplicación del cliente**.

Esto genera una **brecha de comunicación** entre las partes. Un propósito fundamental de la especificación de requerimientos de software es **cerrar esa brecha**.

El SRS es el medio a través del cual las necesidades del cliente y del usuario se especifican con precisión para el desarrollador.

De allí se derivan varias ventajas importantes:

- **Un SRS establece la base de acuerdo entre el cliente y el proveedor sobre lo que hará el producto de software.**

Este acuerdo suele formalizarse en un **contrato legal** entre el cliente (o consumidor) y el desarrollador (el proveedor). A través del SRS, el cliente expresa claramente lo que espera, y el desarrollador entiende con claridad qué capacidades debe construir en el software.

Sin un acuerdo de este tipo, casi con seguridad, al finalizar el desarrollo habrá un **cliente insatisfecho**, lo que suele llevar a **desarrolladores insatisfechos**.

(La situación clásica: Cliente: “¡Oye! esto es un error”; Desarrollador: “No, es una característica del software”).

- **Un SRS provee una referencia para la validación del producto final.**

Es decir, ayuda al cliente a determinar si el software cumple con los requerimientos.

Sin un SRS adecuado, el cliente no tiene manera de comprobar si el software entregado es el que se pidió, ni el desarrollador tiene cómo convencer al cliente de que todos los requerimientos han sido satisfechos.

Estas dos razones —**acuerdo y validación**— deberían ser suficientes para justificar la necesidad de un SRS riguroso. Sin embargo, hay razones aún más prácticas y urgentes.

Errores en los requerimientos y su impacto

Sabemos que los principales factores que impulsan un proyecto son: **costo, cronograma y calidad**.

En un estudio (Boehm, citado en [45]):

- En algunos proyectos, el **54% de todos los errores detectados** se encontraron después de la codificación y pruebas unitarias.
- El **45% de esos errores** se originaron en la etapa de requerimientos y en el diseño temprano.
- En otras palabras, **aprox. 25% de los errores ocurren en las etapas iniciales de requerimientos y diseño**.

Ejemplos reportados:

- En el proyecto A-7 se encontraron unos **80 errores** en el documento de requerimientos en pocos meses, lo que generó solicitudes de cambio [8].
- Otro informe indica que se encontraron más de **500 errores** en un SRS previamente aprobado [45].
- En otro proyecto, más de **250 errores** fueron encontrados en un SRS revisado anteriormente, al reescribirlo de forma estructurada y usar herramientas de análisis [47].

Está claro que muchos errores se cometen en la fase de requerimientos.

Y un error en el SRS **casi siempre se manifestará en el sistema final**, porque si el documento

especifica un sistema incorrecto (que no satisface los objetivos del cliente), entonces incluso una implementación “correcta” del SRS dará como resultado un sistema que no sirve al cliente.

De ahí que podamos concluir:

- **Un SRS de alta calidad es un requisito previo para obtener un software de alta calidad.**

Impacto en costos

El costo de corregir un error **crece exponencialmente** a medida que pasa el tiempo.

Un error en los requerimientos, si se detecta después del desarrollo, puede costar **hasta 100 veces más** que corregirlo en la fase de requerimientos.

Ejemplo con datos:

- Corregir un error de requerimientos en la fase temprana: **~2 horas-persona** en promedio.
- Si se detecta más tarde, el costo crece de acuerdo con el avance del proyecto.

Phase	Cost (person-hours)
Requirements	2
Design	5
Coding	15
Acceptance test	50
Operation and maint.	150

Tabla 3.1: Costo de corregir errores de requisitos

Tabla 3.1 resume los costos relativos de corregir errores dependiendo de la fase (según estudios multicorporativos [17, 20]).

Ejemplo simplificado:

- Invertir **100 horas-persona adicionales** en la fase de requerimientos permite detectar unos **50 errores nuevos**.
- Si esos errores no se corrigen ahí, serán encontrados más adelante, con el siguiente patrón (como en el proyecto A-7 [8]):
 - 65% en diseño
 - 2% en codificación
 - 30% en pruebas
 - 3% en operación y mantenimiento

El costo total de corregirlos más tarde sería:

$$32.5 * 5 + 1 * 15 + 15 * 50 + 1.5 * 150 = 1152 \text{ horas-persona}$$

En otras palabras: invertir **100 horas-persona adicionales en requerimientos** reduce el costo total en **~1052 horas-persona**.

Otra manera de verlo:

La probabilidad de detectar un error en fases posteriores es:

- 0.4 en diseño,
- 0.1 en codificación y pruebas unitarias,
- 0.4 en pruebas de aceptación,
- 0.1 en operación/mantenimiento.

El **costo esperado** de corregir un error no eliminado en requerimientos es:

$$0.4 * 5 + 0.1 * 15 + 0.4 * 50 + 0.1 * 150 = 38.5 \text{ horas-persona}$$
$$0.4 * 5 + 0.1 * 15 + 0.4 * 50 + 0.1 * 150 = 38.5 \text{ horas-persona}$$

Entonces, **si detectar y corregir un error en requerimientos cuesta menos que 38.5 horas, conviene hacerlo ahí mismo.**

Requerimientos cambiantes y re-trabajo

Los requerimientos cambian con frecuencia. Aunque algunos cambios son inevitables (porque cambian las necesidades y percepciones), **muchos cambios ocurren porque los requerimientos no fueron bien analizados ni validados.**

Un SRS de alta calidad reduce significativamente esos cambios innecesarios.

Esto es crítico porque:

- los cambios aumentan los costos,
- y pueden descontrolar el cronograma del proyecto.

Ejemplo simplificado:

- Se estima que entre **20% y 40% del esfuerzo total** en un proyecto de software se debe a **re-trabajo**, gran parte causado por cambios en requerimientos.
- Según el modelo **COCOMO** [20], el costo de la fase de requerimientos suele ser de ~6% del costo total del proyecto.

Supongamos un proyecto de **50 meses-persona**:

- Requerimientos consumen **3 meses-persona**.
- Si invertimos un **33% adicional de esfuerzo** en esa fase, y logramos reducir los cambios en requerimientos en 33%, el retrabajo bajará de **10–20 meses-persona** a **6–12 meses-persona**.
- Esto implica un ahorro de **5–11 meses-persona**, es decir, un **10%–22% del costo total del proyecto**.

Conclusión

De todo lo anterior se desprende que:

- **Un SRS de alta calidad reduce los costos de desarrollo.**

Por lo tanto, la calidad del SRS impacta directamente en:

- la **satisfacción del cliente (y del desarrollador)**,
- la **validación del sistema**,
- la **calidad del software final**,
- y el **costo del desarrollo**.

El papel crítico que desempeña el SRS en un proyecto de desarrollo de software debería ser evidente.

3.1.2 Proceso de Requerimientos

El proceso de requerimientos es la secuencia de actividades que deben realizarse en la fase de requerimientos y que culminan en la producción de un documento de alta calidad que contiene la **Especificación de Requerimientos de Software (ERS o SRS por sus siglas en inglés)**.

El proceso de requerimientos típicamente consiste en tres tareas básicas:

- **análisis del problema o requerimientos**,
- **especificación de requerimientos**, y
- **validación de requerimientos**.

El **análisis del problema** suele comenzar con una "declaración del problema" a alto nivel. Durante el análisis, se modela el dominio del problema y el entorno en un esfuerzo por entender el comportamiento del sistema, las restricciones sobre el sistema, sus entradas y salidas, etc. El propósito básico de esta actividad es obtener una comprensión profunda de lo que el software debe proveer.

La comprensión obtenida mediante el análisis del problema forma la base para la **especificación de requerimientos**, en la cual el enfoque está en especificar claramente los requerimientos en un documento. En esta actividad se abordan temas como la representación, los lenguajes de especificación y las herramientas. Dado que el análisis produce grandes cantidades de información y conocimiento con posibles redundancias, **organizar y describir los requerimientos de manera adecuada** es un objetivo importante de esta actividad.

La **validación de requerimientos** se centra en asegurar que lo especificado en la ERS son efectivamente todos los requerimientos del software y en garantizar que la ERS sea de buena calidad. El proceso de requerimientos termina con la producción de la **ERS validada**.

Aunque parece que el proceso de requerimientos es una secuencia lineal de estas tres actividades, en realidad no es así, excepto para sistemas triviales. En la mayoría de sistemas reales, hay una superposición considerable y retroalimentación entre estas actividades. Así, algunas partes del sistema se analizan y luego se especifican mientras el análisis de otras partes sigue en curso.

Además, si las actividades de validación revelan problemas en la ERS, es probable que conduzcan a más análisis y especificación. Sin embargo, en general, para una parte del sistema, **el análisis precede a la especificación y la especificación precede a la validación**.

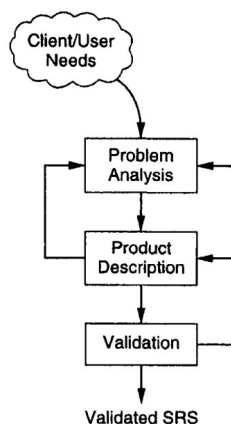


Figura 3.1: El proceso de requerimientos

Este proceso de requerimientos se muestra en la **Figura 3.1**.

Como se muestra en la figura, desde la actividad de especificación podemos regresar a la actividad de análisis. Esto sucede con frecuencia, ya que algunas partes del problema se analizan y especifican antes de que otras lo sean. Asimismo, el proceso de especificación suele mostrar deficiencias en el conocimiento del problema, lo que requiere más análisis. Una vez que la especificación está "completa", pasa a la validación. Esta actividad puede revelar problemas en las especificaciones mismas (lo que requiere regresar a la especificación) o deficiencias en la comprensión del problema (lo que requiere regresar al análisis).

Durante el **análisis de requerimientos**, el enfoque está en **comprender el sistema y sus requerimientos**. Para un sistema complejo, esta es una tarea difícil, y el método probado en el tiempo de **“dividir y conquistar”** (es decir, descomponer el problema o sistema en partes más pequeñas y luego entender esas partes y sus relaciones) se aplica inevitablemente para manejar la complejidad. Además, para manejar la complejidad y el gran volumen de información disponible durante el análisis, se usan diversas **estructuras de representación** que ayudan a visualizar el sistema como una serie de abstracciones. Ejemplos de estas estructuras son los **diagramas de flujo de datos** y los **diagramas de objetos** (más sobre esto en la próxima sección). Para una parte del sistema, el análisis normalmente precede a la especificación. Una vez finalizado el análisis y construidas las estructuras, esa parte del sistema debe especificarse.

La transición del **análisis** a la **especificación**, sin embargo, aunque parece que debería ser sencilla, no lo es. De hecho, puede ser bastante difícil. La razón es que los objetivos de ambas actividades son distintos. En la **especificación** debemos indicar únicamente lo que el software debe hacer, es decir, enfocarnos en el **comportamiento externo** del sistema. Para identificar todos los comportamientos externos, es necesario comprender claramente la estructura del problema y sus componentes, además de sus entradas y salidas. Sin embargo, la estructura en sí puede no ser muy útil en la especificación, ya que el enfoque está exclusivamente en el comportamiento externo o en el sistema final, no en la estructura interna del dominio del problema. Por esto, no se debe esperar que, una vez realizado el análisis, la especificación sea directa.

Además, muchos de los “resultados” del análisis no se utilizan directamente en la ERS. Esto no significa que no sean útiles; son esenciales para modelar el problema, lo cual conduce a una

comprensión adecuada de los requerimientos, que es un **prerrequisito** para la especificación. Por tanto, el uso de las actividades de análisis y las estructuras que se construyen pueden ser **indirectos**, ayudando a la comprensión más que directamente a la especificación.

Vale la pena señalar que existen similitudes entre la **actividad de análisis** y la **actividad de diseño**. Como señaló Davis [45], el problema básico durante el diseño de software es el mismo: **manejar la complejidad**. El enfoque usado es similar: descomposición y construcción de estructuras para representar el sistema como una serie de abstracciones. Debido a esta similitud, los enfoques usados para el análisis del problema y el diseño son frecuentemente similares (por ejemplo, los diagramas de flujo de datos y diagramas de objetos se utilizan en análisis y también en diseño). Sin embargo, aunque los enfoques son similares, los objetivos de ambas actividades son completamente diferentes:

- el **análisis** trata con el dominio del problema, con el objetivo de entenderlo,
- el **diseño** trata con el dominio de la solución, con el objetivo de **optimizar el diseño** [45].

Debido a esto, la aplicación de enfoques similares produce estructuras diferentes durante el análisis y durante el diseño. A veces se cree erróneamente que las estructuras producidas en el análisis se deben arrastrar hasta el diseño. Esto proviene de una **mala comprensión** de los objetivos de ambas actividades. Aunque algunas estructuras eventualmente se usen en el diseño, esto debe hacerse solo si son **consistentes con el objetivo del diseño**.

Finalmente, está el tema del **nivel de detalle** que debe descubrirse y especificarse en el proceso de requerimientos. Este es un asunto que no puede resolverse fácilmente y que depende del objetivo de la fase de especificación de requerimientos.

- Si el objetivo es **definir las necesidades generales del sistema**, los requerimientos pueden expresarse en forma abstracta. Estos se usan generalmente para realizar un **análisis de factibilidad** o para **licitaciones competitivas**.
- En el otro extremo están los requerimientos en los que todo el comportamiento y las interfaces externas del software están claramente especificados. Estos son muy detallados y adecuados para el **desarrollo del software**.

Un ejemplo puede ilustrar este punto. Supongamos que un fabricante de automóviles desea tener un sistema de **control de inventario**.

- A nivel abstracto, los requerimientos podrían expresarse en términos del número de piezas a rastrear, el nivel de concurrencia que debe soportar, si será en línea o por lotes, qué tipo de información y reportes debe proporcionar (por ejemplo: estado de cada ítem bajo demanda, órdenes de compra de artículos con inventario bajo, patrones de consumo), etc. Estos requerimientos abstractos pueden usarse para **estimar costos y realizar un análisis costo-beneficio**, así como para invitar a distintos desarrolladores a presentar propuestas.
- Sin embargo, tal especificación de requerimientos es de poca utilidad para un desarrollador contratado para construir el software. Ese desarrollador necesita saber el **formato exacto de los reportes**, todas las **consultas posibles y su estructura**, el **número total de terminales** que el sistema debe soportar, la **estructura de las bases de datos principales**, etc.

Todo esto sigue describiendo el **comportamiento externo del software**, pero desde un nivel más bajo de abstracción que define cómo se satisfacen los requerimientos generales.

Como debería quedar claro, el nivel abstracto **no es adecuado para el desarrollo de software**. Por eso, nos enfocaremos principalmente en los requerimientos de **nivel detallado**, en los cuales se especifica toda la información sobre el comportamiento externo que el desarrollador necesita para construir el sistema. Es decir, consideramos la ERS como un documento que **proporciona toda la información necesaria para que el desarrollador construya el sistema correctamente**.

Sin embargo, es importante señalar que incluso cuando el objetivo es obtener los requerimientos detallados, los **requerimientos abstractos** pueden desempeñar un papel útil en sistemas complejos. Dado que el análisis del problema comienza con una descripción inicial del comportamiento o necesidades del sistema, los requerimientos abstractos pueden desempeñar este papel (además de servir para licitaciones competitivas o análisis de factibilidad).

En otras palabras, **especificar los requerimientos a nivel abstracto puede ser útil para producir la ERS** que contiene los requerimientos detallados del sistema.

Las siguientes secciones estarán dedicadas a proporcionar una descripción más detallada de las actividades en las tres principales de la fase de requerimientos:

- análisis,
- especificación, y
- verificación.

3.2 Análisis del Problema

El objetivo básico del análisis del problema es obtener una comprensión clara de las necesidades de los clientes y los usuarios, qué es exactamente lo que se desea del software, y cuáles son las restricciones de la solución [45].

Con frecuencia, el cliente y los usuarios no entienden o no conocen todas sus necesidades, porque el potencial del nuevo sistema no suele apreciarse completamente. Los analistas deben asegurarse de descubrir las verdaderas necesidades de los clientes y usuarios, incluso si ellos mismos no las conocen con claridad. Es decir, los analistas no solo recopilan y organizan información sobre la organización del cliente y sus procesos, sino que también actúan como consultores que desempeñan un papel activo ayudando a los clientes y usuarios a identificar sus necesidades.

El principio básico utilizado en el análisis es el mismo que en cualquier tarea compleja: **divide y vencerás**. Es decir, dividir el problema en subproblemas e intentar comprender cada subproblema y su relación con los demás, en un esfuerzo por entender el problema total.

Los conceptos de *estado* y *proyección* también pueden usarse eficazmente en el proceso de partición. Un estado de un sistema representa ciertas condiciones sobre el mismo. Frecuentemente, al usar estados, se ve primero al sistema como si operara en uno de los varios estados posibles, y luego se realiza un análisis detallado de cada estado. Este enfoque se utiliza a veces en software en tiempo real o en software de control de procesos.

En la *proyección*, un sistema se define desde múltiples puntos de vista [152]. Al usar proyección, se definen diferentes perspectivas del sistema y luego se analiza el sistema desde esas diferentes perspectivas. Las distintas “proyecciones” obtenidas se combinan para formar el análisis completo

del sistema. Analizar el sistema desde distintas perspectivas suele ser más sencillo, ya que limita y enfoca el alcance del estudio.

En el resto de esta sección discutiremos algunos métodos para el análisis del problema. Como el objetivo del análisis es comprender el dominio del problema, un analista debe estar familiarizado con diferentes métodos de análisis y elegir el enfoque que considere más adecuado para el problema en cuestión.

3.2.1 Enfoque Informal

El enfoque informal de análisis es aquel en el que no se utiliza ninguna metodología definida. Como en cualquier enfoque, la información sobre el sistema se obtiene mediante interacción con el cliente, usuarios finales, cuestionarios, estudio de documentos existentes, sesiones de *brainstorming*, etc. Sin embargo, con este enfoque no se construye ningún modelo formal del sistema. El problema y el modelo del sistema se construyen esencialmente en la mente de los analistas (o los analistas pueden usar alguna notación informal para este fin) y se traducen directamente de la mente de los analistas al SRS (*Software Requirements Specification* o Especificación de Requerimientos de Software).

Con frecuencia, en este tipo de enfoque, el analista tendrá una serie de reuniones con los clientes y usuarios finales. En las primeras reuniones, los clientes y usuarios finales le explicarán al analista acerca de su trabajo, su entorno y sus necesidades según las perciben. Se pueden entregar documentos que describan el trabajo o la organización, junto con salidas de los métodos actuales de realización de tareas. En estas reuniones iniciales, el analista es básicamente un oyente, absorbiendo la información proporcionada.

Una vez que el analista entiende el sistema hasta cierto punto, usa las siguientes reuniones para aclarar las partes que no comprende. Puede documentar la información de alguna manera (incluso puede construir un modelo si lo desea) y puede realizar sesiones de *brainstorming* o reflexionar sobre lo que el sistema debería hacer. En las últimas reuniones, el analista básicamente explica al cliente lo que entiende que el sistema debe hacer y utiliza estas reuniones como un medio de verificación para comprobar si lo que propone que el sistema haga es consistente con los objetivos del cliente. En estas reuniones finales puede usarse un borrador inicial del SRS.

El enfoque informal de análisis se usa ampliamente y puede ser bastante útil. La razón de su utilidad es que los enfoques basados en modelado conceptual frecuentemente no modelan todos los aspectos del problema y no siempre son adecuados para todos los problemas. Además, dado que el SRS debe ser validado y la retroalimentación de la actividad de validación puede requerir más análisis o especificación (ver Figura 3.1), elegir un enfoque informal de análisis no es muy riesgoso: los errores que puedan introducirse no necesariamente pasarán desapercibidos en la fase de requerimientos. Por lo tanto, este tipo de enfoques pueden ser la alternativa más práctica para el análisis en algunas situaciones.

3.2.2 Modelado de Flujo de Datos

El modelado basado en flujo de datos, a menudo referido como la técnica de análisis estructurado, utiliza descomposición basada en funciones mientras modela el problema. Se enfoca en las funciones realizadas en el dominio del problema y en los datos consumidos y producidos por estas funciones. Es un enfoque de refinamiento de arriba hacia abajo, que originalmente se llamó análisis

y especificación estructurada, y fue propuesto para producir las especificaciones. Sin embargo, limitaremos nuestra atención al aspecto de análisis del enfoque. Antes de describir el enfoque, describamos el diagrama de flujo de datos y el diccionario de datos en los que la técnica se apoya fuertemente.

Diagramas de Flujo de Datos y Diccionario de Datos

Los **diagramas de flujo de datos** (también llamados *data flow graphs*) se utilizan comúnmente durante el análisis de problemas. Estos diagramas (DFD) son bastante generales y no están limitados únicamente al análisis de problemas para la especificación de requerimientos de software. De hecho, ya se utilizaban mucho antes de que la ingeniería de software existiera como disciplina. Los DFD resultan muy útiles para comprender un sistema y pueden emplearse eficazmente durante la etapa de análisis.

Un DFD muestra el **flujo de datos a través de un sistema**. Considera al sistema como una función que transforma las entradas en salidas deseadas. Un sistema complejo no realiza esta transformación en un “único paso”; normalmente, los datos pasan por una serie de transformaciones antes de convertirse en la salida final. El objetivo de un DFD es capturar esas transformaciones que ocurren dentro del sistema sobre los datos de entrada, hasta que finalmente se obtiene la salida.

El agente que realiza la transformación de los datos de un estado a otro se denomina **proceso** (o *burbuja*). Por lo tanto, un DFD muestra cómo se mueven los datos a través de las diferentes transformaciones o procesos del sistema. Los procesos se representan mediante **círculos con nombre**, mientras que los **flujos de datos** se representan con **flechas con nombre** que entran o salen de esas burbujas. Un **rectángulo** representa una fuente o un destino, que son los puntos donde los datos se originan o se consumen. Usualmente, una fuente o destino se encuentra fuera del sistema principal que se está estudiando.

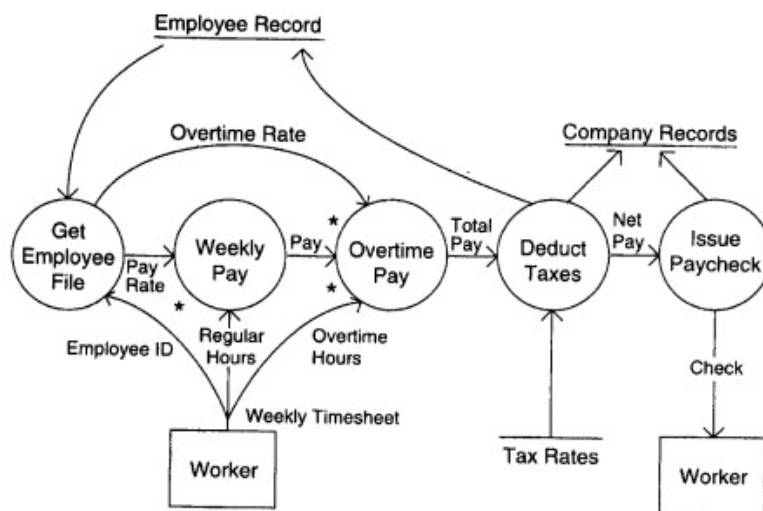


Figura 3.2: DFD de un sistema de pago de empleados

Un ejemplo de DFD para un sistema que paga a los empleados se muestra en la Figura 3.2.

En este DFD hay un flujo de entrada básico: la **planilla semanal de horas**, que proviene de la fuente “trabajador”. La salida principal es el **cheque de pago**, cuyo destino también es el trabajador.

En este sistema, primero se recupera el registro del empleado utilizando el número de identificación que aparece en la planilla. A partir de ese registro se obtienen la tarifa de pago y las horas extras. Con esa información, junto a las horas normales y extras de la planilla, se calcula el sueldo. Una vez determinado el pago total, se deducen los impuestos. Para calcular la deducción de impuestos se consulta el archivo de tasas impositivas. El monto descontado se registra tanto en los registros del empleado como en los de la compañía. Finalmente, se emite el cheque con el pago neto, y el monto abonado también queda registrado en los archivos de la empresa.

Algunas convenciones utilizadas en el dibujo de este DFD merecen explicación.

- Todos los **archivos externos** (como el registro de empleados, registro de la compañía y tasas de impuestos) se muestran como una **línea recta con etiqueta**.
- Cuando un proceso requiere **múltiples flujos de datos**, esto se indica con un ‘*’ entre los flujos. Este símbolo representa la relación **AND**. Por ejemplo, si un proceso tiene como entrada A * B, significa que necesita A **Y** B. En el DFD, para el proceso “pago semanal”, tanto las “horas” como la “tarifa” son necesarias, lo cual se muestra con este símbolo.
- De manera similar, la relación **OR** se representa con un ‘+’ entre los flujos de datos.

Este DFD es una **descripción abstracta** del sistema de pagos. No importa si el sistema es automático o manual. De hecho, este diagrama podría representar perfectamente un sistema manual donde los cálculos se hacen con calculadoras y los registros son carpetas físicas o libros contables. Los detalles y caminos de datos menores no aparecen en este nivel. Por ejemplo, no se muestra qué ocurre si hay errores en la planilla semanal. Esto se hace a propósito, para no atascarse con detalles al construir un DFD del sistema general. Si se desean más detalles, el DFD puede refinarse en niveles posteriores.

Es importante aclarar que un **DFD no es un diagrama de flujo (flowchart)**. El DFD representa **flujos de datos**, mientras que un flowchart representa **flujos de control**. Un DFD **no incluye información procedimental**. Por eso, al dibujar un DFD, no se debe pensar en detalles de procedimiento, y es necesario evitar conscientemente ese tipo de razonamiento. Consideraciones como bucles o decisiones deben ignorarse. Lo único que debe especificar el diseñador en el DFD son las **transformaciones principales** que los datos atraviesan desde la entrada hasta la salida. Cómo se realizan esas transformaciones no es relevante en este nivel.

No existen procedimientos estrictos para dibujar un DFD de un problema dado, solo **algunas pautas generales**.

Una manera de construir un DFD es comenzar identificando las **entradas y salidas principales**, ignorando inicialmente las menores (como mensajes de error). Después, partiendo desde las entradas, se trabaja hacia las salidas, identificando las principales transformaciones intermedias. Otra alternativa es trabajar en sentido inverso: desde las salidas hacia las entradas.

(Recuerda que la información procedimental, como bucles y decisiones, no debe aparecer en el DFD, y el diseñador no debe preocuparse por esos aspectos al dibujar el diagrama de flujo de datos).

A continuación, se presentan algunas sugerencias para la construcción de un DFD [154, 48]:

Trabaja de manera consistente desde las entradas hacia las salidas, o viceversa.

Si te atascas, cambia la dirección. Empieza con un **DFD de alto nivel**, con pocas transformaciones

principales que describan toda la conversión de entradas en salidas, y luego refina cada transformación en transformaciones más detalladas.

- **Nunca intentes mostrar lógica de control.**

Si te descubres pensando en términos de bucles o decisiones, es momento de detenerte y comenzar de nuevo.

- **Etiqueta cada flecha con los elementos de datos adecuados.**

Las entradas y salidas de cada transformación deben estar cuidadosamente identificadas.

- **Haz uso de las operaciones * y +, y muestra suficiente detalle en el diagrama de flujo de datos.**

- **Intenta dibujar DFDs alternativos antes de decidirte por uno.**

Muchos sistemas son demasiado grandes como para que un solo DFD pueda describir claramente el procesamiento de datos. En tales casos, es necesario utilizar algún mecanismo de **descomposición y abstracción**.

Los DFDs pueden organizarse jerárquicamente, lo que ayuda a ir particionando y analizando progresivamente sistemas grandes. A este conjunto de DFDs se le llama un **conjunto de DFDs nivelados** (*leveled DFD set* [48]).

Un **conjunto de DFDs nivelados** comienza con un DFD inicial, que es una representación muy abstracta del sistema, mostrando las entradas y salidas principales y los procesos más relevantes. Luego, cada proceso se refina, dibujando un nuevo DFD para ese proceso. En otras palabras, una **burbuja en un DFD se expande en otro DFD** durante el refinamiento.

Para que la jerarquía sea consistente, es importante que las **entradas y salidas netas** de un DFD de proceso coincidan con las entradas y salidas del proceso en el DFD de nivel superior. El refinamiento se detiene cuando cada burbuja se considera “atómica”, es decir, cuando puede especificarse o comprenderse fácilmente.

Es importante señalar que durante el refinamiento, aunque las entradas y salidas netas se preservan, también puede ocurrir una **descomposición de los datos**. Por ejemplo, una unidad de datos puede dividirse en sus componentes para el procesamiento cuando se dibuja el DFD detallado de un proceso. Así, a medida que se descomponen los procesos, también se descomponen los datos.

En un DFD, los **flujos de datos** se identifican con **nombres únicos**. Estos nombres deben transmitir algún significado sobre lo que representan. Sin embargo, la **estructura precisa** de los flujos de datos no se especifica en un DFD.

El **diccionario de datos** es un repositorio que contiene la definición de los diferentes flujos de datos presentes en un DFD. En él se establece con precisión la estructura de cada flujo. También se especifican los componentes de un flujo, así como la estructura de los archivos mostrados en el DFD.

Para definir la estructura de los datos se usan diferentes **notaciones**, similares a las de las expresiones regulares (que se discutirán más adelante en este capítulo). En esencia, además de la **secuencia o composición** (representada con ‘+’), se incluyen **selección e iteración**:

- La **selección** (representada con la barra vertical ‘|’) significa *uno u otro*.
- La **repetición** (representada con ‘*’) significa *una o más ocurrencias*.

En el DFD mostrado anteriormente, se utilizan los flujos de datos de la **planilla semanal**. El diccionario de datos correspondiente a este DFD se muestra en la Figura 3.3.

La mayoría de los flujos de datos del DFD están especificados allí. Algunos de los más obvios no se muestran. Por ejemplo, la entrada del diccionario para la **planilla semanal** indica que este flujo de datos está compuesto por tres entidades básicas:

1. nombre del empleado,
2. ID del empleado,
3. y múltiples ocurrencias de un par de datos que incluyen horas normales y horas extras.

La última entidad representa las horas de trabajo diarias del empleado. El diccionario de datos también contiene entradas que especifican los diferentes elementos de cada flujo de datos.

Verificación de un DFD y su diccionario de datos

Una vez que se ha construido un DFD y su correspondiente diccionario de datos, es necesario **verificar su corrección**.

No puede haber una verificación formal de un DFD, porque lo que se modela en él no está definido formalmente en ningún otro lugar contra lo cual pueda compararse. Por lo tanto, deben usarse **procesos humanos y reglas prácticas** para validarlo.

Además de la revisión con el cliente (*walkthrough*), el analista debe buscar errores comunes. Algunos de los más frecuentes son:

- Flujos de datos **sin etiqueta**.
- Flujos de datos **faltantes**: información que un proceso necesita pero que no está disponible.
- Flujos de datos **innecesarios**: información que no se usa en el proceso.
- Falta de **consistencia** durante el refinamiento.
- **Procesos faltantes**.
- Inclusión de información de control dentro de los flujos de datos.

Quizás el error más común es el **flujo de datos sin etiqueta**. Si un analista no puede nombrar un flujo de datos, es probable que no comprenda su propósito ni su estructura. Una buena prueba para este tipo de error es revisar que todas las entradas del diccionario de datos sean precisas para cada flujo de datos.

Para comprobar si faltan flujos de datos, el analista debe preguntar, para cada proceso:

“¿Puede este proceso generar las salidas mostradas a partir de las entradas dadas?”

De manera similar, para verificar flujos de datos redundantes, se debe preguntar:

“¿Todos los flujos de datos de entrada son realmente necesarios para calcular las salidas?”

En un conjunto de DFDs nivelados es esencial mantener la **consistencia**. Esta puede perderse fácilmente si se agregan flujos nuevos durante una modificación. En tal caso, se deben realizar los cambios correspondientes en el **DFD padre** o en el **DFD hijo**.

Es decir:

- Si se añade un flujo nuevo en un DFD de nivel bajo, este debe reflejarse también en los DFDs de niveles superiores.
- Si se añade un flujo nuevo en un DFD de nivel alto, los DFDs de los procesos afectados también deben modificarse.

Los DFDs deben ser cuidadosamente revisados para asegurar que **todos los procesos del entorno físico** estén representados. Ningún flujo de datos debe en realidad transportar **información de control**. Un flujo sin estructura o composición es un **candidato potencial** a contener información de control.

El Método de Análisis Estructurado

Ahora volvamos al método de análisis estructurado. La visión básica de este enfoque es que cada sistema puede verse como una función de transformación que opera dentro de un entorno, tomando algunas entradas de ese entorno y produciendo algunas salidas para él. Y como la función global de transformación de todo el sistema puede ser demasiado compleja para comprenderla como una sola función, ésta debe dividirse en **subfunciones** que, en conjunto, formen la función global.

Las subfunciones pueden dividirse aún más y el proceso repetirse hasta llegar a un punto en el que cada función pueda comprenderse con facilidad. El enfoque básico utilizado para descubrir las funciones que se realizan en el sistema (o las que forman parte de la función global del sistema) es rastrear los datos a medida que fluyen a través de éste: **desde la entrada hasta la salida**.

Se cree que, en cualquier sistema complejo, la transformación de los datos desde la entrada hasta la salida no ocurre en un solo paso; más bien, los datos se transforman en una serie de pasos, comenzando desde la entrada y culminando en la salida deseada. Al comprender los “**estados**” por los que pasan los datos en esa serie de transformaciones, pueden identificarse las funciones del sistema; cada transformación de los datos en esa serie es realizada por una función de transformación.

Por lo tanto, al rastrear los datos a través del sistema, pueden identificarse las diversas funciones que éste ejecuta. Como este enfoque puede modelarse fácilmente mediante **diagramas de flujo de datos (DFD)**, éstos se utilizan ampliamente en este método.

Etapas del método

1. Estudiar el entorno físico:

- En esta etapa se dibuja un **DFD del sistema actual no automatizado** (o parcialmente automatizado), mostrando los flujos de datos de entrada y salida, cómo circulan por el sistema y qué procesos actúan sobre ellos.
- Este DFD puede contener nombres específicos de flujos de datos y procesos usados en el entorno físico, como nombres de departamentos, personas, procedimientos locales o archivos organizacionales.
- Para elaborarlo, el analista interactúa con los usuarios para determinar el proceso general desde el punto de vista de los datos.

- Esta etapa se considera terminada cuando el usuario acepta que el DFD refleja fielmente el funcionamiento del sistema actual.
- Puede comenzar con un **diagrama de contexto**, donde el sistema entero se trata como un solo proceso y se identifican todas sus entradas, salidas, fuentes y destinos.

2. Obtener el DFD lógico del sistema actual:

- El objetivo es llegar a un DFD donde cada flujo de datos y cada proceso sean entidades **lógicas**, no físicas.
- El DFD físico sirve solo como punto de partida.
- En esta fase se reemplazan los nombres físicos por equivalentes lógicos.
 - Ejemplo: “archivo 12.3.2” → “archivo de salarios de empleados”.
 - Ejemplo: un proceso llamado “A la oficina de Juan” → “emitir cheques”.
- Además, se eliminan las burbujas que no transforman los datos.
- La fase concluye cuando el DFD lógico ha sido verificado por el usuario.

3. Diseñar el DFD lógico del nuevo sistema:

- Una vez comprendido el sistema actual, se pasa a modelar el **nuevo sistema** con los cambios incorporados.
- El analista trabaja en modo lógico, especificando **qué debe hacerse, no cómo se hará**.
- No se distingue aún entre procesos automatizados y manuales.
- Este paso requiere determinar cuáles partes del DFD actual cambiarán (definir los **límites de cambio**).
- El nuevo DFD reemplazará solo esa parte del existente, manteniendo las mismas entradas y salidas en los límites.

4. Definir la frontera hombre-máquina:

- Aquí se especifica qué procesos se automatizarán y cuáles permanecerán manuales.
- Incluso los procesos manuales pueden diferir de los originales en el nuevo sistema.
- Puede haber varias opciones según el grado de automatización posible.
- El analista debe explorar y presentar estas alternativas.

5. Evaluar opciones y presentar especificaciones:

- Una vez definidos los posibles escenarios de automatización, deben evaluarse y documentarse para presentar la mejor propuesta.

Recomendaciones de construcción

- El análisis estructurado sugiere un **enfoque de arriba hacia abajo (top-down)** para dibujar los DFD.
- Se comienza con un **diagrama de contexto**, que lista las entradas y salidas principales del sistema.
- Luego este diagrama se descompone en partes más detalladas, formando un **conjunto jerárquico de DFDs**.
- Durante la refinación debe mantenerse la **consistencia**, es decir, preservar entradas y salidas netas.

Fortalezas y limitaciones

- El análisis estructurado ofrece métodos claros para **organizar y representar información** sobre sistemas.
- También proporciona **guías para verificar la precisión** de la información obtenida.
- Es especialmente útil en las **primeras dos etapas**, cuando se construye el DFD del sistema actual.
- Sin embargo, para la tercera etapa (analizar el sistema objetivo y construir su DFD o diccionario de datos), la técnica no ofrece tanta ayuda.
- Aun así, el conocimiento adquirido del sistema actual sirve de base para diseñar el nuevo sistema.

3.2.3 Modelado Orientado a Objetos

En el modelado orientado a objetos, un sistema se ve como un conjunto de **objetos**. Los objetos interactúan entre sí a través de los servicios que proveen. Algunos objetos también interactúan con los usuarios a través de sus servicios de tal manera que los usuarios obtienen los servicios deseados. Por lo tanto, el objetivo del modelado es identificar los objetos (en realidad, las clases de objetos) que existen en el dominio del problema, definir las clases especificando qué información de estado encapsulan y qué servicios proveen, e identificar las relaciones que existen entre objetos de diferentes clases, de tal manera que el modelo en su conjunto apoye los servicios que el usuario desea.

El modelado orientado a objetos y los sistemas han recibido mucha atención en el pasado reciente. La razón básica para esto es la creencia de que los sistemas orientados a objetos van a ser más fáciles de construir y mantener. También se cree que la transición desde el análisis orientado a objetos al diseño (e implementación) orientado a objetos será fácil, y que el análisis orientado a objetos es más inmune al cambio porque los objetos son más estables que las funciones. Es decir, en un dominio de problema, es probable que los objetos permanezcan iguales incluso si la naturaleza exacta del problema cambia, mientras que esto no ocurre con el modelado orientado a funciones.

Algunos enfoques para el modelado y diseño orientado a objetos fueron propuestos tempranamente. Los objetivos de muchas de estas técnicas, respecto a qué producir, son bastante similares, y sus enfoques y notaciones también lo son. Aquí, describimos brevemente el enfoque propuesto en [36];

la notación que usamos es **UML** ([24, 64] o www.uml.org), que ahora es la notación estándar de facto para el modelado OO. Discutiremos UML más a fondo en el Capítulo 7 cuando hablemos de diseño OO; aquí discutimos algunos conceptos necesarios para hablar del análisis.

Conceptos Básicos y Notación

Al comprender o modelar un sistema utilizando una técnica de modelado orientado a objetos, el sistema se ve como compuesto de objetos. Cada objeto tiene ciertos **atributos**, que en conjunto definen al objeto. La separación de un objeto de sus atributos es un método natural que usamos para entender sistemas (un hombre está separado de sus atributos como altura, peso, etc.). En sistemas orientados a objetos, los atributos mantienen el estado (o definen el estado) de un objeto. Un atributo es un valor puro de datos (como entero, cadena, etc.), no un objeto.

Objetos de tipo similar se agrupan para formar una **clase de objetos** (o simplemente clase). Una clase es esencialmente una definición de tipo, que define el espacio de estado de los objetos de ese tipo y las operaciones (y su semántica) que pueden aplicarse a los objetos de ese tipo. La formación de clases también es una técnica general utilizada por los humanos para entender sistemas y diferenciar entre clases (por ejemplo, un manzano es una instancia de la clase de árboles, y la clase de árboles es diferente de la clase de aves).

Un objeto también provee algunos **servicios u operaciones**. Estos servicios son el único medio por el cual el estado del objeto puede ser modificado o visto desde el exterior. Para operar un servicio, se envía un mensaje al objeto para ese servicio. En general, estos servicios se definen para una clase y son provistos para cada objeto de esa clase. Encapsular servicios y atributos juntos en un objeto es una de las principales características que distingue un enfoque de modelado orientado a objetos de los enfoques de modelado de datos, como los diagramas ER.

Los **diagramas de clases** representan gráficamente la estructura del problema utilizando una notación precisa. En un diagrama de clases, una clase se representa como un rectángulo en formato vertical dividido en tres partes. La parte superior contiene el nombre de la clase. La parte del medio lista los atributos que poseen los objetos de esta clase. Y la tercera parte lista los servicios provistos por los objetos de esta clase.

Para modelar las **relaciones entre clases**, se usan algunas estructuras.

- La estructura de **generalización-especialización** puede ser usada por una clase para heredar todos o algunos atributos y servicios de una clase general y agregar más atributos y servicios. Esta estructura se modela en el modelado orientado a objetos mediante **herencia**. Usando una clase general e incorporando algunos de sus atributos y servicios y agregando más, se puede crear una clase que es una versión especializada de la clase general. Y muchas clases especializadas pueden crearse a partir de una clase general, dándonos jerarquías de clases.
- La estructura de **agregación** modela la relación todo-parte. Un objeto puede estar compuesto de muchos objetos; esto se modela mediante la estructura de agregación.

La representación de estas relaciones en un diagrama de clases se muestra en la Figura 3.8.

Además de estas, las **instancias** de una clase pueden estar relacionadas con objetos de alguna otra clase. Por ejemplo, un objeto de la clase **Employer** puede estar relacionado con muchos objetos de la clase **Employee**. Esta relación entre objetos también debe ser capturada si se quiere modelar

adecuadamente un sistema. Esto se captura a través de **asociaciones**. Una asociación se muestra en el diagrama de clases mediante una línea entre las dos clases. La **multiplicidad** de una asociación especifica cuántas instancias de una clase pueden relacionarse con instancias de la otra clase a través de esa asociación. Una asociación entre dos clases puede ser de **uno a uno** (es decir, una instancia de una clase está relacionada exactamente con una instancia de la otra clase), de **uno a muchos**, o de algunos otros casos especiales. La multiplicidad se especifica colocando una estrella (*) en la línea adyacente a la clase, representando que cero o más instancias de la clase pueden estar relacionadas con una instancia de la otra clase.

3.2.4 Prototipado

El **prototipado** adopta un enfoque diferente para el análisis de problemas en comparación con los enfoques basados en modelos. En el prototipado, se construye un **sistema parcial**, que luego es utilizado por el cliente, los usuarios y los desarrolladores para obtener una mejor comprensión del problema y de las necesidades. Por lo tanto, la experiencia real con un prototipo que implementa parte del sistema de software eventual se usa para analizar el problema y entender los requisitos del sistema final.

Un **prototipo de software** se puede definir como una implementación parcial de un sistema cuyo propósito es **aprender algo sobre el problema que se está resolviendo o sobre el enfoque de solución** [46]. Como indica esta definición, el prototipado también puede usarse para evaluar o verificar una alternativa de diseño (a este tipo se le llama **prototipo de diseño** [46]). Aquí nos enfocamos en el prototipado utilizado principalmente para comprender los requisitos.

La **justificación** de usar prototipado para el entendimiento y análisis de problemas es que los clientes y usuarios, a menudo, encuentran difícil visualizar cómo funcionará el sistema de software final en su entorno solo leyendo un documento de especificación. Imaginar la operación del software que aún no se ha construido, y si cumplirá con los objetivos finales, únicamente leyendo y discutiendo requisitos en papel, es muy difícil. Esto es particularmente cierto si el sistema es completamente nuevo y muchos usuarios o clientes no tienen una idea clara de sus necesidades.

La idea del prototipado es que **clientes y usuarios pueden evaluar mucho mejor sus necesidades si pueden ver un sistema en funcionamiento**, aunque este sea solo parcial. El prototipado enfatiza que la **experiencia práctica real es la mejor ayuda para comprender las necesidades**. Al experimentar con un sistema, la gente puede decir:

- “No quiero esta característica”,
- “Me gustaría que tuviera esta otra”, o
- “Esto es excelente”.

Existen dos enfoques de prototipado: **desechable y evolutivo** [44, 46]. En el **enfoque desechable**, el prototipo se construye con la idea de que será descartado después de completar el análisis, y el sistema final se construirá desde cero. En el **enfoque evolutivo**, el prototipo se construye con la idea de que eventualmente se convertirá en el sistema final. Desde el punto de vista del análisis de problemas y su comprensión, los **prototipos desechantes** son los más adecuados. En lo que sigue, nos enfocaremos en los prototipos desechantes.

La primera pregunta que debe responderse es **si conviene o no hacer un prototipo**. En otras palabras, es importante entender claramente cuándo debe hacerse. Los requisitos de un sistema pueden dividirse en tres conjuntos [46]:

- Los bien entendidos,
- Los poco entendidos, y
- Los desconocidos.

En un **prototipo desechable**, los requisitos poco entendidos son los que deben incorporarse. Con base en la experiencia obtenida con el prototipo, estos requisitos poco entendidos se transforman en **bien entendidos**, como se muestra en la Figura 3.12.

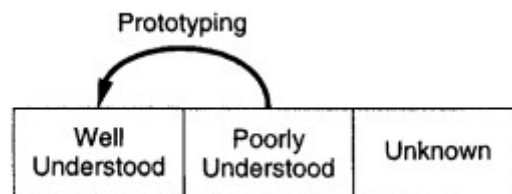


Figura 3.12: Prototipo desechable.

Es posible dividir aún más el conjunto de requisitos poco entendidos en dos subconjuntos:

- Los **críticos para el diseño**, y
- Los **no críticos para el diseño** [46].

Los requisitos que pueden incorporarse fácilmente al sistema más adelante se consideran **no críticos**. Si es posible determinar cuáles requisitos poco entendidos son críticos y cuáles no, entonces el prototipo desechable debe enfocarse principalmente en los **críticos**.

En general, podemos decir que si el conjunto de requisitos poco entendidos es sustancial (en particular, la parte de los críticos), entonces conviene construir un prototipo desechable.

El **proceso de desarrollo** usando un prototipo desechable ya fue descrito en el Capítulo 2. La actividad comienza con un **ERS (Especificación de Requerimientos de Software)** para el prototipo. Sin embargo, desarrollar este ERS requiere identificar qué funciones deben incluirse en el prototipo. Esta decisión depende de la aplicación. En general, como se mencionó antes, los requisitos que tienden a ser **inciertos, vagos o cambiantes** son los que deben implementarse en el prototipo.

Ejemplos comunes de estos son:

- La **interfaz de usuario**,
- Las **nuevas características** a agregar (más allá de automatizar lo que ya se hace), y
- Las características que podrían ser **inviables**.

Dependiendo de qué aspectos del sistema se incluyan en el prototipo, este puede clasificarse como **vertical** o **horizontal** [110].

- En el **prototipado horizontal**, el sistema se ve como una serie de capas y se selecciona una de ellas para el prototipo (ejemplo: la **interfaz de usuario**, donde casi toda la interfaz se incluye en el prototipo).
- En el **prototipado vertical**, se construye por completo una parte específica del sistema que no se entiende bien. Este enfoque se usa para validar alguna **funcionalidad o capacidad** del sistema.

El **desarrollo de un prototipo desechable** es fundamentalmente distinto del desarrollo de software de calidad final para producción. El foco principal durante el prototipado es **mantener bajos los costos y minimizar el tiempo de producción**.

Por esta razón:

- Muchas de las tareas de **documentación, control de calidad y gestión** se reducen al mínimo.
- Las **preocupaciones de eficiencia** pasan a un segundo plano.
- Con frecuencia se usan **lenguajes interpretados de alto nivel** para construir el prototipo.

Debido a esto, la **tentación de convertir el prototipo en el sistema final debe evitarse**.

La experiencia se obtiene al poner el prototipo en manos del cliente y los usuarios reales. Durante esta actividad es necesaria una **interacción constante** para comprender sus reacciones. Se pueden usar **cuestionarios e entrevistas** para recopilar retroalimentación.

El **ERS final** se desarrolla de manera similar a cualquier ERS, pero con la gran ventaja de que los clientes y usuarios podrán **responder mejor a las preguntas y explicar sus necesidades**, gracias a la experiencia con el prototipo.

Para que el prototipado en análisis de requisitos sea viable, su **costo debe mantenerse bajo**. Por lo tanto, solo se incluyen en el prototipo aquellas funciones cuyo **retorno en términos de experiencia del usuario sea valioso**.

En general, no se incluyen características como:

- Manejo de excepciones,
- Recuperación ante fallos,
- Conformidad con estándares y formatos.

Como el prototipo será desechado, la cantidad de documentos producidos se mantiene al mínimo: no se necesitan documentos de diseño, plan de pruebas ni especificaciones de casos de prueba. Otra forma importante de reducir costos es disminuir el **nivel de pruebas**, ya que las pruebas consumen gran parte del presupuesto en el desarrollo normal de software.

Gracias a estas medidas, es posible mantener el costo del prototipo en **menos de unos pocos puntos porcentuales** del costo total del desarrollo.

El costo de construir y ejecutar un prototipo puede estar alrededor del **10% del costo total del desarrollo** [72]. Sin embargo, esto **no significa** que el costo total aumente en esa proporción. La razón principal es que los **beneficios del prototipado** —en forma de reducción de errores de

requisitos y disminución en la cantidad de cambios posteriores— suelen ser muy significativos, reduciendo así el costo global del desarrollo.

3.3 Especificación de Requisitos

El resultado final es el documento de especificación de requisitos de software (SRS).

En problemas pequeños, o en aquellos que pueden comprenderse fácilmente, la actividad de especificación puede realizarse una vez que el análisis está completamente terminado. Sin embargo, lo más común es que el análisis del problema y la especificación se hagan de manera simultánea. Normalmente, un analista estudia una parte del problema y luego escribe los requisitos correspondientes a esa parte. En la práctica, las actividades de análisis y de especificación se superponen, y se avanza de una a otra constantemente, como se muestra en la Figura 3.1. No obstante, dado que toda la información de la especificación proviene del análisis, conceptualmente podemos ver la especificación como una etapa posterior al análisis.

La primera pregunta que surge es: si durante el análisis se realiza un modelado formal, ¿por qué no se consideran los resultados de ese modelado —las estructuras creadas (por ejemplo, DFD y DD, diagramas de objetos)— como la SRS?

La razón principal es que el modelado generalmente se centra en la estructura del problema, no en su comportamiento externo. Por eso, elementos como la **interfaz de usuario** rara vez se modelan, aunque suelen ser una parte fundamental de la SRS. De manera similar, para simplificar el modelado, a menudo no se consideran en detalle “situaciones menores” como los errores en la salida, mientras que en la SRS sí es necesario especificar cómo debe comportarse el sistema en esos casos. Además, aspectos como restricciones de rendimiento, limitaciones de diseño, cumplimiento de estándares, recuperación ante fallos, etc., no se incluyen en los modelos, pero deben estar claramente definidos en la SRS, ya que el diseñador necesita conocerlos para poder diseñar el sistema adecuadamente.

Por lo tanto, queda claro que los modelos no pueden reemplazar una SRS.

Por estas razones, la transición del análisis a la especificación no debe pensarse como un paso directo, incluso si se usaron modelos formales en el análisis. No se trata simplemente de tomar las estructuras del modelado y escribirlas de forma más formal. Una buena SRS debe cubrir muchos aspectos que el modelado no resuelve satisfactoriamente. Además, a veces las estructuras producidas en el análisis no sirven para traducirse directamente en especificaciones de comportamiento externo (que es justamente lo que se define en una SRS). Por ejemplo, un diagrama de objetos hecho durante un análisis orientado a objetos tiene una utilidad limitada a la hora de especificar el comportamiento externo del sistema deseado.

En esencia, lo que pasa del análisis de requisitos a la especificación es el **conocimiento adquirido sobre el sistema**. El modelado es una herramienta para ayudar a obtener un conocimiento profundo y completo del sistema propuesto. La SRS se redacta en base a ese conocimiento. Como transformar ese conocimiento en un documento estructurado no es algo simple, la especificación en sí misma es una tarea importante y relativamente independiente.

Una consecuencia de esto es que es menos crítico tener un modelo “completo” que una especificación completa. El objetivo principal del análisis es comprender el problema, mientras que el objetivo principal de la fase de requisitos es producir la SRS. Por lo tanto, los modelos de análisis exhaustivos y detallados no son esenciales. De hecho, es posible crear una SRS sin usar técnicas de modelado formal. El objetivo central de los modelos es apoyar la representación del conocimiento y la división del problema; los modelos no son un fin en sí mismos.

Con esto en mente, comencemos la discusión sobre la especificación de requisitos. Empezaremos analizando las características deseables de una SRS.

3.3.1 Características de un SRS

Para cumplir correctamente con los objetivos básicos, un **SRS** (Especificación de Requisitos de Software) debería tener ciertas propiedades y contener distintos tipos de requisitos. En esta sección vamos a hablar sobre algunas de las características deseables de un SRS y de los componentes que lo conforman.

Un buen SRS debe ser:

1. **Correcto**
2. **Completo**
3. **No ambiguo**
4. **Verificable**
5. **Consistente**
6. **Clasificado por importancia y/o estabilidad**
7. **Modificable**
8. **Trazable**

Un **SRS es correcto** si cada requisito incluido en él representa algo que realmente se necesita en el sistema final.

Un **SRS es completo** si especifica todo lo que el software debe hacer y también las respuestas del software para todas las clases de datos de entrada.

La **corrección** y la **completitud** van de la mano: la corrección asegura que lo que está especificado esté bien hecho, mientras que la completitud asegura que todo esté especificado.

La corrección es más fácil de comprobar que la completitud, ya que implica revisar cada requisito y asegurarse de que refleja lo que el usuario necesita. La completitud, en cambio, es la más difícil de garantizar, porque implica detectar lo que **falta**, y comprobar una ausencia es mucho más complicado que verificar lo que ya está presente.

Un **SRS es no ambiguo** si, y solo si, cada requisito tiene una sola interpretación posible. Como los requisitos suelen escribirse en lenguaje natural (que es inherentemente ambiguo), si se usan frases en lenguaje común el redactor del SRS debe tener especial cuidado en evitar ambigüedades. Una forma de reducirlas es usar un lenguaje formal de especificación de requisitos. Sin embargo, su

desventaja es que escribir un SRS en un lenguaje formal requiere mucho esfuerzo, tiene un alto costo y además es difícil de leer y entender (especialmente para clientes y usuarios).

Un **SRS es verificable** si, y solo si, cada requisito puede ser comprobado.

Un requisito es verificable si existe algún proceso, razonablemente económico, que pueda comprobar si el software final cumple con ese requisito. Esto implica que los requisitos deben ser lo menos subjetivos posible, porque lo subjetivo es difícil de verificar. La ausencia de ambigüedad es esencial para que algo sea verificable.

Dado que la verificación de requisitos suele hacerse mediante revisiones, también implica que un SRS debe ser comprensible, al menos para el desarrollador, el cliente y los usuarios. La **entendibilidad** es clave, porque uno de los objetivos de la fase de requisitos es producir un documento en el que cliente, usuarios y desarrolladores estén de acuerdo.

Un **SRS es consistente** si no existen requisitos que entren en conflicto entre sí. Las inconsistencias pueden aparecer por la terminología (cuando diferentes requisitos usan distintos términos para referirse a lo mismo) o por conflictos lógicos o temporales.

Esto ocurre si el SRS contiene dos o más requisitos que no pueden cumplirse juntos en ningún sistema de software.

Por ejemplo: si un requisito dice que un evento **e** debe ocurrir antes que un evento **f**, pero otro requisito (directo o indirecto) dice que **f** debe ocurrir antes que **e**, entonces hay una inconsistencia. Estas inconsistencias pueden reflejar problemas serios en la especificación.

En general, no todos los requisitos del software tienen la misma importancia. Algunos son críticos, otros son importantes pero no críticos, y otros simplemente deseables. Del mismo modo, algunos requisitos son “centrales” y poco probables de cambiar con el tiempo, mientras que otros dependen más de las circunstancias.

Un **SRS está clasificado por importancia y/o estabilidad** si para cada requisito se indica qué tan importante es y qué tan probable es que cambie en el futuro. La estabilidad de un requisito se puede expresar en términos del volumen esperado de cambios.

Escribir un SRS es un **proceso iterativo**. Incluso cuando los requisitos de un sistema se especifican, más tarde se modifican a medida que cambian las necesidades del cliente. Por eso, un **SRS debe ser fácil de modificar**.

Un SRS es modificable si su estructura y estilo permiten hacer cambios fácilmente, sin perder completitud ni consistencia. La redundancia es un gran obstáculo para la modificabilidad, ya que puede generar errores.

Por ejemplo: si un requisito está escrito en dos lugares distintos y luego hay que cambiarlo, pero solo se modifica en uno de los dos, el SRS resultante queda inconsistente.

Un **SRS es trazable** si se conoce claramente el origen de cada requisito y si facilita la referencia de cada requisito durante el desarrollo posterior [91].

La **trazabilidad hacia adelante** significa que cada requisito puede vincularse con elementos de diseño y código.

La **trazabilidad hacia atrás** implica que es posible rastrear elementos de diseño y código hacia los requisitos que soportan. La trazabilidad facilita la verificación y la validación.

De todas estas características, la **completitud** es quizás la más importante (y la más difícil de garantizar).

Uno de los problemas más comunes en la especificación de requisitos ocurre cuando algunos

requisitos del cliente no se incluyen. Esto obliga a hacer agregados y modificaciones más adelante en el ciclo de desarrollo, y esos cambios suelen ser costosos.

La incompletitud también es una fuente importante de conflictos entre cliente y proveedor.

Por eso, la importancia de tener requisitos completos no puede enfatizarse lo suficiente.

3.3.2 Componentes de un SRS

La completitud de las especificaciones es difícil de lograr y aún más difícil de verificar.

Tener guías sobre qué cosas debería especificar un SRS ayuda a detallar los requisitos de manera completa.

Aquí describimos algunas de las propiedades del sistema que un SRS debería especificar.

Los aspectos básicos que un SRS debe cubrir son:

- **Funcionalidad**
- **Desempeño (Performance)**
- **Restricciones de diseño impuestas en la implementación**
- **Interfaces externas**

Conceptualmente, cualquier SRS debería tener estos componentes.

Si se sigue el enfoque tradicional de análisis de requisitos, entonces el SRS incluso podría tener secciones que correspondan a cada uno de ellos.

Sin embargo, los requisitos funcionales pueden especificarse de manera indirecta, ya sea indicando los servicios sobre los objetos o describiendo los casos de uso.

Requisitos Funcionales

Los **requisitos funcionales** especifican qué salidas (outputs) deben producirse a partir de las entradas (inputs) dadas.

Describen la relación entre la entrada y la salida del sistema.

Para cada requisito funcional, se debe incluir una descripción detallada de:

- Todos los datos de entrada y su origen.
- Las unidades de medida.
- El rango de valores válidos para las entradas.

También deben especificarse todas las operaciones que se realizan sobre los datos de entrada para obtener la salida.

Esto incluye:

- Las verificaciones de validez en los datos de entrada y salida.
- Los parámetros afectados por la operación.
- Las ecuaciones u otras operaciones lógicas que deben usarse para transformar las entradas en las salidas correspondientes.

Por ejemplo, si existe una fórmula para calcular una salida, debe estar especificada.

Es importante **no especificar algoritmos de implementación** que no formen parte del sistema en sí, aunque puedan ser necesarios para implementarlo.

Las decisiones de este tipo deben quedar a cargo del diseñador.

Un aspecto crucial de la especificación es el **comportamiento del sistema en situaciones anormales**, como:

- Entradas inválidas (que pueden presentarse de muchas formas).
- Errores durante el cómputo.

El requisito funcional debe indicar claramente qué debe hacer el sistema si ocurren estas situaciones.

En particular, debe especificar:

- El comportamiento frente a entradas inválidas.
- El comportamiento frente a salidas inválidas.
- El comportamiento cuando la entrada es válida, pero la operación normal no puede realizarse.

Ejemplo: en un sistema de reservas de vuelos, aunque un pasajero válido intente reservar, la operación no puede completarse si el avión ya está totalmente ocupado.

En resumen:

El comportamiento del sistema debe especificarse para **todas las entradas previstas** y para **todos los estados posibles del sistema**.

Estas condiciones especiales suelen pasarse por alto, lo que termina generando sistemas poco robustos.

Requisitos de Rendimiento

Esta parte de un SRS especifica las restricciones de rendimiento del sistema de software. Todos los requisitos relacionados con las características de rendimiento del sistema deben estar claramente definidos. Existen dos tipos de requisitos de rendimiento: estáticos y dinámicos.

Los **requisitos estáticos** son aquellos que no imponen restricciones sobre las características de ejecución del sistema. Entre ellos se incluyen, por ejemplo, el número de terminales que el sistema debe soportar, el número de usuarios simultáneos que puede manejar y el número de archivos que debe procesar, así como sus tamaños. Estos también se conocen como requisitos de capacidad del sistema.

Por otro lado, los **requisitos dinámicos** especifican restricciones sobre el comportamiento en ejecución del sistema. Normalmente incluyen el tiempo de respuesta y el rendimiento o throughput. El tiempo de respuesta se refiere al tiempo esperado para completar una operación bajo ciertas circunstancias, mientras que el throughput indica el número esperado de operaciones que se pueden realizar en una unidad de tiempo. Por ejemplo, un SRS puede especificar cuántas transacciones deben procesarse por unidad de tiempo o cuál debe ser el tiempo de respuesta de un comando específico. Además, se deben definir los rangos aceptables para estos parámetros de rendimiento, considerando tanto condiciones de carga normales como máximas.

Es fundamental que todos estos requisitos estén expresados en términos medibles. Requisitos vagos como “el tiempo de respuesta debe ser bueno” o “el sistema debe procesar todas las transacciones rápidamente” no son deseables, ya que son imprecisos y no verificables. En cambio, se deben usar declaraciones claras y cuantificables, por ejemplo: “El tiempo de respuesta del comando X debe ser menor a un segundo el 90% de las veces” o “Una transacción debe procesarse en menos de un segundo el 98% de las veces”. Esto asegura que los requisitos de rendimiento sean objetivos y verificables.

3.3.3 Lenguaje de Especificación

La especificación de requisitos requiere el uso de algún **lenguaje de especificación**. Este lenguaje debe soportar las cualidades deseadas del SRS, como modificabilidad, comprensibilidad, ausencia de ambigüedad, entre otras. Además, debe ser fácil de aprender y usar. Como es de esperar, muchas de estas características pueden entrar en conflicto al momento de elegir un lenguaje de especificación. Por ejemplo, para evitar ambigüedad, lo ideal sería usar un lenguaje formal. Pero para facilitar la comprensión, un lenguaje natural podría ser preferible.

Aunque existen notaciones formales para especificar propiedades específicas del sistema, hoy en día los **lenguajes naturales** son los más utilizados para especificar requisitos. Si se usan lenguajes formales, normalmente se emplean para especificar propiedades particulares o partes específicas del sistema, y estas especificaciones formales generalmente se incluyen dentro del SRS general, que está escrito en lenguaje natural. En otras palabras, el SRS completo suele estar en lenguaje natural y, cuando es factible y conveniente, algunas partes pueden usar notaciones formales.

La ventaja principal de usar un lenguaje natural es que tanto el cliente como el proveedor lo entienden. Sin embargo, por la naturaleza misma del lenguaje natural, este es impreciso y puede ser ambiguo. Para reducir estos inconvenientes, generalmente se usa el **lenguaje natural de manera estructurada**. Por ejemplo, en inglés estructurado, los requisitos se dividen en secciones y párrafos, y cada párrafo se subdivide en subpárrafos. Muchas organizaciones también establecen reglas estrictas para el uso de palabras como "shall", "perhaps" o "should", y restringen ciertas frases comunes para mejorar la precisión y reducir la ambigüedad y la verbosidad. Una regla general al usar lenguaje natural es ser preciso, factual y conciso, organizando los requisitos jerárquicamente siempre que sea posible y asignando un número único a cada requisito.

En un SRS, como se discutió, algunas partes pueden especificarse mejor usando notaciones formales. Por ejemplo, para definir los formatos de entradas o salidas, las expresiones regulares son muy útiles. De manera similar, para sistemas como protocolos de comunicación, se pueden usar autómatas finitos. Las **tablas de decisión** son útiles para especificar formalmente el comportamiento del sistema ante diferentes combinaciones de entradas o configuraciones. Además, algunos aspectos del sistema pueden especificarse o explicarse mediante los modelos que se hayan construido durante el análisis del problema. A veces, estos modelos se incluyen como documentos de apoyo para clarificar mejor los requisitos y su motivación.

3.4 Especificación Funcional con Casos de Uso

Los **requisitos funcionales** a menudo constituyen el núcleo de un documento de requisitos.

El enfoque tradicional para especificar la funcionalidad consiste en describir cada función que el sistema debe proporcionar.

Los **casos de uso** especifican la funcionalidad de un sistema describiendo su **comportamiento**, capturado como las **interacciones entre los usuarios y el sistema**. Los casos de uso pueden utilizarse para describir los **procesos de negocio** de una organización que despliega el software, o simplemente **para describir el comportamiento del sistema de software en sí**. En este caso, nos centraremos en describir el comportamiento de los sistemas de software que se van a construir.

Aunque los casos de uso se utilizan principalmente para especificar comportamiento, también pueden ser efectivos durante la fase de análisis. Más adelante, al discutir cómo desarrollar casos de uso, veremos cómo pueden ayudar también a **obtener los requisitos**.

Los casos de uso ganaron relevancia después de ser utilizados como parte del enfoque de **modelado orientado a objetos** propuesto por Jacobson. Debido a esta conexión con el enfoque orientado a objetos, a veces se consideran parte de este enfoque para el desarrollo de software. Sin embargo, los casos de uso son un **método general** para describir la interacción de un sistema, incluso de sistemas no tecnológicos.

La discusión sobre casos de uso aquí se basa en los conceptos y procesos presentados en las referencias mencionadas.

3.5 Validación

El desarrollo de software comienza con un **documento de requisitos**, que también se utiliza posteriormente para determinar si el software entregado es aceptable o no. Por lo tanto, es importante que la especificación de requisitos no contenga errores y refleje correctamente las necesidades del cliente. Además, como hemos visto, cuanto más tiempo permanezca un error sin ser detectado, mayor será el costo de corregirlo. Por esta razón, es muy deseable detectar los errores en los requisitos **antes** de que comience el diseño y desarrollo del software.

Debido a la naturaleza de la fase de especificación de requisitos, existe mucho margen para malentendidos y errores, y es bastante posible que la especificación de requisitos no represente con exactitud las necesidades del cliente. El objetivo principal de la **validación de requisitos** es garantizar que el SRS refleje los requisitos reales de manera precisa y clara. Un objetivo relacionado es comprobar que el documento SRS en sí sea de **“buena calidad”**.

Antes de discutir la validación, conviene analizar los tipos de errores que típicamente ocurren en un SRS. Existen muchos tipos de errores posibles, pero los más comunes se pueden clasificar en cuatro categorías: **omisión, inconsistencia, hecho incorrecto y ambigüedad**.

- **Omisión:** Es un error común donde algún requisito del usuario simplemente no se incluye en el SRS. El requisito omitido puede estar relacionado con el comportamiento del sistema, su rendimiento, restricciones u otros factores. La omisión afecta directamente la **completitud externa** del SRS.

- **Inconsistencia:** Puede deberse a contradicciones dentro de los propios requisitos o a la incompatibilidad entre los requisitos declarados y los reales del cliente o del entorno en que operará el sistema.
- **Hecho incorrecto:** Ocurre cuando algún dato registrado en el SRS no es correcto.
- **Ambigüedad:** Ocurre cuando algún requisito tiene múltiples interpretaciones posibles, es decir, su interpretación no es única.

Algunos proyectos han recopilado datos sobre los errores en los requisitos. Por ejemplo, se han detectado más de 250 errores en aplicaciones de procesamiento de datos, y en el proyecto A-7 (software de control de vuelo en tiempo real) se detectaron alrededor de 80 errores, de los cuales aproximadamente un 23% eran de naturaleza administrativa. A pesar de que la distribución de errores varía según la aplicación y el método de detección, los principales problemas, además de los errores administrativos, son **omisiones, hechos incorrectos, inconsistencias y ambigüedades**. Esto indica que la validación debe centrarse en descubrir estos tipos de errores además de mejorar la calidad del SRS.

Como los requisitos generalmente son documentos textuales que no se pueden ejecutar, las **inspecciones** son muy adecuadas para la validación de requisitos. Las inspecciones del SRS, frecuentemente llamadas **revisiones de requisitos**, son el método más común de validación. Dado que la especificación de requisitos formaliza algo que originalmente existía de manera informal en la mente de las personas, la validación debe involucrar tanto a los clientes como a los usuarios. Por ello, el equipo de revisión suele incluir representantes del cliente y de los usuarios.

La **revisión de requisitos** es un proceso en el que un grupo de personas busca errores y señala otros aspectos importantes en el SRS. El grupo de revisión debería incluir: el autor del documento de requisitos, alguien que conozca las necesidades del cliente, un miembro del equipo de diseño y las personas responsables de mantener el documento de requisitos. También es recomendable incluir a alguien que no esté directamente involucrado en el desarrollo, como un ingeniero de calidad de software.

Aunque el objetivo principal del proceso de revisión es detectar errores, también se considera la calidad del SRS, como su **capacidad de prueba y legibilidad**. Durante la revisión, los revisores deben identificar requisitos demasiado subjetivos o difíciles de probar.

Se suelen usar **listas de verificación** para enfocar el esfuerzo de revisión y asegurar que no se pase por alto ninguna fuente importante de errores. Algunos ejemplos de preguntas de la lista de verificación son:

- ¿Se han definido todos los recursos de hardware?
- ¿Se han especificado los tiempos de respuesta de las funciones?
- ¿Se han definido todas las interfaces de hardware, software externo y datos?
- ¿Se han especificado todas las funciones requeridas por el cliente?
- ¿Cada requisito es verificable?
- ¿Se ha definido el estado inicial del sistema?
- ¿Se han especificado las respuestas a condiciones excepcionales?

- ¿Contiene el requisito restricciones controlables por el diseñador?
- ¿Se han especificado posibles modificaciones futuras?

Las revisiones de requisitos son probablemente el medio más efectivo para detectar errores. En el proyecto A-7, aproximadamente el 33% de los errores se detectaron mediante revisiones, y alrededor del 45% durante la fase de diseño, cuando el documento de requisitos se utilizó como referencia. Esto sugiere que revisar los requisitos no solo detecta una fracción sustancial de errores, sino que la mayoría de los errores restantes se detectan poco después en la fase de diseño.

Aunque las revisiones siguen siendo el método más común y viable, existen otras posibilidades si se utilizan herramientas especiales de modelado y análisis. Por ejemplo, si los requisitos están escritos en un lenguaje formal o diseñado para procesamiento por máquina, es posible usar herramientas para verificar algunas propiedades, como **consistencia interna** y **completitud**, aunque no pueden comprobar directamente la **completitud externa** (por ejemplo, no pueden detectar requisitos completamente omitidos). Por esta razón, incluso cuando se usan herramientas o notaciones formales, las revisiones siguen siendo necesarias.

3.6 Métricas

Como mencionamos anteriormente, el propósito básico de las métricas en cualquier punto durante un proyecto de desarrollo es proporcionar información cuantitativa al proceso de gestión, de modo que dicha información pueda ser utilizada para controlar efectivamente el proceso de desarrollo.

A menos que la métrica sea útil de alguna manera para monitorear o controlar el costo, el cronograma o la calidad del proyecto, tiene poca utilidad para el mismo.

Existen muy pocas métricas definidas para los requerimientos, y se ha realizado poco trabajo para estudiar la relación entre los valores de las métricas y las propiedades del proyecto que resultan de interés.

Esto dice más sobre el estado actual del arte en métricas de software que sobre la utilidad de contar con tales métricas.

En esta sección, discutiremos algunas de las métricas y cómo pueden ser utilizadas.

3.6.1 Tamaño — Puntos de Función

Un problema importante después de completar los requerimientos es estimar el esfuerzo y el cronograma del proyecto. Para ello, se necesitan algunas métricas que puedan extraerse de los requerimientos y utilizarse para estimar el costo y el cronograma (mediante el uso de algún modelo). Dado que el factor principal que determina el costo (y el cronograma) de un proyecto de software es su tamaño, una métrica que permita obtener una idea del tamaño del proyecto será útil para estimar el costo.

Esto implica que, durante la fase de requerimientos, medir el tamaño de la especificación de requerimientos (SRS) en sí misma carece de sentido, a menos que el tamaño de la SRS refleje el esfuerzo requerido para el proyecto. También se requiere que las relaciones entre cualquier medida de tamaño propuesta y el esfuerzo final del proyecto se establezcan antes de hacer un uso general de la métrica.

Una métrica de tamaño comúnmente utilizada para los requerimientos es el tamaño del texto de la SRS. El tamaño puede medirse en número de páginas, número de párrafos, número de requerimientos funcionales, etc. Como puede imaginarse, estas medidas dependen en gran medida de los autores del documento. Un analista verboso, que gusta de usar muchas ilustraciones, puede producir una SRS que tenga un tamaño varias veces mayor que la de un analista más conciso. De manera similar, el grado en que un analista refine los requerimientos tiene impacto en el tamaño del documento. En general, tales métricas no pueden ser indicadores precisos del tamaño del proyecto. Se utilizan principalmente para dar una idea general sobre el tamaño del mismo.

Los puntos de función son una de las medidas de tamaño de software más ampliamente utilizadas. La base de los puntos de función es que la “funcionalidad” de un sistema —es decir, lo que el sistema realiza— es la medida del tamaño del sistema. Y dado que la funcionalidad es independiente de cómo se especifiquen los requerimientos del sistema o incluso de cómo se implementen finalmente, esta medida tiene la ventaja de depender únicamente de las capacidades del sistema. En los puntos de función, la funcionalidad del sistema se calcula en términos del número de funciones que implementa, el número de entradas, el número de salidas, etc.; parámetros que pueden obtenerse después del análisis de requerimientos y que son independientes del lenguaje de especificación (y de implementación).

La formulación original para calcular los puntos de función utiliza el conteo de cinco parámetros diferentes: tipos de entradas externas, tipos de salidas externas, tipos de archivos internos lógicos, tipos de archivos de interfaz externa y tipos de consultas externas. Según el enfoque de puntos de función, estos cinco parámetros capturan toda la funcionalidad de un sistema. Sin embargo, dos elementos del mismo tipo pueden diferir en su complejidad y, por lo tanto, no deberían aportar la misma cantidad a la “funcionalidad” del sistema. Para tener en cuenta la complejidad, cada parámetro dentro de un tipo se clasifica como simple, promedio o complejo. La definición de cada uno de estos tipos y la interpretación de sus niveles de complejidad se dan más adelante.

Cada tipo de entrada (de datos o de control) única que se proporciona al sistema desde el exterior se considera de tipo de entrada externa y se cuenta. Una entrada externa se considera única si su formato es diferente de otras o si las especificaciones requieren un procesamiento diferente de otros tipos de entrada del mismo formato. La fuente de la entrada externa puede ser el usuario u otra aplicación o archivo. Una entrada externa se considera simple si tiene pocos elementos de datos y afecta solo a unos pocos archivos internos de la aplicación. Se considera compleja si tiene muchos elementos de datos y requiere muchos archivos internos lógicos para su procesamiento. La complejidad es promedio si se encuentra entre ambos extremos. Nótese que los archivos necesarios para el sistema operativo o el hardware (por ejemplo, archivos de configuración) no se cuentan como archivos de entrada externa, ya que no pertenecen a la aplicación sino a la tecnología subyacente.

De manera similar, cada salida única que deja el límite del sistema se cuenta como un tipo de salida externa. Una salida externa se considera única si su formato o su procesamiento difieren. Los reportes o mensajes a los usuarios u otras aplicaciones se cuentan como tipos de salida externa. Los criterios de complejidad son similares a los de las entradas externas. En un reporte, si contiene pocas columnas se considera simple; si tiene múltiples columnas, se considera promedio; y si contiene estructuras de datos complejas y hace referencia a muchos archivos para su generación, se considera complejo.

Cada aplicación mantiene información internamente para realizar sus funciones. Cada grupo lógico de información de datos o control que sea generado, usado y mantenido por la aplicación se cuenta como un tipo de archivo interno lógico. Un archivo interno lógico es simple si contiene pocos tipos de registros, complejo si tiene muchos, y promedio si se encuentra entre ambos.

Los archivos que son compartidos entre aplicaciones se cuentan como tipos de archivos de interfaz externa. Nótese que cada archivo de este tipo se cuenta para todas las aplicaciones que lo comparten. Los niveles de complejidad se definen igual que para los archivos internos lógicos.

Un sistema también puede tener consultas, donde una consulta se define como una combinación entrada-salida en la que la entrada provoca que la salida se genere casi de inmediato. Cada par único entrada-salida se cuenta como tipo de consulta externa. Una consulta es única si difiere de otras en el formato de entrada o salida o si requiere un procesamiento diferente. Para clasificar el tipo de consulta, la entrada y la salida se clasifican igual que los tipos de entrada y salida externa, respectivamente. La complejidad de la consulta se toma como el mayor de los dos valores.

Cada elemento del mismo tipo y complejidad aporta una cantidad fija y constante al conteo total de puntos de función del sistema (que es una medida de la funcionalidad del sistema), pero la contribución varía entre los distintos tipos, y dentro de un tipo varía según el nivel de complejidad. La cantidad de contribución de un elemento se muestra en la Tabla 3.3 [2, 113].

Una vez que se conocen los conteos para los cinco tipos diferentes en las tres clases de complejidad, el punto de función bruto o no ajustado (UFP) puede calcularse como una suma ponderada de la siguiente manera:

$$UFP = \sum_{i=1}^5 \sum_{j=1}^3 w_{ij} C_{ij}$$

donde i representa la fila y j la columna en la Tabla 3.3; w_{ij} es la entrada en la fila i y columna j de la tabla (es decir, representa la contribución de un elemento del tipo i y complejidad j); y C_{ij} es el conteo del número de elementos del tipo i que han sido clasificados con la complejidad correspondiente a la columna j .

Una vez obtenido el UFP, este se ajusta según la complejidad del entorno. Para ello, se consideran 14 características del sistema: comunicaciones de datos, procesamiento distribuido, objetivos de rendimiento, carga de configuración operativa, tasa de transacciones, entrada de datos en línea, eficiencia del usuario final, actualización en línea, lógica de procesamiento compleja, reusabilidad, facilidad de instalación, facilidad operativa, múltiples sitios y facilidad de cambio. El grado de influencia de cada factor se mide de 0 a 5, representando seis niveles: no presente (0), influencia insignificante (1), influencia moderada (2), influencia promedio (3), influencia significativa (4) e influencia fuerte (5). La suma de los 14 grados de influencia da un total N (que varía de 0 a 70). Este N se utiliza para obtener el factor de ajuste de complejidad (CAF) como sigue:

$$CAF = 0.65 + 0.01N$$

Con esta ecuación, el valor de CAF varía entre 0.65 y 1.35. Los puntos de función entregados (DFP) se calculan simplemente multiplicando el UFP por el CAF, es decir:

Puntos de Función Entregados = CAF × Puntos de Función No Ajustados.

Como se observa, tras el ajuste por complejidad del entorno, el DFP puede diferir del UFP en un máximo de 35%. El conteo final de puntos de función para una aplicación es el DFP calculado.

Los puntos de función se han utilizado extensamente como medida de tamaño y para la estimación de costos. También se han realizado estudios para establecer correlaciones entre el DFP y el tamaño final del software (medido en líneas de código). Por ejemplo, según una conversión dada en www.theadvisors.com/langcomparison.htm, un punto de función equivale aproximadamente a 125 líneas de código C, y a unas 50 líneas de código C++ o Java. Construyendo modelos entre puntos de función y líneas de código entregadas (y dado que se ha demostrado que existe una correlación razonablemente fuerte entre DFP y KLOC), se puede estimar el tamaño del software en KLOC, si se desea.

Como puede verse por la manera en que se define la funcionalidad del sistema, el enfoque de puntos de función fue diseñado para aplicaciones de procesamiento de datos. Para este tipo de aplicaciones, los puntos de función funcionan muy bien [106] y han sido ampliamente aceptados. En tales casos, los puntos de función se usan eficazmente para estimar costos y evaluar productividad. Sin embargo, su utilidad como medida de tamaño para aplicaciones que no son de procesamiento de datos (por ejemplo, software en tiempo real, sistemas operativos y aplicaciones científicas) no ha sido bien establecida, y generalmente se considera que los puntos de función no se adaptan bien a esos casos.

Una desventaja importante del enfoque de puntos de función es que el proceso de cálculo involucra evaluaciones subjetivas en varios puntos, y el valor final calculado puede no ser único y depender del analista. Algunos ejemplos donde aparece la subjetividad son:

- (1) Diferentes interpretaciones de la SRS (por ejemplo, si algo debe contarse como tipo de entrada externa o como tipo de interfaz externa; si algo constituye o no un archivo interno lógico; si dos reportes difieren muy poco, si deben contarse como uno o dos).
- (2) La estimación de complejidad de una función de usuario es totalmente subjetiva y depende del analista (uno puede clasificar algo como complejo mientras otro lo considera promedio), y la complejidad puede tener un impacto sustancial en el conteo final, ya que los pesos entre simple y complejo suelen diferir por un factor de 2.
- (3) Juicios de valor para la complejidad del entorno.

Estos factores hacen que el proceso de conteo de puntos de función sea algo subjetivo. Las organizaciones que usan puntos de función intentan especificar un conjunto más preciso de reglas de conteo para reducir esta subjetividad. También se ha encontrado que, con la experiencia, dicha subjetividad disminuye [113]. En general, a pesar de esta subjetividad, el uso de puntos de función para aplicaciones de procesamiento de datos sigue creciendo.

La principal ventaja de los puntos de función sobre la métrica de tamaño en KLOC (la otra aproximación común) es que la definición del DFP depende solo de la información disponible en las especificaciones, mientras que el tamaño en KLOC no puede determinarse directamente a partir de ellas. Además, el conteo DFP es independiente del lenguaje en que se implemente el proyecto. Aunque estas son ventajas importantes, otra desventaja del enfoque de puntos de función es que, incluso cuando el proyecto está terminado, el DFP no se conoce de manera única y sigue siendo subjetivo. Esto dificulta la construcción de modelos para estimación de costos, ya que dichos modelos se basan en información de proyectos completados (los modelos de costos se discuten más adelante en el próximo capítulo).

Además, determinar el DFP —ya sea a partir de los requerimientos o de un proyecto terminado— no puede automatizarse. Es decir, se requiere un esfuerzo considerable para obtener el tamaño, incluso para un proyecto completado. Esto representa una desventaja respecto a la métrica KLOC,

ya que KLOC puede determinarse de manera única mediante herramientas automáticas una vez finalizado el proyecto.

4 Arquitectura de Software

Un **sistema** es una entidad que provee algún tipo de **comportamiento a su entorno**, el cual puede estar formado por personas u otros sistemas. En el capítulo anterior vimos que el **comportamiento esperado** de un sistema de software propuesto se define a través de un documento de **especificación de requisitos de software (SRS)**. Para construir el sistema especificado, un paso clave es el **diseño de la arquitectura de software**, tema central de este capítulo.

Cualquier sistema complejo está compuesto por **subsistemas** que interactúan bajo el control del diseño del sistema, de modo que el conjunto provea el comportamiento esperado. Al diseñar un sistema, el enfoque lógico es **identificar los subsistemas que lo compondrán, las interfaces entre ellos y las reglas de interacción**. Justamente esto es lo que busca lograr la **arquitectura de software**.

La arquitectura de software es un área relativamente reciente. A medida que los sistemas de software se vuelven **más distribuidos y complejos**, la arquitectura se convierte en un paso esencial para su construcción. Debido a la **gran cantidad de opciones** disponibles para configurar y conectar un sistema, el diseño cuidadoso de la arquitectura resulta muy importante. Es durante este proceso donde se toman decisiones como el uso de cierto **middleware**, de una **base de datos**, de un **servidor** o de un **componente de seguridad**. No es posible diseñar todos los detalles y luego intentar incorporar estas decisiones; la arquitectura debe contemplarlas desde el inicio en la **estructura del sistema**.

Además, la arquitectura es el punto más temprano en el que se pueden evaluar propiedades como **fiabilidad** y **rendimiento**, algo cada vez más relevante en los sistemas modernos.

En este capítulo nos enfocaremos principalmente en los **conceptos de arquitectura** y en cierta **notación para describirla**. El tema de la **metodología** (es decir, cómo debe crearse la arquitectura) no se aborda, ya que se trata de una **actividad creativa de alto nivel** para la cual no existen metodologías estrictas. Sin embargo, discutiremos algunos **estilos arquitectónicos**, que sugieren formas de estructurar sistemas. Una combinación o variación de estos estilos puede resultar útil para muchos casos.

También trataremos temas relacionados con la **documentación de arquitecturas**, su **relación con el diseño** y una forma de **analizar arquitecturas**. Finalmente, cerraremos el capítulo con una discusión sobre la arquitectura en los **dos estudios de caso**.

4.1 El Rol de la Arquitectura de Software

Antes de hablar de su papel en la construcción de un sistema, debemos responder: **¿qué es la arquitectura?**

A un nivel alto, la **arquitectura** es el diseño de un sistema que ofrece una **vista general de sus partes y de cómo se relacionan** para formar el sistema completo. Esto implica **particionar** el sistema en componentes lógicos que pueden entenderse de forma independiente, y luego describir tanto esas partes como las relaciones entre ellas.

En un sistema complejo puede haber muchas maneras de dividirlo, y lo mismo ocurre con un software: **no existe una única arquitectura posible**, sino múltiples estructuras válidas.

Una de las definiciones más aceptadas es:

La arquitectura de software de un sistema es la estructura o estructuras del sistema, compuestas por elementos de software, las propiedades visibles externamente de esos elementos y las relaciones entre ellos.

Esto significa que la arquitectura se enfoca en las **abstracciones necesarias para especificar interacciones**: qué servicios brinda cada componente, qué propiedades de calidad ofrece (rendimiento, seguridad, fiabilidad, etc.) y cómo se relaciona con otros. **No es necesario entrar en los detalles internos de implementación** en esta etapa.

La arquitectura, entonces, representa de forma **concisa y comprensible** un sistema complejo. Eso sí, esta definición no evalúa si una arquitectura es buena o mala; eso requiere **análisis posterior**.

Principales usos de la descripción arquitectónica

1. Comprensión y comunicación

- Sirve como medio de **comunicación entre stakeholders**: usuarios, clientes, desarrolladores y arquitectos.
- Ofrece un **lenguaje común** para negociar y alinear expectativas.
- Reduce la complejidad mostrando el sistema como subsistemas y sus interacciones, ocultando los detalles de implementación.
- También puede documentar sistemas ya existentes, facilitando su entendimiento.

2. Reutilización (Reuse)

- La arquitectura es clave para decidir dónde y cómo **reutilizar componentes existentes**.
- Permite construir **familias de productos** (product lines), especificando qué partes son fijas y cuáles variables.
- La reutilización es más factible y eficaz a nivel arquitectónico que a nivel de detalle.

3. Construcción y evolución

- Una buena partición arquitectónica facilita que distintos equipos trabajen en paralelo en subsistemas relativamente independientes.
- Define **restricciones estructurales** que deben respetarse en el desarrollo.
- Durante la evolución del software, ayuda a decidir dónde agregar nuevas funcionalidades con el **mínimo impacto y complejidad**, y qué partes se verán afectadas por cambios.

4. Análisis

- Permite **predecir propiedades del sistema antes de construirlo**.

- Ejemplo: analizar confiabilidad, rendimiento o tiempos de respuesta de una arquitectura propuesta.
- Así se pueden explorar alternativas y elegir la que mejor satisfaga los requisitos de calidad.
- Por ejemplo, al diseñar un sitio de e-commerce, la arquitectura puede analizarse para estimar throughput y tiempos de respuesta según la carga esperada, y decidir si se necesita un servidor más potente o una arquitectura distinta.

No todos estos usos son igual de relevantes en todos los proyectos:

- En proyectos pequeños, la **comunicación** puede ser lo más importante.
- En sistemas grandes o críticos, el **análisis de rendimiento y confiabilidad** puede ser el foco central.

4.2 Vistas de la Arquitectura

Está surgiendo una visión general de que no existe una arquitectura única de un sistema. La definición que hemos adoptado (dada arriba) también expresa este sentimiento. En consecuencia, **no existe un único dibujo arquitectónico del sistema**. La situación es similar a la de la construcción civil, una disciplina que es la usuaria original del concepto de arquitectura y de donde se ha tomado el concepto de arquitectura de software. Para un edificio, si usted quiere ver el plano del piso, se le muestra un conjunto de dibujos. Si usted es un ingeniero eléctrico y quiere ver cómo se ha planificado la distribución eléctrica, se le mostrará otro conjunto de dibujos. Y si está interesado en seguridad y en la lucha contra incendios, se usa otro conjunto de dibujos. Estos dibujos no son independientes unos de otros: todos tratan sobre el mismo edificio. Sin embargo, cada dibujo proporciona una vista diferente del edificio, una vista que se enfoca en explicar un aspecto del edificio y trata de hacer un buen trabajo en eso, sin divulgar mucho sobre los otros aspectos. Y ningún dibujo puede expresar todos los aspectos diferentes: un dibujo así sería demasiado complejo para resultar útil.

La situación es similar con la arquitectura de software. **En software, los diferentes dibujos se llaman vistas. Una vista representa el sistema como compuesto por ciertos tipos de elementos y relaciones entre ellos. Qué elementos utiliza una vista depende de qué quiere resaltar la vista. Diferentes vistas exponen diferentes propiedades y atributos, permitiendo así que los involucrados y los analistas evalúen adecuadamente esos atributos para el sistema. Al enfocarse solo en algunos aspectos del sistema, una vista reduce la complejidad con la que el lector debe lidiar en un momento dado, ayudando así a la comprensión y al análisis del sistema.**

Una vista describe una estructura del sistema. Usaremos estos dos conceptos —vistas y estructuras— de manera intercambiable. También usaremos el término vista arquitectónica para referirnos a una vista. Se han propuesto muchos tipos de vistas. La mayoría de las vistas propuestas generalmente pertenecen a uno de estos tres tipos [35, 9]:

- Módulo
- Componente y conector
- Asignación

En una vista de módulo, el sistema es visto como una colección de unidades de código, cada una implementando alguna parte de la funcionalidad del sistema. Es decir, los elementos principales en esta vista son los módulos. Estas vistas están basadas en el código y no representan explícitamente ninguna estructura de tiempo de ejecución del sistema. Ejemplos de módulos son paquetes, una clase, un procedimiento, un método, una colección de funciones y una colección de clases. Las relaciones entre estos módulos también están basadas en el código y dependen de cómo el código de un módulo interactúa con otro.

Si el edificio fuera solo una casa pequeña, entonces quizás no haya necesidad de tener una vista separada que describa el sistema de emergencia y de incendios. De manera similar, si no hay aire acondicionado en el edificio, entonces no se necesita ninguna vista para eso. Por otro lado, un edificio de oficinas tal vez requiera ambas vistas, además de otras vistas que describan plomería, espacio, cableado, etc.

La situación con el software es similar: qué vistas se necesitan para un proyecto depende del proyecto y del sistema que se está construyendo. Dependiendo de las necesidades del proyecto, se puede decidir qué vistas son necesarias. Por ejemplo, si se va a realizar un análisis de rendimiento, entonces la arquitectura debe describir alguna vista de componentes y conectores para capturar la estructura de tiempo de ejecución del sistema, así como describir la vista de asignación para especificar en qué hardware se ejecutan los diferentes componentes. Si la arquitectura se utiliza para planificar el desarrollo, entonces se debe proporcionar una vista de módulo para que los diferentes programadores o equipos puedan ser asignados a diferentes módulos. En general, un proyecto grande y complejo donde hay mucho dinero en juego requerirá muchas vistas diferentes para que pueda ser analizado desde muchos ángulos distintos, y al hacerlo se reduzcan los riesgos de fallos. Por otro lado, para un proyecto más pequeño, quizás una sola vista, o un par de vistas, será suficiente.

Sin embargo, a pesar de que existen múltiples dibujos que muestran diferentes vistas de un edificio, hay una vista que predomina en la construcción: la de la estructura física. Esta vista constituye la base de otras vistas, en el sentido de que otras vistas no pueden realmente completarse a menos que esta vista pueda realizarse. Otras vistas pueden o no ser necesarias para construir un edificio, dependiendo de la naturaleza del proyecto. Por lo tanto, en cierto sentido, la vista que muestra la estructura del edificio puede considerarse como la vista principal, en el sentido de que casi siempre se usa, y otras vistas dependen sustancialmente de ella. La vista también captura quizás la propiedad más importante que se debe analizar en las primeras etapas, a saber, la organización del espacio.

La situación con la arquitectura de software es también algo similar. Como hemos dicho, dependiendo de qué propiedades sean de interés, se necesitan diferentes vistas de la arquitectura del software. Sin embargo, de estas vistas, la vista de componentes y conectores (C&C) se ha convertido en la vista primaria de facto, una que casi siempre se prepara cuando se diseña una arquitectura (algunas definiciones incluso ven la arquitectura solo en términos de vistas C&C). En este capítulo, nos centraremos principalmente en la vista de C&C, y discutiremos los otros dos tipos solo brevemente. La vista de módulo se discutirá más adelante en los capítulos siguientes cuando tratemos el diseño de alto nivel, que se enfoca en identificar los diferentes módulos en el software.

4.3 Vista de Componentes y Conectores

La vista de arquitectura de Componentes y Conectores (C&C) de un sistema tiene dos elementos principales: componentes y conectores. Los componentes son usualmente elementos computacionales o almacenes de datos que tienen alguna presencia durante la ejecución del sistema. Los conectores definen los medios de interacción entre estos componentes. Una vista C&C del sistema define los componentes, y qué componente está conectado con cuál y a través de qué conector. Una vista C&C describe una estructura en tiempo de ejecución del sistema —qué componentes existen cuando el sistema se está ejecutando y cómo interactúan durante la ejecución. La estructura C&C es esencialmente un grafo, con componentes como nodos y conectores como aristas.

La vista C&C es quizás la vista más común de la arquitectura y la mayoría de los diagramas de cajas y líneas que representan la arquitectura intentan capturar esta vista. Muy a menudo, cuando las personas hablan de la arquitectura, se refieren a la vista C&C. La mayoría de los lenguajes de descripción de arquitecturas también se enfocan en la vista C&C.

4.3.1 Componentes

Los componentes son generalmente unidades de computación o almacenes de datos en el sistema. Un componente tiene un nombre, que en general se elige para representar el rol del componente o la función que realiza. El nombre también proporciona una identidad única para el componente, lo cual es necesario para referenciar los detalles sobre el componente en los documentos de apoyo, ya que un dibujo C&C solo mostrará los nombres de los componentes.

Un componente es de un tipo de componente, donde el tipo representa un componente genérico, definiendo la computación general y las interfaces que un componente de ese tipo debe tener. Nótese que aunque un componente tiene un tipo, en la vista de arquitectura C&C tenemos componentes (es decir, instancias reales) y no tipos. Ejemplos de estos tipos son clientes, servidores, filtros, etc. Diferentes dominios pueden tener otros tipos genéricos como controladores, actuadores y sensores (para un dominio de sistemas de control).

En un diagrama que representa una vista de arquitectura C&C de un sistema, es altamente deseable tener una representación diferente para los distintos tipos de componentes, de manera que los diferentes tipos puedan identificarse visualmente. En un diagrama de cajas y líneas, a menudo todos los componentes se representan como cajas rectangulares. Tal enfoque requerirá que los tipos de los componentes se describan por separado y que el lector tenga que leer la descripción para averiguar los tipos de los componentes. Es mucho mejor usar un símbolo/una notación diferente para cada tipo de componente. Si hay múltiples componentes del mismo tipo, entonces cada uno de estos componentes será representado usando el mismo símbolo; se distinguirán entre sí por sus nombres.

Los componentes usan interfaces para comunicarse con otros componentes. Las interfaces a veces se llaman puertos. Un componente debe especificar claramente sus puertos. En un diagrama, esto se hace típicamente colocando marcas adecuadas en los bordes del símbolo que se usa para el componente.

Sería útil que hubiera una lista de símbolos estándar que pudieran usarse para construir un diagrama de arquitectura. Sin embargo, como no existe una lista estándar de tipos de componentes, no existe

tal lista estándar. Algunos de los símbolos comunes usados para representar tipos de componentes comúnmente encontrados se muestran en la Figura 4.2.

Como no hay notaciones estándar para los diferentes tipos de componentes y un arquitecto puede usar sus propios símbolos, la información del tipo no puede obtenerse por el lector a partir de los símbolos usados. Para asegurarse de que los significados de los diferentes símbolos sean claros para el lector, es por lo tanto necesario tener una clave de los distintos símbolos para describir qué tipo de componente representa un símbolo.

Un componente es esencialmente un sistema por sí mismo que provee algún comportamiento en interfaces definidas (es decir, puertos) hacia su entorno. Como cualquier sistema, un componente puede ser complejo y tener una estructura propia, que puede determinarse descomponiendo el componente. En muchas situaciones, particularmente para sistemas que no son demasiado grandes, puede no ser necesario descomponer los componentes para determinar su arquitectura interna. Principalmente trabajaremos con componentes atómicos, es decir, componentes cuya estructura interna no es necesaria para describir o analizar una vista de arquitectura.

4.3.2 Conectores

Los diferentes componentes de un sistema probablemente interactúen mientras el sistema está en operación para proporcionar los servicios esperados del sistema. Después de todo, los componentes existen para proveer partes de los servicios y características del sistema, y estos deben combinarse para entregar la funcionalidad general del sistema. Para componer un sistema a partir de sus componentes, es necesaria información sobre la interacción entre componentes.

La interacción entre componentes puede darse mediante un mecanismo simple soportado por la infraestructura de ejecución del proceso del sistema operativo. Por ejemplo, un componente puede interactuar con otro usando el mecanismo de llamada a procedimiento (un conector), que es provisto por el entorno de ejecución del lenguaje de programación. Sin embargo, la interacción también puede involucrar mecanismos más complejos. Ejemplos de tales mecanismos son la llamada a procedimiento remoto (RPC), los puertos TCP/IP y un protocolo como HTTP. Estos mecanismos requieren una cantidad considerable de infraestructura de ejecución subyacente, así como programación especial dentro de los componentes para usar esa infraestructura. En consecuencia, es extremadamente importante identificar y representar explícitamente estos conectores. La especificación de conectores ayudará a identificar la infraestructura adecuada necesaria para implementar una arquitectura, así como a aclarar las necesidades de programación para los componentes que los utilicen. Sin una comprensión adecuada de los conectores, la implementación de los componentes que los usan puede no ser posible.

Cabe destacar que los conectores no necesitan ser binarios, y un conector puede proveer comunicación de tipo n-vías entre múltiples componentes. Por ejemplo, se puede usar un bus de difusión (broadcast bus) como un conector, que permite a un componente transmitir su mensaje a todos los demás componentes. (Por supuesto, cómo será implementado tal conector es otra cuestión que debe resolverse antes de que la arquitectura pueda ser implementada. Generalmente, al crear una arquitectura, es recomendable que el arquitecto use los conectores que están disponibles en los sistemas donde se desplegará el software. De lo contrario, deben existir planes para construir esos conectores o comprarlos si están disponibles).

Un conector también tiene un nombre que debe describir la naturaleza de la interacción que soporta. Un conector también tiene un tipo, que es una descripción genérica de la interacción, especificando propiedades como si es binario o n-vías, los tipos de interfaces que soporta, etc. A veces, la interacción soportada por un conector se representa mejor como un protocolo. Un protocolo implica que cuando dos o más componentes usan el conector empleando el protocolo para comunicarse, deben seguir ciertas convenciones sobre el orden de eventos o comandos, el orden en que los datos deben agruparse para su envío, condiciones de error, etc. Por ejemplo, si se van a usar puertos TCP para enviar información de un proceso a otro (los puertos TCP son el conector entre dos componentes de tipo proceso), el protocolo requiere que primero se establezca una conexión y se obtenga un número de puerto antes de enviar la información, y que la conexión sea cerrada al final. Una descripción de protocolo hace explícitas todas estas restricciones y define las condiciones de error y escenarios especiales. Si un conector usa un protocolo, debe ser declarado explícitamente.

Así como con los componentes, en un diagrama de arquitectura C&C (Componentes y Conectores) de un sistema, es mejor usar una notación diferente para cada tipo de conector. Es un error común usar una línea simple o una flecha para representar todos los tipos de conectores, obligando al lector a obtener la información del tipo desde otro lugar. Sin embargo, no siempre es necesario distinguir múltiples instancias del mismo tipo de conector mediante nombres, ya que a menudo los componentes a los cuales conectan pueden proporcionar una identificación única. Como con los componentes, dado que no existen notaciones comúnmente aceptadas, es mejor proporcionar una clave de las notaciones utilizadas. Algunos ejemplos de conectores se muestran en la Figura 4.3.

Vale la pena señalar que la implementación de un conector puede ser bastante compleja y puede estar distribuida. Por ejemplo, un middleware como CORBA provee conectores que pueden ser usados por objetos para interactuar. Sin embargo, hay mucho código en forma de ORB (Object Request Broker) que es necesario para soportar este conector. Es el software ORB el que realiza las traducciones de formato entre el componente emisor y el receptor, y lleva a cabo toda la comunicación entre ellos (usando un protocolo llamado IIOP), además de proporcionar varios otros servicios que los objetos pueden necesitar para cooperar. Por esto, la representación explícita de conectores es importante, particularmente en sistemas distribuidos donde los conectores juegan roles que no pueden ser fácilmente reemplazados por mecanismos implícitos del lenguaje o del sistema operativo.

Si el conector es provisto por el sistema subyacente, entonces los componentes solo deben asegurarse de usar el conector conforme a sus especificaciones. Si, sin embargo, el sistema subyacente no provee un conector usado en una arquitectura, entonces, como se mencionó anteriormente, el conector deberá ser implementado como parte del proyecto para construir el sistema. Es decir, durante el desarrollo, no solo será necesario desarrollar los componentes, sino que también deberán asignarse recursos para desarrollar el conector. (Esta situación puede surgir en sistemas especializados que requieren conectores específicos para el dominio del problema).

4.4 Estilos de Arquitectura para la Vista de Componentes y Conectores (C&C)

Como se mencionó anteriormente, una vista de arquitectura describe una estructura del sistema en términos de sus elementos y de las relaciones entre ellos. Claramente, diferentes sistemas tendrán

diferentes estructuras, incluso para una misma vista. Sin embargo, existen algunas estructuras y restricciones asociadas que se han observado en muchos sistemas y que parecen representar estructuras generales que resultan útiles para la arquitectura de una clase de problemas. Estas se denominan las restricciones de ese estilo [35, 9, 135].

Por ejemplo, para las vistas de módulos, algunos de los estilos comunes son descomposición, usos, generalización y en capas. En el estilo de descomposición, un módulo se descompone en submódulos, y el sistema se convierte en una jerarquía de módulos. En el estilo de usos, los módulos no son partes unos de otros, sino que un módulo utiliza los servicios de otros módulos (por ejemplo, una llamada a función o una invocación de método) para realizar correctamente su propio trabajo. En el estilo de generalización, los módulos suelen ser clases, y una clase hija hereda las propiedades de la clase padre y las especializa. Esto soporta una jerarquía del tipo *es-un*, donde un módulo hijo también es del tipo del módulo padre. En el estilo en capas, el sistema se estructura como una pila de capas, donde cada capa representa una máquina virtual que provee un conjunto claro de servicios. Además, a una capa solo se le permite utilizar los servicios de las capas adyacentes. Algunos de estos estilos se discutirán más adelante en los capítulos de diseño.

En esta sección discutimos algunos estilos comunes para la vista C&C. Se han propuesto muchos estilos para la vista de Componentes y Conectores, algunos para dominios específicos. Aquí discutimos solo algunos que son ampliamente tratados en la literatura y que pueden ser útiles para un amplio conjunto de problemas [135, 35]. Estos estilos pueden proporcionar ideas para crear una vista de arquitectura para el problema en cuestión. Los estilos pueden combinarse para formar vistas más ricas. De hecho, es probable que para un problema, una combinación de estilos proporcione la arquitectura deseada. Muchos sistemas utilizan múltiples estilos y diferentes partes de un sistema pueden usar distintos estilos.

4.4.1 Tubos y Filtros

El estilo de arquitectura de tubos y filtros es muy adecuado para **sistemas que realizan principalmente transformación de datos**: se recibe algún dato de entrada y el objetivo del sistema es producir algún dato de salida transformando adecuadamente los datos de entrada. Un sistema que utiliza una arquitectura de tubos y filtros logra la transformación deseada aplicando una red de transformaciones más pequeñas y componiéndolas de tal manera que, en conjunto, se logra la transformación global deseada.

El estilo de tubos y filtros **tiene solo un tipo de componente llamado filtro**. También **tiene solo un tipo de conector, llamado tubo**. **Un filtro realiza una transformación de datos y envía los datos transformados a otros filtros para su procesamiento posterior utilizando el conector tubo**. En otras palabras, un filtro recibe los datos que necesita desde algunos tubos de entrada definidos, realiza la transformación de datos y luego envía los datos de salida a otros filtros a través de los tubos de salida definidos. **Un filtro puede tener más de una entrada y más de una salida**. Los filtros pueden ser entidades independientes y asíncronas y, como solo se preocupan por los datos que llegan a través del tubo, **un filtro no necesita conocer la identidad del filtro que envió los datos de entrada ni la identidad del filtro que consumirá los datos que produce**.

El **conector tubo es un canal unidireccional que transporta flujos de datos recibidos en un extremo hacia el otro extremo**. Un **tubo no modifica los datos de ninguna manera**, sino que simplemente los

transporta al filtro del extremo receptor en el orden en que se reciben los elementos de datos. Dado que los filtros pueden ser asincrónicos y deben funcionar sin conocer la identidad del productor o del consumidor, el almacenamiento en búfer y la sincronización necesarios para asegurar el funcionamiento fluido de la relación productor–consumidor, materializada al conectar dos filtros mediante un tubo, son garantizados por el tubo. Los filtros simplemente consumen y producen datos.

Existen algunas restricciones que este estilo impone. Primero, como se mencionó anteriormente, los filtros deben funcionar sin conocer la identidad del consumidor o del productor; solo deben requerir los elementos de datos que necesitan. Segundo, un tubo, que es un conector unidireccional, debe conectar un puerto de salida de un filtro con un puerto de entrada de otro filtro.

Una estructura pura de tubos y filtros generalmente también tendrá la restricción de que un filtro tenga un hilo de control independiente que procese los datos a medida que llegan. Implementar esto requerirá una infraestructura subyacente adecuada para soportar un mecanismo de tubos que almacene los datos en búfer y realice la sincronización necesaria (por ejemplo, bloquear al productor cuando el búfer está lleno y bloquear al filtro consumidor cuando el búfer está vacío). Para utilizar este tubo, el constructor del filtro debe ser plenamente consciente de las propiedades del tubo, particularmente en lo que respecta al almacenamiento en búfer y la sincronización, los mecanismos de entrada y salida y los símbolos de fin de datos.

Sin embargo, puede haber situaciones en las que la restricción de que un filtro procese los datos a medida que llegan no sea necesaria. Sin esta restricción, una vista de estilo de tubos y filtros puede tener filtros que produzcan completamente los datos antes de pasarlos, o que comiencen su procesamiento solo después de que la entrada completa esté disponible. En un sistema de este tipo, los filtros no pueden operar de manera concurrente y el sistema es como un sistema de procesamiento por lotes. Sin embargo, esto puede simplificar considerablemente los tubos y se pueden utilizar mecanismos más simples para soportarlos.

Consideremos un ejemplo de un sistema necesario para contar la frecuencia de diferentes palabras en un archivo. Una arquitectura que utiliza el estilo de tubos y filtros para lograr esto se muestra en la Figura 4.7.

Esta arquitectura propone que los datos de entrada se dividan primero en una secuencia de palabras mediante un componente Sequencer. Esta secuencia de palabras es luego ordenada por el componente Sorting, que pasa la salida de palabras ordenadas a otro filtro (Counting) que cuenta el número de ocurrencias de las diferentes palabras. Esta estructura de ordenar primero las palabras ha sido elegida ya que hará que la tarea de determinar la frecuencia sea más eficiente, aunque implique una operación de ordenamiento. Debería quedar claro que esta arquitectura propuesta puede implementar la funcionalidad deseada. Más adelante en el capítulo discutiremos con mayor detalle algunos aspectos de implementación relacionados con esta arquitectura.

Como se puede observar a partir de este ejemplo, el estilo arquitectónico de tubos y filtros es muy adecuado para el procesamiento y la transformación de datos. En consecuencia, es útil en aplicaciones de procesamiento de texto. Las aplicaciones de procesamiento de señales también lo encuentran útil, ya que este tipo de aplicaciones típicamente realizan codificación, corrección de errores y otras transformaciones sobre los datos.

El estilo de tubos y filtros, debido a sus restricciones, permite que la transformación global de un sistema se componga de transformaciones más pequeñas. O, visto de otra manera, permite que una

transformación deseada se descomponga en transformaciones más pequeñas y luego se construyan filtros para dichas transformaciones. Es decir, permite utilizar técnicas de composición y descomposición funcional, algo que resulta matemáticamente atractivo.

5 Planificación de un Proyecto de Software

Para que un proyecto tenga éxito, **tanto una buena gestión del proyecto como una buena ingeniería son esenciales.**

La falta de cualquiera de las dos puede hacer que un proyecto fracase.

Hemos visto que las actividades de gestión de proyectos pueden considerarse que tienen **tres fases principales:**

- **planificación del proyecto,**
- **monitoreo y control del proyecto, y**
- **finalización del proyecto.**

En términos generales, la **planificación** abarca todas las actividades que deben realizarse **antes de comenzar el trabajo de desarrollo.**

Una vez que el proyecto ha comenzado, **inicia el control del proyecto.**

En otras palabras, durante la planificación se **planifican todas las actividades que la gestión necesita realizar**, mientras que durante el control del proyecto, el **plan se ejecuta y se actualiza.**

La planificación puede ser **la actividad de gestión más importante.**

Sin un plan adecuado, **no es posible un monitoreo ni un control reales del proyecto.**

A menudo los proyectos se apresuran hacia la implementación **sin dedicar suficiente tiempo y esfuerzo a la planificación.**

Ninguna cantidad de esfuerzo técnico posterior puede compensar la falta de una planificación cuidadosa.

La falta de una planificación adecuada es una garantía de fracaso para un proyecto de software grande.

Por esta razón, tratamos la planificación de proyectos como un capítulo independiente.

El **objetivo básico de la planificación** es mirar hacia el futuro, **identificar las actividades necesarias para completar el proyecto con éxito, y planificar el cronograma y los recursos.**

Las **entradas** a la actividad de planificación son la **especificación de requisitos** y la **descripción de la arquitectura.**

No se necesita un documento de requisitos muy detallado para la planificación, pero para tener un buen plan es necesario que **se conozcan todos los requisitos importantes**, y es **altamente deseable que ya se hayan tomado las decisiones de arquitectura.**

Los **principales temas** que aborda la planificación del proyecto son:

- **Planificación del proceso**
- **Estimación del esfuerzo**
- **Estimación del cronograma y de los recursos**
- **Planes de calidad**
- **Planes de gestión de la configuración**
- **Gestión de riesgos**

- **Planes de monitoreo del proyecto**

En el resto de este capítulo discutiremos cada uno de estos temas y algunas **técnicas para tratarlos**.

5.1 Planificación del Proceso

Ya hemos discutido en detalle el proceso de desarrollo y los diversos modelos de proceso.

Para un proyecto, durante la planificación, una actividad clave es planificar y especificar el proceso que debe usarse en el proyecto.

Esto significa especificar las distintas etapas del proceso, los criterios de entrada para cada etapa, los criterios de salida, y las actividades de verificación que se realizarán al final de la etapa.

Como se ha discutido, algún modelo de proceso establecido puede usarse como un proceso estándar y adaptarse para satisfacer las necesidades del proyecto.

En una organización, a menudo se definen procesos estándar, y un proyecto puede usar cualquiera de estos procesos estándar y ajustarlo (tailor) para adecuarlo a las necesidades específicas del proyecto.

Por lo tanto, la actividad de planificación del proceso consiste principalmente en seleccionar un proceso estándar y adaptarlo para el proyecto.

El ajuste (tailoring) es un tema avanzado que no discutiremos en detalle.

En general, sin embargo, basándose en el tamaño, complejidad y naturaleza del proyecto, así como en las características del equipo —como la experiencia de los miembros del equipo con el dominio del problema y con la tecnología utilizada— se adapta el proceso estándar.

Las acciones comunes de ajuste son:

- modificar un paso,
- omitir un paso,
- agregar un paso, o
- cambiar el grado de formalidad con el que se realiza un paso.

Después de realizar el ajuste, se obtiene la especificación del proceso para el proyecto.

Este proceso guía el resto de la planificación, particularmente la planificación detallada del cronograma, donde se definen las tareas detalladas que deben realizarse en el proyecto y se asignan a las personas encargadas de ejecutarlas.

5.2 Estimación del Esfuerzo

Para un conjunto dado de requisitos, es deseable saber cuánto costará desarrollar el software y cuánto tiempo tomará el desarrollo.

Estas estimaciones son necesarias antes de iniciar el desarrollo.

La razón principal para estimar costo y cronograma es el análisis de costo-beneficio y el monitoreo y control del proyecto.

Un uso más práctico de estas estimaciones es en la licitación de proyectos de software, donde se deben proporcionar estimaciones de costo a un posible cliente para el contrato de desarrollo.

La mayor parte del costo del desarrollo de software se debe a los recursos humanos necesarios, y por lo tanto, la mayoría de los procedimientos de estimación de costos se enfocan en estimar el

esfuerzo en términos de persona-mes (PM).

Al incluir correctamente los “sobrecostos” (es decir, el costo de hardware, software, espacio de oficina, etc.) en el costo de un persona-mes, las estimaciones de esfuerzo pueden convertirse en costo.

Para un proyecto de desarrollo de software, las estimaciones de esfuerzo y cronograma son prerequisites esenciales para la gestión del proyecto.

De lo contrario, incluso preguntas simples como:

- “¿Está atrasado el proyecto?”
- “¿Hay sobrecostos?”
- “¿Cuándo es probable que se complete el proyecto?”

no pueden responderse.

Las estimaciones de esfuerzo y cronograma también son necesarias para determinar el nivel de personal de un proyecto durante sus distintas fases.

Las estimaciones pueden basarse en la opinión subjetiva de una persona o determinarse mediante el uso de modelos.

Aunque existen enfoques para estructurar las opiniones de las personas y lograr un consenso sobre la estimación de esfuerzo (por ejemplo, el enfoque Delphi), generalmente se acepta que es importante tener un enfoque más científico mediante el uso de modelos.

En esta sección, solo discutimos el enfoque basado en modelos para la estimación del esfuerzo.

Antes de discutir los modelos, primero comprendamos las limitaciones de cualquier procedimiento de estimación de esfuerzo.

5.2.1 Incertidumbres en la Estimación del Esfuerzo

Se puede realizar una estimación del esfuerzo en cualquier punto del ciclo de vida del software. Como el esfuerzo del proyecto depende de la naturaleza y las características del proyecto, en cualquier momento, la precisión de la estimación dependerá de la cantidad de información confiable que tengamos sobre el producto final.

Claramente, cuando el producto se entrega, el esfuerzo puede determinarse con precisión, ya que todos los datos sobre el proyecto y los recursos utilizados pueden conocerse completamente para ese momento.

Esta es una estimación del esfuerzo con conocimiento completo del proyecto.

En el otro extremo está el punto en el que el proyecto se está iniciando o durante el estudio de factibilidad.

En ese momento, solo tenemos una idea de las clases de datos que el sistema recibirá y producirá, y de la funcionalidad principal del sistema.

Existe una gran incertidumbre sobre las especificaciones reales del sistema.

Las especificaciones con incertidumbre representan un rango de posibles productos finales, no un producto definido con precisión.

Por lo tanto, la estimación del esfuerzo basada en este tipo de información no puede ser precisa.

Las estimaciones en esta fase del proyecto pueden desviarse hasta en un factor de cuatro con respecto al esfuerzo final real.

A medida que especificamos el sistema de manera más completa y precisa, las incertidumbres se reducen y pueden hacerse estimaciones de esfuerzo más exactas.

Por ejemplo, una vez que los requerimientos están completamente especificados, pueden hacerse estimaciones de esfuerzo más precisas que las estimaciones después del estudio de factibilidad. Una vez que el diseño está completo, las estimaciones pueden realizarse con aún mayor precisión. La precisión alcanzable de las estimaciones, a medida que varía en las diferentes fases, se muestra en la Figura 5.1 [20, 21].

Debe notarse que esta figura simplemente especifica las limitaciones de las estrategias de estimación de esfuerzo —la mejor precisión que una estrategia de estimación de esfuerzo puede esperar lograr.

No dice nada sobre la existencia de estrategias que puedan proporcionar la estimación con esa precisión.

Para la estimación real del esfuerzo, deben desarrollarse modelos o procedimientos de estimación. La precisión de las estimaciones dependerá de la efectividad y precisión de los procedimientos o modelos de estimación empleados y del proceso (es decir, qué tan predecible sea).

A pesar de las limitaciones, los modelos de estimación han madurado considerablemente y en general brindan estimaciones bastante precisas.

Por ejemplo, cuando el modelo COCOMO (que se discutirá más adelante) fue verificado con datos de algunos proyectos, se descubrió que las estimaciones estaban dentro del 20% del esfuerzo real el 68% de las veces.

También debe mencionarse que lograr una estimación dentro del 20% después de que se han especificado los requerimientos es en realidad bastante bueno.

Con una estimación así, ni siquiera es necesario que haya sobrecostos o retrasos en el cronograma, ya que generalmente hay suficiente tiempo libre o de holgura disponible (recuerda el estudio mencionado anteriormente que encontró que un programador pasa más del 30% de su tiempo en tareas personales o misceláneas) que puede usarse para cumplir los objetivos establecidos para el proyecto basados en las estimaciones.

En otras palabras, si la estimación está dentro del 20% del valor real, el efecto de esta inexactitud ni siquiera se reflejará en el costo y el cronograma finales.

Generalmente no se necesitan estimaciones altamente precisas.

Las estimaciones razonables en un proyecto de software tienden a convertirse en una profecía autocumplida —las personas trabajan para cumplir los cronogramas (que derivan de las estimaciones de esfuerzo).

5.2.2 Construcción de Modelos de Estimación del Esfuerzo

Un modelo de estimación puede verse como una “función” que produce como salida la estimación del esfuerzo.

Claramente, esta función de estimación necesitará entradas sobre el proyecto, a partir de las cuales pueda generar la estimación.

La idea básica de tener un modelo o procedimiento para la estimación es que reduce el problema de estimar a determinar el valor de los “parámetros clave” que caracterizan el proyecto, sobre cuya base puede estimarse el esfuerzo.

Debe notarse que un modelo de estimación no funciona ni puede funcionar en el vacío; necesita entradas para producir como salida la estimación del esfuerzo.

Al inicio de un proyecto, cuando los detalles del software aún no se conocen, se espera que el modelo de estimación requiera valores de parámetros que puedan medirse en esa etapa.

Aunque el esfuerzo de un proyecto es una función de muchos parámetros, generalmente se acepta que el factor principal que controla el esfuerzo es el tamaño del proyecto; es decir, cuanto más grande sea el proyecto, mayor será el esfuerzo requerido.

Por lo tanto, un enfoque común para estimar el esfuerzo es hacerlo una función del tamaño del proyecto, y la ecuación del esfuerzo se considera como:

$$\text{EFFORT} = a \times \text{SIZE}^b$$

donde a y b son constantes [5], y el tamaño del proyecto generalmente se mide en KLOC o puntos de función.

Los valores de estas constantes para un proceso en particular se determinan mediante análisis de regresión, que se aplica a los datos de los proyectos realizados en el pasado.

Por ejemplo, Watson y Felix [142] analizaron los datos de más de 60 proyectos realizados en la División de Sistemas Federales de IBM, que iban desde 4,000 hasta 467,000 líneas de código fuente entregadas, y encontraron que si la estimación de $SIZE$ está en miles de líneas de código entregadas (KLOC), el esfuerzo total E , en persona-meses (PM), puede darse por la ecuación $E = 5.2 \times (SIZE)^{0.91}$.

A menudo, sin embargo, se puede usar la productividad simple para determinar la estimación total a partir del tamaño.

Es decir, si la productividad es P KLOC/PM, entonces la estimación del esfuerzo para el proyecto puede ser $SIZE / P$ persona-meses.

Este enfoque funcionará si el tamaño y tipo del proyecto son similares al conjunto de proyectos a partir de los cuales se obtuvo la productividad P .

Este enfoque de determinar el esfuerzo total a partir del tamaño total es lo que llamamos el **enfoque de arriba hacia abajo (top-down)**, ya que primero se determina el esfuerzo total y luego, a partir de este, se obtiene el esfuerzo para las diferentes partes.

En un modelo de estimación de arriba hacia abajo, al usar el tamaño como la entrada principal del modelo, hemos reemplazado el problema de estimar el esfuerzo por el de estimar el tamaño.

Entonces uno puede preguntar: ¿por qué no hacer directamente la estimación del esfuerzo en lugar de la estimación del tamaño?

La respuesta es que la estimación del tamaño suele ser más fácil que la estimación directa del esfuerzo.

Para estimar el tamaño, el sistema generalmente se divide en los componentes que probablemente tenga.

Una vez que se conocen los componentes del sistema, como estimar algo sobre una unidad pequeña suele ser mucho más fácil que hacerlo para un sistema más grande, los tamaños de los componentes pueden estimarse con bastante precisión.

Una vez que se tienen las estimaciones de tamaño de los componentes, para obtener la estimación total de tamaño del sistema, las estimaciones de todos los componentes pueden sumarse.

Esta propiedad similar **no se cumple** para la estimación del esfuerzo, ya que el esfuerzo para desarrollar un sistema **no es** la suma del esfuerzo para desarrollar sus componentes (ya que se necesita esfuerzo adicional para integración y otras actividades al construir un sistema a partir de componentes desarrollados).

Esta característica clave —que la propiedad del sistema sea la suma de las propiedades de sus partes— se cumple para el tamaño, pero no para el esfuerzo, y es la razón principal por la cual la estimación del tamaño se considera más fácil que la estimación del esfuerzo.

Con los modelos de arriba hacia abajo, si la estimación del tamaño es inexacta, la estimación del esfuerzo producida por los modelos también será inexacta.

Por lo tanto, es importante obtener buenas estimaciones para el tamaño del software.

No existe un método “simple” conocido para estimar el tamaño con precisión.

Al estimar el tamaño del software, la mejor forma puede ser obtener la mayor cantidad posible de detalles sobre el software que se desarrollará y ser conscientes de nuestros sesgos al estimar el tamaño de los distintos componentes.

Al obtener detalles y usarlos para la estimación del tamaño, las estimaciones probablemente estarán más cerca del tamaño real del software final.

En general, suele haber una tendencia de las personas a subestimar el tamaño del software [20].

Un enfoque algo diferente para la estimación del esfuerzo es el **enfoque de abajo hacia arriba (bottom-up)**.

En este enfoque, el proyecto primero se divide en tareas, y luego se obtienen estimaciones para las diferentes tareas del proyecto.

A partir de las estimaciones de las distintas tareas, se determina la estimación total.

Es decir, la estimación general del proyecto se deriva de las estimaciones de sus partes.

Este tipo de enfoque también se llama **estimación basada en actividades**.

Esencialmente, en este enfoque, el tamaño y la complejidad del proyecto se capturan en el conjunto de tareas que el proyecto debe realizar.

El enfoque de abajo hacia arriba permite una estimación directa del esfuerzo; una vez que el proyecto se divide en tareas más pequeñas, es posible estimar directamente el esfuerzo requerido para ellas, especialmente si las tareas son relativamente pequeñas.

Un riesgo de los métodos de abajo hacia arriba es que se puede omitir alguna actividad importante en la lista de tareas.

Además, estimar directamente el esfuerzo de algunas tareas de soporte, como la gestión del proyecto, que abarcan todo el proyecto, puede ser difícil.

Tanto los enfoques de arriba hacia abajo como los de abajo hacia arriba requieren información sobre el proyecto: el **tamaño** (para los enfoques de arriba hacia abajo) o una **lista de tareas** (para los de abajo hacia arriba).

En muchos sentidos, estos enfoques son complementarios, y a menudo puede ser conveniente determinar el esfuerzo utilizando ambos enfoques y luego usar esas estimaciones para obtener la estimación final.

5.2.3 Un Enfoque de Estimación de Abajo Hacia Arriba

Si ya se ha desarrollado la arquitectura del sistema que se va a construir y se cuenta con información previa sobre cómo se distribuye el esfuerzo entre las distintas fases, entonces el enfoque de abajo hacia arriba no necesita listar completamente todas las tareas, y es posible aplicar un método menos tedioso.

Aquí describimos uno de esos enfoques utilizado en una organización comercial.

En este enfoque, primero se determinan los programas principales (o unidades o módulos) del software que se va a construir.

Cada unidad de programa se clasifica como **simple**, **media** o **compleja**, según ciertos criterios. Para cada categoría de complejidad, se define un esfuerzo promedio para la codificación (y las pruebas unitarias).

Este esfuerzo estándar de codificación puede basarse en datos anteriores de un proyecto similar, en algunas guías establecidas, o en una combinación de ambos.

Una vez que se conoce el número de unidades en las tres categorías de complejidad y se ha seleccionado el esfuerzo estimado de codificación para cada tipo de programa, se obtiene el **esfuerzo total de codificación** del proyecto.

A partir del esfuerzo de codificación, se determina el esfuerzo requerido para las demás fases y actividades como un **porcentaje** del esfuerzo de codificación.

Usando información sobre el desempeño pasado del proceso, se decide la posible **distribución del esfuerzo** entre las distintas fases de este proyecto, y luego se utiliza para estimar el esfuerzo de las demás fases y actividades.

A partir de estas estimaciones, se obtiene el **esfuerzo total del proyecto**.

Este enfoque permite una combinación equilibrada de **experiencia y datos históricos**.

Si no se dispone de datos previos adecuados (por ejemplo, al iniciar un tipo de proyecto nuevo), puede estimarse el esfuerzo de codificación basándose en la experiencia, una vez que se haya especificado la naturaleza de los distintos tipos de unidades.

Con esta estimación, se pueden obtener las estimaciones de las demás actividades trabajando con una **distribución razonable o estándar del esfuerzo**.

Esta estrategia también puede contemplar actividades que a veces son difíciles de enumerar al inicio, pero que consumen esfuerzo, asignando un presupuesto al rubro “**otras**” o “**misceláneas**”.

El procedimiento de estimación puede resumirse en la siguiente secuencia de pasos:

1. Identificar los módulos del sistema y clasificarlos como simples, medios o complejos.
2. Determinar el esfuerzo promedio de codificación para los módulos simples, medios y complejos.
3. Obtener el esfuerzo total de codificación utilizando el esfuerzo de codificación de los distintos tipos de módulos y sus cantidades.
4. Usando la distribución de esfuerzo de proyectos similares, estimar el esfuerzo para las demás tareas y el esfuerzo total.
5. Refinar las estimaciones según los factores específicos del proyecto.

Este procedimiento combina de manera prudente los **datos históricos** (en forma de distribución del esfuerzo) y la **experiencia de los programadores**.

Este enfoque también es **simple** y se asemeja a cómo muchos de nosotros planificamos cualquier proyecto.

Por esta razón, para **proyectos pequeños**, muchas personas encuentran este enfoque **natural y cómodo**.

Cabe señalar que este método de clasificar los programas en unas pocas categorías y usar un esfuerzo promedio de codificación para cada una se emplea **solo para la estimación del esfuerzo**.

En la **planificación detallada**, cuando un gerente de proyecto asigna cada unidad a un miembro del equipo para su codificación y asigna tiempo para la actividad, **se tienen en cuenta las características específicas de la unidad**, otorgando **más o menos tiempo** según corresponda.

5.2.4 Modelo COCOMO

Un modelo de arriba hacia abajo puede depender de muchos factores diferentes, en lugar de depender solo de una variable, dando lugar a modelos multivariantes.

Un enfoque para construir modelos multivariantes es comenzar con una estimación inicial determinada utilizando las ecuaciones estáticas de modelos de una sola variable, que dependen del tamaño, y luego ajustar las estimaciones basándose en otras variables.

Este enfoque implica que el tamaño es el factor principal para el costo; otros factores tienen un efecto menor.

Aquí discutiremos uno de esos modelos llamado **Modelo Constructivo de Costos (COCOMO)** desarrollado por **Boehm**.

Este modelo también estima el esfuerzo total en términos de **persona-meses** (*person-months*).

Los pasos básicos en este modelo son:

1. Obtener una estimación inicial del esfuerzo de desarrollo a partir de la estimación de miles de líneas de código fuente entregadas (**KLOC**).
2. Determinar un conjunto de **15 factores multiplicadores** provenientes de diferentes atributos del proyecto.
3. Ajustar la estimación del esfuerzo multiplicando la estimación inicial con todos los factores multiplicadores.

La estimación inicial (también llamada **estimación nominal**) se determina mediante una ecuación del tipo usado en los modelos estáticos de una sola variable, usando **KLOC** como la medida del tamaño.

Para determinar el esfuerzo inicial **E_i** en persona-meses, se usa una ecuación del tipo:

$$E_i = a \times (KLOC)^b \quad E_i = a \times (KLOC)^b$$

El valor de las constantes **a** y **b** depende del tipo de proyecto.

En COCOMO, los proyectos se categorizan en tres tipos: **orgánico**, **semisuspendido (semidetached)** y **embebido (embedded)**.

Estas categorías caracterizan, de manera aproximada, la complejidad del proyecto:

- Los proyectos **orgánicos** son aquellos relativamente sencillos y desarrollados por un equipo pequeño.
- Los **embebidos** son ambiciosos y novedosos, con restricciones severas impuestas por el entorno y altos requerimientos en aspectos como interfase y confiabilidad.

El valor de las constantes para un modelo de costos depende del proceso y debe determinarse a partir de datos pasados.

COCOMO, en cambio, ha proporcionado valores **“globales”** para estas constantes.

Estos valores deben considerarse como un punto de partida hasta que se disponga de datos de algunos proyectos.

Con datos de proyectos, el valor de las constantes puede determinarse mediante **análisis de regresión**.

Hay **15 atributos diferentes**, llamados **atributos conductores de costo (cost driver attributes)**, que determinan los factores multiplicadores.

Estos factores dependen de atributos del **producto**, la **computadora**, el **personal** y la **tecnología** (llamados **atributos del proyecto**).

Ejemplos de atributos son:

- **Confiabilidad de software requerida (RELY)**,
- **Complejidad del producto (CPLX)**,
- **Capacidad del analista (ACAP)**,
- **Experiencia en la aplicación (AEXP)**,
- **Uso de herramientas modernas (TOOL)**,
- y **Calendario de desarrollo requerido (SCHD)**.

Cada conductor de costo tiene una **escala de calificación**, y para cada calificación se proporciona un **factor multiplicador**.

Por ejemplo, para el atributo del producto **RELY**, la escala de calificación es: **muy baja, baja, nominal, alta y muy alta** (y en algunos casos **extra alta**).

Los factores multiplicadores para estas calificaciones son **0.75, 0.88, 1.00, 1.15 y 1.40**, respectivamente.

Entonces, si el requisito de confiabilidad para el proyecto se juzga **bajo**, el factor multiplicador es **0.75**; mientras que si se juzga **muy alto**, el factor es **1.40**.

Los atributos y sus factores multiplicadores para las diferentes calificaciones se muestran en la **Tabla 5.1 [20, 21]**.

El enfoque COCOMO también proporciona **guías** para evaluar la calificación de los diferentes atributos [20].

Los factores multiplicadores para los 15 conductores de costo se multiplican entre sí para obtener el **factor de ajuste del esfuerzo (EAF, Effort Adjustment Factor)**.

La estimación final del esfuerzo, **E**, se obtiene multiplicando la estimación inicial por el EAF. Es decir:

$$E = EAF \times E_i \quad E = EAF \times E_i$$

Por este método, se puede estimar el costo total del proyecto.

Para fines de **planificación y monitoreo**, también son deseables las estimaciones del esfuerzo requerido para las diferentes fases.

En COCOMO, el esfuerzo para una fase se define como un **porcentaje del esfuerzo total**.

El porcentaje del esfuerzo total invertido en una fase varía según el **tipo y tamaño del proyecto**.

Los porcentajes para un proyecto de software orgánico se muestran en la **Tabla 5.2**.

Usando esta tabla, se puede determinar la estimación del esfuerzo requerido para cada fase a partir de la estimación total del esfuerzo.

Por ejemplo, si la estimación total del esfuerzo para un sistema de software orgánico es de **20 PM** y la estimación de tamaño es **20 KLOC**, entonces el porcentaje de esfuerzo para la fase de **codificación y prueba unitaria** será:

$$40 + \frac{(38 - 40)}{(32 - 8)} \times 20 = 39\% \quad 40 + \frac{(38 - 40)}{(32 - 8)} \times 20 = 39\%$$

La estimación para el esfuerzo necesario en esta fase es entonces **7.8 PM**.

Esta tabla no incluye el costo de los requisitos como porcentaje del costo total estimado, porque la **planificación del proyecto (y la estimación de costos)** se realiza **después** de que los requisitos estén completos.

En COCOMO, el **diseño detallado** y la **codificación y prueba unitaria** a veces se combinan en una sola fase llamada la **fase de programación**.

Como ejemplo, supongamos que debe diseñarse un sistema para la **automatización de oficinas**. A partir de los requisitos, queda claro que habrá **cuatro módulos principales** en el sistema: **entrada de datos, actualización de datos, consulta, y generador de reportes**.

También es claro, a partir de los requisitos, que este proyecto caerá en la **categoría orgánica**.

5.3 Planificación y Dotación de Personal del Proyecto

Una vez que se estima el esfuerzo, son posibles varios **cronogramas** (o duraciones del proyecto), dependiendo de la cantidad de **recursos** (personas) asignadas al proyecto.

Por ejemplo, para un proyecto cuyo esfuerzo estimado es de **56 persona-meses**, es posible un cronograma total de **8 meses** con **7 personas**.

También es posible un cronograma de **7 meses** con **8 personas**, o un cronograma de aproximadamente **9 meses** con **6 personas**.

Sin embargo, como es bien sabido, **la mano de obra y los meses no son completamente intercambiables** en un proyecto de software.

No se puede obtener un cronograma simplemente a partir de la estimación total del esfuerzo, decidiendo un tamaño promedio del equipo y luego determinando el tiempo total requerido dividiendo el esfuerzo total por el tamaño promedio del personal.

Brooks señaló que las personas y los meses (tiempo) **no son intercambiables**.

Según Brooks [25]:

“... los hombres y los meses son intercambiables solo para actividades que no requieren comunicación entre las personas, como sembrar trigo o cosechar algodón.
Esto no es ni siquiera aproximadamente cierto en el software...”

Por ejemplo, en este caso, **un cronograma de 1 mes con 56 personas no es posible**, aunque el esfuerzo total coincida con el requerido.

De manera similar, nadie ejecutaría el proyecto en **28 meses con 2 personas**.

En otras palabras, una vez que el esfuerzo está fijado, hay **cierta flexibilidad** al establecer el cronograma ajustando adecuadamente el personal del proyecto, **pero esta flexibilidad no es ilimitada**.

Los **datos empíricos** también sugieren que **ninguna ecuación simple** entre esfuerzo y cronograma se ajusta bien [127].

En un proyecto, la actividad de **planificación del cronograma (scheduling)** puede dividirse en dos subactividades:

1. Determinar el **cronograma general** (la duración total del proyecto) con los **principales hitos**, y
2. Desarrollar el **cronograma detallado** de las diversas tareas.

5.3.1 Planificación General

Un método para determinar el cronograma general normal (o nominal) es establecerlo como una función del esfuerzo.

Cualquier función de este tipo debe determinarse a partir de datos de proyectos completados, utilizando técnicas estadísticas como el ajuste de una curva de regresión sobre el diagrama de dispersión obtenido al graficar el esfuerzo y el cronograma de proyectos pasados.

Esta curva suele ser no lineal, ya que el cronograma no crece linealmente con el esfuerzo. Muchos modelos siguen este enfoque.

Por ejemplo, la División de Sistemas Federales de IBM encontró que la duración total (M), en meses calendario, puede estimarse mediante una relación de este tipo.

En el modelo COCOMO, la ecuación para estimar el cronograma de un software de tipo orgánico es:

$$M = 2.5 \times E^{0.38} \quad M = 2.5 \times E^{0.38}$$

Debe quedar claro que el cronograma no depende únicamente del esfuerzo.

Por lo tanto, el cronograma determinado de esta manera no está realmente fijado, pero puede usarse como una guía o verificación de la razonabilidad del cronograma propuesto, que podría decidirse en base a otros factores.

Una regla práctica, llamada el “control de la raíz cuadrada” (square root check), se usa a veces para verificar el cronograma de proyectos medianos [97].

Esta regla sugiere que el cronograma propuesto puede ser aproximadamente igual a la raíz cuadrada del esfuerzo total en persona-meses.

Dicho cronograma puede cumplirse si se asignan recursos adecuados al proyecto.

Por ejemplo, si el esfuerzo estimado es de 50 persona-meses, un cronograma de aproximadamente 7 u 8 meses sería adecuado.

A partir de esta estimación macro del cronograma, debemos determinar el cronograma de los principales hitos del proyecto.

Para determinar estos hitos, primero debemos entender cómo ocurre el incremento progresivo del personal (manpower ramp-up) que típicamente se da en un proyecto.

El número de personas en un proyecto de software tiende a seguir la curva de Rayleigh [126, 127]. Es decir, al principio y al final trabajan pocas personas en el proyecto; el tamaño máximo del equipo (PTS, Peak Team Size) se alcanza aproximadamente en la mitad del proyecto.

Esto ocurre porque solo se necesitan unas pocas personas en las fases iniciales de análisis de requisitos y diseño.

La necesidad de recursos humanos alcanza su punto máximo durante la codificación y las pruebas unitarias, y luego vuelve a disminuir durante las fases de integración y pruebas del sistema.

A menudo, el nivel de personal no cambia continuamente, sino que se usa una aproximación de la curva de Rayleigh:

se asignan pocas personas al inicio, se alcanza el máximo durante la fase de codificación y luego el equipo se reduce para las fases finales de integración y pruebas del sistema.

Si consideramos las tres fases principales —diseño y análisis, construcción, y pruebas—, el aumento de personal en los proyectos típicamente se asemeja a la función mostrada en la Figura 5.2 [97].

Por simplicidad en la planificación, especialmente en proyectos más pequeños, a menudo se asigna todo el personal necesario desde el inicio.

Este enfoque puede generar que algunas personas estén desocupadas al comienzo o al final, pero ese tiempo libre suele aprovecharse en actividades de apoyo al proyecto, como capacitación o documentación.

Dado el esfuerzo estimado para una fase, se puede determinar la duración de dicha fase si se conoce el patrón de incremento del personal.

En estas tres fases principales, el porcentaje del cronograma ocupado por la fase de construcción suele ser menor que su porcentaje de esfuerzo, ya que involucra a más personas.

Por el contrario, el porcentaje del cronograma de las fases de diseño y pruebas supera su porcentaje de esfuerzo.

El cronograma exacto depende de cómo se planifique el aumento del personal y de cuántos recursos pueden utilizarse eficazmente en cada fase del proyecto.

En términos generales:

- Diseño: ocupa alrededor de una cuarta parte del cronograma,
- Construcción: consume aproximadamente la mitad, y
- Integración y pruebas del sistema: utilizan el cuarto restante.

El modelo COCOMO propone una distribución más específica:

- 19% para el diseño,
- 62% para la programación, y
- 18% para la integración.

5.3.2 Planificación Detallada

Una vez que los hitos y los recursos están definidos, es momento de establecer la planificación detallada.

Para elaborar los cronogramas detallados, las tareas principales fijadas durante la planificación de los hitos se dividen en actividades más pequeñas y programables de manera jerárquica.

Por ejemplo, la fase de diseño detallado puede dividirse en tareas como:

- desarrollar el diseño detallado de cada módulo,
- revisar cada diseño detallado,
- corregir los defectos encontrados, y así sucesivamente.

Para cada tarea detallada, el gerente del proyecto estima el tiempo necesario para completarla y asigna un recurso adecuado, de manera que el cronograma general pueda cumplirse.

En cada nivel de refinamiento, el gerente del proyecto determina el esfuerzo total de la tarea global a partir del cronograma detallado y lo compara con las estimaciones de esfuerzo.

Si este cronograma detallado no es consistente con el cronograma y las estimaciones globales de esfuerzo, el cronograma detallado debe modificarse.

Si se descubre que el mejor cronograma detallado posible no puede ajustarse al esfuerzo y al cronograma establecidos en los hitos, entonces deben revisarse las estimaciones previas.

Por lo tanto, la planificación es un proceso iterativo.

En general, el gerente del proyecto refina las tareas hasta alcanzar un nivel en el que cada actividad de nivel más bajo pueda programarse para ocupar solo unos pocos días del tiempo de un único recurso.

Actividades relacionadas con tareas como gestión del proyecto, coordinación, administración de bases de datos y gestión de configuración también pueden incluirse en el cronograma, aunque tengan menos efecto directo en su determinación, ya que son tareas continuas más que actividades programables.

Aun así, estas consumen recursos, por lo que suelen incluirse en el cronograma del proyecto.

Rara vez un gerente de proyecto completa el cronograma detallado del proyecto entero de una sola vez.

Una vez que el cronograma general está fijado, la planificación detallada de una fase puede hacerse al inicio de esa fase.

Para la planificación detallada, herramientas como Microsoft Project o una hoja de cálculo pueden ser muy útiles.

Para cada actividad de nivel más bajo, el gerente del proyecto especifica:

- el esfuerzo,
- la duración,
- la fecha de inicio,
- la fecha de finalización, y
- los recursos asignados.

También pueden especificarse dependencias entre actividades, ya sea debido a una dependencia inherente (por ejemplo, solo se puede realizar una prueba unitaria de un programa después de haberlo codificado), o a una dependencia relacionada con los recursos (la misma persona tiene asignadas dos tareas).

A partir de estas herramientas, se puede determinar el esfuerzo total y el cronograma de las tareas de nivel superior.

Un cronograma detallado de proyecto nunca es estático.

Pueden requerirse cambios si el progreso real del proyecto difiere de lo planeado, si se agregan nuevas tareas en respuesta a solicitudes de cambio, o debido a otras situaciones imprevistas.

Las modificaciones se realizan siempre que sea necesario.

El cronograma final, generalmente mantenido mediante alguna herramienta adecuada, suele ser el documento del plan de proyecto más “vivo”.

Durante el proyecto, si los planes deben cambiarse y deben añadirse actividades adicionales, una vez tomada la decisión, los cambios deben reflejarse en el cronograma detallado, ya que este refleja las tareas realmente planificadas para realizarse.

Por esta razón, el cronograma detallado se convierte en el documento principal para rastrear las actividades y el progreso del cronograma.

5.4 Plan de Gestión de la Configuración del Software

A partir de las discusiones anteriores sobre la gestión de la configuración del software, debería quedar algo claro qué deben contener los planes de SCM (Software Configuration Management). El plan de SCM, al igual que otros planes, debe identificar las actividades que deben realizarse, proporcionar pautas para llevarlas a cabo y asignar recursos para ellas.

La planificación de la gestión de la configuración implica identificar los ítems de configuración y especificar los procedimientos que se usarán para controlar e implementar los cambios en ellos. Hemos discutido la planificación de CM (Configuration Management) al hablar del proceso de CM en el Capítulo 2.

Para resumir, el controlador de configuración realiza la planificación de CM cuando el proyecto ha sido iniciado y se conocen el entorno operativo y las especificaciones de requisitos.

Las actividades en esta etapa incluyen lo siguiente [97]:

- Identificar los ítems de configuración, incluidos los proporcionados por el cliente y los adquiridos.
- Definir un esquema de nombres para los ítems de configuración.
- Definir la estructura de directorios necesaria para la CM.
- Definir los procedimientos de gestión de versiones y los métodos para rastrear cambios en los ítems de configuración.
- Definir las restricciones de acceso.
- Definir los procedimientos de control de cambios.
- Identificar y definir la responsabilidad del CC (Configuration Controller).
- Identificar los puntos en los que se crearán las líneas base (baselines).
- Definir un procedimiento de respaldo y un procedimiento de reconciliación, si es necesario.
- Definir un procedimiento de liberación.

El resultado de esta fase es el plan de CM.

Un ejemplo de un plan de CM completo para un proyecto en una organización comercial se proporciona en.

5.5 Planeamiento de Control de Calidad

Anteriormente, en el Capítulo 1, discutimos la noción de calidad del software.

Aunque existen diferentes dimensiones de la calidad, en la práctica, la gestión de la calidad suele girar en torno a los defectos.

Por lo tanto, usamos la “densidad de defectos entregados” —el número de defectos por unidad de tamaño en el software entregado— como definición de calidad.

Esta definición es actualmente el estándar de facto en la industria.

Por defecto entendemos algo en el software que provoca que éste se comporte de una manera inconsistente con los requisitos o las necesidades del cliente.

Un defecto en el software implica que su eliminación resultará en algún cambio realizado al software.

Para asegurar que el producto final sea de alta calidad, deben realizarse algunas actividades de control de calidad (QC, Quality Control) a lo largo del desarrollo.

Una tarea de QC es aquella cuyo propósito principal es identificar defectos.

El propósito de un plan de calidad en un proyecto es especificar las actividades que deben realizarse para identificar y eliminar defectos, así como las herramientas y métodos que pueden usarse para ese propósito.

En un proyecto, es muy poco probable que los productos intermedios sean de baja calidad, pero el producto final sea de alta calidad.

Por lo tanto, para garantizar que el software entregado sea de buena calidad, es esencial asegurar que todos los productos de trabajo —como la especificación de requisitos, el diseño y el plan de pruebas— también sean de buena calidad.

Por esta razón, un plan de calidad debe contener actividades de calidad a lo largo de todo el proyecto.

El plan de calidad especifica las tareas que deben realizarse en diferentes momentos del proyecto para mejorar la calidad del software mediante la eliminación de defectos, y cómo deben gestionarse. Antes de discutir esto, primero comprendamos el ciclo de inyección y eliminación de defectos.

5.5.1 Ciclo de Inyección y Eliminación de Defectos

El desarrollo de software es una actividad altamente orientada a las personas, y por lo tanto, propensa a errores.

Los defectos pueden inyectarse en el software en cualquier etapa durante su evolución.

Es decir, durante la transformación de las necesidades del usuario en software que satisfaga esas necesidades, los defectos pueden ser introducidos en todas las actividades de transformación que se realicen.

Estas etapas de inyección son principalmente:

- la especificación de requisitos,
- el diseño de alto nivel,

- el diseño detallado, y
- la codificación.

Para lograr software de alta calidad, el producto final debe tener la menor cantidad de defectos posible.

Por lo tanto, para entregar software de alta calidad, es necesario eliminar activamente los defectos mediante las actividades de control de calidad (QC).

Las actividades de QC para la eliminación de defectos incluyen:

- revisiones de requisitos,
- revisiones de diseño,
- revisiones de código,
- pruebas unitarias,
- pruebas de integración,
- pruebas de sistema, y
- pruebas de aceptación.

(No se incluyen las revisiones de los documentos de planificación, aunque tales revisiones también ayudan a mejorar la calidad del software).

La Figura 5.3 muestra el proceso de inyección y eliminación de defectos.

La tarea de la gestión de calidad es planificar actividades adecuadas de control de calidad y luego ejecutarlas y controlarlas correctamente para que se logren los objetivos de calidad del proyecto.

En relación con el control de calidad, a menudo se utilizan los términos verificación y validación.

- **Verificación** es el proceso de determinar si los productos de una fase dada del desarrollo de software cumplen con las especificaciones establecidas durante la fase anterior.
- **Validación** es el proceso de evaluar el software al final del desarrollo para asegurar su cumplimiento con los requisitos del software.

Claramente, para lograr alta confiabilidad, necesitamos realizar ambas actividades.

En conjunto, estas se denominan frecuentemente actividades de V&V (*Verification and Validation*).

Las principales actividades de V&V en el desarrollo de software son la inspección y las pruebas (tanto estáticas como dinámicas).

El plan de calidad identifica las diferentes tareas de V&V para las distintas fases y especifica cómo contribuyen estas tareas a los objetivos de V&V del proyecto.

También se especifican los métodos que se usarán para realizar estas actividades de V&V, las responsabilidades y los hitos de cada una, los insumos y productos de cada tarea de V&V, y los criterios para evaluar los resultados.

5.5.2 Enfoques para la Gestión de la Calidad

Las revisiones y las pruebas son las dos actividades más comunes de control de calidad (QC). Mientras que las revisiones son procesos estructurados y orientados a las personas, las pruebas consisten en ejecutar el software (o partes de él) con el objetivo de identificar defectos.

En el enfoque procedimental de la gestión de calidad, se planifican procedimientos y guías para las actividades de revisión y prueba.

Durante la ejecución del proyecto, estas actividades se llevan a cabo de acuerdo con los procedimientos definidos.

En resumen, el enfoque procedimental consiste en ejecutar ciertos procesos en momentos específicos con el fin de detectar defectos.

Sin embargo, este enfoque no permite afirmar qué porcentaje de defectos fue eliminado ni cuál es la calidad del software luego de completar los procedimientos.

En otras palabras, simplemente ejecutar un conjunto de procedimientos de eliminación de defectos no proporciona una base para evaluar su efectividad ni para medir la calidad del código final.

Además, este enfoque depende en gran medida de la calidad del procedimiento y de la calidad de su ejecución.

Por ejemplo, si la planificación de las pruebas se realiza cuidadosamente y el plan de pruebas es revisado exhaustivamente, la calidad del software después de las pruebas será mejor que si las pruebas se realizan con un plan mal diseñado o no revisado.

Para evaluar mejor la efectividad de los procesos de detección de defectos, es necesario aplicar una evaluación basada en métricas.

A partir del análisis de los datos, se puede decidir si se necesitan más pruebas o revisiones.

Cuando se aplican controles cuantitativos durante el proyecto —basados en datos numéricos— para alcanzar objetivos cuantitativos de calidad, se dice que se está aplicando un enfoque de gestión cuantitativa de la calidad.

Este tipo de gestión es un concepto avanzado, y aquí se describe solo brevemente.

► Enfoque de predicción de defectos

En este método, el objetivo de calidad se establece en términos de la densidad de defectos entregados.

Se fijan metas intermedias estimando la cantidad de defectos que podrían ser identificados por las distintas actividades de detección de defectos.

Luego, se compara el número real de defectos encontrados con los niveles estimados.

La efectividad de este enfoque depende de qué tan bien se pueden predecir los niveles de defectos en las distintas etapas del proyecto.

Este método requiere datos históricos para hacer las estimaciones (un ejemplo puede encontrarse en [97]).

► Enfoque de control estadístico del proceso (SPC)

En este método, se establecen expectativas de desempeño para los distintos procesos de control de calidad —como las revisiones y las pruebas— en forma de límites de control.

Si el desempeño real de una actividad de QC no se mantiene dentro de los límites establecidos, la situación se analiza y se toman acciones correctivas.

Los límites de control son similares a las predicciones de niveles de defectos basadas en el desempeño pasado, pero también pueden utilizarse para monitorear la calidad en un nivel más detallado, por ejemplo, en la revisión o prueba unitaria de un módulo.

5.5.3 Plan de Calidad

El plan de calidad de un proyecto es el documento que guía las actividades de calidad dentro del mismo.

El nivel de sofisticación del plan depende del tipo de datos o de los modelos de predicción disponibles.

En su forma más simple, el plan de calidad especifica las tareas de control de calidad (QC) que se llevarán a cabo en el proyecto.

Normalmente, estas tareas se incluyen como actividades programables dentro del cronograma detallado del proyecto.

Por ejemplo, el plan puede indicar:

- qué documentos serán inspeccionados,
- qué partes del código serán revisadas, y
- qué niveles de prueba se realizarán.

El plan se vuelve mucho más útil si incluye una estimación de los niveles de defectos esperados para las distintas tareas de control de calidad, ya que esos valores pueden utilizarse para monitorear la calidad del proyecto a medida que avanza.

Gran parte del plan de calidad gira en torno a las pruebas y las revisiones.

Las pruebas se analizarán con más detalle en un capítulo posterior.

La efectividad de las revisiones depende en gran medida de cómo se llevan a cabo.

Un proceso particular de revisión llamado inspección ya fue discutido en el Capítulo 2.

Este proceso puede aplicarse a cualquier producto de trabajo, como:

- las especificaciones de requisitos,
- los documentos de diseño,
- los planes de prueba,
- los planes de proyecto, y
- el código fuente.

5.6 Gestión de Riesgos

Un proyecto de software es una empresa compleja.

Los eventos imprevistos pueden afectar negativamente el costo, el cronograma o la calidad de un proyecto.

La **gestión de riesgos** es un **intento de minimizar las probabilidades de fracaso causadas por eventos no planificados**.

El **objetivo** de la gestión de riesgos **no es evitar proyectos con riesgos, sino minimizar el impacto de los riesgos en los proyectos que se llevan a cabo**.

Un riesgo es un evento probabilístico: puede ocurrir o no ocurrir.

Por esta razón, frecuentemente existe una tendencia optimista a ignorar los riesgos o esperar que no se materialicen.

Factores sociales y organizativos también pueden estigmatizar los riesgos y desalentar su identificación clara.

Esta actitud puede causar problemas si los riesgos realmente se materializan, lo cual es muy probable en proyectos grandes.

No es sorprendente, entonces, que la gestión de riesgos se considere una de las mejores prácticas para manejar grandes proyectos de software.

Su relevancia se destacó por primera vez con el tutorial de Boehm sobre gestión de riesgos.

Desde entonces, varios libros se han enfocado en la gestión de riesgos en software.

5.6.1 Conceptos de Gestión de Riesgos

El riesgo se define como la exposición a la posibilidad de daño o pérdida.

Es decir, el riesgo implica que algo negativo puede ocurrir.

En el contexto de proyectos de software, “negativo” significa que hay un efecto adverso sobre el costo, la calidad o el cronograma.

La gestión de riesgos es el área que intenta asegurar que el impacto de los riesgos en el costo, la calidad y el cronograma sea mínimo.

Se puede considerar que la gestión de riesgos trata sobre la posibilidad y la ocurrencia real de eventos que no son “regulares” o comúnmente esperados, es decir, que son probabilísticos.

Los eventos comúnmente esperados, como licencias del personal o cambios menores en los requisitos, se manejan mediante la gestión normal de proyectos.

En este sentido, la gestión de riesgos comienza donde termina la gestión normal de proyectos. Trata sobre eventos infrecuentes, parcialmente fuera del control de la gerencia y que pueden tener un impacto importante en el proyecto.

La mayoría de los proyectos tienen riesgos.

La idea de la gestión de riesgos es minimizar la posibilidad de que los riesgos se materialicen, si es posible, o minimizar sus efectos si efectivamente ocurren.

Por ejemplo, al construir un edificio, existe un riesgo de que el edificio pueda colapsar por un terremoto.

Si se trata de un complejo residencial grande, el costo potencial si el riesgo se materializa puede ser

enorme.

Este riesgo se puede reducir ubicando el edificio en una zona no sísmica.

Si esto no es aceptable, entonces los efectos del riesgo se minimizan construyendo el edificio adecuadamente (como se hace en Japón y California).

En cambio, si se construye un pequeño depósito, no se aplicarán tales medidas, ya que el impacto financiero y otros efectos de un terremoto son bajos y no justifican acciones especiales.

Queda claro que la **gestión de riesgos** debe ocuparse de:

- **Identificar los eventos indeseables que pueden ocurrir.**
- **Evaluar la probabilidad de que ocurran.**
- **Determinar la pérdida si el evento indeseable ocurre.**

Una vez que se conoce esto, se pueden **formular estrategias para reducir la probabilidad de materialización del riesgo o reducir sus efectos si se materializa.**

Por lo tanto, la gestión de riesgos gira en torno a:

Evaluación de riesgos

Control de riesgos

Para cada una de estas actividades principales, deben realizarse subactividades específicas.

Una descomposición de estas actividades se muestra en la Figura 5.4.

5.6.2 Evaluación de Riesgos

La evaluación de riesgos es una actividad que **debe realizarse durante la planificación del proyecto.** Esto implica identificar los riesgos, analizarlos y priorizarlos según el análisis.

Debido a la naturaleza de un proyecto de software, las incertidumbres son mayores al inicio del proyecto, de manera similar a la estimación de costos.

Por ello, aunque la evaluación de riesgos **debe realizarse durante todo el proyecto, es más necesaria en las fases iniciales.**

El **objetivo** de la evaluación de riesgos es **priorizar los riesgos para que la atención y los recursos se enfoquen en los elementos más riesgosos.**

Identificación de riesgos

La identificación de riesgos es el primer paso y consiste en detectar todos los riesgos posibles para un proyecto específico.

Estos **riesgos dependen del proyecto**, y su identificación **requiere visualizar lo que podría salir mal.**

Métodos que pueden ayudar incluyen:

- Listas de verificación de posibles riesgos (checklists)
- Encuestas
- Reuniones y sesiones de lluvia de ideas (brainstorming)
- Revisión de planes, procesos y productos de trabajo

Las listas de riesgos frecuentes son probablemente la herramienta más común para la identificación. La mayoría de las organizaciones preparan una lista de riesgos típicos basada en proyectos anteriores, que sirve como punto de partida para el proyecto actual.

Boehm, basándose en encuestas a gerentes de proyectos experimentados, elaboró una lista de los 10 riesgos principales que probablemente comprometan el éxito de un proyecto de software.

Aunque los riesgos son específicos de cada proyecto, esta lista es un buen punto de partida.

- **Riesgo principal:** déficit de personal. Esto ocurre al tener menos personas de las necesarias o falta de habilidades específicas. Se gestiona contratando talento adecuado, ajustando las necesidades del proyecto a las capacidades del personal, ofreciendo capacitación y manteniendo personal clave en áreas críticas.
- **Segundo riesgo:** cronogramas y presupuestos poco realistas. Sucede con frecuencia debido a presiones de negocio o decisiones de la gerencia. Puede haber subestimación por inexperiencia u optimismo.
- **Riesgos relacionados con los requisitos:**
 - Desarrollo de software incorrecto si el análisis de requisitos no se realiza correctamente o si el desarrollo comienza demasiado pronto.
 - Interfaces de usuario mal diseñadas que requieren retrabajo extensivo.
 - Gold plating: agregar funciones marginalmente útiles, lo que consume tiempo y recursos sin retorno significativo.
 - Cambios frecuentes en requisitos, especialmente si el cliente no tiene claridad sobre sus necesidades, lo que impacta el costo, especialmente en fases avanzadas.
- **Riesgos relacionados con componentes externos:**
 - Retrasos si un componente externo no está disponible a tiempo.
 - Mala calidad o incompatibilidad de componentes externos.
- **Riesgos tecnológicos:** uso de tecnología poco desarrollada que puede provocar fallas por sobrepasar las capacidades de ingeniería de software.

Métodos para identificación de riesgos

Además de checklists, se pueden usar:

- **Análisis de decisiones:** cuestionar decisiones clave del proyecto.
- **Análisis de supuestos:** identificar suposiciones optimistas sobre el proyecto, como que nada fallará o que todos los recursos estarán disponibles.
- **Descomposición:** dividir el proyecto en partes claramente definidas y analizar módulos críticos (20% de los módulos suelen causar 80% de los problemas).

Análisis y priorización de riesgos

La identificación de riesgos solo enumera los eventos indeseables.

Para analizar riesgos, se debe estimar:

- La probabilidad de ocurrencia

- La pérdida potencial si el riesgo se materializa

Esto se puede hacer mediante:

- Discusión basada en experiencia y entendimiento
- Modelos de costo (como COCOMO) para evaluar riesgos de costo y cronograma
- Estimaciones de peor caso para tamaño y drivers de costo

Otros enfoques incluyen:

- Análisis de decisiones
- Análisis de redes (dependencias de tareas críticas y probabilidad de no completarlas)
- Análisis de factores de calidad (confiabilidad, usabilidad)
- Análisis de rendimiento mediante simulación, si hay restricciones fuertes de desempeño

Priorización mediante Exposición al Riesgo (RE)

Se puede priorizar riesgos usando el concepto de Exposición al Riesgo (RE):

$$RE = \text{Prob}(\text{UO}) \times \text{Loss}(\text{UO})$$

- Prob(UO): probabilidad de que ocurra el riesgo (resultado indeseable)
- Loss(UO): pérdida total, incluyendo financieras, de credibilidad, negocios futuros o incluso pérdidas de propiedad o vidas

El RE es el valor esperado de la pérdida por un riesgo particular.

Cuanto mayor sea el RE, mayor prioridad tendrá el riesgo.

Evaluación subjetiva

No siempre es posible usar modelos para estimar probabilidades y pérdidas.

Cuando no hay modelos disponibles, la evaluación de riesgos puede ser subjetiva, mediante:

- Estimación de una persona
- Consenso grupal usando técnicas como el método Delphi, donde un grupo discute y converge en una estimación consensuada.

5.6.3 Control de Riesgos

El objetivo principal de la gestión de riesgos es identificar los principales riesgos y enfocarse en ellos.

Una vez que el gerente del proyecto ha identificado y priorizado los riesgos, los riesgos más importantes se pueden detectar fácilmente. La pregunta siguiente es: ¿qué hacer con ellos?

Conocer los riesgos solo tiene valor si se puede preparar un plan para minimizar sus consecuencias, que es el **objetivo básico** de la gestión de riesgos.

Estrategias de control de riesgos

1. Evitación del riesgo:

Consiste en tomar acciones para eliminar el riesgo por completo, como en el ejemplo

anterior de mover un edificio a una zona no sísmica. Para algunos riesgos, la evitación es posible.

2. Mitigación del riesgo:

Para la mayoría de los riesgos, la estrategia consiste en reducir la probabilidad de que ocurra o reducir la pérdida si se materializa.

- Estas acciones se llaman pasos de mitigación del riesgo.
- Para decidir qué pasos tomar, es útil tener una lista de medidas de mitigación comúnmente utilizadas para distintos riesgos.
- Normalmente, las tablas compiladas de riesgos frecuentes incluyen también los métodos de mitigación aplicados en proyectos previos.

A diferencia de la evaluación de riesgos, que es principalmente un ejercicio analítico, la mitigación de riesgos comprende acciones activas que deben realizarse para minimizar el impacto.

- Seleccionar un paso de mitigación no es solo un ejercicio intelectual; debe ejecutarse y monitorearse.
- Para asegurar que las acciones se realicen correctamente, deben incorporarse al cronograma detallado del proyecto.

Dinamismo y monitoreo de riesgos

- La priorización de riesgos y la planificación se basan en la percepción de riesgos en el momento del análisis.
- Como los riesgos son eventos probabilísticos que dependen de factores externos, la amenaza puede cambiar con el tiempo.
- La percepción del riesgo también puede cambiar a lo largo del proyecto.
- Además, las acciones de mitigación realizadas pueden afectar la percepción del riesgo.

Esto implica que los riesgos no deben tratarse como estáticos, sino que deben monitorearse y reevaluarse periódicamente.

- Monitoreo de riesgos: actividad de seguir el estado de los riesgos y las acciones de control implementadas.
- Un enfoque simple para el monitoreo consiste en analizar los riesgos nuevamente en cada hito importante y modificar los planes según sea necesario.

5.6.4 Un Enfoque Práctico de Gestión de Riesgos

Aunque el concepto de exposición al riesgo es muy completo, una manera práctica y sencilla de planificar riesgos es categorizar los riesgos y sus impactos en unos pocos niveles, y luego usar esto para la priorización. Este enfoque se utiliza en muchas organizaciones.

Aquí se describe un enfoque simple usado en una organización:

- La probabilidad de que un riesgo ocurra se clasifica como baja, media o alta.
- El impacto del riesgo también se clasifica como bajo, medio o alto.

Con estas clasificaciones, se puede seguir el siguiente método simple para priorizar riesgos:

1. Para cada riesgo, calificar la probabilidad de ocurrencia como baja, media o alta.
2. Para cada riesgo, evaluar su impacto en el proyecto como bajo, medio o alto.
3. Ordenar los riesgos según la probabilidad y el impacto en el proyecto; por ejemplo, un riesgo de alta probabilidad y alto impacto tendrá mayor prioridad que un riesgo de probabilidad media y alto impacto. En caso de conflicto, se usa el juicio del gerente del proyecto.
4. Seleccionar los principales riesgos para mitigación y seguimiento.

Un ejemplo de este enfoque se muestra en la Tabla 5.5, que presenta las diferentes calificaciones y los pasos de mitigación del riesgo.

Como se puede observar, la sección de gestión de riesgos del plan de gestión del proyecto, que esencialmente es esta tabla, puede ser muy breve y enfocada.

Para el monitoreo de riesgos, una manera es revisar la planificación de riesgos en los hitos importantes, prestando más atención a los riesgos listados en el plan del proyecto. Durante este monitoreo, puede ocurrir repriorización y los planes de mitigación para el resto del proyecto pueden cambiar, dependiendo de la situación actual y del efecto de las medidas de mitigación aplicadas previamente.

5.7 Plan de Monitoreo del Proyecto

Un plan de gestión de proyectos es simplemente un documento que puede usarse para guiar la ejecución de un proyecto. Incluso un buen plan es inútil si no se ejecuta correctamente. Y la ejecución no puede ser guiada apropiadamente por el plan a menos que se monitoree cuidadosamente y se haga un seguimiento del desempeño real en comparación con lo planeado.

El monitoreo requiere que se realicen mediciones para evaluar la situación del proyecto. Si se van a tomar mediciones durante la ejecución del proyecto, debemos planificar cuidadosamente qué medir, cuándo medir y cómo medir. Por lo tanto, la planificación de mediciones es un elemento clave dentro de la planificación del proyecto.

Además, también se debe planificar de antemano cómo se analizarán y reportarán los datos de medición, para evitar la situación de recopilar datos sin saber qué hacer con ellos. Sin una planificación cuidadosa de la recolección de datos y su análisis, es poco probable que ambos ocurran efectivamente.

En esta sección se discutirán los aspectos relacionados con las mediciones y el seguimiento del proyecto.

5.7.1 Mediciones

El propósito fundamental de las mediciones en un proyecto es monitorear y controlar el proyecto de manera efectiva. Para el seguimiento del cronograma de un proyecto, los elementos básicos que deben medirse son: tamaño, esfuerzo y defectos.

- Cronograma: Es una de las métricas más importantes porque la mayoría de los proyectos están impulsados por plazos. Solo monitoreando el cronograma real podemos evaluar si el

proyecto está a tiempo o si existe algún retraso. Es fácil de medir, ya que generalmente se usa el tiempo calendario en todos los planes.

- **Esfuerzo:** Es el recurso principal consumido en un proyecto de software. Por lo tanto, seguir el esfuerzo es una actividad clave durante el monitoreo; es esencial para evaluar si el proyecto se está ejecutando dentro del presupuesto. Para los datos de esfuerzo, se requiere algún tipo de sistema de hojas de tiempo donde cada persona que trabaja en el proyecto registre la cantidad de tiempo dedicado al proyecto. Para un monitoreo más efectivo, el esfuerzo dedicado a diferentes tareas debe registrarse por separado. Generalmente, el esfuerzo se registra a través de algún sistema en línea (como un sistema de reporte semanal de actividades), lo que permite que cada persona registre el tiempo invertido en una actividad particular. En cualquier momento, se puede agregar el esfuerzo total de una actividad.
- **Defectos:** Dado que los defectos tienen una relación directa con la calidad del software, su seguimiento es crítico para garantizar la calidad. Un proyecto de software grande puede incluir miles de defectos, encontrados por diferentes personas en distintas etapas. Para llevar un control, los defectos deben registrarse y seguirse hasta su cierre. Una vez que cada defecto se registra (y luego se cierra), se puede analizar cuántos defectos se han encontrado hasta ahora, qué porcentaje aún está abierto y otros aspectos. El seguimiento de defectos se considera una de las mejores prácticas para gestionar un proyecto [26]. Se discutirá en el Capítulo 10.
- **Tamaño:** Es otra métrica fundamental porque muchos datos (por ejemplo, densidad de defectos entregados) se normalizan con respecto al tamaño. El tamaño del software entregado puede medirse en líneas de código (LOC) (determinadas mediante editores y contadores de líneas) o en puntos de función. A un nivel más general, puede bastar con el número de módulos o el número de funcionalidades.

5.7.2 Monitoreo y Seguimiento del Proyecto

El objetivo principal del monitoreo es que los gerentes de proyecto tengan visibilidad sobre la ejecución del proyecto, de modo que puedan determinar si es necesario tomar alguna acción para asegurar que se cumplan los objetivos del proyecto. Se pueden realizar diferentes tipos de monitoreo en un proyecto. Los tres niveles principales son: nivel de actividad, informes de estado y análisis de hitos. Para el monitoreo se emplean las mediciones tomadas durante el proyecto.

- **Monitoreo a nivel de actividad:** Garantiza que cada actividad en el cronograma detallado se haya realizado correctamente y dentro del tiempo previsto. Este tipo de monitoreo puede realizarse diariamente en reuniones del equipo de proyecto o mediante la verificación del estado de todas las tareas programadas para completarse ese día. Una tarea completada generalmente se marca como 100% completa en el cronograma detallado; herramientas como Microsoft Project usan esto para seguir el porcentaje de avance del proyecto general o de tareas de nivel superior.
- **Informes de estado:** Se preparan semanalmente para hacer un balance de lo que ha sucedido y lo que queda por hacer. Estos informes típicamente contienen: un resumen de las actividades completadas desde el último informe, cualquier actividad que haya tenido

retraso, problemas del proyecto que requieran atención y si todo está listo para la semana siguiente.

- **Análisis de hitos:** Se realiza en cada hito o cada pocas semanas si los hitos están muy separados. En este análisis se compara lo real versus lo estimado en términos de esfuerzo y cronograma. Si la desviación es significativa, puede indicar que el proyecto podría tener problemas y no cumplir sus objetivos. Esto obliga a los gerentes a entender las causas de la variación y a aplicar acciones correctivas y preventivas si es necesario.

Un método gráfico para capturar el progreso básico de un proyecto en comparación con su planificación es el gráfico de costo-cronograma-hito [20]. Este gráfico muestra el cronograma y costo planificado de los diferentes hitos, junto con el costo y cronograma real de los hitos alcanzados hasta el momento. Al mostrar ambos en el mismo gráfico, es fácil visualizar el progreso del proyecto.

- **Ejes del gráfico:**
 - El eje X representa el tiempo, marcado por los meses del cronograma del proyecto.
 - El eje Y representa el costo, en dólares o en persona-meses (PM).

Se trazan dos curvas:

1. **Curva planificada:** Representa el costo y cronograma planificados, donde se marcan los hitos importantes del proyecto. Esta curva se completa una vez que se hace el plan del proyecto.
2. **Curva real:** Representa el costo y cronograma reales, marcando la fecha real de logro de cada hito y el costo real para alcanzarlo.

Por ejemplo, un gráfico de costo-cronograma-hito para un proyecto hipotético con un costo estimado de \$100K se muestra en la Figura 5.6. Los hitos identificados son: PDR (revisión preliminar de diseño), CDR (revisión crítica de diseño), finalización del Módulo 1, finalización del Módulo 2, pruebas de integración y pruebas de aceptación. Para cada hito se asigna un presupuesto basado en estimaciones.

- El presupuesto planificado se muestra con una línea punteada.
- El gasto real se muestra con una línea gruesa.

El gráfico indica que solo se han alcanzado los hitos PDR y CDR, y aunque el proyecto estaba dentro del presupuesto al completar PDR, ahora está ligeramente por encima del presupuesto.

6 Diseño Orientado a Funciones

Generalmente, el diseño se enfoca en lo que hemos llamado la vista de módulos en el Capítulo 4. Es decir, durante el diseño determinamos qué módulos debe tener el sistema y cuáles deben ser desarrollados. A veces, la vista de módulos puede ser efectivamente una estructura de módulos de cada componente en la arquitectura. Es decir, el ejercicio de diseño determina la estructura de módulos de los componentes. Sin embargo, esta correspondencia simple entre componentes y módulos puede no mantenerse siempre. En ese caso debemos asegurarnos de que la vista de módulos creada en el diseño sea consistente con la arquitectura.

El diseño de un sistema es esencialmente un plano o una guía para una solución del sistema. Aquí consideramos un sistema como un conjunto de módulos con un comportamiento claramente definido que interactúan entre sí de una manera determinada para producir algún comportamiento o servicios para su entorno. Un módulo de un sistema puede considerarse a su vez un sistema, con sus propios módulos.

El proceso de diseño para sistemas de software suele tener dos niveles. En el primer nivel, el enfoque está en decidir qué módulos se necesitan para el sistema, las especificaciones de estos módulos y cómo deben estar interconectados. Esto es lo que se llama diseño del sistema o diseño de alto nivel. En el segundo nivel, se decide el diseño interno de los módulos, o cómo pueden satisfacerse las especificaciones del módulo. Este nivel de diseño suele llamarse diseño detallado o diseño lógico. El diseño detallado esencialmente amplía el diseño del sistema para contener una descripción más detallada de la lógica de procesamiento y las estructuras de datos, de modo que el diseño sea lo suficientemente completo para la codificación.

Una metodología de diseño es un enfoque sistemático para crear un diseño mediante la aplicación de un conjunto de técnicas y guías. La mayoría de las metodologías de diseño se enfocan en el diseño del sistema y no reducen la actividad de diseño a una secuencia de pasos que el diseñador pueda seguir ciegamente.

En este capítulo discutimos los métodos orientados a funciones para el diseño y describimos una metodología particular —la metodología de diseño estructurado— en cierto detalle. En un enfoque de diseño orientado a funciones, un sistema se ve como una función de transformación, transformando las entradas en las salidas deseadas. El propósito de la fase de diseño es especificar los componentes de esta función de transformación, de modo que cada componente sea también una función de transformación. Es decir, cada módulo en el diseño representa una abstracción funcional. El resultado básico de la fase de diseño del sistema, cuando se sigue un enfoque de diseño orientado a funciones, es la definición de todas las estructuras de datos principales del sistema, todos los módulos principales del sistema y cómo interactúan entre sí los módulos.

En este capítulo primero discutimos algunos principios generales de diseño. Luego discutimos una notación para expresar diseños orientados a funciones y describimos la metodología de diseño estructurado para desarrollar un diseño. Después discutimos algunos métodos de verificación para el diseño y algunas métricas que son aplicables a los diseños orientados a funciones. Como en la mayoría de los capítulos, terminaremos con los estudios de caso.

6.1 Principios de Diseño

El diseño de un sistema es correcto si un sistema construido exactamente de acuerdo con el diseño satisface los requerimientos de dicho sistema. Claramente, el objetivo durante la fase de diseño es producir diseños correctos. Sin embargo, la corrección no es el único criterio durante esta fase, ya

que pueden existir muchos diseños correctos. El objetivo del proceso de diseño no es simplemente producir un diseño para el sistema, sino encontrar el mejor diseño posible dentro de las limitaciones impuestas por los requerimientos y el entorno físico y social en el cual el sistema operará.

Para evaluar un diseño, debemos especificar algunas propiedades y criterios que puedan utilizarse para la evaluación. Idealmente, estas propiedades deberían ser lo más cuantitativas posible. En tal situación podríamos evaluar con precisión la “bondad” de un diseño y determinar cuál es el mejor. Sin embargo, los criterios de calidad del diseño de software suelen ser subjetivos o no cuantificables. En tales casos, los criterios son esencialmente reglas empíricas que ayudan a evaluar el diseño. Un diseño debe ser claramente verificable, completo (implementa todas las especificaciones) y rastreable (todos los elementos del diseño pueden rastrearse hasta algún requerimiento). Sin embargo, las dos propiedades más importantes que preocupan a los diseñadores son la eficiencia y la simplicidad. La eficiencia de cualquier sistema se refiere al uso adecuado de los recursos escasos por parte del sistema. La necesidad de eficiencia surge por consideraciones de costo. Si algunos recursos son escasos y costosos, es deseable que esos recursos se utilicen de manera eficiente. En los sistemas computacionales, los recursos que más frecuentemente se consideran en relación con la eficiencia son el tiempo del procesador y la memoria. Un sistema eficiente es aquel que consume menos tiempo de procesador y requiere menos memoria. En épocas anteriores, el uso eficiente de la CPU y la memoria era importante debido al alto costo del hardware. Ahora que los costos de hardware son bajos en comparación con los costos del software, para muchos sistemas de software las preocupaciones tradicionales sobre eficiencia han pasado a un segundo plano frente a otras consideraciones. Una excepción son los sistemas en tiempo real, para los cuales existen restricciones estrictas de tiempo de ejecución.

La simplicidad es quizás el criterio de calidad más importante para los sistemas de software. Hemos visto que el mantenimiento del software suele ser bastante costoso. La mantenibilidad del software es uno de los objetivos que hemos establecido. El diseño de un sistema es uno de los factores más importantes que afectan la mantenibilidad del mismo. Durante el mantenimiento, el primer paso que debe realizar quien mantiene el sistema es comprenderlo. Solo después de que haya entendido a fondo los diferentes módulos del sistema, cómo están interconectados y cómo una modificación en uno afectará a los demás, debería llevarse a cabo la modificación. Un diseño simple y comprensible facilitará enormemente la tarea del mantenedor.

Estos criterios no son independientes, y aumentar uno puede tener un efecto desfavorable sobre otro. Por ejemplo, a menudo los “trucos” utilizados para aumentar la eficiencia de un sistema resultan en que el sistema sea más complejo. Por lo tanto, las decisiones de diseño con frecuencia implican compromisos. Es tarea de los diseñadores reconocer esos compromisos y lograr el mejor equilibrio posible. Para nuestros propósitos, la simplicidad es la propiedad principal de interés, y por lo tanto el objetivo del proceso de diseño es producir diseños que sean simples de entender. Crear un diseño simple (y eficiente) de un sistema grande puede ser una tarea extremadamente compleja que requiere buen juicio ingenieril. Dado que el diseño es fundamentalmente una actividad creativa, no puede reducirse a una serie de pasos que simplemente puedan seguirse, aunque sí pueden proporcionarse guías. En esta sección examinaremos algunos principios básicos que pueden usarse para producir el diseño de un sistema. Algunos de estos principios de diseño se relacionan con proporcionar medios para manejar eficazmente la complejidad del proceso de diseño. Manejar eficazmente la complejidad no solo reducirá el esfuerzo necesario para el diseño (es decir, reducirá el costo de diseño), sino que también puede reducir la posibilidad de introducir errores durante el diseño. Los principios discutidos aquí forman la base de la mayoría de las metodologías de diseño.

Debe notarse que los principios que pueden usarse en el diseño son los mismos que se usan en el análisis de problemas. De hecho, los métodos también son similares, ya que tanto en el análisis como en el diseño estamos esencialmente construyendo modelos. Sin embargo, existen algunas diferencias fundamentales. Primero, en el análisis de problemas, estamos construyendo un modelo del dominio del problema, mientras que en el diseño estamos construyendo un modelo para el dominio de la solución. Segundo, en el análisis de problemas, el analista tiene un grado limitado de libertad para seleccionar los modelos, ya que el problema está dado y el modelado debe representarlo. En el diseño, el diseñador tiene un alto grado de libertad para decidir los modelos, ya que el sistema que el diseñador está modelando no existe; de hecho, el diseñador está creando un modelo para el sistema que servirá de base para construirlo. Es decir, en el diseño, el sistema depende del modelo, mientras que en el análisis de problemas el modelo depende del sistema. Finalmente, como se mencionó anteriormente, el objetivo básico del modelado en el análisis de problemas es comprender, mientras que el objetivo básico del modelado en el diseño es optimizar (en nuestro caso, simplicidad y rendimiento). En otras palabras, aunque los principios y técnicas básicos puedan parecer similares, las actividades de análisis y diseño son muy diferentes.

6.2 Conceptos a Nivel de Módulo

En la sección anterior discutimos algunos principios generales de diseño. Ahora dirigimos nuestra atención a algunos conceptos específicos del diseño orientado a funciones.

Antes de discutirlos, definamos qué entendemos por un módulo. **Un módulo es una parte lógicamente separable de un programa.** Es una unidad de programa que es discreta e identificable con respecto a la compilación y la carga. **En términos de construcciones comunes de los lenguajes de programación, un módulo puede ser una macro, una función, un procedimiento (o subrutina), un proceso o un paquete.** En sistemas que utilizan abstracción funcional, un módulo suele ser un procedimiento o función, o una colección de estos.

Para producir diseños modulares, deben utilizarse ciertos criterios para seleccionar los módulos de manera que estos soporten abstracciones bien definidas y puedan resolverse y modificarse por separado. En un sistema que utiliza abstracción funcional, el acoplamiento y la cohesión son dos criterios de modularización que suelen emplearse conjuntamente.

6.2.1 Acoplamiento

Dos módulos se consideran independientes si uno puede funcionar completamente sin la presencia del otro. Obviamente, si dos módulos son independientes, pueden resolverse y modificarse por separado. Sin embargo, **no todos los módulos de un sistema pueden ser independientes entre sí, ya que deben interactuar para producir juntos el comportamiento externo deseado del sistema.**

Cuanto más conexiones existan entre los módulos, más dependientes serán, en el sentido de que se necesita más conocimiento sobre un módulo para entender o resolver el otro. Por lo tanto, **cuanto menos numerosas y más simples sean las conexiones entre los módulos, más fácil será entender uno sin necesidad de entender el otro.** La noción de *acoplamiento* intenta capturar este concepto de “qué tan fuertemente” están interconectados los distintos módulos.

El **acoplamiento entre módulos es la fuerza de las interconexiones entre ellos o una medida de la interdependencia entre módulos.** En general, cuanto más debemos saber sobre el módulo A para entender el módulo B, más estrechamente conectado está A con B. Los módulos “altamente acoplados” están unidos por interconexiones fuertes, mientras que los “débilmente acoplados”

tienen interconexiones débiles. Los módulos independientes no tienen interconexiones. Para poder resolver y modificar un módulo por separado, querríamos que estuviera débilmente acoplado con los demás.

La elección de los módulos determina el acoplamiento entre ellos. Dado que los módulos del sistema de software se crean durante el diseño del sistema, el acoplamiento entre módulos se decide en gran medida durante el diseño y no puede reducirse en la implementación.

El acoplamiento aumenta con la complejidad y la falta de claridad de la interfaz entre módulos. Para mantener bajo el acoplamiento, debemos minimizar el número de interfaces por módulo y la complejidad de cada una. Una interfaz de un módulo se utiliza para pasar información hacia y desde otros módulos. El acoplamiento se reduce si solo se usa la interfaz de entrada definida del módulo (por ejemplo, pasando información exclusivamente a través de parámetros). El acoplamiento aumenta si un módulo es utilizado por otros mediante una interfaz indirecta u oculta, como el uso directo de sus elementos internos o el empleo de variables compartidas.

Tipo de conexión entre módulos

La complejidad de la interfaz es otro factor que afecta el acoplamiento. Cuanto más compleja sea cada interfaz, mayor será el grado de acoplamiento. Por ejemplo, la complejidad de la interfaz de entrada de un procedimiento depende del número de elementos que se pasan como parámetros y de la complejidad de esos elementos. Cierta nivel de complejidad en las interfaces es necesario para permitir la comunicación entre módulos. Sin embargo, a menudo se usa más de lo necesario. Por ejemplo, si un procedimiento necesita un solo campo de un registro, frecuentemente se pasa el registro completo en lugar de solo ese campo. Al pasar todo el registro, estamos aumentando el acoplamiento innecesariamente. En esencia, debemos mantener la interfaz de un módulo tan simple y pequeña como sea posible.

El tipo de flujo de información a través de las interfaces es el tercer factor principal que afecta el acoplamiento. Existen dos tipos de información que pueden fluir por una interfaz: *datos* o *control*. Pasar o recibir información de control significa que la acción del módulo dependerá de esa información, lo que dificulta comprender el módulo y definir su abstracción. La transferencia de información de datos significa que un módulo pasa ciertos datos como entrada a otro módulo y recibe a cambio otros datos como salida. Esto permite tratar un módulo como una simple función de entrada-salida que realiza una transformación sobre los datos de entrada para producir los datos de salida.

En general, las interfaces que solo comunican datos resultan en el grado más bajo de acoplamiento, seguidas por las interfaces que solo transfieren información de control. El acoplamiento se considera más alto si los datos son *híbridos*, es decir, si se pasan entre los módulos tanto elementos de datos como de control. El efecto de estos tres factores sobre el acoplamiento se resume en la Tabla 6.1.

6.2.2 Cohesión

Hemos visto que el acoplamiento se reduce cuando se minimizan las relaciones entre elementos de distintos módulos. Es decir, el acoplamiento se reduce cuando los elementos de diferentes módulos tienen pocos o ningún vínculo entre sí.

Otra forma de lograr este efecto es fortalecer el vínculo entre los elementos del mismo módulo, maximizando la relación entre los elementos internos. La cohesión es el concepto que intenta capturar esta relación intramódulo. Con la cohesión, nos interesa determinar qué tan estrechamente relacionados están los elementos de un módulo entre sí.

La **cohesión de un módulo** representa qué tan fuertemente unidos están sus elementos internos. Le da al diseñador una idea de si los distintos elementos de un módulo realmente pertenecen juntos en ese mismo módulo. La cohesión y el acoplamiento están claramente relacionados. Generalmente, cuanto mayor es la cohesión de cada módulo en el sistema, menor es el acoplamiento entre módulos.

Esta correlación no es perfecta, pero se ha observado en la práctica.

Existen varios niveles de cohesión:

- Casual
- Lógica
- Temporal
- Procedural
- Comunicacional
- Secuencial
- Funcional

Casual es el nivel más bajo, y funcional es el más alto. Estos niveles **no forman una escala lineal**: la cohesión funcional es mucho más fuerte que el resto, mientras que los dos primeros se consideran mucho más débiles que los demás.

A menudo, varios niveles pueden ser aplicables al analizar la cohesión entre dos elementos de un módulo. En tales casos, se considera el nivel más alto. La cohesión de un módulo se toma como el nivel más alto de cohesión aplicable a todos sus elementos.

La **cohesión casual** ocurre cuando **no existe una relación significativa** entre los elementos del **módulo**. Puede aparecer si un programa existente es “modularizado” simplemente cortándolo en partes y convirtiendo esas partes en módulos.

Si se crea un módulo solo para evitar código duplicado combinando fragmentos que aparecen en muchos lugares, dicho módulo probablemente tenga cohesión casual.

En este caso, las sentencias del módulo no tienen relación entre sí, y si un módulo que usa ese código necesita modificarse, es probable que los demás módulos que también lo usan no quieran ese cambio.

Como resultado, la modificación del “módulo común” puede causar fallas en otros módulos. Por lo tanto, esos módulos no pueden modificarse por separado y tienen una fuerte interconexión.

Generalmente, es mala práctica crear un módulo solo para evitar código duplicado (a menos que ese código común realmente realice una función identificable), o dividir un módulo en partes más pequeñas únicamente para reducir su tamaño.

Un módulo tiene **cohesión lógica** si **existe alguna relación lógica entre sus elementos** y estos realizan funciones que pertenecen a la misma categoría lógica.

Un ejemplo típico es un módulo que realiza **todas las entradas** o **todas las salidas**.

En tal caso, si queremos ingresar o imprimir un registro, debemos indicar al módulo qué operación hacer. A menudo esto se logra pasando una bandera de estado, usada para decidir qué sentencias ejecutar.

Además de producir un flujo de información híbrido entre módulos —que suele ser la forma más mala de acoplamiento— este tipo de módulo suele tener código complicado y torpe.

En general, los módulos con cohesión lógica deberían evitarse.

La **cohesión temporal** es similar a la lógica, excepto que los elementos también están relacionados en el **tiempo** y se ejecutan juntos.

Módulos que realizan actividades como “inicialización”, “limpieza” o “finalización” son temporales.

Aunque los elementos están lógicamente relacionados, la cohesión temporal es mayor porque todos se ejecutan juntos. Esto evita el paso de banderas y el código suele ser más simple.

Un módulo con **cohesión procedural** agrupa elementos que pertenecen a una misma unidad procedimental. Por ejemplo, un bucle o **una secuencia de decisiones podría extraerse para formar un módulo separado.**

Los módulos procedurales suelen aparecer cuando la estructura modular se deriva de un diagrama de flujo. A menudo atraviesan límites funcionales: un módulo con cohesión procedural puede contener solo parte de una función completa o partes de varias funciones.

Un módulo con **cohesión comunicacional** tiene **elementos relacionados por el uso del mismo conjunto de datos de entrada o salida.**

Es decir, los elementos están juntos porque operan sobre los mismos datos.

Por ejemplo, un módulo “imprimir y perforar registro”.

Los módulos comunicacionales pueden realizar más de una función, pero su cohesión es suficientemente alta como para ser aceptable si no existen estructuras alternativas con cohesión mayor.

Cuando los elementos están juntos porque **la salida de uno es la entrada del siguiente**, se obtiene **cohesión secuencial.**

Si existe una secuencia donde la salida de un elemento alimenta al siguiente, la cohesión secuencial **no** indica cómo deben agruparse en módulos. Existen muchas posibilidades: ponerlos todos en un módulo, la mitad en uno y la mitad en otro, un tercio en uno y el resto en otro, etc.

Por lo tanto, un módulo secuencial puede contener varias funciones o partes de funciones distintas. Aunque se asemejan mucho a la estructura del problema, están lejos del ideal: la cohesión funcional.

La **cohesión funcional** es la más fuerte.

En un módulo funcional, **todos sus elementos están relacionados con la realización de una única función.**

No significa solo funciones matemáticas; incluyen objetivos específicos como “calcular raíz cuadrada” o “ordenar un arreglo”.

¿Cómo se determina el nivel de cohesión de un módulo?

No existe una fórmula matemática: debemos usar juicio.

Una técnica útil para evaluar cohesión funcional es escribir una oración que describa completa y exactamente la función o propósito del módulo. Luego pueden aplicarse las siguientes pruebas:

1. Si la oración debe ser compuesta, contiene una coma, o más de un verbo, el módulo probablemente realiza más de una función y tenga cohesión secuencial o comunicacional.
2. Si contiene palabras relacionadas con tiempo como “primero”, “después”, “cuando”, “luego”, probablemente tenga cohesión secuencial o temporal.
3. Si el predicado de la oración no contiene un único objeto específico después del verbo (como “editar todos los datos”), el módulo probablemente tenga cohesión lógica.

4. Palabras como “inicializar” y “limpiar” implican cohesión temporal.

Los módulos con cohesión funcional pueden describirse con una oración simple.

Sin embargo, si la descripción es compuesta, no significa necesariamente que el módulo no tenga cohesión funcional.

Pero si **no puede** describirse con una oración simple, es poco probable que tenga cohesión funcional.

7 Diseño Orientado a Objetos

Los enfoques orientados a objetos (OO) para el desarrollo de software se han vuelto extremadamente populares en los últimos años. Gran parte del nuevo desarrollo se realiza ahora utilizando técnicas y lenguajes OO. Hay muchas ventajas que los sistemas OO ofrecen.

Un modelo OO representa de manera cercana el dominio del problema, lo cual hace más fácil producir y entender los diseños. A medida que los requerimientos cambian, los objetos en un sistema son menos vulnerables a estos cambios, permitiendo así cambios más fácilmente. La herencia y la estrecha asociación de los objetos en el diseño con las entidades del dominio del problema fomentan una mayor reutilización, es decir, nuevas aplicaciones pueden usar los módulos existentes más efectivamente, reduciendo así el costo de desarrollo y el tiempo del ciclo.

Se cree que los enfoques orientados a objetos son más naturales y proporcionan estructuras más ricas para el pensamiento y la abstracción. También se han descubierto patrones de diseño comunes que permiten la reutilización a un nivel más alto.

El enfoque de diseño orientado a objetos es fundamentalmente diferente de los enfoques de diseño orientado a funciones, principalmente debido a la diferente abstracción que se utiliza. Requiere una forma distinta de pensar y de particionar. Puede decirse que pensar en términos orientados a objetos es lo más importante para producir diseños verdaderamente orientados a objetos.

Durante el diseño, como se mencionó en el capítulo anterior, nuestro enfoque está en lo que se llama la vista de módulos en la arquitectura. Es decir, el objetivo es identificar los módulos que el sistema debe tener, y sus interfaces y relaciones. En OOD, por lo tanto, estamos identificando las clases que deben existir en el software y la relación entre estas clases. Durante el diseño arquitectónico, la vista de componentes y conectores suele estar fijada. Un objetivo del diseño es asegurar que la arquitectura se preserve, y que la relación entre los componentes y los módulos esté clara.

En este capítulo, discutiremos algunos conceptos importantes que forman la base de la orientación a objetos. Luego discutiremos algunos conceptos que influyen a un diseñador al crear un diseño orientado a objetos (OOD). Después describiremos la notación UML que puede utilizarse mientras se realiza un diseño orientado a objetos, seguida por una metodología OOD. Luego discutiremos algunas métricas que son aplicables al OOD y que pueden utilizarse para evaluar la calidad del diseño. No discutimos métodos de verificación, ya que los métodos de verificación del diseño discutidos en el capítulo anterior son métodos generales que pueden utilizarse sin importar el enfoque usado para producir el diseño. Finalmente, como con otros capítulos, terminaremos haciendo el diseño OO de los estudios de caso.

Antes de proceder, entendamos la relación entre el análisis OO y el diseño OO.

7.1 Análisis OO y Diseño OO

El desarrollo puramente orientado a objetos requiere que se utilicen técnicas orientadas a objetos durante el análisis, el diseño y la implementación del sistema. Sin embargo, gran parte del enfoque del enfoque orientado a objetos para el desarrollo de software ha estado en el análisis y el diseño. Se han propuesto varios métodos para análisis y diseño, muchos de los cuales proponen una técnica

combinada de análisis y diseño. Nos referiremos a un método combinado como análisis y diseño orientado a objetos (OOAD). En OOAD, el límite entre análisis y diseño es difuso. Una razón de esta difuminación es la similitud de las construcciones básicas (es decir, objetos y clases) que se utilizan en el análisis y el diseño.

Aunque no hay acuerdo sobre qué partes del proceso de desarrollo orientado a objetos pertenecen al análisis y cuáles al diseño, hay algún acuerdo general sobre los dominios de las dos actividades.

La diferencia fundamental entre el análisis orientado a objetos (OOA) y el diseño orientado a objetos (OOD) es que el primero modela el dominio del problema, llevando a una comprensión y especificación del problema, mientras que el segundo modela la solución al problema. Es decir, el análisis trata con el dominio del problema, mientras que el diseño trata con el dominio de la solución. Sin embargo, en OOAD se cree que la representación del dominio del problema creada por OOA generalmente queda subsumida en la representación del dominio de la solución. Es decir, la representación del dominio de la solución, creada por OOD, generalmente contiene gran parte de la representación creada por OOA, y más. Esto se muestra en la Figura 7.1 [118].

Como el objetivo tanto de OOA como de OOD es modelar algún dominio, frecuentemente los procesos de OOA y OOD (es decir, las metodologías) y las representaciones se ven bastante similares. Esto contribuye a la difuminación de los límites entre análisis y diseño. A menudo no está claro dónde termina el análisis y dónde comienza el diseño. La línea divisoria es una cuestión de percepción. La falta de separación clara entre análisis y diseño también puede considerarse uno de los puntos fuertes del enfoque orientado a objetos: la transición del análisis al diseño es "sin fisuras". Esta es también la razón principal por la que se han propuesto métodos OOAD —donde se realizan tanto análisis como diseño—.

A pesar de las diferencias en las percepciones sobre el límite entre OOA y OOD, una cosa está clara. La principal diferencia entre OOA y OOD, debido a los diferentes dominios de modelado, está en el tipo de objetos que surgen de los procesos de análisis y diseño. Los objetos durante OOA se enfocan en el dominio del problema y generalmente representan algunas cosas o conceptos en el problema. Estos objetos a veces se llaman objetos semánticos ya que tienen un significado en el dominio del problema [118]. El dominio de la solución, por otro lado, consiste en objetos semánticos, así como otros objetos. Durante el diseño, como el enfoque está en encontrar y definir una solución, los objetos semánticos identificados durante OOA pueden refinarse y extenderse desde el punto de vista de la implementación, y se agregan otros objetos que son específicos del dominio de la solución. Los objetos del dominio de la solución incluyen objetos de interfaz, de aplicación y utilitarios [118]. Los objetos de interfaz tratan con la interfaz de usuario, que no es directamente parte del dominio del problema pero representa algún aspecto de la solución deseada por el usuario. Los objetos de aplicación especifican los mecanismos de control para la solución propuesta. Son objetos controladores que son específicos de las necesidades de la aplicación. Los objetos utilitarios son aquellos necesarios para apoyar los servicios de los objetos semánticos o para implementarlos eficientemente (por ejemplo, colas, árboles y tablas). Estos objetos son frecuentemente de propósito general y no dependen de la aplicación.

El objetivo básico de las actividades de análisis y diseño es identificar las clases en el sistema y sus relaciones, frecuentemente representadas por diagramas de clases. Sin embargo, el sistema debe soportar cierta funcionalidad y comportamiento. Por lo tanto, además de concentrarse en la estructura estática del dominio del problema o de la solución, el comportamiento dinámico del sistema debe estudiarse para asegurar que el diseño final soporte los comportamientos dinámicos

deseados. Debido a esto, algún modelado dinámico del sistema es deseable antes de que el diseño esté completo. Si este tipo de modelado es parte del análisis o del diseño —es decir, dónde en el proceso global de OOAD está el límite entre análisis y diseño— no está generalmente acordado.

Otra forma de ver la diferencia entre modelado y diseño es que en el diseño se construye un modelo para la (eventual) implementación. Como consecuencia, los problemas de implementación guían el proceso de modelado durante el diseño. Mientras que en el análisis, los problemas de comprensión y representación guían el proceso. Esto también da como resultado que OOA a veces use primitivas que son algo más ricas que las que se usan en OOD, ya que las primitivas de OOD tienden a estar estrechamente asociadas con las características del lenguaje de programación que se utilizará para implementar el diseño. Los modelos construidos durante el análisis orientado a objetos forman el punto de partida del diseño orientado a objetos, y el modelo construido por OOD forma la base para la implementación orientada a objetos.

7.2 Conceptos OO

Aquí discutimos los conceptos principales detrás de la orientación a objetos.

Aunque estos conceptos también se utilizaron durante el análisis orientado a objetos, aquí se discuten en términos más concretos, ya que el diseño trata con el dominio de la solución y por lo tanto está más cerca de la implementación final. Como la discusión gira en torno a los conceptos OO tal como son soportados en los lenguajes de programación, los lectores que estén muy familiarizados con los lenguajes OO y sus conceptos pueden omitir esta sección. En la siguiente sección discutimos algunos conceptos de diseño.

7.3 Conceptos de Diseño

En un sistema OO, el módulo básico es una clase, y durante el diseño la actividad clave es identificar y especificar los módulos que deberían estar en el sistema que se está construyendo. El objetivo de la actividad de diseño es crear un diseño que, además de ser correcto, tenga otros atributos que lo conviertan en un buen diseño.

Hay muchos atributos deseables para un sistema OO. Sin embargo, aquí nos centraremos en tres conceptos principales. Si podemos crear un diseño que sea satisfactorio desde estas tres perspectivas (y que sea correcto), entonces podemos estar bastante seguros de que tenemos un buen diseño. Estos conceptos clave gobiernan la calidad de un diseño, y por lo tanto deben guiar el proceso de diseño y las decisiones de diseño. Los tres conceptos son cohesión, acoplamiento (coupling) y el principio abierto-cerrado (open-closed principle). Nuestro objetivo es crear un diseño en el que los módulos tengan bajo acoplamiento, alta cohesión (pronto entenderemos qué significan “bajo” y “alto”), y que satisfagan el principio abierto-cerrado. Además de estos, también discutimos algunas guías de diseño que sugieren formas más concretas de poner estos principios en práctica.

7.3.1 Acoplamiento

Como se mencionó en el capítulo anterior, el acoplamiento es un concepto entre módulos que captura la fortaleza de la interconexión entre módulos. Cuanto más fuertemente acoplados están los módulos, más dependientes son entre sí, y más difícil es entenderlos y modificarlos. Un **bajo acoplamiento es deseable para hacer el sistema más entendible y modificable**.

El grado de acoplamiento entre un módulo y otro módulo depende de cuánta información se necesita sobre el otro módulo para entender y modificar este módulo, y cuán compleja y explícita es esa información. El acoplamiento bajo ocurre cuando esta información es lo más mínima posible, lo más simple posible, y fácilmente visible o identificable. En el capítulo anterior discutimos este concepto para sistemas con módulos funcionales. Aunque el concepto sigue siendo el mismo, su manifestación en sistemas OO es algo diferente, ya que los objetos son semánticamente más ricos que las funciones. En sistemas OO, existen tres tipos diferentes de acoplamiento entre módulos [53]:

- **Acoplamiento por interacción**
- **Acoplamiento por componentes**
- **Acoplamiento por herencia**

El **acoplamiento por interacción** ocurre debido a **métodos de una clase que invocan métodos de otras clases**. Nótese que, como estamos observando el acoplamiento entre clases, nos enfocamos en la interacción entre clases y no dentro de una clase. En muchos aspectos, esta situación es similar a que una función llame a otra función, y por lo tanto este acoplamiento es similar al acoplamiento entre módulos funcionales. Como con las funciones, la **peor forma de acoplamiento aquí es si los métodos acceden directamente a partes internas de otros métodos**. (Este tipo de interacción está deshabilitada en muchos lenguajes, pero es permitida en aquellos donde existen conceptos como clases amigas —friend classes— que permiten a un amigo acceder a los internos de una clase).

El acoplamiento por interacción se reduce, aunque aún es muy alto, si los métodos de una clase interactúan con métodos en otra clase manipulando directamente variables de instancia o atributos de objetos de la otra clase. Esta forma de interacción también es mala, ya que uno debe entender el código de las otras clases para comprender qué cambios están haciendo en la clase. También viola el principio de encapsulación de OO. Esta forma de interacción es peor si las variables se usan para comunicar datos temporales, es decir, si las variables se usan no para mantener el estado del objeto sino para pasar el estado de la computación de un objeto a otro. Si esta variable usada como contenedor temporal se encuentra en la superclase, entonces el acoplamiento empeora ya que la variable es visible para todas las subclases.

El **acoplamiento es menor** (igual que en acoplamiento con módulos funcionales) **si los métodos se comunican directamente a través de parámetros**. Dentro de esta categoría, el acoplamiento es menor si solo se pasa información de datos, pero mayor si se pasa información de control, ya que el método invocado afecta la secuencia de ejecución en el método que llama. Además, el acoplamiento es mayor si la cantidad de datos que se pasa es mayor. Entonces, si se pasan estructuras de datos completas cuando solo se necesitan algunas partes, el acoplamiento aumenta innecesariamente. De manera similar, si se pasa un objeto a un método cuando solo se usan algunos de sus objetos componentes (o los objetos a los que el objeto pasado hace referencia), el acoplamiento aumenta innecesariamente. La situación de menor acoplamiento, por lo tanto, es cuando la comunicación se

realiza solo mediante parámetros, pasando únicamente las variables necesarias, y estos parámetros solo transmiten datos.

El **acoplamiento por componentes** se refiere a la **interacción entre dos clases cuando una clase tiene variables de la otra clase**. Tres situaciones claras existen cuando esto puede ocurrir. Una clase C puede tener acoplamiento por componentes con otra clase C' si C tiene una variable de instancia de tipo C', o C tiene un método cuyo parámetro es de tipo C', o si C tiene un método que contiene una variable local de tipo C' (que luego puede ser pasada como parámetro a algún método que invoque). Nótese que cuando C tiene acoplamiento por componentes con C', tiene el potencial de estar acoplada por componentes con todas las subclases de C', ya que en tiempo de ejecución puede usarse un objeto de cualquier subclase. Debería ser claro que cuando existe acoplamiento por componentes, es probable que también haya acoplamiento por interacción. El acoplamiento por componentes se considera el más débil (es decir, el más deseable) si en una clase C, las variables de clase C' están en las firmas de los métodos de C, o algunos atributos son de tipo C'. Si la interacción es a través de variables locales, entonces esta interacción no es visible desde afuera, y por lo tanto aumenta el acoplamiento.

El **acoplamiento por herencia** se debe a la **relación de herencia entre clases**. **Dos clases se consideran acopladas por herencia si una clase es una subclase directa o indirecta de la otra**. Si la herencia agrega acoplamiento, uno podría preguntarse por qué no eliminar la herencia completamente. La razón es que la herencia puede reducir el acoplamiento general en el sistema. Consideremos dos situaciones. Si una clase A está acoplada con otra clase B, y si B es una jerarquía con B₁ y B₂ como dos subclases, entonces si un método m se extrae de B₁ y B₂ y se coloca en la superclase B, el acoplamiento se reduce ya que A ahora solo está acoplada con B, mientras que antes estaba acoplada con B₁ y con B₂. Similarmente, si B es una jerarquía de clases que soporta la relación especialización-generalización, entonces si se agregan nuevas subclases a B, no es necesario hacer cambios en la clase A que llama métodos en B. Es decir, para modificar la jerarquía de B, A no necesita ser afectada. Sin esta jerarquía, cambios en B probablemente resultarían en cambios en A.

Dentro del acoplamiento por herencia existen algunas situaciones peores que otras. La peor forma es cuando una subclase B₁ modifica la firma de un método en B (o elimina el método). Esta situación puede llevar fácilmente a un error en tiempo de ejecución, además de violar el verdadero espíritu de la relación es-un (is-a). Si la firma se preserva pero la implementación de un método se cambia, eso también viola la relación es-un, aunque puede no causar un error en tiempo de ejecución, y debería evitarse. El escenario de menor acoplamiento es cuando una subclase solo agrega variables de instancia y métodos pero no modifica ninguno de los heredados.

7.3.2 Cohesión

Mientras que el acoplamiento es un concepto inter-módulo, la cohesión es un concepto intra-módulo. Se enfoca en por qué los elementos de un módulo están juntos en el mismo módulo. El **objetivo aquí es que los elementos que están estrechamente relacionados pertenezcan al mismo módulo**. Esto hará que los módulos sean más fáciles de entender y, al capturar conceptos y abstracciones claras, más fáciles de modificar. Generalmente, una mayor cohesión llevará a un menor acoplamiento, ya que muchos elementos que necesitan interactuar mucho residirán juntos en módulos fuertemente cohesivos, reduciendo la necesidad de interacción con otros módulos. Por otro

lado, los módulos que tienen baja cohesión a menudo necesitarán interactuar con otros módulos para realizar su tarea. Claramente, para hacer un sistema más entendible y modificable, nos gustaría que consistiera en módulos altamente cohesivos. En otras palabras, la meta es tener un alto grado de cohesión en los módulos del sistema. La cohesión en sistemas OO también tiene tres aspectos:

- Cohesión de métodos
- Cohesión de clases
- Cohesión de herencia

La cohesión de métodos es la misma que la cohesión en módulos funcionales, que discutimos extensamente en el capítulo anterior. Se enfoca en por qué los diferentes elementos de código de un método están juntos dentro del método. La forma más alta de cohesión es si cada método implementa una función claramente definida, y todas las sentencias en el método contribuyen a implementar esa función. En general, con métodos funcionalmente cohesivos, lo que hace el método puede expresarse fácilmente con una declaración simple. Es decir, en una declaración corta y simple del tipo “este método hace...”, podemos expresar la funcionalidad del método.

La cohesión de clases se enfoca en por qué diferentes atributos y métodos están juntos en esta clase. La meta es tener una clase que implemente un único concepto o abstracción, con todos los elementos contribuyendo a soportar ese concepto. En general, cuando hay múltiples conceptos encapsulados dentro de una clase, la cohesión de la clase no es tan alta como podría ser, y un diseñador debería tratar de cambiar el diseño para que cada clase encapsule un único concepto.

Un síntoma de la situación en la que una clase tiene múltiples abstracciones es que el conjunto de métodos puede dividirse en dos (o más) grupos, cada uno accediendo a un subconjunto distinto de atributos. Es decir, el conjunto de métodos y atributos puede particionarse en grupos separados, cada uno encapsulando un concepto diferente. Claramente, en tal situación, al tener clases separadas encapsulando conceptos separados, podemos tener módulos con cohesión mejorada.

En muchas situaciones, aunque dos (o más) conceptos estén encapsulados dentro de una clase, hay algunos métodos que acceden a atributos de ambos conceptos encapsulados. Esto sucede cuando la clase representa diferentes entidades que tienen una relación entre ellas. Para la cohesión, es mejor representarlas como dos clases separadas con una relación entre ellas. Es decir, deberíamos tener múltiples clases, con algunos métodos en estas clases accediendo a objetos de la otra clase. En cierto modo, esta mejora en cohesión resulta en un aumento del acoplamiento. Sin embargo, para la modificabilidad y entendibilidad, es mejor si cada clase encapsula un solo concepto.

La cohesión de herencia se enfoca en por qué las clases están juntas en una jerarquía. Las dos razones principales para la herencia son modelar la relación generalización-especialización y la reutilización de código. Se considera que la cohesión es alta si la jerarquía soporta la generalización-especialización de algún concepto (lo cual probablemente llevará de forma natural a la reutilización de código). Se considera menor si la jerarquía existe principalmente para compartir código con una relación conceptual débil entre la superclase y las subclasses. En otras palabras, se desea que en un sistema OO las jerarquías de clases sean tales que soporten relaciones de generalización-especialización claramente identificadas.

8 Diseño Detallado

En capítulos anteriores se presentaron dos enfoques distintos para el **diseño de sistemas**.

En esa etapa del proceso, el objetivo principal es **definir los módulos** que componen el sistema y **cómo interactúan entre sí**.

Una vez que un módulo ha sido **especificado con precisión**, se puede pasar a determinar **la lógica interna** necesaria para implementar esas especificaciones. Ese es el tema central de este capítulo.

Aquí se explican los **métodos para desarrollar y describir el diseño detallado** de un módulo, junto con las **métricas** que pueden obtenerse a partir de dicho diseño.

8.1 Diseño Detallado y PDL

La mayoría de las técnicas de diseño, como el diseño estructurado, identifican los módulos principales y el flujo de datos más importante entre ellos. Los métodos utilizados para especificar el diseño del sistema suelen centrarse en las interfaces externas de los módulos y no pueden ampliarse fácilmente para describir su funcionamiento interno.

El *Process Design Language* (PDL) es una forma de comunicar el diseño de manera precisa y completa, con el nivel de detalle que el diseñador desee. Es decir, puede emplearse tanto para especificar el diseño del sistema como para extenderlo e incluir el diseño lógico.

El PDL resulta especialmente útil cuando se aplican técnicas de refinamiento descendente (*top-down refinement*) para diseñar un sistema o un módulo.

8.1.1 PDL

Una forma de **comunicar un diseño es especificarlo en un lenguaje natural, como el inglés**. Sin embargo, este enfoque **suele generar malentendidos y resulta impreciso**, lo que lo hace poco útil al momento de convertir el diseño en código. En el **extremo opuesto, se podría expresar el diseño en un lenguaje formal, como un lenguaje de programación**. Este tipo de representación **suele contener un nivel de detalle muy alto** —necesario para la implementación— pero no esencial para comunicar la estructura del diseño. Esos **detalles pueden dificultar la comprensión general**.

Lo ideal sería poder expresar el diseño en un lenguaje que sea lo más preciso y claro posible, pero sin caer en excesivos detalles, y que además pueda transformarse fácilmente en una implementación. Ese es el objetivo del *Process Design Language* (PDL).

El **PDL tiene una sintaxis externa similar a la de un lenguaje de programación estructurado, pero con un vocabulario de lenguaje natural** (en este caso, inglés). Puede entenderse como una especie de “inglés estructurado”. Dado que la estructura de un diseño expresado en PDL es formal, mediante el uso de construcciones típicas de lenguajes formales, es posible realizar cierto procesamiento automatizado sobre tales diseños.

Por ejemplo, supongamos que queremos encontrar el valor mínimo y máximo de un conjunto de números contenidos en un archivo y mostrar esos resultados. En PDL, el procedimiento podría describirse de manera completa en cuanto a su lógica, pero sin entrar en los detalles específicos de

implementación. Cada sentencia del PDL podría luego traducirse directamente a un lenguaje de programación concreto.

Otro ejemplo sería un programa que toma un texto en un archivo con una separación de un espacio entre palabras y lo formatea en líneas de 80 caracteres (excepto la última). Ninguna palabra debe dividirse entre dos líneas, y los espacios adicionales necesarios para completar la línea se añaden al final, sin usar más de dos espacios entre palabras. En PDL, este diseño se puede expresar mediante procedimientos que describen claramente el comportamiento deseado.

El PDL permite expresar el diseño con el nivel de detalle que se considere adecuado. Una forma común de usarlo es comenzar con un esquema general de la solución y luego ir refinándolo progresivamente, agregando más detalle conforme se avanza. Este enfoque de refinamiento sucesivo permite detectar errores de diseño temprano y facilita la verificación fase por fase, ayudando a obtener diseños más confiables. Además, su sintaxis estructurada fomenta el uso de construcciones de programación bien organizadas al implementar el diseño.

Las construcciones básicas del PDL son similares a las de un lenguaje estructurado. Por ejemplo, incluye el condicional **IF**, que se asemeja al **if - then - else** de Pascal, y estructuras de repetición flexibles que pueden adaptarse al problema sin necesidad de una formulación estricta. Algunos ejemplos válidos son:

DO WHILE there are characters in input file

DO UNTIL the end of file is reached

DO FOR each item in the list EXCEPT when item is zero

Además, en PDL pueden definirse y usarse diversas estructuras de datos, como listas, tablas, escalares o enteros. Existen varias versiones del PDL, y con el apoyo de herramientas automáticas, este lenguaje se utiliza ampliamente para comunicar y documentar diseños de software.

8.1.2 Diseño de lógica o de algoritmos

El objetivo principal del diseño detallado es definir la lógica de los distintos módulos que se especificaron durante el diseño del sistema. Especificar la lógica implica desarrollar un algoritmo que implemente esas especificaciones.

A continuación se presentan algunos principios para diseñar algoritmos o lógicas que cumplan con las especificaciones dadas.

El término *algoritmo* es bastante general y puede aplicarse a una amplia variedad de áreas. En esencia, un algoritmo es una secuencia de pasos que deben ejecutarse para resolver un problema determinado. El problema no tiene por qué ser necesariamente de programación. Por ejemplo, podemos diseñar algoritmos para actividades como cocinar un plato (una receta no es más que un algoritmo) o construir una mesa.

En el ciclo de desarrollo de software, solo nos interesan los algoritmos relacionados con el software. Para este contexto, un algoritmo se define como un procedimiento no ambiguo para resolver un problema. Un procedimiento es una secuencia finita de pasos u operaciones bien definidas, cada una de las cuales requiere una cantidad finita de memoria y tiempo para completarse. En esta definición

se asume que la terminación (es decir, que el proceso finaliza) es una propiedad esencial de todo procedimiento. A partir de aquí, usaremos los términos *procedimiento*, *algoritmo* y *lógica* como sinónimos.

El desarrollo de un algoritmo implica una serie de pasos. El primero es **definir el problema**. El problema para el cual se va a crear un algoritmo debe estar claramente expresado y bien entendido por la persona responsable de diseñarlo. En el diseño detallado, la definición del problema proviene del diseño del sistema, por lo tanto, ya se dispone de ella al comenzar esta etapa.

El siguiente paso es **desarrollar un modelo matemático del problema**. En esta fase se deben elegir las estructuras matemáticas más adecuadas para representarlo. Puede ser útil observar cómo se resolvieron problemas similares. En la mayoría de los casos, los modelos se construyen a partir de otros ya existentes, adaptándolos a las necesidades del problema actual.

Después se pasa al **diseño del algoritmo en sí**, donde se deciden las estructuras de datos y la estructura del programa. Una vez diseñado el algoritmo, es importante verificar su corrección.

No existe un procedimiento claro y universal para diseñar algoritmos. Tener uno equivaldría a automatizar completamente el proceso de desarrollo de algoritmos, algo que no es posible con los métodos actuales. Sin embargo, existen algunas heurísticas o métodos que pueden ayudar al diseñador.

El método más común para diseñar algoritmos o la lógica de un módulo es la **técnica de refinamiento paso a paso (stepwise refinement)**.

Esta técnica divide el problema de diseño lógico en una serie de pasos para poder desarrollarlo de manera gradual. El proceso comienza convirtiendo las especificaciones del módulo en una descripción abstracta del algoritmo que contiene unas pocas instrucciones generales. En cada paso, una o varias de esas instrucciones se descomponen en pasos más detallados. El refinamiento continúa hasta que todas las instrucciones son lo suficientemente precisas como para poder traducirse fácilmente a un lenguaje de programación.

Durante el refinamiento, tanto los datos como las instrucciones deben detallarse progresivamente. Una guía útil es que, en cada paso, la cantidad de descomposición sea manejable y represente una o dos decisiones de diseño.

El refinamiento paso a paso es un método **de arriba hacia abajo (top-down)** para desarrollar el diseño detallado. Como ya vimos, los métodos de arriba hacia abajo también se usan en el diseño de sistemas.

Para aplicar el refinamiento paso a paso se necesita un lenguaje que permita expresar la lógica de un módulo en distintos niveles de detalle, desde las especificaciones generales hasta la descripción detallada. Dicho lenguaje debe tener la flexibilidad suficiente para adaptarse a diferentes niveles de precisión. Los lenguajes de programación comunes no suelen ser adecuados para esto.

Por esa razón, se utiliza el **PDL (Process Design Language o Lenguaje de Diseño de Procesos)**, ya que tiene una **sintaxis externa formal** que asegura que el diseño desarrollado sea un “algoritmo computacional” que luego puede traducirse fácilmente a código, y una **sintaxis interna flexible basada en lenguaje natural**, lo que permite expresar las ideas con distintos grados de detalle y facilita el refinamiento.

Ejemplo:

Consideremos nuevamente el problema de contar las palabras distintas en un archivo de texto. Supongamos que en el diseño de alto nivel de un gran sistema de procesamiento de texto se ha especificado un módulo llamado COUNT, cuya tarea es determinar el número de palabras diferentes.

Durante el diseño detallado, debemos definir la lógica de este módulo para que cumpla con sus especificaciones. Usaremos el método de refinamiento paso a paso y, para especificar, el PDL con una sintaxis similar a C.

```
int count (file)
FILE file;
word_list wl;
{
    read file into wl
    sort (wl);
    count = different_words (wl);
    printf (count);
}
```

Figura 8.3: Estrategia para el primer paso en el refinamiento gradual.

Una estrategia sencilla para el primer paso se muestra en la Figura 8.3.

Esta estrategia es simple y fácil de entender; es la misma que se propuso en el diagrama de flujo de datos anterior. Las operaciones “primitivas” usadas en esta estrategia son de alto nivel y necesitan ser refinadas. En particular, tres operaciones requieren más detalle:

1. Leer el archivo y crear la lista de palabras.
2. Ordenar la lista de palabras en orden ascendente.
3. Contar las palabras distintas en una lista de palabras ordenada.

Hasta ahora, solo se ha definido una estructura de datos: la lista de palabras. A medida que se refine el diseño, podrían necesitarse más estructuras.

En el siguiente paso de refinamiento, elegimos una de las tres operaciones para desarrollarla con más detalle. Supongamos que refinamos el proceso de lectura. Una estrategia sería leer palabra por palabra y agregarlas a la lista de palabras (Figura 8.4).

```
read_from_file (file, wl)
FILE file;
word_list wl;
{
    initialize wl to empty;
    while not end-of-file {
        get_a_word from file
        add word to wl
    }
}
```

Figura 8.4: Refinamiento de la operación de lectura.

Esta estrategia es directa y puede completarse fácilmente en un paso. Otra opción sería leer grandes bloques de datos en un búfer y formar la lista de palabras desde allí, lo que podría ser más eficiente.

El siguiente paso podría ser refinar la función de conteo. Una estrategia posible para implementarla se muestra en la Figura 8.5.

```
int different_words (wl)
word_list wl;
{
    word last, cur;
    int cnt;

    last = first word in wl
    cnt = 1;
    while not end of list {
        cur = next word from wl
        if (cur <> last) {
            cnt = cnt + 1;
            last = cur;
        }
    }
    return (cnt)
}
```

Figura 8.5: Refinamiento de la función different_words.

De la misma forma, se puede refinar la función de ordenamiento. Una vez realizadas estas refinaciones, el diseño será suficientemente detallado y no requerirá más pasos.

En problemas más complejos, una sola operación puede necesitar muchos niveles de refinamiento. En estos casos, el diseño puede avanzar de dos maneras:

- **Profundidad primero (depth-first):** se termina completamente el refinamiento de una operación antes de pasar a otra.
- **Amplitud primero (breadth-first):** se refinan todas las operaciones una vez, y luego se continúa refinando cada una en sucesivas rondas.

También se puede combinar ambos enfoques.

Por último, vale la pena comparar la estructura de los programas PDL producidos con este método y la de los obtenidos mediante diseño estructurado. Las estructuras no son iguales. La diferencia principal es que, en el refinamiento paso a paso, la función `sort` depende directamente del módulo principal, mientras que en el diseño estructurado sería un submódulo del módulo de entrada.

Esta diferencia no es menor: refleja una distinción en los enfoques. En el refinamiento paso a paso, en cada etapa se especifican las operaciones necesarias (como al crear un diagrama de flujo de datos), mientras que en el diseño estructurado el foco está en dividir el problema en módulos de entrada, salida y transformación, lo que produce una estructura diferente.

8.1.3 Modelado de estado de las clases

En el diseño orientado a objetos, el enfoque recién discutido para obtener el diseño detallado puede no ser suficiente, ya que se centra en especificar la lógica o el algoritmo de los módulos

identificados en el diseño de alto nivel (orientado a funciones). Pero una clase no es una abstracción funcional y no puede verse como un algoritmo.

Un **método** de una clase sí puede considerarse un módulo funcional, y las técnicas mencionadas pueden aplicarse para especificar la lógica de dichos métodos. Sin embargo, para comprender mejor una **clase en su totalidad**, sin entrar en la lógica de cada método individual, se requiere un enfoque distinto del basado en refinamiento.

Un objeto de una clase tiene un **estado** y múltiples **operaciones** que pueden realizarse sobre él. Para entender una clase, es necesario comprender la relación entre su estado y las operaciones, así como el efecto que produce la interacción entre esas operaciones. Este es, de hecho, uno de los objetivos del **diseño detallado** en el desarrollo orientado a objetos. Una vez comprendida la clase en su conjunto, se pueden desarrollar los algoritmos para cada uno de sus métodos.

Cabe recordar que el **enfoque de especificación axiomática** de una clase, discutido anteriormente en este capítulo, también adopta esta perspectiva: en lugar de especificar la funcionalidad de cada operación, define —mediante axiomas— las interacciones entre diferentes operaciones.

Una forma de entender el comportamiento de una clase es modelarla como un **autómata finito (FSA, Finite State Automaton)**.

Un FSA está compuesto por **estados** y **transiciones** entre ellos, las cuales ocurren cuando se producen ciertos **eventos**.

Al modelar un objeto:

- El **estado** corresponde al valor de sus atributos.
- Un **evento** corresponde a la ejecución de una operación sobre el objeto.

Un **diagrama de estados** muestra la relación entre los eventos y los estados, indicando cómo cambia el estado cuando ocurre un evento.

Un diagrama de estados de un objeto suele incluir un **estado inicial**, desde el cual todos los demás estados son alcanzables (es decir, existe un camino desde el estado inicial a cualquier otro estado del autómata).

Sin embargo, el diagrama de estados **no representa todos los estados reales** del objeto, ya que estos pueden ser infinitos o muy numerosos. En su lugar, intenta representar solo los **estados lógicos** del objeto.

Un **estado lógico** es una agrupación de todos aquellos estados desde los cuales el comportamiento del objeto es similar para todos los eventos posibles. Dos estados lógicos son diferentes si difieren en su comportamiento para al menos un evento.

Por ejemplo, para un objeto que representa una **pila (stack)**:

- Todos los estados en los que la pila tiene entre 1 y un número máximo de elementos son equivalentes, ya que las operaciones (**push**, **pop**, **top**, etc.) se comportan de la misma manera en todos ellos.
- El estado que representa una **pila vacía** es distinto, ya que ahora las operaciones **top** y **pop** pueden generar un mensaje de error.
- Del mismo modo, el estado que representa una **pila llena** también es diferente.

El modelo de estados finitos para esta pila de tamaño limitado se muestra en la Figura 8.6.

El **modelado mediante autómatas finitos** de los objetos ayuda a comprender el efecto que las distintas operaciones definidas en una clase tienen sobre el estado del objeto. Una buena comprensión de esto puede facilitar el desarrollo de la lógica de cada operación.

Para desarrollar la lógica de las operaciones pueden utilizarse los métodos habituales de diseño de algoritmos. Además, el modelo de estados puede emplearse para **validar la corrección** de la lógica de una operación.

Como hemos visto, para una clase normalmente no se dispone de una especificación completa de entrada-salida para sus operaciones; por ello, el modelo FSA puede servir como **referencia de validación** para las distintas implementaciones de los métodos.

Más adelante, en el **Capítulo 10**, veremos que un modelo de estados también puede utilizarse para **generar casos de prueba** para la validación.

El **modelado de estado de clases** también ha sido propuesto como una técnica de análisis. Sin embargo, se considera que su utilidad durante la fase de análisis es **limitada** y que su papel es más apropiado durante el **diseño detallado**, cuando se necesita entender en profundidad el funcionamiento interno de una clase.

Incluso en esta etapa, su alcance sigue siendo restringido: resulta más útil cuando existe **una fuerte interacción entre los métodos a través del estado**, o cuando hay **muchos estados distintos** en los que los métodos deben comportarse de manera diferente.

8.2 Verificación

Existen algunas técnicas disponibles para **verificar que el diseño detallado sea consistente con el diseño del sistema**.

El enfoque de la verificación en la fase de diseño detallado se centra en **demostrar que el diseño detallado cumple con las especificaciones establecidas en el diseño del sistema**.

En esta etapa, **no se hace hincapié en validar** que el sistema, tal como ha sido diseñado, **sea consistente con los requisitos del sistema** (esa validación se realiza en fases anteriores).

Los tres métodos de verificación que se consideran son:

- **Revisiones de diseño (design walkthroughs)**
- **Revisiones críticas de diseño (critical design review)**
- **Comprobadores de consistencia (consistency checkers)**

8.2.1 Revisiones de diseño (Design Walkthroughs)

Una **revisión de diseño** o *walkthrough* es un **método manual de verificación**.

La forma en que se definen y utilizan los *walkthroughs* puede variar de una organización a otra. Aquí se describe un modelo general de este tipo de revisión.

Un *walkthrough* se realiza durante una **reunión informal** convocada por el diseñador o por el líder del grupo de diseño.

El grupo de revisión suele ser **pequeño** y está compuesto, además del diseñador, por el líder del grupo y/o otro diseñador del equipo.

A veces, el diseñador puede reunirse simplemente con un colega para realizar el *walkthrough*, o el líder puede solicitarle que lo haga con él.

Durante la sesión, **el diseñador explica paso a paso la lógica del diseño**, mientras que los demás participantes **hacen preguntas, señalan posibles errores o piden aclaraciones**.

Un efecto secundario positivo de este proceso es que, al **explicar y detallar su propio diseño**, el diseñador puede **descubrir errores por sí mismo**.

En esencia, los *walkthroughs* son una **forma de revisión entre pares** (*peer review*).

Sin embargo, debido a su **naturaleza informal**, suelen ser **menos efectivos que una revisión de diseño formal**.

8.2.2 Revisión Crítica del Diseño

El propósito de la revisión crítica del diseño es asegurar que el diseño detallado satisfaga las especificaciones establecidas durante el diseño del sistema. El proceso de revisión crítica del diseño es el mismo que el proceso de inspección, en el cual un grupo de personas se reúne para discutir el diseño con el objetivo de revelar errores de diseño o propiedades indeseables.

El grupo de revisión incluye, además del autor del diseño detallado, a un miembro del equipo de diseño del sistema, al programador responsable de codificar finalmente el(los) módulo(s) en revisión, y a un ingeniero independiente de calidad de software. Durante la revisión del diseño, se debe tener en cuenta que el objetivo es descubrir errores de diseño, no intentar corregirlos. La corrección se realiza más adelante.

El uso de listas de verificación, como en otras revisiones, se considera importante para el éxito de la revisión. La lista de verificación sirve como un medio para enfocar la discusión o la “búsqueda” de errores. Las listas pueden ser utilizadas por cada miembro durante el estudio privado del diseño y durante la reunión de revisión. Para obtener los mejores resultados, la lista de verificación debe adaptarse al proyecto en cuestión, para descubrir errores específicos del proyecto. A continuación se enumeran algunos elementos generales que pueden utilizarse para construir una lista de verificación para una revisión de diseño.

Ejemplo de Lista de Verificación

- ¿Existe cada uno de los módulos del diseño del sistema en el diseño detallado?
- ¿Hay análisis que demuestren que se pueden cumplir los requisitos de rendimiento?
- ¿Están todas las suposiciones explícitamente declaradas y son aceptables?
- ¿Se reflejan todos los aspectos relevantes del diseño del sistema en el diseño detallado?
- ¿Se han manejado las condiciones excepcionales?
- ¿Son todos los formatos de datos consistentes con el diseño del sistema?
- ¿Está el diseño estructurado y cumple con los estándares locales?
- ¿Se han estimado los tamaños de las estructuras de datos? ¿Se han tomado precauciones para evitar desbordamientos?
- ¿Está cada enunciado especificado en lenguaje natural fácilmente codificable?
- ¿Están correctamente especificadas las condiciones de terminación de los bucles?
- ¿Son correctas las condiciones en los bucles?
- ¿Son correctas las condiciones en las sentencias “if”?

- ¿Es adecuada la anidación?
- ¿Es la lógica del módulo demasiado compleja?
- ¿Son los módulos altamente cohesivos?

8.2.3 Verificadores de Consistencia

Las revisiones y recorridos del diseño son procesos manuales; las personas involucradas en ellos son quienes determinan los errores en el diseño. Si el diseño está especificado en un **PDL** (Lenguaje de Descripción de Programa) u otro lenguaje de diseño formalmente definido, es posible detectar algunos defectos de diseño utilizando **verificadores de consistencia**.

Los verificadores de consistencia son esencialmente compiladores que toman como entrada el diseño especificado en un lenguaje de diseño (PDL, en nuestro caso). Claramente, no pueden producir código ejecutable porque la sintaxis interna del PDL permite el uso de lenguaje natural y muchas actividades se especifican en lenguaje natural. Sin embargo, las especificaciones de las interfaces de los módulos (que pertenecen a la sintaxis externa) están especificadas formalmente.

Un verificador de consistencia puede asegurarse de que todos los módulos invocados o utilizados por un módulo dado realmente existan en el diseño, y que la interfaz usada por el llamador sea consistente con la definición de la interfaz del módulo llamado. También puede verificar si los elementos de datos globales utilizados están efectivamente definidos globalmente en el diseño.

Dependiendo de la precisión y la sintaxis del lenguaje de diseño, los verificadores de consistencia pueden generar otra información adicional. Además, estas herramientas pueden utilizarse para calcular la complejidad de los módulos y otras métricas, ya que dichas métricas se basan en construcciones alternativas y de bucles, las cuales tienen una sintaxis formal en PDL.

El compromiso aquí es que, cuanto más formal sea el lenguaje de diseño, más verificaciones podrán realizarse durante la etapa de diseño; pero el costo es que el lenguaje de diseño se vuelve menos flexible y tiende a parecerse más a un lenguaje de programación.

8.3 Métricas

Después del diseño detallado, la lógica del sistema y las estructuras de datos quedan en gran medida especificadas. Solo los detalles orientados a la implementación —que a menudo dependen del lenguaje de programación utilizado— necesitan definirse con mayor precisión.

Por lo tanto, muchas de las métricas que tradicionalmente se asocian con el código pueden utilizarse de manera efectiva después del diseño detallado. Durante esta etapa, todas las métricas tratadas en el diseño del sistema siguen siendo aplicables y útiles.

Con la lógica de los módulos ya disponible tras el diseño detallado, tiene sentido hablar de la **complejidad de un módulo**. Tradicionalmente, las métricas de complejidad se aplican al código, pero también pueden aplicarse fácilmente al diseño detallado. A continuación, se describen algunas métricas aplicables al diseño detallado.

8.3.1 Complejidad Ciclomática

Basado en la capacidad del cerebro humano y en la experiencia práctica, se reconoce generalmente que las condiciones y las sentencias de control agregan complejidad a un programa. Dado dos programas del mismo tamaño, el que tiene un mayor número de sentencias de decisión probablemente será más complejo.

La medida más simple de complejidad, entonces, es el número de construcciones que representan **ramificaciones en el flujo de control** del programa, como las sentencias `if-then-else`, `while-do`, `repeat-until` y `goto`.

Una medida más refinada es la **complejidad ciclomática** propuesta por *McCabe*, basada en conceptos de teoría de grafos. Para un grafo GGG con n nodos, e aristas y p componentes conectados, el número ciclomático $V(G)$ se define como:

$$V(G) = e - n + p$$

Para definir la complejidad ciclomática de un módulo, primero se dibuja el **grafo de flujo de control** GGG del módulo. Para construirlo, se divide el módulo en bloques delimitados por sentencias que afectan el flujo de control (como `if`, `while`, `repeat`, `goto`). Estos bloques forman los nodos del grafo. Si el control puede pasar de un bloque i a un bloque j , se dibuja un arco de i a j .

El grafo tiene un **nodo de entrada** y un **nodo de salida**, correspondientes al primer y último bloque de sentencias. En tal grafo, debe existir un camino desde la entrada hacia cualquier nodo y desde cualquier nodo hacia la salida (suponiendo que no haya código inalcanzable).

Si el código es una secuencia lineal sin estructuras de control, el número ciclomático puede ser 0. Sin embargo, se agrega un arco del nodo de salida al de entrada para hacer el grafo *fuertemente conexo*, garantizando así una complejidad distinta de cero incluso para programas simples. Por lo tanto, la **complejidad ciclomática de un módulo** se define como el número ciclomático de este grafo fuertemente conexo.

El número ciclomático de un módulo equivale al **número máximo de circuitos linealmente independientes** del grafo. Un conjunto de circuitos es linealmente independiente si ningún circuito está totalmente contenido en otro ni puede expresarse como combinación de otros. Por lo tanto, para calcular la complejidad ciclomática, se puede:

1. Dibujar el grafo,
2. Conectarlo agregando un arco de salida a entrada, y
3. Contar los circuitos o aplicar la fórmula $V(G) = e - n + p$.

Por ejemplo, en un grafo con 10 aristas, 7 nodos y un componente, se obtiene:

$$V(G') = 10 - 7 + 1 = 4$$

Los circuitos independientes serían:

- ckt 1: b c e b
- ckt 2: b c d e b

- ckt 3: a b f a
- ckt 4: a g a

También puede demostrarse que la **complejidad ciclomática** de un módulo es igual al **número de decisiones más uno**, donde una decisión corresponde a cualquier sentencia condicional en el módulo.

En el ejemplo dado, el módulo tiene tres decisiones (dos `while` y un `if`), por lo que la complejidad ciclomática es $3+1=4$.

El número ciclomático es una medida cuantitativa de la **complejidad de un módulo**. Aunque puede extenderse al programa completo, resulta más útil a nivel modular. McCabe propuso mantener la complejidad ciclomática de los módulos **por debajo de 10**.

Además, este número puede utilizarse como una **guía para determinar cuántos caminos deben probarse** durante las pruebas. La complejidad ciclomática es una de las métricas de complejidad más utilizadas.

Los experimentos muestran que la complejidad ciclomática está altamente correlacionada con el tamaño del módulo (en escala logarítmica), ya que más líneas de código suelen implicar más decisiones. También se ha encontrado correlación con el **número de errores** detectados en los módulos.

8.3.2 Vínculos de datos

Ya hemos visto que el **acoplamiento** y la **cohesión** son conceptos clave para evaluar un diseño. Sin embargo, para que esta evaluación sea realmente efectiva, es necesario contar con **métricas que permitan medir** el grado de acoplamiento entre módulos o la cohesión dentro de un módulo.

Durante el **diseño del sistema**, intentamos cuantificar el acoplamiento basándonos en el flujo de información entre módulos. Ahora que también conocemos la lógica interna de los módulos, podemos definir métricas que consideren tanto la estructura como el comportamiento. Una de estas métricas es el **vínculos de datos** (*data binding*), que busca capturar las interacciones de datos entre distintas partes del sistema de software. En otras palabras, los enlaces de datos miden **qué tan fuertemente acoplados están los diferentes módulos** del sistema. Existen distintos tipos de enlaces de datos.

Un **enlace de datos potencial** se define como un triplete (p, x, q) , donde p y q son módulos, y x es una variable que está dentro del **ámbito estático** de ambos. Esto refleja la posibilidad de que los módulos p y q puedan comunicarse mediante la variable compartida x . Este tipo de enlace no considera la lógica interna de los módulos para verificar si realmente acceden a x ; se basa únicamente en la declaración de la variable.

Un **enlace de datos usado** es un enlace potencial en el que tanto p como q efectivamente **usan la variable x** (ya sea para leer o asignar un valor). Este tipo de enlace es más difícil de determinar que el potencial, ya que requiere conocer más detalles sobre la lógica interna de los módulos.

Un **enlace de datos real** es un enlace usado con la condición adicional de que **el módulo p asigna un valor a x y el módulo q utiliza ese valor**. Este es el tipo más complejo de calcular, pues representa el flujo real de información desde el módulo p hacia el módulo q a través de la variable compartida x . Su cálculo requiere una descripción detallada de la lógica de ambos módulos.

Todos estos tipos de enlaces buscan **representar la fuerza de las interconexiones entre módulos**. Cuanto mayor sea el número de enlaces entre dos módulos, más estrechamente conectados estarán.

Para un tipo específico de enlace, puede construirse una **matriz** que contenga el número de enlaces entre los distintos módulos. Dicha matriz puede utilizarse luego en **análisis estadísticos** para evaluar el grado de interconexión del sistema o de alguno de sus subsistemas.

8.3.3 Métrica de Cohesión

Aquí se presenta un intento de **cuantificar la cohesión de un módulo**.

Para calcular el valor de la métrica de cohesión de un módulo M, se construye un **grafo de flujo** G para dicho módulo. Cada **vértice** en G representa una instrucción ejecutable de M. Para cada nodo, se registra también la **variable referenciada** en esa instrucción. Existe un **arco** desde un nodo S_i hacia otro nodo S_j si la instrucción S_j puede ejecutarse inmediatamente después de S_i en alguna ejecución del módulo.

Además, se agregan dos nodos especiales:

- un **nodo inicial** I, desde el cual comienza la ejecución del módulo, y
- un **nodo final** T, donde termina la ejecución.

Para las instrucciones de terminación (por ejemplo, `return`, `exit`), se dibuja un arco desde la instrucción correspondiente hacia T.

A partir de G, se construye un **grafo de flujo reducido** eliminando los nodos que **no hacen referencia a ninguna variable** (como los `goto` sin restricciones). Todos los arcos que llegaban al nodo eliminado se redirigen hacia su **sucesor** (estos nodos tienen solo un sucesor).

Supongamos que las variables están numeradas secuencialmente como 1, 2, ..., n.

Para una variable i:

- R_i es el **conjunto de referencia**, es decir, el conjunto de todas las instrucciones ejecutables que hacen referencia a la variable i.
La unión de todos los R_i constituye el conjunto de todos los nodos del grafo (excepto el nodo T, que no se ejecuta).

En esencia, esta métrica intenta medir la cohesión de un módulo observando **cuántos caminos independientes del módulo atraviesan las distintas instrucciones**.

La idea es que, si un módulo tiene **alta cohesión**, la mayoría de las variables serán utilizadas en la mayoría de los caminos.

Por lo tanto, en un módulo con alta cohesión, la cohesión de los conjuntos de referencia de cada variable será alta.

El **valor máximo posible** de cohesión ocurre cuando la dimensión de todos los conjuntos de referencia es igual al número total de caminos independientes, es decir, igual a la **complejidad ciclomática** del módulo.

En otras palabras, la cohesión máxima se alcanza cuando **todos los caminos independientes utilizan todas las variables** del módulo.

9 Codificación

El objetivo de la actividad de codificación o programación es **implementar el diseño de la mejor manera posible**.

La actividad de codificación influye profundamente tanto en las pruebas como en el mantenimiento. Como vimos antes, el tiempo dedicado a la codificación representa **un pequeño porcentaje del costo total del software**, mientras que las pruebas y el mantenimiento **consumen el mayor porcentaje**.

Por lo tanto, debe quedar claro que **el objetivo durante la codificación no debe ser reducir el costo de implementación**, sino **reducir el costo de las fases posteriores**, incluso si eso significa que el costo de esta fase debe aumentar. En otras palabras, **el objetivo durante esta fase no es simplificar el trabajo del programador**, sino **simplificar el trabajo del tester y del encargado del mantenimiento**.

Esta distinción es importante, ya que los programadores a menudo se preocupan por cómo terminar su trabajo rápidamente, sin tener en cuenta las fases posteriores. Durante la codificación, se debe tener presente que **los programas no deben construirse para que sean fáciles de escribir, sino para que sean fáciles de leer y entender**.

Un programa se lee con mucha más frecuencia y por muchas más personas en las fases posteriores.

Existen muchos criterios diferentes para juzgar un programa, entre ellos: **su legibilidad, tamaño, tiempo de ejecución y memoria requerida**.

Tener la **legibilidad y la comprensibilidad** como objetivos claros de la actividad de codificación puede, por sí mismo, **contribuir a producir software más mantenible**.

Un famoso experimento realizado por **Weinberg** mostró que si a los programadores se les especifica un objetivo claro para el programa, **suelen cumplirlo**. En el experimento, **cinco equipos diferentes** recibieron el mismo problema para desarrollar un programa, pero **a cada equipo se le indicó un objetivo distinto** que debía cumplir.

Los diferentes objetivos dados fueron:

- Minimizar el esfuerzo necesario para completar el programa.
- Minimizar el número de sentencias.
- Minimizar la memoria requerida.
- Maximizar la claridad del programa.
- Maximizar la claridad de la salida.

Se observó que, en la mayoría de los casos, **cada equipo obtuvo el mejor resultado en el objetivo que se le había asignado**.

La clasificación de los diferentes equipos para los distintos objetivos se muestra en la **Figura 9.1**.

	Resulting Rank (1 = Best)				
	O1	O2	O3	O4	O5
Minimize effort to complete (O1)	1	4	4	5	3
Minimize number of statements (O2)	2-3	1	2	3	5
Minimize memory required (O3)	5	2	1	4	4
Maximize program clarity (O4)	4	3	3	2	2
Maximize output clarity (O5)	2-3	5	5	1	1

Figure 9.1: El experimento de Weinberg.

El experimento muestra claramente que, **si los objetivos están bien definidos, los programadores tienden a alcanzarlos.**

Por lo tanto, si la **legibilidad** es un objetivo de la actividad de codificación, es probable que los programadores desarrollen programas **fáciles de entender.**

Para nuestros propósitos, la **facilidad de comprensión y de modificación** son los **objetivos básicos** de la actividad de programación.

En este capítulo, primero discutiremos **algunas prácticas y guías de programación**, en las que también mencionaremos algunos **errores comunes de codificación** para que los estudiantes tomen conciencia de ellos.

Luego discutiremos **algunos procesos que se siguen durante la codificación.**

A continuación, hablaremos sobre la **refactorización**, que se realiza durante la codificación, pero constituye una actividad distinta.

Después trataremos **algunos métodos de verificación**, seguidos por una **discusión sobre métricas.**

Finalmente, **concluiremos el capítulo** con una discusión sobre la **implementación de los estudios de caso.**

9.1 Principios y guías de programación

La tarea principal de un programador es **escribir código de calidad con pocos errores.**

La restricción adicional es hacerlo **rápidamente.**

Escribir código sólido es una **habilidad que solo puede adquirirse con la práctica.**

Sin embargo, basándose en la experiencia, se pueden establecer **algunas reglas y guías generales** para el programador.

La **buena programación** (es decir, producir programas **correctos y simples**) es una práctica **independiente del lenguaje de programación utilizado**, aunque los lenguajes de programación bien estructurados **facilitan el trabajo del programador.**

En esta sección, discutiremos **algunos conceptos y prácticas** que pueden ayudar al programador a **escribir código de mayor calidad.**

Dado que una de las tareas clave de un programador es **evitar errores en los programas**, comenzaremos analizando **algunos errores comunes de codificación.**

9.1.1 Errores comunes de codificación

Los errores de software (en esta sección utilizaremos los términos *errores*, *defectos* y *bugs* indistintamente; las definiciones precisas se dan en el siguiente capítulo) son una realidad con la que todos los programadores deben lidiar.

Gran parte del esfuerzo en el desarrollo de software se dedica a identificar y eliminar errores. Existen diversas prácticas que pueden reducir la aparición de bugs, pero sin importar las herramientas o métodos que usemos, los errores van a seguir apareciendo en los programas.

Aunque los errores pueden surgir de muchas maneras diferentes, ciertos tipos ocurren con mayor frecuencia.

A continuación, presentamos una lista de algunos de los errores más comunes.

El principal objetivo de discutirlos es educar a los programadores sobre estos errores, de modo que puedan evitarlos.

Fugas de memoria (Memory Leaks)

Una fuga de memoria ocurre cuando se asigna memoria a un programa pero luego no se libera correctamente.

Este error es una fuente común de fallos de software, y aparece con frecuencia en lenguajes que no tienen recolección automática de basura, como C o C++.

En programas cortos, las fugas de memoria tienen poco impacto, pero en sistemas que se ejecutan durante largos períodos pueden tener efectos catastróficos.

Un sistema de software con fugas de memoria consume cada vez más memoria, hasta que en algún momento el programa puede detenerse abruptamente debido a la falta de memoria libre.

Un ejemplo de este tipo de error es el siguiente:

```
char* foo(int s)
{
    char *output;
    if (s>0)
        output=(char*) malloc (size);
    if (s==1)
        return NULL; /* if s==1 then mem leaked */
    return(output);
}
```

Liberar un recurso ya liberado

En general, en los programas, los recursos primero se asignan y luego se liberan. Por ejemplo, la memoria se asigna primero y después se desasigna.

Este error ocurre cuando el programador intenta liberar un recurso que ya fue liberado previamente.

El impacto de este error común puede ser catastrófico.

Un ejemplo de este tipo de error es el siguiente:

```

main()
{
    char *str;
    str=(char *)malloc(10);
    if (global==0)
        free(str);
    free(str); /* str is already freed
}

```

El impacto de este error puede ser aún más grave si existe alguna instrucción `malloc` entre las dos llamadas a `free`.

En ese caso, existe la posibilidad de que la ubicación de memoria liberada por primera vez sea reasignada a una nueva variable, y que la segunda llamada a `free` libere esa nueva variable por error.

Desreferenciación de NULL

Este error ocurre cuando se intenta acceder al contenido de una ubicación de memoria que apunta a **NULL**.

Es un error muy común que puede hacer fallar completamente un sistema de software. Además, es difícil de detectar, ya que la desreferenciación de **NULL** puede ocurrir solo en ciertos caminos de ejecución y bajo determinadas condiciones.

A menudo, este problema surge debido a una inicialización incorrecta en distintos caminos del programa, lo que lleva a que una variable no se asigne correctamente antes de ser utilizada.

También puede deberse al uso de alias, es decir, cuando dos variables apuntan al mismo objeto: si una de ellas se libera y luego se intenta desreferenciar la otra, se produce el error.

El siguiente fragmento de código muestra dos casos de desreferenciación de **NULL**.

```

char *ch=NULL;
if (x>0)
{
    ch='c';
}
printf("\%c", *ch); /* ch may be NULL
*ch=malloc(size);
ch = 'c'; /* ch will be NULL if malloc returns NULL

```

Similar al error de **desreferenciación de NULL** está el error de **acceso a memoria no inicializada**. Este problema ocurre con frecuencia cuando los datos **se inicializan en la mayoría de los casos**, pero **algunas situaciones no previstas** quedan **sin cubrir** y, por lo tanto, **la variable no se inicializa antes de ser usada**.

Un ejemplo de este error es el siguiente:

```

switch( i )
{
    case 0: s=OBJECT_1; break;
    case 1: s=OBJECT_2; break;
}
return (s); /* s not initialized for values
            other than 0 or 1 */

```

Falta de direcciones únicas

El **aliasing** genera muchos problemas, y uno de ellos es la **violación de la unicidad de direcciones** cuando se espera que sean diferentes.

Por ejemplo, en una función de **concatenación de cadenas**, se espera que las **direcciones de origen y destino sean distintas**.

Si esto no ocurre —como en el segmento de código que se muestra a continuación— puede provocar **errores en tiempo de ejecución**.

```
strcat(src, destn);  
/* In above function, if src is aliased to destn,  
 * then we may get a runtime error */
```

Errores de sincronización

En un programa paralelo, donde hay múltiples hilos que pueden acceder a recursos compartidos, pueden ocurrir errores de sincronización. Estos errores son muy difíciles de detectar, ya que no se manifiestan fácilmente. Sin embargo, cuando aparecen, pueden causar daños serios al sistema.

Existen diferentes categorías de errores de sincronización, entre las cuales se encuentran:

1. Interbloqueos (**Deadlocks**)
2. Condiciones de carrera (**Race conditions**)
3. Sincronización inconsistente
 - Un **interbloqueo(dl)** ocurre cuando **uno o más hilos se bloquean mutuamente**. La causa más frecuente es la secuencia de bloqueo inconsistente, donde los hilos en interbloqueo esperan recursos que, a su vez, están bloqueados por otro hilo.
 - Las **condiciones de carrera** ocurren cuando **dos hilos intentan acceder al mismo recurso** y el **resultado de la ejecución depende del orden en que se ejecutan los hilos**.
 - La sincronización inconsistente es un error común que ocurre cuando hay una mezcla de accesos bloqueados y no bloqueados a algunas variables compartidas, especialmente si los accesos implican actualizaciones.

9.1.2 Programación Estructurada

Como se mencionó anteriormente, el objetivo básico de la actividad de codificación es producir **programas que sean fáciles de entender**. Muchos han argumentado que la práctica de la **programación estructurada** ayuda a desarrollar programas más comprensibles. El movimiento de programación estructurada comenzó en la década de 1970, y se ha hablado y escrito mucho al respecto. Hoy en día, el concepto está tan difundido que se acepta generalmente —incluso se da por sentado— que la programación debe ser estructurada.

Aunque se ha hecho mucho hincapié en la programación estructurada, el concepto y la motivación detrás de ella a menudo no se entienden completamente. A menudo se considera que la programación estructurada es programación "sin goto". Aunque el uso extensivo de **goto** ciertamente no es deseable, **se pueden escribir programas estructurados usando goto**. Aquí ofrecemos una breve explicación de qué es la programación estructurada.

Un programa tiene una **estructura estática** y una **estructura dinámica**.

- La **estructura estática** es la **organización del texto del programa**, que generalmente es una organización lineal de las sentencias del programa.
- La **estructura dinámica** es la **secuencia de sentencias que se ejecutan durante la ejecución del programa**.

En otras palabras, tanto la estructura estática como el comportamiento dinámico son secuencias de sentencias; mientras que la secuencia que representa la estructura estática es fija, la secuencia de sentencias que se ejecuta puede cambiar de una ejecución a otra.

La noción general de corrección de un programa significa que, cuando el programa se ejecuta, produce el comportamiento deseado. Para demostrar que un programa es correcto, necesitamos mostrar que **su comportamiento durante la ejecución es el esperado**. Por lo tanto, al analizar un programa, ya sea formalmente para probar su corrección o informalmente para depurarlo o convencernos de que funciona, **estudiamos la estructura estática del programa (su código) pero tratamos de razonar sobre su comportamiento dinámico**.

En otras palabras, **gran parte de la actividad de entender un programa consiste en entender su comportamiento dinámico a partir del texto del programa**. Será más fácil entender el comportamiento dinámico si este se asemeja a la estructura estática. Cuanto más cercana sea la correspondencia entre la ejecución y la estructura del texto, más fácil será entender el programa; mientras más diferente sea la estructura durante la ejecución, más difícil será razonar sobre el comportamiento a partir del código.

El **objetivo de la programación estructurada es garantizar que la estructura estática y la dinámica sean las mismas**. Es decir, se busca escribir programas de manera que la secuencia de sentencias ejecutadas durante la ejecución coincida con la secuencia de sentencias en el texto del programa. Dado que las sentencias en un texto de programa están organizadas linealmente, el objetivo de la programación estructurada se convierte en desarrollar programas cuyo flujo de control durante la ejecución sea lineal y siga la organización lineal del texto.

Claramente, ningún programa significativo puede escribirse como una secuencia de sentencias simples sin ningún desvío o repetición (lo cual también implica desvío). Entonces, ¿cómo se logra linearizar el flujo de control?

Mediante el uso de constructos estructurados.

En la programación estructurada, una sentencia no es solo una sentencia de asignación simple, sino una **sentencia estructurada**. La **propiedad clave** de una sentencia estructurada es que tiene **una sola entrada y una sola salida**. Es decir, durante la ejecución, la sentencia comienza desde un punto definido y termina en un punto definido.

Con sentencias de **entrada y salida única**, podemos ver un programa como una **secuencia de sentencias estructuradas**. Y si todas las sentencias son estructuradas, entonces durante la ejecución, la secuencia de ejecución será la misma que la secuencia en el texto del programa. Por lo tanto, usando sentencias de entrada y salida única, se puede lograr la correspondencia entre la estructura estática y la dinámica. Las sentencias de una sola entrada y una sola salida más comúnmente utilizadas son:

Selection: if B then S1 else S2

if B then S1

Iteration: While B do S

repeat S until B

Sequencing: S1; S2; S3;...

Se puede demostrar que estas tres construcciones básicas son suficientes para programar cualquier algoritmo concebible.

Los lenguajes modernos tienen otras construcciones similares que ayudan a linearizar el flujo de control de un programa, lo cual, en términos generales, facilita su comprensión.

Por lo tanto, los programas deberían escribirse de modo que, en la medida de lo posible, se utilicen estructuras de control de una sola entrada y una sola salida.

El objetivo principal, como hemos tratado de enfatizar, es hacer que la lógica del programa sea fácil de entender.

No se puede formular una regla estricta que sea aplicable en todas las circunstancias.

La práctica de la programación estructurada constituye una buena base y una guía útil para escribir programas de forma clara.

Debe señalarse que la razón principal por la cual se promovió la programación estructurada es la verificación formal de programas.

Como veremos más adelante en este capítulo, durante la verificación, un programa se considera una secuencia de sentencias ejecutables, y la verificación avanza paso a paso, considerando una sentencia de la lista (el programa) a la vez.

En estos métodos de verificación se asume que, durante la ejecución, las sentencias se ejecutarán en el mismo orden en que aparecen en el texto del programa.

Si esta suposición se cumple, la tarea de verificación se simplifica.

Por lo tanto, incluso desde el punto de vista de la verificación, es importante que la secuencia de ejecución de las sentencias coincida con su orden en el texto.

Una nota final sobre las construcciones estructuradas:

no cualquier fragmento de código con una sola entrada y una sola salida puede considerarse una construcción estructurada.

Si así fuera, uno siempre podría definir unidades apropiadas en cualquier programa para que pareciera una secuencia de dichas unidades (en el peor de los casos, todo el programa podría definirse como una sola unidad).

El objetivo fundamental de utilizar construcciones estructuradas es linearizar el flujo de control para que el comportamiento en tiempo de ejecución sea más fácil de entender y razonar sobre él.

En un flujo de control linealizado, si comprendemos adecuadamente el comportamiento de cada una de las construcciones básicas, entonces el comportamiento del programa puede considerarse una composición de los comportamientos de las distintas sentencias.

Para que este enfoque funcione, se implica que podamos entender con claridad el comportamiento de cada construcción.

Esto requiere que seamos capaces de describir de manera concisa el comportamiento de cada construcción.

A menos que podamos hacer esto, no será posible componerlas.

Claramente, para una estructura arbitraria no podemos hacer esto simplemente porque tenga una sola entrada y una sola salida.

Desde este punto de vista, las estructuras mencionadas anteriormente son las que se eligen como sentencias estructuradas, ya que existen reglas bien definidas que especifican cómo se comportan durante la ejecución, lo que nos permite razonar sobre programas más grandes.

En general, puede decirse que la programación estructurada conduce a programas más fáciles de entender que los no estructurados, y que dichos programas son también más fáciles (relativamente

hablando) de verificar formalmente.

Sin embargo, debe tenerse en cuenta que la programación estructurada no es un fin en sí misma.

Nuestro objetivo básico es que el programa sea fácil de comprender, y la programación estructurada es una manera segura de lograr este objetivo.

Aun así, existen algunas prácticas comunes de programación actualmente bien entendidas que hacen uso de construcciones no estructuradas (por ejemplo, las sentencias `break` o `continue`).

Aunque se debe procurar evitar el uso de sentencias que violen efectivamente la propiedad de una sola entrada y una sola salida, si el uso de tales sentencias es la forma más simple de organizar el programa, entonces, desde el punto de vista de la legibilidad, deberían utilizarse.

El punto principal es que una construcción no estructurada solo debería usarse si la alternativa estructurada es más difícil de entender.

Esta visión puede adoptarse únicamente porque estamos poniendo el foco en la legibilidad.

Si el objetivo fuera la verificabilidad formal, la programación estructurada sería probablemente necesaria.

9.1.3 Ocultamiento de Información

Una solución de software a un problema siempre contiene estructuras de datos que están destinadas a representar la información del dominio del problema.

Es decir, cuando se desarrolla un software para resolver un problema, este utiliza ciertas estructuras de datos para capturar la información presente en dicho dominio.

En general, solo ciertas operaciones se realizan sobre una determinada información.

Es decir, una pieza de información en el dominio del problema se usa solo de un número limitado de maneras.

Por ejemplo, un libro contable en una oficina de contabilidad tiene usos muy definidos: debitación, acreditación, consulta del saldo actual, etc.

Una operación como “multiplicar todos los débitos entre sí y luego dividirlos por la suma de todos los créditos” no suele realizarse.

Por lo tanto, cualquier tipo de información en el dominio del problema normalmente tiene un pequeño conjunto de operaciones definidas que se aplican sobre ella.

Cuando esa información se representa mediante estructuras de datos, debe aplicarse el mismo principio:

solo deben realizarse operaciones definidas sobre esas estructuras de datos.

Esto, esencialmente, es el principio del ocultamiento de información.

La información capturada en las estructuras de datos debe ocultarse del resto del sistema, y solo las funciones de acceso que representen las operaciones válidas sobre esa información deben ser visibles.

En otras palabras, cuando la información del dominio del problema se captura en estructuras de datos, para cada operación que se pueda realizar sobre dicha información, se debe proporcionar una función de acceso.

Y dado que, en el dominio del problema, el resto del sistema solo realiza esas operaciones definidas sobre la información, los demás módulos del software deben usar únicamente esas funciones de acceso para manipular las estructuras de datos.

El ocultamiento de información puede reducir el acoplamiento entre módulos y hacer que el sistema sea más fácil de mantener.

También es una herramienta eficaz para manejar la complejidad del desarrollo de software: al utilizar el ocultamiento de información, se separa la preocupación de manejar los datos de la preocupación de usarlos para producir los resultados deseados.

Muchos de los lenguajes más antiguos, como Pascal, C y FORTRAN, no proporcionan mecanismos para soportar la abstracción de datos.

Con esos lenguajes, el ocultamiento de información solo puede lograrse mediante un uso disciplinado del lenguaje, es decir, imponiendo manualmente restricciones de acceso por parte de los programadores, ya que el lenguaje no las proporciona.

En cambio, la mayoría de los lenguajes modernos orientados a objetos sí ofrecen mecanismos lingüísticos que permiten implementar el ocultamiento de información.

9.2 Proceso de Codificación

La actividad de codificación comienza cuando ya se ha realizado alguna forma de diseño y se dispone de las especificaciones de los módulos que deben desarrollarse.

Con el diseño en mano, los módulos suelen asignarse a desarrolladores individuales para su implementación.

En una implementación de arriba hacia abajo (top-down), se comienza asignando los módulos que están en la parte superior de la jerarquía y luego se avanza hacia los niveles inferiores.

En cambio, en una implementación de abajo hacia arriba (bottom-up), el desarrollo empieza por implementar los módulos de nivel más bajo de la jerarquía y luego se avanza hacia los superiores. La forma en que se elija proceder tiene un impacto directo en la integración y las pruebas.

Una vez que los módulos se asignan a los desarrolladores, estos utilizan algún proceso para desarrollar el código.

A continuación, se presentan algunos de los procesos que los desarrolladores suelen usar durante la codificación, o que han sido propuestos para guiar esta etapa.

9.2.1 Un Proceso de Codificación Incremental

El proceso que muchos desarrolladores siguen consiste en escribir el código del módulo que tienen asignado, y cuando lo terminan, realizar pruebas unitarias sobre él y corregir los errores que se encuentren. Luego, el código se sube al repositorio del proyecto para ponerlo a disposición de los demás integrantes del equipo.

(Explicaremos más adelante el proceso de *check-in* o subida del código).

Un proceso mejor para la codificación —que suelen seguir los desarrolladores con más experiencia— es desarrollar el código de manera incremental.

Esto significa escribir código que implemente solo una parte de la funcionalidad del módulo.

Ese código se compila y se prueba rápidamente con algunos tests simples, para verificar que la parte escrita funcione correctamente.

Una vez que el código pasa esas pruebas, el desarrollador agrega más funcionalidad, y vuelve a probar.

En otras palabras, el código se construye de manera incremental, probándolo a medida que se

desarrolla.

Este proceso de codificación se muestra en la Figura 9.2.

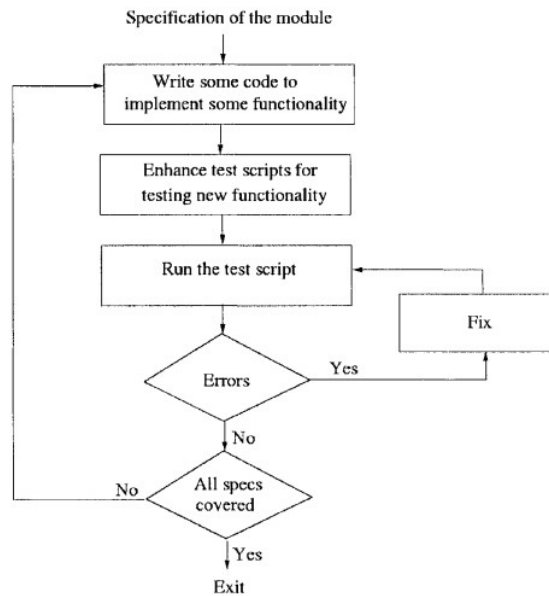


Figure 9.2: Un proceso de codificación incremental.

La ventaja principal de desarrollar código de forma incremental, realizando pruebas después de cada ronda de codificación, es que facilita la depuración (debugging):

si se encuentra un error durante las pruebas, se puede atribuir con seguridad al código que se agregó desde la última prueba exitosa.

Para seguir este proceso, es fundamental disponer de scripts de prueba automatizados, que permitan ejecutar los casos de prueba con solo presionar un botón.

Con estos scripts, las pruebas pueden realizarse tan frecuentemente como se desee, y además es fácil agregar nuevos casos de prueba.

Estos scripts de prueba son de gran ayuda cuando, en el futuro, se mejora o modifica el código debido a cambios en los requisitos: gracias a ellos se puede verificar rápidamente que la funcionalidad anterior sigue funcionando correctamente.

Incluso, estos scripts pueden reutilizarse —con algunas mejoras— para realizar las pruebas unitarias finales, que suelen hacerse antes de subir el módulo al repositorio.

9.2.2 Desarrollo Guiado por Pruebas (Test Driven Development)

El Desarrollo Guiado por Pruebas (TDD) es un proceso de codificación que invierte el enfoque tradicional de desarrollo.

En TDD, el programador primero escribe los scripts de prueba, y luego escribe el código necesario para que esas pruebas pasen exitosamente.

Todo el proceso se realiza de manera incremental, escribiendo las pruebas a partir de las especificaciones, y desarrollando el código para satisfacerlas.

El proceso de TDD se muestra en la Figura 9.3.

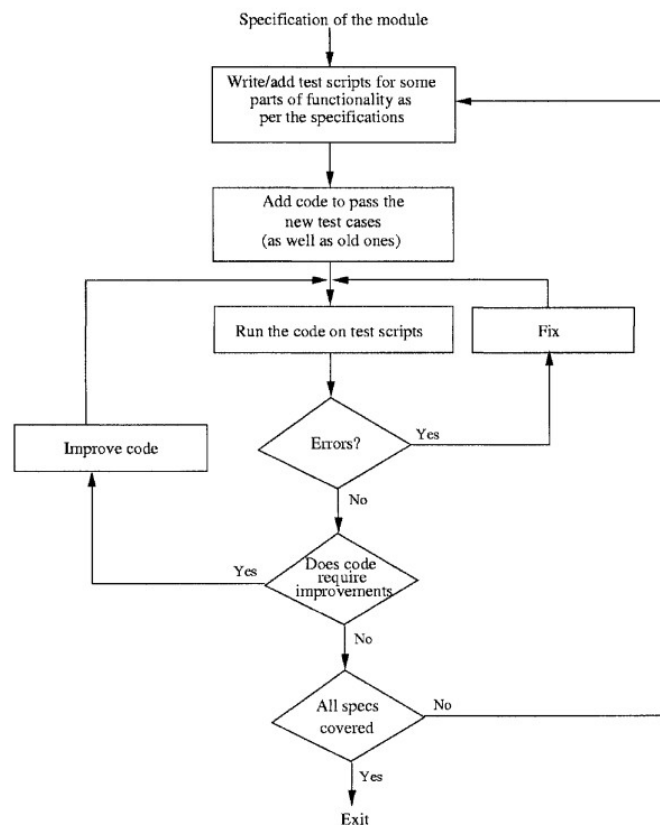


Figure 9.3: Proceso de desarrollo guiado por pruebas.

Este es un enfoque relativamente nuevo, que ha sido adoptado dentro de la metodología de Programación Extrema (Extreme Programming, XP).

Sin embargo, el concepto de TDD es general y no está limitado a ninguna metodología en particular.

Hay algunos puntos importantes que destacar sobre el TDD:

Primero, este enfoque establece que solo se debe escribir el código necesario para pasar las pruebas. Siguiendo esta idea, el código siempre se mantiene sincronizado con las pruebas.

Esto no siempre sucede con el enfoque tradicional de “código primero”, donde es común escribir una gran cantidad de código y luego crear solo unas pocas pruebas que cubren una parte del mismo.

Al promover que el código se escriba únicamente para pasar los tests, la responsabilidad de garantizar que se implemente toda la funcionalidad requerida recae en la etapa de escritura de los casos de prueba.

Es decir, son los casos de prueba los que deben verificar que el código desarrollado cubra toda la funcionalidad necesaria.

El hecho de escribir los casos de prueba antes del código hace que el desarrollo sea guiado por el uso (*usage-driven*).

Primero, se define cómo se espera que el código sea utilizado, lo cual se extrae de las especificaciones, y las interfaces de uso se definen con precisión al escribir los casos de prueba. Esto ayuda a asegurar que las interfaces se diseñen desde la perspectiva del usuario del código, y que los escenarios clave de uso se consideren antes de comenzar a programar.

El enfoque se centra en los usuarios del código, y el código se escribe para satisfacer sus necesidades, lo que puede reducir los errores de interfaz.

En TDD, también ocurre de forma natural cierta priorización del desarrollo.

Es muy probable que las primeras pruebas se enfoquen en la funcionalidad principal, mientras que las pruebas de características secundarias o de menor prioridad se escriban más adelante.

Como resultado, el código de las funcionalidades más importantes se desarrolla primero, y las características de menor prioridad se implementan después.

Esto tiene la ventaja de que las funciones más críticas se completan antes, pero también el inconveniente de que algunos casos especiales o características de baja prioridad pueden quedar sin manejar, si no se escriben pruebas para ellos.

Dado que el código se escribe para satisfacer los casos de prueba, la completitud del código depende directamente de la exhaustividad de las pruebas.

Sin embargo, escribir casos de prueba para todos los escenarios posibles o condiciones especiales puede resultar difícil y tedioso, y es poco probable que el desarrollador los cubra todos.

En TDD, dado que el objetivo es escribir solo el código necesario para pasar las pruebas existentes, algunos casos especiales podrían no ser tratados.

Además, como en cada paso el código se escribe principalmente para pasar los tests actuales, puede ocurrir que con el tiempo se descubra que algunos algoritmos implementados previamente no eran los más adecuados.

En ese caso, el código debería mejorarse o refactorizarse antes de agregar nueva funcionalidad, tal como se ilustra en la Figura 9.3.

9.2.3 Programación en Pareja (Pair Programming)

La programación en pareja es también un proceso de codificación que se ha propuesto como una técnica clave en la metodología de programación extrema (XP). En la programación en pareja, el código no es escrito por programadores individuales, sino por una pareja de programadores. Es decir, el trabajo de codificación no se asigna a un individuo, sino a una pareja de personas. Esta pareja escribe el código conjuntamente.

El proceso previsto es que una persona teclee el programa mientras la otra participa activamente y revisa constantemente lo que se está escribiendo. Cuando se notan errores, se señalan y corrigen. Cuando es necesario, la pareja discute los algoritmos, estructuras de datos o estrategias que se utilizarán en el código. Los roles se rotan con frecuencia, haciendo que ambos sean socios iguales y desempeñen roles similares.

La motivación básica para la programación en pareja es que, dado que la lectura y revisión de código han demostrado ser muy efectivas para detectar defectos, al hacer que una pareja realice la programación tenemos la situación en la que el código se revisa a medida que se escribe. Es decir, en lugar de escribir el código y luego hacer que otro programador lo revise, tenemos un programador que revisa constantemente el código que se está escribiendo. Al igual que en el desarrollo e implementación incremental de pruebas, ahora tenemos una revisión incremental en curso.

Además de la revisión continua del código, el hecho de que dos programadores se dediquen juntos a la tarea de programación probablemente dé lugar a mejores decisiones sobre las estructuras de datos, algoritmos, interfaces, lógica, etc. Las condiciones especiales, que con frecuencia resultan en errores, también es más probable que se manejen de una mejor manera.

El posible inconveniente de la programación en pareja es que puede resultar en una pérdida de productividad al asignar dos personas a una misma tarea de programación. Está claro que una pareja producirá un código de mejor calidad en comparación con el desarrollado por un solo programador. La pregunta abierta es si este aumento en la productividad debido a la mejora en la calidad del código compensa la pérdida ocasionada por asignar dos personas a una tarea. También existen cuestiones de responsabilidad y propiedad del código, particularmente cuando las parejas no son fijas y rotan (como se propone en XP). El impacto de la programación en pareja es un área activa de investigación, especialmente dentro de la ingeniería de software experimental.

9.2.4 Control del Código Fuente y Construcción

En un proyecto, muchas personas diferentes desarrollan código fuente. Cada programador crea distintos archivos fuente, los cuales finalmente se combinan para crear los ejecutables. Los programadores van modificando sus archivos fuente a medida que el código evoluciona, como hemos visto en los procesos mencionados anteriormente, y con frecuencia también realizan cambios en otros archivos fuente. Para mantener el control sobre los archivos fuente y su evolución, casi siempre se utiliza un sistema de control de versiones en los proyectos, empleando herramientas como CVS en UNIX (www.cvshome.org) o Visual SourceSafe (VSS) en Windows (msdn.microsoft.com/vstudio/previous/ssafe). A continuación, se ofrece una breve descripción de cómo se utilizan estas herramientas en el proceso de codificación.

En el Capítulo 2 discutimos los conceptos generales de un proceso de gestión de configuración (CM). Nuestra explicación aquí se basa en CVS.

Un sistema moderno de control de código fuente contiene un repositorio, que es esencialmente una estructura de directorios controlada que mantiene el historial completo de revisiones de todos los archivos. Por eficiencia, el historial de un archivo generalmente se guarda como diferencias o incrementos respecto al archivo base. Esto permite recrear cualquier versión anterior del archivo, proporcionando así la flexibilidad de descartar fácilmente un cambio si es necesario. El repositorio es también la fuente “oficial” de todos los archivos.

Para un proyecto, debe configurarse un repositorio con permisos adecuados para las distintas personas del equipo. También se especifican los archivos que contendrá el repositorio —es decir, los archivos cuya evolución se desea mantener. Los programadores utilizan el repositorio tanto para hacer disponibles los cambios en sus archivos fuente como para obtener los archivos fuente de otros. Algunos de los comandos típicos que un programador ejecuta son:

Obtener una copia local. Un programador trabaja en una copia local de los archivos. Existen comandos para crear una copia local desde el repositorio. Obtener una copia local generalmente se denomina “checkout”. Un ejemplo de comando es `cvsv checkout <module>`, que copia en la máquina local el conjunto de archivos pertenecientes al módulo indicado. El usuario obtendrá la versión más reciente del archivo. Sin embargo, si lo desea, puede obtener cualquier versión anterior del archivo, ya que el repositorio mantiene el historial completo. Varios usuarios pueden realizar el checkout de un mismo archivo.

Realizar cambios en los archivos. Los cambios que un programador hace en su copia local permanecen locales hasta que se “confirman” (commit) en el repositorio. Al confirmar (por ejemplo, con `cvsv commit <file>`), los cambios realizados localmente se actualizan en el repositorio y se vuelven disponibles para los demás. Esta operación también se conoce como “check in”.

Actualizar una copia local. Los cambios que otros miembros del proyecto confirman en el repositorio no se reflejan automáticamente en las copias locales que se obtuvieron antes. Para obtener estos cambios, las copias locales deben actualizarse (por ejemplo, con el comando `cvsv update`). Con esta operación, todos los cambios realizados en los archivos se reflejan en la copia local.

Obtener reportes. Las herramientas de control de versiones ofrecen una variedad de comandos para generar informes sobre la evolución de los archivos. Estos incluyen reportes que muestran las diferencias entre la versión local y la versión más reciente, o listan todos los cambios realizados en un archivo junto con las fechas y las razones de dichos cambios (que suelen proporcionarse al confirmar una modificación).

Cabe destacar que, una vez que los cambios se confirman, se vuelven disponibles para todos los miembros del equipo, quienes deben usar los archivos del repositorio al verificar sus propios programas. Por lo tanto, es esencial que un programador confirme un archivo fuente solo cuando esté en un estado utilizable por otros. En un estado estable, el comportamiento típico de un miembro del proyecto será el siguiente: obtener (checkout) la versión más reciente de los archivos que planea modificar, realizar los cambios planeados, verificar que los cambios tengan el efecto deseado (para lo cual puede copiar todos los archivos y probar el sistema localmente), y luego confirmar los cambios en el repositorio.

Está claro que si dos personas obtienen el mismo archivo y luego hacen cambios, puede surgir un conflicto —es decir, diferentes modificaciones en las mismas partes del archivo. Todas las herramientas detectarán el conflicto cuando la segunda persona intente confirmar los cambios y notificarán al usuario. Este debe resolver manualmente el conflicto, es decir, modificar el archivo de manera que las modificaciones no se contradigan con los cambios existentes, y luego confirmar el archivo. Los conflictos son poco frecuentes, ya que solo ocurren cuando se realizan cambios en las mismas líneas de un archivo.

Con un sistema de control de código fuente, un programador no necesita mantener todas las versiones por su cuenta —en cualquier momento, si es necesario deshacer algún cambio, las versiones anteriores pueden recuperarse fácilmente. Los repositorios siempre se respaldan, lo que también protege contra pérdidas accidentales. Además, se mantiene un registro detallado de los cambios —quién los hizo y cuándo, por qué se realizaron, qué se modificó exactamente, etc. Lo más importante es que el repositorio proporciona un lugar centralizado para almacenar los archivos más recientes y oficiales del proyecto. Esto es invaluable para productos con una vida larga y que evolucionan durante muchos años.

Además de utilizar el repositorio para mantener diferentes versiones, también se usa para construir el sistema de software a partir de los archivos fuente —una actividad que suele llamarse “build”. El proceso de build obtiene la versión más reciente (o una versión específica) de los archivos fuente del repositorio y genera los ejecutables a partir de ellos.

La construcción de los ejecutables finales a partir de los archivos fuente suele realizarse mediante herramientas como *Makefile* [62], que especifican las dependencias entre archivos y cómo deben combinarse los archivos fuente para producir los ejecutables finales. Estas herramientas pueden detectar cuándo un archivo ha cambiado y recompilar automáticamente los necesarios para crear los ejecutables. Con el control de código fuente, estas herramientas generalmente obtienen la copia más reciente del repositorio antes de generar los ejecutables.

Este es uno de los enfoques más simples para el control de código fuente y el proceso de construcción. Sin embargo, cuando se construyen sistemas grandes, se requieren métodos más elaborados para el control del código y la compilación. Estos métodos suelen incluir una jerarquía de áreas controladas, cada una con distintos niveles de control y fuentes diferentes, donde la parte superior de la jerarquía contiene todos los archivos necesarios para construir el sistema “oficial”. Los niveles inferiores pueden ser utilizados por diferentes grupos para crear versiones “locales” destinadas a pruebas u otros propósitos. En tales sistemas, se requiere integración directa e inversa (forward y reverse integration) para transferir los cambios entre las distintas áreas controladas de la jerarquía. Una herramienta avanzada como *ClearCase* ofrece estas capacidades.

9.3 Refactorización

Hemos visto que la codificación a menudo implica realizar cambios en código existente. El código también cambia cuando cambian los requisitos o cuando se agrega nueva funcionalidad. Debido a las modificaciones que se realizan en los módulos, incluso si comenzamos con un buen diseño, con el tiempo solemos terminar con un código cuyo diseño ya no es tan bueno como podría ser. Y una vez que el diseño incorporado en el código se vuelve complejo, mejorar el código para adaptarlo a los cambios necesarios se vuelve más difícil, lleva más tiempo y es más propenso a errores. En otras palabras, la productividad y la calidad comienzan a disminuir.

La **refactorización** es la técnica utilizada para mejorar el código existente y evitar que el diseño se deteriore con el tiempo. La refactorización forma parte de la actividad de codificación, ya que se realiza durante ella, pero no es lo mismo que programar normalmente. La refactorización ha sido practicada por programadores desde hace tiempo, pero recientemente ha tomado una forma más concreta y se ha propuesto como un paso clave en la metodología de **Programación Extrema (XP)**.

La refactorización también cumple un papel importante en el desarrollo guiado por pruebas (**TDD**), ya que el paso de mejora del código en el proceso de TDD consiste, en realidad, en realizar refactorización. En esta sección discutiremos algunos conceptos clave y métodos para llevar a cabo la refactorización.

9.3.1 Conceptos Básicos

La refactorización se define como un cambio realizado en la estructura interna del software para hacerlo más fácil de entender y más barato de modificar, sin cambiar su comportamiento observable. Un punto clave aquí es que el cambio se realiza sobre el diseño incorporado en el código fuente (es decir, su estructura interna) con el único propósito de **mejorarlo**.

El objetivo principal de la refactorización es **mejorar el diseño**. Sin embargo, es importante notar que no se trata de mejorar un diseño durante las etapas de diseño para crear un modelo que luego se implementará (que es el objetivo de las metodologías de diseño), sino de **mejorar el diseño del código que ya existe**. En otras palabras, aunque la refactorización se realice sobre el código fuente, su propósito es optimizar el diseño que dicho código implementa. Por lo tanto, los principios fundamentales del diseño guían el proceso de refactorización.

Como resultado, una refactorización suele producir uno o más de los siguientes efectos:

1. **Reducción del acoplamiento**
2. **Aumento de la cohesión**
3. **Mejor adherencia al principio abierto/cerrado** (en sistemas orientados a objetos)

La refactorización implica modificar el código para mejorar alguna propiedad del diseño, **manteniendo el mismo comportamiento externo**. A menudo, se **inicia cuando es necesario realizar ciertos cambios en el código**. Si se requiere agregar una mejora o modificación, y se percibe que la estructura actual del código dificulta ese cambio, ese es el momento adecuado para refactorizar y mejorar su estructura.

Aunque la refactorización suele surgir a partir de la necesidad de cambiar el software (y su comportamiento externo), **no debe confundirse ni mezclarse con la implementación de nuevas funcionalidades**. Lo ideal es mantener ambos tipos de cambios separados. Así, cuando un programador detecta la necesidad de refactorizar, **debe detener el desarrollo de nueva funcionalidad, realizar primero la refactorización y recién después continuar con la nueva implementación**.

El principal riesgo de refactorizar es que el código existente —que funciona correctamente— **puede romperse** debido a los cambios realizados. Esta es la razón más común por la que la refactorización no se hace con frecuencia. (La otra razón es que suele considerarse un costo adicional innecesario). Para mitigar este riesgo, existen dos **reglas de oro**:

1. **Refactorizar en pasos pequeños.**
2. **Contar con scripts de prueba que verifiquen la funcionalidad existente.**

Si se dispone de una buena batería de pruebas automatizadas, es fácil comprobar si la refactorización mantiene el comportamiento anterior. Sin una suite de pruebas, determinar si el comportamiento externo se ha alterado sería costoso y poco práctico. Al refactorizar en pequeños pasos y ejecutar pruebas tras cada uno, los errores pueden detectarse y corregirse fácilmente. Cada

refactorización realiza un cambio pequeño, pero una **serie de refactorizaciones puede transformar significativamente la estructura de un programa.**

Gracias a la refactorización, el código puede mejorar continuamente. En lugar de degradarse con el tiempo, **el diseño evoluciona y se fortalece.** Esto mejora la calidad del código, facilita la incorporación de cambios y la detección de errores. El costo adicional de refactorizar se compensa luego con los ahorros obtenidos en menor tiempo de prueba y depuración, mayor calidad y menor esfuerzo para futuras modificaciones.

Si la refactorización se aplica de forma sistemática, también puede **simplificar la tarea de diseño** durante las etapas iniciales. A menudo, los diseñadores dedican mucho esfuerzo a tratar de crear un diseño “perfecto” o lo bastante flexible como para soportar todo tipo de cambios futuros. Esto puede volver el diseño demasiado complejo. Con la refactorización, el diseñador puede centrarse en crear un diseño **simple y correcto**, sabiendo que si surgen nuevas necesidades o limitaciones, el diseño podrá ajustarse más adelante mediante refactorización.

Finalmente, es importante aclarar que **la refactorización no es una técnica para corregir errores ni para reparar código en mal estado.** Se aplica a código que ya funciona correctamente. Su propósito es **prolongar la vida útil del software** manteniendo su estructura saludable. En lugar de dejar que el código se deteriore con el tiempo, la refactorización busca mantenerlo constantemente en buena forma.

9.3.2 Un ejemplo

9.3.3 Malos olores

Ahora hablaremos de las **señales en el código que pueden indicarnos que podría ser necesario aplicar refactorización.** A estas señales a veces se las llama **“malos olores”** (*bad smells*). Básicamente, **son señales fáciles de detectar en el código que suelen indicar que algunas de las propiedades deseables del diseño pueden estar siendo violadas, o que existe la posibilidad de mejorar el diseño.** En otras palabras, si “huelo” uno de estos “malos olores”, puede ser una señal de que se necesita refactorizar. Por supuesto, si la refactorización es realmente necesaria deberá decidirse caso por caso, observando el código y las oportunidades de mejora que puedan existir. A continuación se presentan algunos de estos malos olores tomados de [65].

1. **Código duplicado.** Esto es bastante común. Una razón puede ser que una pequeña funcionalidad se ejecuta en varios lugares (por ejemplo, calcular la edad a partir de la fecha de nacimiento en cada sitio donde se necesita). Otra razón frecuente es que, cuando hay múltiples subclases de una clase, cada subclase termina haciendo algo similar. El código duplicado implica que, si se necesita cambiar esa lógica o función, habrá que modificarla en todos los lugares donde aparece, lo que hace los cambios mucho más difíciles y costosos.
2. **Método largo.** Si un método es grande, a menudo representa una situación en la que intenta hacer demasiadas cosas y, por lo tanto, carece de cohesión.
3. **Clase larga.** De manera similar, una clase grande puede indicar que está encapsulando múltiples conceptos, haciendo que la clase no sea cohesionada.

4. **Lista de parámetros larga.** Las interfaces complejas claramente no son deseables: hacen que el código sea más difícil de entender. A menudo, esta complejidad no es intrínseca, sino una señal de un diseño inadecuado.
5. **Sentencias switch.** En los programas orientados a objetos, si el polimorfismo no se usa adecuadamente, es probable que aparezcan sentencias *switch* en cada lugar donde el comportamiento dependa de alguna propiedad. La presencia de sentencias *switch* similares en diferentes partes del código indica que se está utilizando una estructura condicional en lugar de una jerarquía de clases. La presencia de una sentencia *switch* hace mucho más difícil extender el código: si se agrega una nueva categoría, será necesario modificar todas las sentencias *switch*.
6. **Generalidad especulativa.** Algunas jerarquías de clases pueden existir porque los objetos de las subclasses parecen diferentes. Sin embargo, si el comportamiento de los objetos de las diferentes subclasses es el mismo y no hay una razón inmediata para pensar que estos comportamientos puedan cambiar, se trata de un caso de complejidad innecesaria.
7. **Demasiada comunicación entre objetos.** Si los métodos de una clase hacen muchas llamadas a métodos de otro objeto para averiguar su estado, esto es un signo de acoplamiento fuerte. Es posible que esto sea innecesario, por lo que tales situaciones deberían analizarse para una posible refactorización.
8. **Encadenamiento de mensajes.** Un método llama a otro método, que simplemente pasa la llamada a otro objeto, y así sucesivamente. Esta cadena puede generar un acoplamiento innecesario.

En general, estos malos olores son indicativos de un mal diseño. Podemos notar que muchos de estos olores estaban presentes en nuestro ejemplo anterior.

9.3.4 Refactorizaciones comunes

Está claro que existen infinitas posibilidades sobre cómo se puede refactorizar el código para mejorar su diseño. Un catálogo de refactorizaciones comunes y los pasos para realizar cada una de ellas se presenta en. Nuevas refactorizaciones se siguen listando continuamente en www.refactoring.com. Como se mencionó antes, una refactorización debe ayudar a que el código sea más fácil de entender y modificar.

Para lograr este objetivo, muchas refactorizaciones se enfocan en mejorar los métodos, las clases o las jerarquías de clases. Aquí discutimos brevemente algunas de las refactorizaciones sugeridas. Existen otras refactorizaciones que tratan con la reducción de cadenas de mensajes (*message chains*), pero no las discutiremos aquí.

El proceso general de refactorización es el mismo para todas ellas: se realiza un paso de refactorización y luego se prueba el nuevo código (con pruebas automatizadas) para asegurarse de que la refactorización no haya alterado el comportamiento y que las pruebas anteriores sigan pasando. Si se van a aplicar múltiples refactorizaciones, deben aplicarse una a la vez, y una nueva solo debe realizarse después de que la anterior haya sido probada con éxito.

Mejorando los métodos

Ya vimos antes que un método puede no ser cohesivo y puede realizar muchas funciones diferentes. Un objetivo principal de la refactorización para mejorar métodos es simplificarlos y hacerlos más cohesivos. El nivel de acoplamiento de un método depende en gran medida de su interfaz, y al simplificar la interfaz, el acoplamiento puede reducirse.

1. Extraer método (Extract Method).

Esta refactorización se realiza con frecuencia cuando un método es demasiado largo, lo que indica que puede no ser cohesivo y estar realizando múltiples funciones. El objetivo es tener métodos cortos cuyas firmas den a los usuarios una idea bastante precisa de lo que hacen. Durante esta refactorización, un fragmento de código dentro de un método se extrae y se convierte en un nuevo método. Las variables a las que hace referencia ese fragmento y que están en el alcance del método original se convierten en los parámetros del nuevo método. Cualquier variable declarada en ese fragmento pero usada en otra parte deberá definirse en el método original. En el método original, el código extraído se reemplaza por una llamada al nuevo método.

A veces, el nuevo método puede ser una función que devuelve un valor. Un ejemplo de esta refactorización fue mostrado anteriormente.

De manera similar, si existe un método que devuelve un valor pero también cambia el estado de algunos objetos, entonces este método debe dividirse en dos: uno que devuelva el valor deseado y otro que realice el cambio de estado necesario. Tener un método que solo devuelve un valor y no tiene efectos secundarios generalmente implica una fuerte cohesión funcional. (Esta refactorización se llama **separar consulta de modificador** —*separate query from modifier*.)

2. Agregar o eliminar parámetros (Add/Remove Parameter).

Si un método necesita más información de su invocador, quizás porque el alcance de lo que debe hacer se ha ampliado, entonces se deben agregar nuevos parámetros. Sin embargo, esto debe hacerse solo si los parámetros existentes no pueden proporcionar la información necesaria.

El caso opuesto es la eliminación de parámetros. A veces, por razones de flexibilidad o futuras extensiones, se piden parámetros que finalmente no se utilizan. Por simplicidad, los parámetros que no se usan deberían eliminarse. Pero si la clase forma parte de una jerarquía, esto debe hacerse con cuidado para asegurar que las relaciones jerárquicas no se vean afectadas. O bien, el cambio tendrá que realizarse también en los niveles superiores o inferiores de la jerarquía.

Mejorando las clases

La mayoría de las refactorizaciones en esta categoría se centran en mejorar la cohesión de las clases o en reducir el acoplamiento entre ellas. Aumentar la cohesión de las clases a menudo implica mover campos y métodos de una clase a otra, de manera que los datos y comportamientos relacionados se encuentren encapsulados en un mismo objeto. La reducción del acoplamiento, en cambio, suele requerir cambios en las clases para disminuir el grado de interacción entre ellas.

1. Mover método (Move Method).

En esta refactorización, un método se mueve de una clase a otra. Es una de las más importantes y se realiza generalmente cuando un método en la clase A interactúa demasiado

con un objeto de la clase B, lo que sugiere que el lugar más natural para el método es B y no A.

A veces no está claro desde el principio a qué clase debería pertenecer un método, por lo que el diseñador puede colocarlo inicialmente en A. Sin embargo, si más adelante se observa que su ubicación lógica es B, entonces debe moverse.

Inicialmente, es recomendable dejar el método original como un método *delegado* que solo invoque al nuevo método. De esta forma, el cambio afecta únicamente a esas dos clases. No obstante, siempre que sea posible, las referencias o llamadas al método deben redirigirse al nuevo, y el método original debe eliminarse. Si el método original no era privado, todas las clases que lo utilizaban deberán actualizarse.

2. Mover campo (Move Field).

Si un campo en la clase A es utilizado con mayor frecuencia por métodos de la clase B, ese campo debería trasladarse a B. Esto reducirá el acoplamiento entre A y B y mejorará la encapsulación de ambas clases.

Esta refactorización es similar a la anterior: en *Mover Método* se reasigna comportamiento, y en *Mover Campo* se reasigna estado. Si el campo era privado, después de moverlo, todas las referencias a él en los métodos de A deberán reemplazarse por llamadas a métodos apropiados de B para obtener o modificar ese estado.

3. Extraer clase (Extract Class).

A menudo un diseñador comienza con una clase que, con el tiempo y la incorporación de nuevas funcionalidades, se amplía tanto que termina teniendo múltiples responsabilidades o conceptos mezclados. Si una clase grande parece carecer de una abstracción clara o está asumiendo demasiadas responsabilidades, esta refactorización es apropiada.

Se crea una nueva clase y se mueven a ella los campos y métodos relacionados. La refactorización se justifica si ambas clases resultantes tienen una abstracción más clara y responsabilidades bien definidas. Sin embargo, si el resultado genera un acoplamiento excesivo entre ambas clases, eso indica que la división fue artificial.

Una manera práctica de aplicar esta refactorización es primero crear la nueva clase y establecer un enlace desde la clase original. Luego se aplican las refactorizaciones *Mover Campo* y *Mover Método* repetidamente para transferir los elementos que correspondan a la nueva clase.

4. Reemplazar un valor de datos con un objeto (Replace Data Value with Object).

Esta refactorización es similar a *Extraer Clase*. A menudo, ciertos datos se tratan inicialmente como campos simples de una clase. Sin embargo, a medida que el desarrollo avanza, esos datos pueden adquirir mayor significado y asociarse con múltiples operaciones. Ejemplos típicos son fechas, números de teléfono, direcciones o números de seguridad social.

Si se realizan múltiples operaciones sobre un mismo dato dentro de la clase, puede tener sentido convertirlo en un objeto propio. Esto mejora la organización del código, facilita la reutilización y refuerza la cohesión.

Mejorando las jerarquías

La jerarquía de clases es un concepto clave en la programación orientada a objetos y constituye la base del *principio abierto/cerrado*. Para cumplir con este principio, es fundamental utilizar

adecuadamente el polimorfismo. El objetivo de las refactorizaciones en esta categoría es aprovechar el polimorfismo para facilitar la extensión futura del código y crear diseños que representen el problema de manera más natural.

1. **Reemplazar condicional con polimorfismo (Replace Conditional with Polymorphism).**

Si una clase contiene métodos cuyo comportamiento depende del valor de algún tipo o código de estado (*type-code*), se está utilizando un enfoque más cercano al estilo funcional que al orientado a objetos. En estos casos, el polimorfismo puede y debe emplearse para representar mejor la situación.

La presencia de una sentencia `switch`, `case` o una serie de condicionales `if/else` dentro de un método, o de variables que actúan como códigos de tipo dentro de la clase, son señales claras de que esta refactorización puede ser necesaria.

Por ejemplo, en el método `printReport()` de una clase `GradReport`, si el comportamiento varía según si el estudiante es de posgrado (PG) o de grado (UG), una mejor solución es crear una jerarquía de clases donde `PGStudent` y `UGStudent` hereden de una clase base `Student`.

Las funcionalidades comunes se ubican en la clase base, mientras que los comportamientos específicos se implementan en las subclases. Así, un objeto del tipo `Student` puede apuntar dinámicamente a cualquiera de las clases derivadas, eliminando la necesidad de sentencias condicionales y capturando las diferencias de comportamiento mediante el polimorfismo.

2. **Elevar campo/método (Pull Up Field/Method).**

Un buen diseño de jerarquía requiere que los elementos comunes se ubiquen en las superclases y los elementos variables en las subclases.

Si se detecta que varias subclases tienen el mismo campo, dicho campo debería moverse hacia arriba en la jerarquía, es decir, a la superclase.

De igual manera, si existen métodos en distintas subclases que realizan esencialmente la misma función (aunque con implementaciones distintas), se está duplicando funcionalidad, lo que dificulta el mantenimiento y la evolución del código. En este caso, el método debe trasladarse a la superclase.

Incluso los constructores pueden refactorizarse de esta forma: si varias subclases comparten constructores similares, estos pueden elevarse a la superclase y ser invocados desde los constructores de las subclases.

La operación inversa se conoce como **empujar hacia abajo (Push Down Field/Method)**.

Si una superclase contiene un campo que solo es utilizado por una o pocas subclases, es mejor mover ese campo hacia la subclase correspondiente.

Del mismo modo, si un método en la superclase solo tiene relevancia para ciertas subclases, debería desplazarse hacia abajo, es decir, a aquellas clases donde realmente se utiliza.

Estas refactorizaciones ayudan a mantener jerarquías limpias, lógicas y fáciles de extender, reduciendo duplicación de código y mejorando la cohesión entre clases.

9.4 Verificación

Una vez que un programador ha escrito el código para un módulo, este debe ser verificado antes de que sea utilizado por otros. Hasta ahora hemos asumido que la verificación se realiza mediante

pruebas. Aunque las pruebas son el método más común de verificación, existen otras técnicas efectivas. Aquí discutiremos algunas de las más comunes.

Es importante aclarar que por verificación no nos referimos a la **demostración formal de la corrección de programas**, que para nuestros propósitos es solo un método dentro del conjunto de técnicas de verificación.

Nos centraremos en las técnicas que se utilizan ampliamente en la práctica, como las **inspecciones** (incluyendo la lectura de código), las **pruebas unitarias** y la **verificación automática de programas (program checking)**. También hablaremos brevemente de la **verificación formal**, aunque esta se usa con menos frecuencia y principalmente en situaciones especiales.

Aunque aquí nos enfocamos en la verificación de programas individuales escritos por los programadores, algunas técnicas —como la verificación automática— también pueden aplicarse a nivel de **sistema completo**.

9.4.1 Inspección de código (Code Inspections)

La **inspección** es una técnica general de verificación que puede aplicarse a cualquier tipo de documento y se ha utilizado ampliamente para detectar defectos. Originalmente se desarrolló para encontrar errores en el código, pero luego se aplicó también al diseño, los requisitos, los planes, etc. El proceso general de inspección ya fue discutido anteriormente, y para la inspección de código se mantiene básicamente igual.

Las **inspecciones de código** suelen realizarse una vez que el código ha sido compilado correctamente y después de aplicar otras herramientas estáticas. Esto se hace para ahorrar tiempo y esfuerzo humano, evitando que las personas busquen errores que ya podrían ser detectados por un compilador o un analizador estático.

La documentación que se entrega al equipo de inspección incluye el **código a revisar** y el **documento de diseño**. El equipo debe incluir al programador, al diseñador y al tester. El objetivo principal de la inspección es **detectar defectos en el código**, aunque también se revisan **aspectos de calidad** como la eficiencia o el cumplimiento de estándares de codificación.

Con frecuencia, las inspecciones se guían por una **lista de verificación (checklist)** que contiene los tipos de defectos en los que deben centrarse los inspectores. Algunos ejemplos de ítems que puede incluir una checklist son:

- ¿Las definiciones de datos aprovechan las capacidades de tipado del lenguaje?
- ¿Todos los punteros apuntan a algún objeto (no hay punteros colgantes)?
- ¿Se asignan los punteros a NULL cuando corresponde?
- ¿Se verifica si un puntero es NULL antes de usarse?
- ¿Los índices de los arreglos están dentro de los límites?
- ¿Los índices están correctamente inicializados?
- ¿Las condiciones de las ramas son correctas (ni demasiado débiles ni demasiado fuertes)?
- ¿Los bucles siempre terminan (no hay bucles infinitos)?
- ¿La condición de terminación de los bucles es correcta?
- ¿El número de ejecuciones del bucle no tiene un error de “uno de más o uno de menos”?
- ¿Se verifica que los divisores no sean cero?

- ¿Se valida la entrada de datos externos?
- ¿Los parámetros reales y formales coinciden?
- ¿Todas las variables se utilizan y todas las variables de salida se asignan?
- ¿Existen sentencias dentro de bucles que podrían moverse fuera de ellos?
- ¿Hay etiquetas sin referencia?
- ¿Se cumplen los requisitos de tiempo de ejecución?
- ¿Se siguen los estándares locales de codificación?

Las inspecciones son **muy efectivas** para detectar defectos y se utilizan ampliamente en muchas organizaciones comerciales. Sin embargo, también **son costosas**, ya que requieren el tiempo de varias personas. Por eso, en algunos casos el costo puede no estar justificado, y en lugar de una inspección grupal puede realizarse una **revisión individual**.

Una técnica común para esto es la **lectura estructurada de código (code reading)**. Esta consiste en leer cuidadosamente el código para detectar discrepancias entre la especificación de diseño y la implementación real. El lector debe identificar la **abstracción del módulo** y luego compararla con su especificación.

El proceso de lectura es el inverso del diseño:

- En el diseño se parte de una abstracción y se avanza hacia los detalles.
- En la lectura de código se parte de los detalles del programa y se asciende hacia una descripción abstracta.

Lo ideal es leer el código **desde adentro hacia afuera**, empezando por la estructura más interna del módulo. Primero se determina su comportamiento abstracto, luego se reemplaza en la estructura de nivel superior, y se repite hasta llegar al módulo completo. Finalmente, el comportamiento abstracto obtenido se compara con las especificaciones para detectar discrepancias.

La **lectura de código** es muy útil y puede detectar errores que no aparecen en las pruebas. Además, fomenta que los programadores escriban código estructurado de manera clara, lo que mejora la calidad general. Este proceso también se conoce como **revisión de escritorio (desk review)**.

10 Pruebas (Testing)

En un proyecto de desarrollo de software, los errores pueden introducirse en cualquier etapa del desarrollo. Aunque los errores se detectan después de cada fase mediante técnicas como las inspecciones, algunos permanecen sin ser detectados. En última instancia, esos errores restantes se reflejarán en el código. Por lo tanto, el código final probablemente contendrá errores de requisitos y errores de diseño, además de los errores introducidos durante la actividad de codificación.

La prueba (testing) es la actividad en la que deben detectarse los errores que permanecen de todas las fases anteriores. Por ello, las pruebas cumplen un rol muy crítico para garantizar la calidad.

El enfoque de este capítulo está principalmente en las pruebas del sistema (system testing), en las que se prueba todo el sistema de software completo, aunque las pruebas también se realizan sobre programas individuales escritos por los programadores, y los conceptos discutidos también son aplicables a las pruebas de programas individuales.

Durante las pruebas, el software que se va a probar se ejecuta con un conjunto de casos de prueba (test cases), y el comportamiento del sistema frente a esos casos se evalúa para determinar si el sistema funciona como se espera. Claramente, el éxito de las pruebas para revelar errores depende críticamente de los casos de prueba. Gran parte de este capítulo está dedicada a la selección de casos de prueba, los criterios para seleccionarlos y su efecto sobre las pruebas.

Comenzamos este capítulo discutiendo algunas definiciones y conceptos relevantes para las pruebas. Luego analizamos los dos enfoques básicos de las pruebas:

- Caja negra (black-box) o pruebas funcionales, y
- Caja blanca (white-box) o pruebas estructurales.

A continuación, se discuten los aspectos del proceso de prueba, seguidos de una discusión sobre cómo los datos obtenidos de las pruebas pueden utilizarse para la prevención de defectos.

Finalmente, se aborda la estimación de confiabilidad (reliability estimation), ya que la confiabilidad es la principal métrica de interés durante las pruebas.

El capítulo concluye con estudios de caso.

10.1 Fundamentos de las pruebas

En esta sección primero definiremos algunos términos que se utilizan comúnmente al hablar de pruebas de software. Luego discutiremos algunos temas básicos relacionados con cómo puede llevarse a cabo el proceso de prueba, la necesidad de oráculos en las pruebas, la importancia de la psicología del probador (tester) y algunas propiedades deseables para los criterios utilizados en las pruebas.

Una vez que estos conceptos se hayan abordado, avanzaremos hacia el tema central: la selección de los casos de prueba.

10.1.1 Error, Falla y Fallo

Hasta ahora, hemos usado el significado intuitivo del término error para referirnos a problemas en los requerimientos, el diseño o el código. Sin embargo, los términos *error*, *falla (fault)* y *fallo*

(*failure*) a veces se usan indistintamente, y otras veces para conceptos diferentes. Comencemos por definirlos claramente, siguiendo las definiciones del IEEE [91].

El término **error** se usa de dos maneras:

1. Para referirse a la **diferencia entre un valor calculado, observado o medido y el valor verdadero**, especificado o teóricamente correcto. Es decir, el error representa la **discrepancia entre la salida real del software y la salida correcta esperada**.
2. También se usa para referirse a la **acción humana que causa que el software contenga un defecto o falla**.

Este segundo significado es más general y abarca todas las fases del desarrollo.

Una **falla** (fault) es una **condición que provoca que un sistema no cumpla su función requerida**. Es la causa básica del mal funcionamiento del software y es **sinónimo del término común *bug***.

En este libro, usaremos los términos error y falla de la manera habitual, sin hacer distinciones explícitas entre ambos salvo que sea necesario.

Cabe notar que las únicas fallas que puede tener un software son “fallas de diseño”, ya que el software no sufre desgaste físico.

Un **fallo** (failure) es la **incapacidad de un sistema o componente para realizar una función requerida según sus especificaciones**.

Un fallo ocurre cuando el comportamiento del software difiere del comportamiento especificado.

Los fallos pueden deberse a razones funcionales o de rendimiento, y solo se producen cuando existe una falla en el sistema.

Sin embargo, la presencia de una falla no garantiza que ocurra un fallo; las fallas tienen el potencial de causar fallos, pero su presencia es una condición necesaria, no suficiente, para que estos se manifiesten.

Es importante notar que la definición de “fallo” depende del proyecto.

Por ejemplo:

- ¿Una línea fuera de lugar en la salida es un fallo? Depende del proyecto: algunos lo considerarán un fallo, otros no.
- ¿Y si el resultado no se produce dentro de un tiempo límite? En un sistema en tiempo real probablemente sí lo sea, pero en un sistema operativo quizás no.

Esto significa que no hay una definición universal de fallo; es el gestor del proyecto o el usuario final quien debe decidir qué se considerará un fallo a efectos de confiabilidad.

Incluso si un defecto se registra y se corrige más tarde, su aparición puede no ser considerada un fallo.

Estas definiciones tienen algunas implicaciones importantes:

- La presencia de un error (en el estado del sistema) implica que ocurrió un fallo.
- La observación de un fallo implica que existe una falla en el sistema.
- Pero la presencia de una falla no implica necesariamente que vaya a ocurrir un fallo.

Que una falla se manifieste o no depende de muchos factores.

Por eso, si observamos un sistema durante un tiempo y no detectamos fallos, no podemos asegurar que esté libre de fallas.

En cambio, si sí se observa un fallo, podemos afirmar que existen fallas en el sistema.

Esto tiene consecuencias directas para las pruebas.

Durante las pruebas, se observa el comportamiento del sistema, y a partir de él se determina si hay o no un fallo.

Por lo tanto, las pruebas solo pueden revelar la presencia de fallas, no su ausencia.

Si se observan fallos, se concluye que existen fallas; pero si no se observan, no se puede afirmar que el sistema esté libre de ellas.

Una consecuencia inmediata de esto es que es difícil decidir cuándo detener las pruebas: ¿cuánto tiempo debemos probar sin observar fallos antes de concluir que el sistema es confiable?

Durante el proceso de pruebas, lo único que se observa son fallos, a partir de los cuales se deduce la existencia de fallas.

La identificación concreta de esas fallas se realiza en una actividad separada llamada depuración (debugging).

En otras palabras, las pruebas revelan la presencia de fallas, pero la depuración las localiza y corrige.

Esta es una de las razones por las que probar software es un método costoso para identificar fallas, en comparación con métodos que las detectan directamente.

10.1.2 Oráculos de Prueba

Para probar cualquier programa, necesitamos tener una descripción de su comportamiento esperado y un método para determinar si el comportamiento observado se ajusta a ese comportamiento esperado. Para esto, necesitamos un oráculo de prueba.

Un oráculo de prueba es un mecanismo, diferente del propio programa, que puede utilizarse para verificar la corrección de la salida del programa para los casos de prueba. Conceptualmente, podemos considerar las pruebas como un proceso en el que los casos de prueba se entregan tanto al oráculo de prueba como al programa bajo prueba. Luego, las salidas de ambos se comparan para determinar si el programa se comportó correctamente para esos casos de prueba, como se muestra en la Figura 10.1.

Los oráculos de prueba son necesarios para la verificación. Idealmente, nos gustaría disponer de un oráculo automatizado, que siempre proporcione una respuesta correcta. Sin embargo, con frecuencia los oráculos son seres humanos, quienes pueden cometer errores. Como resultado, cuando hay una discrepancia entre los resultados del programa y los del oráculo, debemos verificar el resultado producido por el oráculo antes de declarar que existe una falla en el programa.

Los oráculos humanos generalmente utilizan las especificaciones del programa para decidir cuál debería ser el comportamiento “correcto” del mismo. Sin embargo, las especificaciones pueden contener errores, ser imprecisas o tener ambigüedades. Estas deficiencias en las especificaciones son la causa principal de situaciones en las que una parte afirma que una condición particular no es un fallo, mientras que otra sostiene que sí lo es. No existe una solución sencilla a este problema, ya

que las pruebas requieren necesariamente unas especificaciones contra las cuales comparar el sistema evaluado.

Existen algunos sistemas en los que los oráculos se generan automáticamente a partir de las especificaciones de los programas o módulos. Con este tipo de oráculos, se garantiza que la salida del oráculo sea consistente con las especificaciones. Estos oráculos también eliminan el esfuerzo de determinar el comportamiento esperado para cada caso de prueba. Sin embargo, incluso este enfoque no resuelve todos los problemas, debido a la posibilidad de errores en las especificaciones. En consecuencia, un oráculo generado a partir de las especificaciones solo producirá resultados correctos si las especificaciones son correctas, y no será confiable en caso de que existan errores en ellas.

Además, estos sistemas que generan oráculos a partir de especificaciones suelen requerir especificaciones formales, las cuales con frecuencia no se producen durante la fase de diseño.

10.1.3 Casos de Prueba y Criterios de Prueba

Tener casos de prueba que sean efectivos para revelar la presencia de fallos es fundamental para realizar pruebas exitosas. La razón es que, aunque exista un fallo en un programa, este puede seguir produciendo el comportamiento esperado para muchos valores de entrada. Solo para el conjunto de entradas que ejerciten ese fallo, la salida del programa diferirá del comportamiento esperado. Por lo tanto, puede decirse que la calidad de las pruebas depende directamente de la calidad de sus casos de prueba.

Idealmente, querríamos determinar un conjunto de casos de prueba tal que, si todos se ejecutan con éxito, se pueda concluir que el programa no contiene errores. Sin embargo, este objetivo ideal rara vez puede alcanzarse debido a limitaciones prácticas y teóricas. Cada caso de prueba tiene un costo asociado: se requiere esfuerzo para generarlo, tiempo de máquina para ejecutarlo, y trabajo adicional para evaluar los resultados. Por ello, también deseamos minimizar la cantidad de casos de prueba necesarios para detectar errores.

De este modo, las dos metas fundamentales de una actividad de prueba práctica son:

1. **Maximizar el número de errores detectados.**
2. **Minimizar el número de casos de prueba (es decir, minimizar el costo).**

Como estos dos objetivos suelen ser contradictorios, el problema de seleccionar el conjunto de casos de prueba adecuados se vuelve más complejo.

Al seleccionar los casos de prueba, el objetivo principal es garantizar que, si existe un error o fallo en el programa, al menos uno de los casos de prueba lo ejecute. Un conjunto de casos de prueba ideal es aquel que solo tiene éxito (no revela errores) si realmente no existen errores en el programa.

Un posible conjunto ideal de pruebas sería aquel que incluye todas las entradas posibles del programa. Esto se denomina prueba exhaustiva. Sin embargo, la prueba exhaustiva es impráctica e inviable, ya que incluso en programas pequeños, el número de posibles entradas puede ser extremadamente grande.

Selección de Casos de Prueba y Criterios

Entonces, ¿cómo debemos seleccionar nuestros casos de prueba?

¿Con qué criterio decidimos incluir ciertos elementos del dominio de entrada y no otros?

Para esto se utilizan los criterios de selección de pruebas (o simplemente criterios de prueba).

Dado un programa P y sus especificaciones S, un criterio de prueba especifica las condiciones que debe cumplir un conjunto de casos de prueba T.

El criterio sirve así como una base para la selección de casos de prueba.

Por ejemplo, si el criterio establece que todas las sentencias del programa deben ejecutarse al menos una vez durante las pruebas, entonces un conjunto de casos de prueba T satisface este criterio para un programa P si la ejecución de P con T garantiza que cada sentencia de P se ejecuta al menos una vez.

Propiedades de un Criterio de Prueba

Existen dos propiedades fundamentales de un criterio de prueba:

- **Confiabilidad** (reliability): un criterio es confiable si todos los conjuntos de casos de prueba que lo satisfacen detectan los mismos errores.
Es decir, no importa qué conjunto particular se elija; todos detectarán exactamente los mismos errores.
- **Validez** (validity): un criterio es válido si, para cualquier error en el programa, existe al menos un conjunto que satisfaga el criterio y que revele dicho error.

Un teorema fundamental de las pruebas establece que, si un criterio es válido y confiable, y un conjunto de pruebas que lo satisface no revela fallos, entonces el programa no contiene errores.

Sin embargo, se ha demostrado que no existe ningún algoritmo que pueda determinar un criterio válido para un programa arbitrario.

Limitaciones Prácticas

Obtener un criterio que sea a la vez confiable, válido y que pueda cumplirse con un número manejable de casos de prueba no suele ser posible. Por ello, a menudo se utilizan criterios más débiles o aproximados, como:

- “El 90 % de las sentencias debe ejecutarse al menos una vez.”
- “Todos los valores especiales del dominio deben incluirse.” (aunque rara vez se define qué constituye un “valor especial”).

Incluso cuando el criterio se especifica claramente, generar casos de prueba que lo satisfagan no es sencillo. En general, no puede automatizarse.

Por ejemplo, incluso para un criterio simple como “cada sentencia del programa debe ejecutarse”, es extremadamente difícil construir un conjunto de casos de prueba que lo cumpla para un programa grande, aun asumiendo que todas las sentencias sean ejecutables.

Relación entre Criterios

Un criterio C_1 incluye (o subsume) a otro criterio C_2 si, para todo programa P y sus especificaciones S , cualquier conjunto de casos de prueba que satisfaga C_1 también satisface C_2 . Esta relación se denota como $C_1 \Rightarrow C_2$ y es transitiva.

Podría pensarse que si $C_1 \Rightarrow C_2$, entonces las pruebas basadas en C_1 serán siempre mejores que las basadas en C_2 .

Desafortunadamente, esto no siempre es cierto, porque la capacidad de detección de fallos de un conjunto de casos de prueba T que satisface un criterio C depende de los casos de prueba específicos incluidos en T , y no solo del criterio C .

Es decir, si dos conjuntos T_1 y T_2 satisfacen el mismo criterio C para un programa P , no significa que ejecuten los mismos caminos ni detecten los mismos errores.

Por lo tanto, incluso si $C_1 \Rightarrow C_2$, es posible que T_2 (que satisface C_2) detecte un error que T_1 (que satisface C_1) no detecte.

En general, si se utilizan métodos similares de generación de casos de prueba, puede afirmarse que C_1 es mejor que C_2 si $C_1 \Rightarrow C_2$.

Conclusión

Lo discutido anteriormente ilustra que **ningún criterio único puede garantizar la detección de un número razonable de errores en un programa.**

Aunque frecuentemente se pone el foco en el criterio, para aplicarlo eficazmente también es importante la estrategia de generación de casos de prueba que permita satisfacerlo.

Dado que en la práctica no es posible revelar todos los fallos de un programa mediante pruebas, y que cada criterio tiene sus limitaciones, **lo más recomendable es utilizar múltiples criterios durante el proceso de prueba.**

10.1.4 Psicología de las pruebas

Como hemos visto, idear un conjunto de casos de prueba que garantice que se detectarán todos los errores no es factible. Además, no existen métodos formales o precisos para seleccionar casos de prueba. Aunque existen una serie de heurísticas y reglas prácticas para decidir los casos de prueba, la selección de los casos de prueba sigue siendo una actividad creativa que depende de la ingeniosidad del probador. Debido a esto, la psicología de la persona que realiza las pruebas se vuelve importante.

El **propósito básico de las pruebas es detectar los errores** que puedan estar presentes en el programa. Por lo tanto, **no se debe comenzar a probar con la intención de demostrar que un programa funciona, sino con la intención de demostrar que un programa no funciona.** Con esto en mente podemos definir las pruebas como el proceso de ejecutar un programa con la intención de encontrar errores.

Este énfasis en la intención adecuada de las pruebas no es un asunto trivial porque los casos de prueba son diseñados por seres humanos, y los seres humanos tienden a realizar acciones para lograr el objetivo que tienen en mente. Entonces, si el objetivo es demostrar que un programa funciona, podemos consciente o inconscientemente seleccionar casos de prueba que intenten

demostrar ese objetivo y que vayan en contra del propósito básico de las pruebas. Por otro lado, si la intención es demostrar que el programa no funciona, pondremos a prueba nuestro intelecto para encontrar casos de prueba con ese fin, y es probable que detectemos más errores. Las pruebas son esencialmente un proceso destructivo, donde el probador debe tratar el programa como un adversario que debe ser vencido mostrando la presencia de errores. Con esto en mente, un caso de prueba es “bueno” si detecta un error que aún no ha sido detectado en el programa, y nuestro objetivo al diseñar casos de prueba debe ser diseñar tales casos “buenos”.

Una de las razones por las que muchas organizaciones requieren que un producto sea probado por personas que no participaron en el desarrollo del programa antes de entregarlo al cliente es este factor psicológico. Es difícil ser destructivo con algo que hemos creado nosotros mismos, y a todos nos gusta creer que el programa que hemos escrito “funciona”. Por lo tanto, no es fácil para alguien probar su propio programa con el estado mental adecuado para las pruebas. Otra razón para las pruebas independientes es que a veces los errores ocurren porque el programador no entendió claramente las especificaciones. Las pruebas de un programa por su propio programador no detectarán tales errores, mientras que las pruebas independientes pueden tener éxito en encontrarlos.

Este enfoque hacia las pruebas es adecuado para las primeras etapas de las pruebas, donde en efecto el objetivo es revelar errores. Sin embargo, a menudo las últimas etapas de las pruebas están destinadas más a evaluar el producto. En estos tipos de pruebas, los casos de prueba se seleccionan principalmente para imitar el comportamiento o los escenarios de los usuarios.

10.2 Pruebas de caja negra (Black-Box Testing)

Existen dos enfoques básicos para las pruebas: caja negra y caja blanca. En las pruebas de caja negra no se considera la estructura del programa. Los casos de prueba se deciden únicamente en función de los requisitos o especificaciones del programa o módulo, y no se tienen en cuenta los aspectos internos del módulo o programa para la selección de los casos de prueba. En esta sección, presentaremos algunas técnicas para generar casos de prueba para las pruebas de caja negra. Las pruebas de caja blanca se discuten en la siguiente sección.

En las pruebas de caja negra, el evaluador solo conoce las entradas que pueden darse al sistema y qué salida debería producir el sistema. En otras palabras, la base para decidir los casos de prueba en las pruebas funcionales son los requisitos o especificaciones del sistema o módulo. Esta forma de prueba también se denomina prueba funcional o de comportamiento.

El procedimiento de prueba funcional más obvio es la prueba exhaustiva, que, como hemos mencionado, es impracticable. Un criterio para generar casos de prueba es generarlos aleatoriamente. Esta estrategia tiene pocas posibilidades de producir un conjunto de casos de prueba cercano al óptimo (es decir, que detecte el máximo de errores con un número mínimo de casos de prueba). Por lo tanto, necesitamos algún otro criterio o regla para seleccionar los casos de prueba. No existen reglas formales para diseñar casos de prueba para las pruebas funcionales. De hecho, no hay criterios precisos para seleccionar los casos de prueba. Sin embargo, existen una serie de técnicas o heurísticas que pueden utilizarse para seleccionar casos de prueba y que han demostrado ser muy exitosas en la detección de errores. A continuación mencionamos algunas de estas técnicas.

10.2.1 Partición en Clases de Equivalencia

Como no podemos realizar pruebas exhaustivas, el siguiente enfoque natural es dividir el dominio de entrada en un conjunto de clases de equivalencia, de modo que si el programa funciona correctamente para un valor, entonces funcionará correctamente para todos los demás valores en esa clase. Si realmente podemos identificar tales clases, entonces probar el programa con un valor de cada clase de equivalencia es equivalente a realizar una prueba exhaustiva del programa.

Sin embargo, sin observar la estructura interna del programa, es imposible determinar tales clases de equivalencia ideales (incluso con la estructura interna, generalmente no puede hacerse). El método de partición en clases de equivalencia busca aproximar este ideal. Una clase de equivalencia se forma con las entradas para las cuales el comportamiento del sistema se especifica o se espera que sea similar. Cada grupo de entradas para las cuales se espera un comportamiento diferente se considera una clase de equivalencia separada. La justificación para formar las clases de equivalencia de esta manera es la suposición de que si las especificaciones requieren el mismo comportamiento para cada elemento de una clase de valores, entonces es probable que el programa esté construido de tal forma que falle o tenga éxito para todos los valores de esa clase. Por ejemplo, las especificaciones de un módulo que determina el valor absoluto de números enteros indican un comportamiento para los enteros positivos y otro para los enteros negativos. En este caso, formaremos dos clases de equivalencia: una que contenga los enteros positivos y otra que contenga los enteros negativos.

Para un software robusto, también debemos considerar las entradas inválidas. Es decir, debemos definir clases de equivalencia también para entradas inválidas.

Las clases de equivalencia suelen formarse considerando cada condición especificada sobre una entrada como la especificación de una clase de equivalencia válida y una o más clases de equivalencia inválidas. Por ejemplo, si una condición de entrada especifica un rango de valores (por ejemplo, $0 < \text{count} < \text{Max}$), entonces se forma una clase de equivalencia válida con ese rango y dos clases de equivalencia inválidas: una con valores menores que el límite inferior (es decir, $\text{count} < 0$) y otra con valores mayores que el límite superior ($\text{count} > \text{Max}$). Si la entrada especifica un conjunto de valores y los requisitos especifican comportamientos distintos para diferentes elementos del conjunto, entonces se forma una clase de equivalencia válida para cada elemento del conjunto y una clase inválida para una entidad que no pertenezca al conjunto.

Un enfoque común para determinar clases de equivalencia es el siguiente. Si existe razón para creer que todo el rango de una entrada no será tratado de la misma manera, entonces el rango debe dividirse en dos o más clases de equivalencia, cada una consistente en valores para los cuales se espera un comportamiento similar. Por ejemplo, para una entrada de tipo carácter, si tenemos razones para creer que el programa actuará de manera diferente si el carácter es una letra, un número o un carácter especial, entonces debemos dividir la entrada en tres clases de equivalencia válidas.

Otro enfoque para formar clases de equivalencia es considerar cualquier valor especial para el cual el comportamiento podría ser diferente como una clase de equivalencia. Por ejemplo, el valor 0 podría ser un valor especial para una entrada entera.

Además, para cada clase de equivalencia válida, deben identificarse una o más clases de equivalencia inválidas.

A menudo es útil considerar también las clases de equivalencia en la salida. Para una clase de equivalencia de salida, el objetivo es tener entradas tales que la salida de ese caso de prueba pertenezca a la clase de salida correspondiente. Como ejemplo, consideremos un programa que determina la tasa de retorno de una inversión. Hay tres clases de equivalencia de salida claras: tasa de retorno positiva, tasa de retorno negativa y tasa de retorno nula. Durante las pruebas, es importante probar cada una de ellas, es decir, proporcionar entradas que generen cada uno de esos tres tipos de salida. Determinar los casos de prueba para las clases de salida puede ser más difícil, pero se ha comprobado que estas clases revelan errores que no se detectan al considerar solo las clases de entrada.

Una vez seleccionadas las clases de equivalencia para cada una de las entradas, el problema es seleccionar los casos de prueba adecuadamente. Hay diferentes maneras de seleccionar los casos de prueba. Una estrategia es seleccionar cada caso de prueba cubriendo tantas clases de equivalencia válidas como sea posible, y un caso de prueba separado para cada clase de equivalencia inválida. Una estrategia algo mejor, que requiere más casos de prueba, es tener un caso de prueba que cubra como máximo una clase de equivalencia válida por cada entrada, y un caso de prueba separado para cada clase de equivalencia inválida. En este último caso, el número de casos de prueba para las clases de equivalencia válidas es igual al número más grande de clases de equivalencia de cualquier entrada, más el número total de clases de equivalencia inválidas.

Como ejemplo, consideremos un programa que toma dos entradas: una cadena s de longitud hasta N y un número entero n . El programa debe determinar los n caracteres que más se repiten en s . El evaluador cree que el programador puede manejar diferentes tipos de caracteres por separado. Un conjunto de clases de equivalencia válidas e inválidas para este caso se muestra en la Tabla 10.1.

Con estas clases de equivalencia, debemos seleccionar los casos de prueba. Un caso de prueba en este ejemplo es un par de valores para s y n . Con la primera estrategia para decidir los casos de prueba, un caso podría ser: s como una cadena de longitud menor que N que contenga letras minúsculas, mayúsculas, números y caracteres especiales; y n como el número 5. Este caso de prueba cubre todas las clases de equivalencia válidas (EQ1 a EQ6). Luego tendremos un caso de prueba para cubrir cada clase de equivalencia inválida (IEQ1, IEQ2 e IEQ3). Es decir, se necesitan un total de 4 casos de prueba.

Con el segundo enfoque, en un caso de prueba solo podemos cubrir una clase de equivalencia por entrada. Entonces, un caso de prueba podría ser: una cadena de números y el valor 5. Esto cubre EQ1 y EQ6. Luego necesitaremos casos de prueba adicionales para EQ2 a EQ5 y casos separados para IEQ1 a IEQ3.

10.2.2 Análisis de Valores Límite

Se ha observado que los programas que funcionan correctamente para un conjunto de valores dentro de una clase de equivalencia fallan en algunos valores especiales. Estos valores a menudo se encuentran en el límite de la clase de equivalencia. Por lo tanto, los casos de prueba que contienen valores en los límites de las clases de equivalencia tienden a ser casos de prueba de “alto rendimiento”, y seleccionar tales casos de prueba es el objetivo del análisis de valores límite. En el análisis de valores límite, se elige una entrada para un caso de prueba dentro de una clase de equivalencia, de modo que la entrada se encuentre en el borde de la clase de equivalencia. Deben cubrirse los valores límite de cada clase de equivalencia, incluidas las clases de equivalencia de salida. Los casos de prueba de valores límite también se denominan “casos extremos”. Por lo tanto,

podemos decir que un caso de prueba de valor límite es un conjunto de datos de entrada que se encuentra en el borde o límite de una clase de datos de entrada o que genera una salida que se encuentra en el límite de una clase de datos de salida.

En el caso de rangos, para el análisis de valores límite es útil seleccionar los elementos límite del rango y un valor inválido justo más allá de los dos extremos (para las dos clases de equivalencia inválidas). Así, si el rango es $0.0 < x < 1.0$, los casos de prueba serían 0.0 y 1.0 (entradas válidas), y -0.1 y 1.1 (entradas inválidas). De manera similar, si la entrada es una lista, se debe prestar atención a los primeros y últimos elementos de la lista.

También debemos considerar las salidas para el análisis de valores límite. Si puede identificarse una clase de equivalencia en la salida, debemos intentar generar casos de prueba que produzcan salidas que se encuentren en los límites de las clases de equivalencia. Además, debemos intentar formar casos de prueba que produzcan una salida que no pertenezca a la clase de equivalencia (si logramos producir una entrada que genere una salida fuera de la clase de equivalencia, habremos detectado un error).

Al igual que en la partición en clases de equivalencia, en el análisis de valores límite primero determinamos los valores de cada una de las variables que deben probarse durante las pruebas. Si hay múltiples entradas, ¿cómo debe formarse el conjunto de casos de prueba que cubra los valores límite? Supongamos que cada variable de entrada tiene un rango definido. Entonces hay 6 valores límite: los extremos del rango, los valores justo más allá de los extremos y los valores justo antes de los extremos. Si el rango de un entero va de min a max, entonces los seis valores son $\text{min} - 1$, min, $\text{min} + 1$, $\text{max} - 1$, max, $\text{max} + 1$. Supongamos que hay n variables de entrada. Existen dos estrategias para combinar los valores límite de las diferentes variables en los casos de prueba.

En la primera estrategia, se seleccionan los diferentes valores límite de una variable y se mantienen las demás variables en algún valor nominal. Además, se selecciona un caso de prueba compuesto por los valores nominales de todas las variables. En este caso, tendremos $6n + 1$ casos de prueba. Para dos variables X y Y , los 13 casos de prueba se muestran en la Figura 10.2.

Una segunda estrategia consiste en probar todas las combinaciones posibles de los valores de las diferentes variables. Dado que hay 7 valores para cada variable (6 valores límite y un valor nominal), si hay n variables, habrá un total de 7^n casos de prueba.

10.2.3 Graficado Causa-Efecto

Una debilidad de los métodos de partición en clases de equivalencia y de análisis de valores límite es que consideran cada entrada por separado. Es decir, ambos se concentran en las condiciones y clases de una sola entrada. No consideran combinaciones de condiciones de entrada que puedan formar situaciones interesantes que deban probarse. Una forma de ejercitar combinaciones de diferentes condiciones de entrada es considerar todas las combinaciones válidas de las clases de equivalencia de las condiciones de entrada. Este enfoque simple resultará en un número inusualmente grande de casos de prueba, muchos de los cuales no serán útiles para revelar nuevos errores. Por ejemplo, si hay n diferentes condiciones de entrada tales que cualquier combinación de ellas es válida, tendremos 2^n casos de prueba.

El graficado causa-efecto es una técnica que ayuda a seleccionar combinaciones de condiciones de entrada de manera sistemática, de modo que el número de casos de prueba no se vuelva inmanejablemente grande. La técnica comienza identificando las causas y efectos del sistema que se

está probando. Una causa es una condición de entrada distinta, y un efecto es una condición de salida distinta. Cada condición forma un nodo en el gráfico causa-efecto. Las condiciones deben expresarse de modo que puedan establecerse como verdaderas o falsas. Por ejemplo, una condición de entrada puede ser “el archivo está vacío”, lo cual puede establecerse como verdadero con un archivo vacío o como falso con un archivo no vacío.

Después de identificar las causas y los efectos, para cada efecto se identifican las causas que pueden producir ese efecto y cómo deben combinarse las condiciones para hacer que el efecto sea verdadero. Las condiciones se combinan utilizando los operadores booleanos “and”, “or” y “not”, que en el gráfico se representan por $\wedge$, $\vee$ y $\neg$. Luego, para cada efecto, se generan todas las combinaciones de causas de las que depende el efecto y que lo hacen verdadero (las causas de las que el efecto no depende se consideran “no importa”). Al hacer esto, se identifican las combinaciones de condiciones que hacen que diferentes efectos sean verdaderos. A partir de allí, se genera un caso de prueba para cada combinación de condiciones que haga verdadero algún efecto.

Ilustremos esta técnica con un pequeño ejemplo. Supongamos que para una base de datos bancaria hay dos comandos permitidos:

```
credit acct_number transaction_amount
debit acct_number transaction_amount
```

Los requisitos establecen que si el comando es *credit* y el *acct_number* es válido, entonces la cuenta se acredita. Si el comando es *debit*, el número de cuenta es válido y el *transaction_amount* es válido (menor que el saldo), entonces la cuenta se debita. Si el comando no es válido, el número de cuenta no es válido o el monto de débito no es válido, se genera un mensaje adecuado.

Podemos identificar las siguientes causas y efectos a partir de estos requisitos:

Causas:

- c1. El comando es *credit*
- c2. El comando es *debit*
- c3. El número de cuenta es válido
- c4. El monto de la transacción es válido

Efectos:

- e1. Imprimir “comando inválido”
- e2. Imprimir “número de cuenta inválido”
- e3. Imprimir “monto de débito no válido”
- e4. Debitar cuenta
- e5. Acreditar cuenta

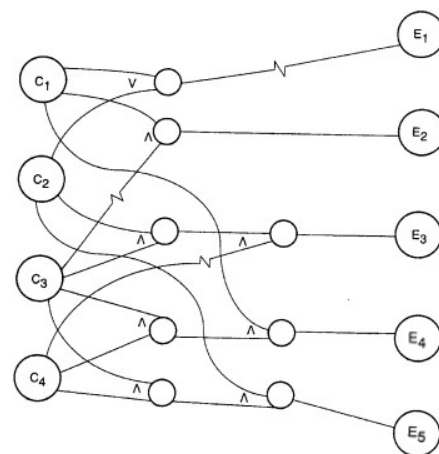


Figura 10.3: El grafo de causa-efecto.

El gráfico causa-efecto de esto se muestra en la Figura 10.3. En el gráfico, se captura la relación causa-efecto de este ejemplo. Para todos los efectos, puede determinarse fácilmente de qué causas depende cada efecto y la naturaleza exacta de la dependencia. Por ejemplo, según este gráfico, el efecto *e5* depende de las causas *c2*, *c3* y *c4* de tal manera que *e5* se activa cuando todas *c2*, *c3* y *c4* son verdaderas. De manera similar, el efecto *e2* se activa si *c3* es falso.

A partir de este gráfico, se puede generar una lista de casos de prueba. La estrategia básica es establecer un efecto en 1 y luego fijar las causas que permiten esa condición. La combinación de causas forma el caso de prueba. Una causa puede establecerse en falso, verdadero o “no importa”

(cuando el efecto no depende de esa causa). Para realizar esto con todos los efectos, resulta conveniente usar una tabla de decisión. La **tabla de decisión** para este ejemplo se muestra en la Figura 10.4.

SNo.	1	2	3	4	5
c1	0	1	x	x	1
C2	0	x	1	1	x
c3	x	0	1	1	1
c4	x	x	0	1	1
e1	1				
e2		1			
e3			1		
e4				1	
e5					1

Figura 10.4: Tabla de decisión para el grafo de causa-efecto.

Esta tabla **enumera las combinaciones de condiciones necesarias para activar diferentes efectos**. Cada combinación de condiciones en la tabla para un efecto constituye un caso de prueba. En conjunto, estas combinaciones de condiciones verifican los diferentes efectos que el software debe mostrar. Por ejemplo, para probar el efecto *e3*, deben establecerse *c2* y *c4*. Es decir, para probar el efecto “Imprimir monto de débito no válido”, el caso de prueba debe ser: el comando es *debit* (configurando *c2* en verdadero), el número de cuenta no es válido (configurando *c3* en falso) y el monto de la transacción no es correcto (configurando *c4* en falso).

El graficado causa-efecto, además de generar casos de prueba de alto rendimiento, también ayuda a comprender la funcionalidad del sistema, ya que el tester debe identificar las causas y efectos distintos. Existen métodos para reducir el número de casos de prueba generados mediante un recorrido adecuado del gráfico. Una vez que se enumeran las causas y efectos y se especifican sus dependencias, gran parte del trabajo restante también puede automatizarse.

10.2.4 Pruebas por pares

Generalmente hay muchos parámetros que determinan el comportamiento de un sistema de software. Estos parámetros pueden ser entradas directas al software o configuraciones implícitas como las de los dispositivos. Estos parámetros pueden tomar diferentes valores, y para algunos de ellos el software puede no funcionar correctamente. Muchos de los defectos en el software generalmente involucran una condición, es decir, algún valor especial de uno de los parámetros. A este tipo de defecto se le llama fallo de modo único. Ejemplos simples de fallos de modo único son un software que no puede imprimir para un tipo particular de impresora, un software que no puede calcular correctamente una tarifa cuando el pasajero es menor de edad, o un software de facturación telefónica que no calcula correctamente la factura para un país en particular.

Los fallos de modo único pueden detectarse probando diferentes valores de distintos parámetros. Entonces, si hay n parámetros en un sistema, y cada uno de ellos puede tomar m valores diferentes (o m clases diferentes de valores, considerando cada clase como equivalente a efectos de prueba, como en la partición en clases de equivalencia), con cada caso de prueba podemos probar un valor

distinto de cada parámetro. En otras palabras, podemos probar todos los diferentes valores en m casos de prueba.

Sin embargo, no todos los fallos son de modo único, y hay combinaciones de entradas que revelan la presencia de fallos. Por ejemplo, un software de facturación telefónica que no calcula correctamente las llamadas nocturnas (un parámetro) hacia un país en particular (otro parámetro). O un sistema de emisión de boletos aéreos que tiene un comportamiento incorrecto cuando un menor (un parámetro) viaja en clase ejecutiva (otro parámetro) y no se queda durante el fin de semana (un tercer parámetro). Estos fallos de modo múltiple pueden detectarse durante las pruebas intentando diferentes combinaciones de valores de parámetros, un enfoque llamado **pruebas combinatorias**.

Desafortunadamente, las pruebas combinatorias completas suelen no ser factibles. Para un sistema con n parámetros, cada uno con m valores, el número de combinaciones diferentes es m^n . Para un sistema simple con 5 parámetros, cada uno con 5 valores distintos, el número total de combinaciones es 3.125. Y si probar cada combinación toma 5 minutos, se necesitaría más de un mes para probarlas todas. Claramente, para sistemas complejos que tienen muchos parámetros y cada parámetro puede tener muchos valores, una prueba combinatoria completa no es factible, por lo que se necesitan técnicas prácticas para reducir el número de pruebas.

Algunas investigaciones han sugerido que la mayoría de los fallos de software se revelan por algunos valores únicos o por la interacción entre pares de valores. Es decir, la mayoría de los fallos tienden a ser de modo único o de doble modo. Para probar los fallos de doble modo, no es necesario probar el sistema con todas las combinaciones de valores de parámetros, sino de forma tal que se ejerciten todas las combinaciones de valores para cada par de parámetros. A esto se le llama **pruebas por pares** (*pair-wise testing*).

En las pruebas por pares, todos los pares de valores deben ejercitarse durante las pruebas. Si hay n parámetros, cada uno con m valores, entre cada dos parámetros hay $m \times m$ pares. El primer parámetro tendrá esa cantidad de pares con cada uno de los $n - 1$ parámetros restantes, el segundo tendrá nuevos pares con $n - 2$ parámetros (ya que sus pares con el primero ya están incluidos en los del primer parámetro), el tercero tendrá pares con $n - 3$ parámetros, y así sucesivamente. Es decir, el número total de pares es $m \times m \times n \times (n - 1) / 2$.

El objetivo de las pruebas por pares es tener un conjunto de casos de prueba que cubran todos los pares. Como hay n parámetros, un caso de prueba es una combinación de valores de esos parámetros y cubrirá $(n - 1) + (n - 2) + \dots = n(n - 1) / 2$ pares. En el mejor de los casos, cuando cada par es cubierto exactamente una vez por un caso de prueba, se necesitarán m^2 casos de prueba diferentes para cubrir todos los pares.

Como ejemplo, consideremos un producto de software que se desarrolla para múltiples plataformas y usa el navegador como interfaz. Supongamos que el software está diseñado para funcionar en tres sistemas operativos diferentes y tres navegadores distintos. Además, como el producto consume mucha memoria, se desea probar su rendimiento bajo distintos niveles de memoria. Entonces tenemos los siguientes tres parámetros con sus valores respectivos:

Sistema Operativo: Windows, Solaris, Linux

Tamaño de Memoria: 128M, 256M, 512M

Navegador: IE, Netscape, Mozilla

Para el análisis, podemos decir que el sistema tiene tres parámetros: A (sistema operativo), B (tamaño de memoria) y C (navegador). Cada uno de ellos puede tener tres valores que llamaremos a_1, a_2, a_3 ; b_1, b_2, b_3 ; y c_1, c_2, c_3 . El número total de combinaciones por pares es $9 \times 3 = 27$. Sin embargo, el número de casos de prueba necesarios para cubrir todos los pares es mucho menor. Un caso de prueba que combine valores de los tres parámetros cubre tres combinaciones (A-B, B-C y A-C). Por lo tanto, en el mejor de los casos, podemos cubrir las 27 combinaciones con $27/3 = 9$ casos de prueba. Estos casos se muestran en la Tabla 10.2 junto con los pares que cubren.

Como debe quedar claro, generar casos de prueba que cubran todos los pares no es una tarea sencilla. El conjunto mínimo de casos de prueba es aquel en el que cada par está cubierto exactamente una vez por un caso. A menudo no será posible generar el conjunto mínimo de casos de prueba, especialmente cuando el número de valores de los distintos parámetros es diferente. Se han propuesto varios algoritmos y algunos programas están disponibles en línea para generar los casos de prueba que cubren todos los pares.

Para muchas situaciones donde la generación manual es factible, puede seguirse el siguiente enfoque: comenzar con una combinación de valores de parámetros. Seguir agregando nuevas combinaciones eligiendo valores tales que no existan dos valores juntos en ningún caso de prueba anterior, hasta cubrir todos los pares. Cuando no sea posible seleccionar tales valores, elegir los valores que tengan la menor cantidad de coincidencias con combinaciones previas. Esencialmente, estamos generando un caso de prueba que puede cubrir la mayor cantidad posible de nuevos pares. Evitando cubrir pares múltiples veces, se puede producir un conjunto pequeño de casos de prueba que cubran todos los pares. Existen algoritmos eficientes para generar el menor número posible de casos de prueba para pruebas por pares. En [40] se da un ejemplo en el cual, para 13 parámetros, cada uno con tres valores distintos, todos los pares se cubren con apenas 15 casos de prueba, mientras que el número total de combinaciones supera el millón.

Las pruebas por pares son una forma práctica de probar sistemas de software grandes que tienen muchos parámetros diferentes con un comportamiento distinto esperado para diferentes valores. Un ejemplo sería un sistema de facturación (telefónico, hotelero, aéreo, etc.) que tiene diferentes tarifas para distintos valores de parámetros. También es un enfoque práctico para probar productos de software de propósito general que se espera funcionen en diferentes plataformas y configuraciones, o un sistema que deba interactuar con distintos tipos de sistemas.

10.2.5 Casos especiales

Se ha observado que los programas a menudo producen un comportamiento incorrecto cuando las entradas forman algunos casos especiales. La razón es que, en los programas, algunas combinaciones de entradas necesitan un tratamiento particular, y brindar un manejo adecuado para estos casos especiales suele pasarse por alto. Por ejemplo, en una rutina aritmética, si hay una división y el divisor es cero, debe tomarse alguna acción especial, la cual fácilmente podría ser olvidada por el programador. Estos casos especiales constituyen casos de prueba particularmente buenos, que pueden revelar errores que normalmente no serían detectados por otros casos de prueba.

Los **casos especiales** a menudo dependerán de las estructuras de datos y de la función del módulo. No existen reglas para determinar los casos especiales, y el tester debe usar su intuición y

experiencia para identificarlos. En consecuencia, **determinar casos especiales también se llama adivinación de errores** (*error guessing*).

La psicología es particularmente importante para la adivinación de errores. El tester debe adoptar el papel de “abogado del diablo” e intentar adivinar las suposiciones incorrectas que el programador podría haber hecho y las situaciones que el programador podría haber pasado por alto o manejado incorrectamente. En esencia, el tester intenta identificar situaciones propensas a errores. Luego, se escriben casos de prueba para estas situaciones. Por ejemplo, en el problema de encontrar el número de palabras diferentes en un archivo (discutido en capítulos anteriores), algunos de los casos especiales pueden ser: el archivo está vacío, solo hay una palabra en el archivo, solo una palabra por línea, algunas líneas vacías en el archivo de entrada, presencia de más de un espacio entre palabras, todas las palabras son iguales, las palabras ya están ordenadas y espacios al inicio o al final del archivo.

Las suposiciones incorrectas suelen hacerse porque las especificaciones no están completas o porque quien las redactó puede no haber mencionado algunas propiedades, asumiendo que eran obvias. Siempre que se dependa de una comprensión tácita en lugar de una declaración explícita en las especificaciones, existe la posibilidad de hacer suposiciones erróneas. Con frecuencia, se hacen suposiciones incorrectas sobre los entornos. Sin embargo, debe señalarse que los casos especiales dependen en gran medida del problema, y el tester realmente debe “ponerse en los zapatos” del diseñador y del programador para determinar estos casos.

10.2.6 Pruebas basadas en estados

Hay algunos sistemas que son esencialmente *sin estado*, en el sentido de que para las mismas entradas siempre dan las mismas salidas o exhiben el mismo comportamiento. Muchos sistemas de procesamiento por lotes, sistemas computacionales y servidores pertenecen a esta categoría. En hardware, los circuitos combinatoriales pertenecen a esta categoría. A un nivel más pequeño, la mayoría de las funciones se supone que se comportan de esta manera.

Sin embargo, existen muchos sistemas cuyo comportamiento **depende del estado**, en el sentido de que para entradas idénticas se comportan de manera diferente en distintos momentos y pueden producir diferentes salidas. La razón de este comportamiento distinto es el estado del sistema; es decir, **el comportamiento y las salidas del sistema dependen no solo de las entradas proporcionadas, sino también del estado del sistema**. El estado del sistema depende de las entradas pasadas que el **sistema ha recibido**. En otras palabras, el estado representa el impacto acumulativo de todas las entradas pasadas en el sistema. En hardware, los sistemas secuenciales pertenecen a esta categoría. En software, muchos sistemas grandes pertenecen a esta categoría, ya que el estado pasado se guarda en bases de datos o archivos y se usa para controlar el comportamiento del sistema. Para estos sistemas, otro enfoque para seleccionar casos de prueba es el enfoque de **pruebas basadas en estados** [34].

Teóricamente, cualquier software que guarde estado puede modelarse como una **máquina de estados**. Sin embargo, el espacio de estados de cualquier programa razonable es casi infinito, ya que es el producto cartesiano de los dominios de todas las variables que conforman el estado. Para muchos sistemas, el espacio de estados puede dividirse en unos pocos estados, cada uno representando una combinación lógica de valores de diferentes variables de estado que comparten alguna propiedad de interés [16]. Si el conjunto de estados de un sistema es manejable, puede construirse un modelo de estados del sistema.

Un modelo de estados de un sistema tiene cuatro componentes:

- **Estados.** Representan el impacto de las entradas pasadas al sistema.
- **Transiciones.** Representan cómo cambia el estado del sistema de uno a otro en respuesta a ciertos eventos.
- **Eventos.** Entradas al sistema.
- **Acciones.** Las salidas ante los eventos.

El modelo de estados muestra qué transiciones de estado ocurren y qué acciones se realizan en el sistema en respuesta a eventos. Cuando un modelo de estados se construye a partir de los requisitos de un sistema, solo podemos incluir los estados, transiciones y acciones que estén indicados en los requisitos o que puedan inferirse de ellos. Si hay más información disponible en las especificaciones de diseño, entonces puede construirse un modelo de estados más completo.

Por ejemplo, consideremos el ejemplo de la encuesta estudiantil discutido en el Capítulo 4. Según los requisitos, se debe crear un sistema para realizar una encuesta a los estudiantes. El estudiante realiza la encuesta y se le devuelve el resultado actual de la encuesta. El resultado de la encuesta puede tener hasta cinco encuestas de antigüedad. Consideramos la última arquitectura mostrada en la Figura 4.11, que tenía una memoria caché entre el servidor y la base de datos, en la cual la encuesta y los resultados se almacenan en caché y se actualizan solo después de 5 encuestas, al llegar una nueva solicitud. La arquitectura propuesta tiene una base de datos en el fondo, la cual puede fallar.

Para crear un modelo de máquina de estados de este sistema, notamos que, en una serie de seis solicitudes, las primeras cinco pueden tratarse de manera diferente. Por lo tanto, dividimos en dos estados: uno que representa la recepción de 1 a 4 solicitudes (estado 1) y otro que representa la recepción de la quinta solicitud (estado 2). Luego vemos que la base de datos puede estar activa o caída, y puede fallar en cualquiera de estos dos estados. Sin embargo, el comportamiento de las solicitudes cuando la base de datos está caída puede ser diferente. Por lo tanto, creamos otro par de estados (estados 3 y 4).

Una vez que la base de datos falla, las primeras 5 solicitudes se atienden usando datos antiguos. Cuando se recibe una solicitud después de recibir 5 solicitudes, el sistema entra en un estado de fallo (estado 5), en el cual no da ninguna respuesta. Cuando el sistema se recupera del estado de fallo, debe actualizar su caché inmediatamente, por lo tanto pasa al estado 2. El modelo de estados para este sistema se muestra en la Figura 10.5 (% representa una entrada del usuario para realizar la encuesta).

Nótese que estamos asumiendo que el modelo de estados del sistema puede crearse a partir de sus especificaciones o diseño. Así es como se realiza la mayoría de los modelados de estados, y así es como se construyó el modelo en el ejemplo. Una vez que el modelo de estados está construido, podemos usarlo para seleccionar casos de prueba. Cuando se implemente el diseño, estos casos de prueba pueden usarse para probar el código. Es por esto que tratamos las pruebas basadas en estados como una estrategia de **caja negra**.

Sin embargo, el modelo de estados a menudo requiere información sobre el diseño del sistema. En el ejemplo anterior, se utilizó cierto conocimiento de la arquitectura. A veces, crear el modelo de estados puede requerir información detallada sobre el diseño del sistema. Por ejemplo, para una clase, hemos visto que el modelado de estados se realiza durante el diseño, cuando ya se conoce bastante sobre la clase, sus atributos y sus métodos. Debido a esto, las pruebas basadas en estados

pueden considerarse como algo intermedio entre las pruebas de caja negra y las de caja blanca. A estas estrategias a veces se las llama **pruebas de caja gris**.

Dado un modelo de estados de un sistema, ¿cómo deberían generarse los casos de prueba? Se han propuesto muchos criterios de cobertura [123]. Aquí discutimos solo algunos. Supongamos que el conjunto de casos de prueba es T. Algunos de los criterios son:

- **Cobertura de todas las transiciones (AT)**. T debe asegurar que cada transición en el grafo de estados sea ejercida.
- **Cobertura de todos los pares de transiciones (ATP)**. T debe ejecutar todos los pares de transiciones adyacentes. (Un par de transiciones adyacentes comprende dos transiciones: una de entrada a un estado y otra de salida desde ese mismo estado).
- **Cobertura de árbol de transiciones (TT)**. T debe ejecutar todos los caminos simples, donde un camino simple es aquel que comienza desde el estado inicial y alcanza un estado que ya ha visitado en ese camino o un estado final.

El primer criterio establece que durante la prueba se activen todas las transiciones. Esto también asegurará que todos los estados sean visitados. La cobertura de pares de transiciones es un criterio más fuerte que requiere que todas las combinaciones de transiciones de entrada y salida para cada estado sean ejercidas por T. Si un estado tiene dos transiciones de entrada t1 y t2, y dos de salida t3 y t4, entonces un conjunto de casos de prueba T que ejecute t1;t3 y t2;t4 satisfará AT. Sin embargo, para satisfacer ATP, T también debe asegurar la ejecución de t1;t4 y t2;t3.

La cobertura de árbol de transiciones recibe ese nombre porque se puede construir un árbol de transiciones a partir del grafo y luego usarlo para identificar los caminos. En ATP, vamos más allá de las transiciones y establecemos que diferentes caminos en el diagrama de estados deben ejercerse durante las pruebas. Generalmente, ATP incluirá AT.

Para el ejemplo anterior, el conjunto de casos de prueba para AT se muestra a continuación en la Tabla 10.3. Aquí `req()` significa que debe hacerse una solicitud para realizar la encuesta, `fail()` significa que la base de datos debe fallar, y `recover()` significa que la base de datos caída debe recuperarse.

Como podemos ver, las pruebas basadas en estados llaman la atención sobre los estados y las transiciones. Incluso en el caso simple anterior, podemos ver que se prueban diferentes escenarios (por ejemplo, el comportamiento del sistema cuando falla la base de datos, y el comportamiento cuando falla y luego se recupera). Muchos de estos escenarios son fáciles de pasar por alto si los casos de prueba se diseñan solo mirando los dominios de entrada. El conjunto de casos de prueba es más completo si se usan otros criterios. Para este ejemplo, dejamos como ejercicio determinar los casos de prueba para los demás criterios.

10.3 Pruebas de caja blanca

En la sección anterior discutimos las pruebas de caja negra, que se centran en la función que el programa probado debe realizar y no se ocupan de la estructura interna del programa responsable de implementar dicha función. Así, las pruebas de caja negra se enfocan en la **funcionalidad** más que en la **implementación** del programa. Por otro lado, las pruebas de caja blanca se ocupan de probar la **implementación** del programa.

El objetivo de este tipo de pruebas no es ejercitar todas las diferentes condiciones de entrada o salida (aunque eso puede ser un subproducto), sino ejercitar las diferentes **estructuras de programación y estructuras de datos** utilizadas en el programa. Las pruebas de caja blanca también se denominan **pruebas estructurales**, y usaremos ambos términos de manera intercambiable.

Para probar la estructura de un programa, las pruebas estructurales buscan lograr casos de prueba que fueren la cobertura deseada de diferentes estructuras. Se han propuesto diversos criterios para esto. A diferencia de los criterios para las pruebas funcionales, que son con frecuencia imprecisos, los criterios para las pruebas estructurales son generalmente bastante precisos, ya que se basan en estructuras de programa que son formales y precisas. Aquí discutiremos tres enfoques diferentes de las pruebas estructurales:

- las pruebas basadas en **flujo de control**,
- las pruebas basadas en **flujo de datos**, y
- las **pruebas de mutación**.

10.3.1 Criterios basados en el flujo de control

Los criterios estructurales más comunes se basan en el **flujo de control** del programa. En estos criterios, se considera el **grafo de flujo de control** de un programa y se especifica la cobertura de diversos aspectos del grafo como criterios. Por lo tanto, antes de considerar los criterios, definamos con precisión un grafo de flujo de control para un programa.

Sea G el grafo de flujo de control (o simplemente grafo de flujo) de un programa P . Un **nodo** en este grafo representa un **bloque de sentencias** que siempre se ejecutan juntas; es decir, cada vez que se ejecuta la primera sentencia, todas las demás también se ejecutan. Una **arista** (i, j) (del nodo i al nodo j) representa una posible **transferencia de control** después de ejecutar la última sentencia del bloque representado por el nodo i , hacia la primera sentencia del bloque representado por el nodo j . Un nodo correspondiente a un bloque cuya primera sentencia es la sentencia de inicio de P se llama **nodo inicial** de G , y un nodo correspondiente a un bloque cuya última sentencia es una sentencia de salida se llama **nodo final**.

Una **ruta** es una secuencia finita de nodos $(n_1, n_2, \dots, n_k)$, $k > 1$, tal que existe una arista (n_i, n_{i+1}) para todos los nodos n_i en la secuencia (excepto el último nodo n_k). Una **ruta completa** es una ruta cuyo primer nodo es el nodo inicial y cuyo último nodo es un nodo final.

Ahora consideremos los criterios basados en el flujo de control. Quizás el criterio de cobertura más simple sea el de **cobertura de sentencias**, que requiere que **cada sentencia del programa se ejecute al menos una vez durante las pruebas**. En otras palabras, exige que las rutas ejecutadas durante las pruebas incluyan todos los nodos del grafo. Este criterio también se denomina **criterio de todos los nodos**.

Este criterio de cobertura **no es muy fuerte y puede dejar errores sin detectar**. Por ejemplo, si hay una sentencia **if** en el programa sin una cláusula **else**, el criterio de cobertura de sentencias se cumplirá con un caso de prueba que evalúe la condición como verdadera. No se necesitará ningún caso de prueba que asegure que la condición en la sentencia **if** se evalúe como falsa. Esta es una limitación seria, ya que las decisiones en los programas son fuentes potenciales de errores. Como ejemplo, consideremos la siguiente función para calcular el valor absoluto de un número:


```

int abs (x)
int x;
{
    if (x >= 0) x = 0 - x;
    return (x)
}

```

Este programa es claramente incorrecto. Supongamos que ejecutamos la función con el conjunto de casos de prueba { $x = 0$ } (es decir, el conjunto tiene solo un caso de prueba). El criterio de **cobertura de sentencias** se cumplirá al probar con este conjunto, pero el error no se revelará.

Un criterio de cobertura un poco más general es la **cobertura de ramas**, que requiere que cada arista en el grafo de flujo de control sea recorrida al menos una vez durante las pruebas. En otras palabras, la **cobertura de ramas** exige que cada decisión en el programa se evalúe como verdadera y como falsa al menos una vez durante las pruebas. Las pruebas basadas en cobertura de ramas suelen llamarse pruebas de ramas. El criterio de 100% de cobertura de ramas también se denomina criterio de todas las aristas [129].

La cobertura de ramas **implica** la cobertura de sentencias, ya que cada sentencia forma parte de alguna rama. En otras palabras, $C_{rama} \Rightarrow C_{sent}$. En el ejemplo anterior, un conjunto de casos de prueba que satisfaga este criterio detectará el error.

El **problema con la cobertura de ramas surge cuando una decisión tiene múltiples condiciones** (es decir, una expresión booleana con operadores lógicos *and* y *or*). En tales situaciones, una decisión puede evaluarse como verdadera y como falsa **sin que se ejerzan realmente todas las condiciones**. Por ejemplo, consideremos la siguiente función que verifica la validez de un dato. El dato es válido si está entre 0 y 100.

```

int check(x)
int x;
{
    if ((x >= ) && (x <= 200))
        check = True;
    else check = False;
}

```

El módulo es incorrecto, ya que está verificando $x < 200$ en lugar de **100** (posiblemente un error de tipeo cometido por el programador). Supongamos que el módulo se prueba con el siguiente conjunto de casos de prueba: { $x = 5$, $x = -5$ }. El criterio de **cobertura de ramas** se cumplirá para este módulo con este conjunto. Sin embargo, el error no será revelado, y el comportamiento del módulo será consistente con sus especificaciones para todos los casos de prueba de este conjunto.

Así, el criterio de cobertura se cumple, pero el error no se detecta. Esto ocurre porque la decisión se evalúa como verdadera y falsa debido a la condición ($x > 0$). La condición ($x < 200$) nunca se evalúa como falsa durante esta prueba, por lo tanto, el error en esta condición no se revela.

Este problema puede resolverse exigiendo que **todas las condiciones se evalúen como verdaderas y falsas**. Sin embargo, pueden darse situaciones en las que una decisión no tome ambos valores, incluso si cada condición individual se evalúa como verdadera y falsa. Una solución obvia a este

problema es exigir **cobertura de decisión/condición**, donde todas las decisiones y todas las condiciones dentro de las decisiones tomen ambos valores (verdadero y falso) durante las pruebas.

Los estudios han indicado que existen muchos errores cuya presencia no es detectada por las pruebas de ramas, ya que algunos errores están relacionados con **combinaciones de ramas**, y su presencia se revela mediante una ejecución que sigue el camino que incluye esas ramas.

Por lo tanto, un criterio de cobertura más general es aquel que requiere que **todas las rutas posibles en el grafo de flujo de control se ejecuten durante las pruebas**. Esto se llama el **criterio de cobertura de rutas** o **criterio de todas las rutas**, y las pruebas basadas en este criterio se denominan **pruebas de rutas**.

La dificultad con este criterio es que los programas que contienen bucles pueden tener un número infinito de posibles rutas. Además, no todas las rutas de un grafo pueden ser *factibles*, en el sentido de que puede no existir ningún conjunto de entradas para el cual esa ruta pueda ser ejecutada.

Debe quedar claro que Cruta $\Rightarrow$ Crama.

Como el criterio de cobertura de rutas conduce a un número potencialmente infinito de rutas, se han realizado algunos esfuerzos para proponer criterios intermedios entre la **cobertura de ramas** y la **cobertura de rutas**. El objetivo básico de estos enfoques es seleccionar un conjunto de rutas que aseguren la cobertura de ramas y, además, incluyan algunas rutas adicionales que puedan ayudar a revelar errores.

Un método para limitar el número de rutas es considerar dos rutas como equivalentes si difieren solo en sus subrutas causadas por los bucles. Aun con esta restricción, el número de rutas puede ser extremadamente grande.

Otro enfoque, basado en la **complejidad ciclomática**, fue propuesto en [116]. El criterio de prueba establece que, si la complejidad ciclomática de un módulo es V , entonces deben ejecutarse al menos **V rutas distintas** durante las pruebas. Ya hemos visto que la complejidad ciclomática V de un módulo es el número de **rutas independientes** en el grafo de flujo de un módulo. Como estas son rutas independientes, todas las demás pueden representarse como una combinación de estas rutas básicas. Estas rutas básicas son finitas, mientras que el número total de rutas en un módulo con bucles puede ser infinito.

Debe señalarse que ninguno de estos criterios es suficiente para detectar **todos los tipos de errores** en los programas. Por ejemplo, si un programa omite algunos caminos de flujo de control necesarios para verificar un valor especial (como *puntero igual a nil* o *divisor igual a cero*), entonces incluso ejecutar todas las rutas no necesariamente detectará el error.

Del mismo modo, si el conjunto de rutas satisface el criterio de todas las rutas pero solo ejerce una parte de una condición compuesta, entonces no revelará ningún error en la parte de la condición que no se haya ejecutado.

Por lo tanto, incluso el **criterio de cobertura de rutas**, que es el más fuerte de los criterios discutidos, **no es lo suficientemente potente como para garantizar la detección de todos los errores**.

10.4 Proceso de Testing

El objetivo básico del proceso de desarrollo de software es producir software que no tenga errores o que tenga muy pocos errores. En un intento por detectar errores poco después de que son introducidos, cada fase termina con una actividad de verificación como una revisión. Sin embargo, la mayoría de estas actividades de verificación en las primeras fases del desarrollo de software se basan en la evaluación humana y no pueden detectar todos los errores. Esta falta de confiabilidad de las actividades de aseguramiento de la calidad en la parte inicial del ciclo de desarrollo coloca una responsabilidad muy alta sobre el testing. En otras palabras, como el testing es la última actividad antes de que el software final sea entregado, tiene la enorme responsabilidad de detectar cualquier tipo de error que pueda estar presente en el software.

Además, sabemos que el software típicamente sufre cambios incluso después de haber sido entregado. Y para validar que un cambio no ha afectado alguna funcionalidad antigua del sistema, se realiza testing de regresión. En el testing de regresión, se ejecutan casos de prueba antiguos con la expectativa de que se produzcan los mismos resultados antiguos. La necesidad de testing de regresión impone requisitos adicionales sobre la fase de testing; debe proporcionar los casos de prueba “antiguos” y sus salidas.

Además, como hemos visto en las discusiones de este capítulo, el testing tiene sus propias limitaciones. Estas limitaciones requieren que se tenga un cuidado adicional al realizar el testing. Como el testing es la actividad más costosa en el desarrollo de software, es importante que se realice de manera eficiente.

Todos estos factores significan que el testing no debe realizarse de manera improvisada, como a veces se hace. Debe ser cuidadosamente planificado y el plan debe ejecutarse correctamente. El proceso de testing se enfoca en cómo debe proceder el testing para un proyecto en particular. Habiendo discutido varios métodos para seleccionar casos de prueba, dirigimos ahora nuestra atención al proceso de testing.

10.4.1 Niveles de Testing

El testing generalmente se utiliza para detectar las fallas que permanecen de etapas anteriores, además de las fallas introducidas durante la codificación misma. Debido a esto, se utilizan diferentes niveles de testing en el proceso de testing; cada nivel de testing apunta a probar diferentes aspectos del sistema.

Los niveles básicos son testing unitario, testing de integración y testing de sistema y de aceptación. Estos diferentes niveles de testing intentan detectar distintos tipos de fallas. La relación entre las fallas introducidas en diferentes fases y los distintos niveles de testing se muestra en la Figura 10.8.

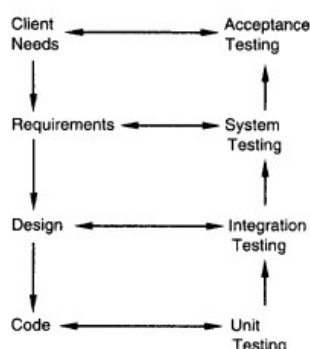


Figura 10.8: Niveles de prueba.

El primer nivel de testing se llama testing unitario. En este, diferentes módulos se prueban contra las especificaciones producidas durante el diseño de los módulos. El testing unitario es esencialmente para la verificación del código producido durante la fase de codificación, y por lo tanto el objetivo es probar la lógica interna de los módulos. Típicamente es realizado por el programador del módulo. Un módulo se considera para integración y uso por otros solo después de que ha sido probado satisfactoriamente a nivel unitario. Esto se ha discutido con más detalle en el capítulo anterior.

El siguiente nivel de testing se denomina frecuentemente testing de integración. En este, muchos módulos que ya han sido probados unitariamente se combinan en subsistemas, los cuales luego son probados. El objetivo aquí es ver si los módulos pueden integrarse correctamente. Por lo tanto, el énfasis está en probar las interfaces entre los módulos. Esta actividad de testing puede considerarse como testing del diseño.

Los siguientes niveles son el testing de sistema y el testing de aceptación. Aquí se prueba el sistema de software completo. El documento de referencia para este proceso es el documento de requisitos, y el objetivo es ver si el software cumple con sus requisitos. Este es esencialmente un ejercicio de validación, y en muchas situaciones es la única actividad de validación. El testing de aceptación a veces se realiza con datos reales del cliente para demostrar que el software funciona de manera satisfactoria. El testing en este nivel se enfoca en el comportamiento externo del sistema; la lógica interna del programa no se enfatiza. En consecuencia, en estos niveles se realiza mayormente testing funcional.

Estos niveles de testing se realizan cuando un sistema está siendo construido a partir de los componentes que han sido codificados. Existe otro nivel de testing, llamado testing de regresión, que se realiza cuando se hacen cambios a un sistema existente. Sabemos que los cambios son fundamentales en el software; cualquier software debe sufrir cambios. Frecuentemente, se realiza un cambio para “actualizar” el software agregando nuevas características y funcionalidades. Claramente, el software modificado necesita ser probado para asegurarse de que las nuevas funcionalidades agregadas efectivamente funcionan. Sin embargo, como se han realizado modificaciones a un sistema existente, también se debe realizar testing para asegurarse de que la modificación no haya tenido ningún efecto secundario no deseado que haga que algunos de los servicios anteriores fallen. Es decir, además de asegurar el comportamiento deseado de los nuevos servicios, el testing debe asegurar que se mantiene el comportamiento deseado de los servicios antiguos. Esta es la tarea del testing de regresión.

Para el testing de regresión, se mantienen algunos casos de prueba que han sido ejecutados sobre el sistema antiguo, junto con la salida producida por ese sistema. Estos casos de prueba se ejecutan nuevamente sobre el sistema modificado y su salida se compara con la salida anterior para asegurarse de que el sistema funciona como antes para estos casos de prueba. Esto frecuentemente es una tarea importante cuando se realizan modificaciones a sistemas existentes.

Una consecuencia de esto es que los casos de prueba de los sistemas deben ser documentados adecuadamente para su uso futuro en el testing de regresión. De hecho, para muchos sistemas que cambian con frecuencia, se utilizan scripts de testing de regresión, los cuales automatizan la ejecución del testing de regresión después de algunos cambios. Un script de testing de regresión ejecuta un conjunto de casos de prueba. Para cada caso de prueba, establece el estado del sistema

para el testing, ejecuta el caso de prueba, determina la salida o algún aspecto del estado del sistema después de ejecutar el caso de prueba, y verifica el estado del sistema o la salida contra los valores esperados. Estos scripts se producen típicamente durante el testing de sistema, ya que el testing de regresión generalmente se realiza solo para sistemas completos. Cuando el sistema es modificado, los scripts se ejecutan nuevamente, proporcionando las entradas especificadas en los scripts y comparando las salidas con las salidas indicadas en los scripts. Dado que existen estos scripts, mediante el uso de herramientas, el testing de regresión puede automatizarse en gran medida.

Incluso con scripts de testing, el testing de regresión de sistemas grandes puede llevar una cantidad considerable de tiempo, particularmente porque la ejecución y verificación de todos los casos de prueba no puede automatizarse completamente. Si se realiza un cambio pequeño en el sistema, a menudo no está justificado ejecutar toda la suite de casos de prueba, y el sistema se prueba solo con un subconjunto de casos de prueba. Esto requiere priorización de los casos de prueba. Para la priorización, generalmente se registra más información sobre cada caso de prueba, la cual luego se utiliza durante el testing de regresión para priorizar. Por ejemplo, un enfoque es registrar el conjunto de bloques que ejecuta cada caso de prueba. Si alguna parte del código ha cambiado, entonces los casos de prueba que ejecutan la parte modificada reciben la mayor prioridad para el testing de regresión. La priorización de casos de prueba es un área activa de investigación y en la literatura se han propuesto muchos enfoques diferentes para esto. No lo discutiremos más.